Sparse Elimination and Applications in Kinematics

by

Ioannis Zacharias Emiris

B.S.E. (Princeton University) 1989
M.S. (University of California at Berkeley) 1991

A dissertation submitted in partial satisfaction of the
requirements for the degree of
Doctor of Philosophy
in
Computer Science
in the
GRADUATE DIVISION
of the
UNIVERSITY of CALIFORNIA at BERKELEY

Committee in charge:

Professor John F. Canny, Chair
Professor Raimund Seidel
Professor Kenneth Ribet

1994

Sparse Elimination and Applications in Kinematics

by

Ioannis Zacharias Emiris

Abstract

Sparse Elimination and Applications in Kinematics

by

Ioannis Zacharias Emiris

Doctor of Philosophy in Computer Science

University of California at Berkeley

Professor John F. Canny, Chair

This thesis proposes efficient algorithmic solutions to problems in computational algebra and computational algebraic geometry. Moreover, it considers their application to different areas where algebraic systems describe kinematic and geometric constraints.

Given an arbitrary system of nonlinear multivariate polynomial equations, its *resultant* serves in eliminating variables and reduces root finding to a linear *eigenproblem*. Our contribution is to describe the first efficient and general algorithms for computing the *sparse resultant*. The sparse resultant generalizes the classical homogeneous resultant and exploits the structure of the given polynomials. Its size depends only on the geometry of the input *Newton polytopes*.

The first algorithm uses a subdivision of the Minkowski sum and produces matrix formulae whose size is well-characterized. The second algorithm uses a heuristic in order to construct smaller matrices, which are typically within a factor of three of optimal size. For most systems an optimal matrix formula does not exist, yet the second algorithm produces optimal matrices in all cases where they are known to exist.

The sparseness of the system is expressed by its *mixed volume*, which gives an upper bound on the number of isolated roots. A refinement of Sturmfels' algorithm is proposed which is very fast and has been used to derive new degree bounds for a class of benchmarks.

Two methods based on the sparse resultant are implemented for calculating the common roots and then applied to concrete problems in robotics, vision and molecular kinematics. This illustrates our claim that problems from these areas often exhibit structure that can be exploited in the context of sparse elimination. Our solver finds all isolated solutions to satisfactory accuracy and speed; furthermore, its generality should establish it as the method of choice for systems of moderate size.

The last chapter discusses the problem of input degeneracy which frequently arises in implementations of geometric as well as algebraic algorithms. The perturbation methods proposed are

computationally optimal for certain classes of algorithms and simple to implement as illustrated by our convex hull implementation.

Approved: John F. Canny

Στους γονείς μου, για την αγάπη τους.

Contents

List of Figures

List of Tables

Acknowledgments

I am most indebted to my advisor, John Canny, for his collaboration and support during these years. His insightful ideas provided invaluable guidance as well as a constant challenge.

I thank Raimund Seidel for his contributions on the subject of geometric degeneracy and perturbations. Sincere thanks go to Ken Ribet for serving on my thesis committee and to Richard Fateman for his assistance over the years. Lastly, I would like to acknowledge financial support by André Galligo for an enjoyable as well as productive semester in Sophia-Antipolis.

I was fortunate to interact with an exceptional group of graduate students at Berkeley. In particular, I am thankful to Ashu Rege for his collaboration on monomial bases, David Parsons for commenting on parts of this thesis and Jim Ruppert and Madhu Sudan for sharing the learning process as well as coffee.

Lastly, I wish to express my gratitude to my parents, my sister Thalia and my aunt Katerina for their support and encouragement. And, of course, I am grateful to Lori for her love.

Chapter 1

Introduction

A central question in computational algebra and computational algebraic geometry is the problem of variable elimination in systems of polynomials. This thesis proposes the first efficient and general algorithms for computing the resultant in the context of *toric varieties* and *sparse elimination theory*. Their complexity is exponential in the number of variables with a linear exponent.

This approach has been active for the past two decades and has already recast several existing results, primarily in intersection and elimination theory, in a radically new language. The new viewpoint takes into account the combinatorial structure of the given objects and, in particular, polynomials. Instead of characterizing them merely by their degree, it considers their *Newton polytope*, thus expressing their structure. The main tool in our approach is the *sparse resultant*, which generalizes the classical homogeneous resultant and exploits the structure defined by the given Newton polytopes.

Besides the theoretical interest of elimination, a wide variety of scientific and engineering applications lead to polynomial systems with small Newton polytopes. Root finding, which may be regarded as a special case of elimination, is thus of paramount importance in scientific computing. The second part of this thesis shows how to apply sparse resultants for reducing root finding to an eigenproblem. Moreover, we argue that certain classes of important problems, especially those expressed in terms of geometric and kinematic constraints, exhibit the type of structure modeled by sparse elimination theory.

The main object of our study is the *polynomial resultant*, or *eliminant*, of a system of multivariate polynomials of arbitrary degree. Historically, the resultant constitutes the first approach to elimination. For homogeneous polynomials, Cayley [Cay48] offered the first general constructive definition more than a century ago and Macaulay [Mac02] proposed his quotient formula in 1902. Familiar examples of the resultant include the determinant of a linear system and the Sylvester resultant of two bivariate forms.

In the mid 1970's, certain Russian mathematicians turned to $\mathbb{C}^*$ and reexamined several fundamental results of algebraic geometry in the context of toric varieties. The impressive connection of the latter to combinatorial geometry has been very fruitful and is discussed, to some extent, in the present thesis. In particular, we are interested in certain remarkable results of Kushnirenko, Bernstein and Khovanskii on bounding the number of common roots [Ber75]. In contrast to the classical theory, here polynomials are not assumed to be homogeneous.

In this context, a generalization of the classical resultant is possible. This has been the contribution of Gelfand, Kapranov and Zelevinsky [GKZ94], who set the foundations of sparse elimination theory and introduced the sparse resultant. For arbitrary systems the sparse resultant was formalized in joint work with Sturmfels [KSZ92]. The term *sparse* does not imply any requirement on the given

polynomials; rather it is meant to indicate that the degree of the resultant polynomial is proportional to the sparseness of the input polynomials. The sparse resultant generalizes its homogeneous counterpart, in much the same way that the Bernstein bound generalizes the Bezout bound.

The second main result of this thesis is an improvement to the original algorithm for the sparse resultant that leads, typically, to smaller resultant matrices. The determinant of a resultant matrix is a multiple of the resultant and is used in order to recover the resultant or directly solve the system. The question of which systems admit exact, or Sylvester-type, formulae is of fundamental significance and is still open. It has been demonstrated, though, that for *multigraded* systems optimal matrices exist [SZ94]; the second algorithm outputs optimal matrices in this case.

A related problem of independent interest is the computation of the Bernstein bound on the number of common zeros. This equals the *mixed volume* of the respective Newton polytopes. We describe what appears to be the fastest current algorithm for computing mixed volumes. We also formalize the relationship of mixed volume to the volume of the Minkowski sum: the former represents the inherent intricacy of the given system, whereas the latter is typically a measure of our algorithms' complexity.

Computer algebra is principally interested in the sparse resultant as a tool for representing as well as calculating the common zeros of systems of polynomial equations. Chapter 4 shows how root finding reduces to computing the eigendecomposition of a square matrix by means of the resultant. Moreover, there has been evidence that whenever a compact formula for the resultant is available, resultant-based methods provide the most efficient means of solution of several problems in the general area of computational science and engineering.

In particular kinematic and geometric problems arising from robotics, vision, modeling as well as other less traditional disciplines are cast in algebraic terms. We argue that the structure exploited by sparse elimination theory often models the polynomials encountered in these applications. We illustrate this claim by calculating all solutions to satisfactory accuracy and speed for certain problems. Lastly, the question of general position, of central importance in implementations that deal with geometric constraints, is addressed.

1.1 Motivation

Resultant-based methods offer the most efficient solution to certain problems in areas ranging from robotics [Can88b] to graphics and modeling [BGW88]. Implicitizing parametric surfaces is a fundamental problem in geometric and solid modeling. Given the parametric expression of a surface

$$(x, y, z, w) = (X(s,t), Y(s,t), Z(s,t), W(s,t)),$$

we wish to find its implicit description as the zero set of a single homogeneous polynomial in x, y, z, w. This is achieved by eliminating the parameters s, t from the system

$$wX(s,t) - xW(s,t) = wY(s,t) - yW(s,t) = wZ(s,t) - zW(s,t) = 0,$$

which is equivalent to computing the system's resultant by considering these equations as polynomials in s, t. For a bicubic surface, methods based on resultants have been shown to run faster by a factor of at least 10^3 compared to Gröbner bases and the Ritt-Wu method [MC92b]. If the parametrization has base points, the implicit form may be computed as a factor of the leading nonvanishing minor of the Dixon resultant. This leads to an algorithm that takes 20.5 seconds on an IBM RS/6000, while most major Gröbner bases packages run out of memory after running for a few days, even when working on a homomorphic image of the problem over a finite field [MC92a, Man94b].

Another example is the inverse kinematics problem for a 6R robot which is solved using a customized resultant in 11 milliseconds [MC92c]. This consists in finding the angles by which each of the six links of the robot must be rotated in order to attain a given final position. Homotopy methods take about 10 seconds, which is unacceptable for real-time industrial manipulators. In general, Gröbner bases and numerical algorithms may also exploit sparseness. Yet, when a compact resultant is known, several problems can be solved much faster.

There exist several classes of problems, such as camera motion reconstruction in vision and the kinematics of mechanisms, the generalized kinematics problem with constraints, the computation of aspect graphs [GCS91, RvE93] as well as problems in computational biology for which sparse methods are expected to be very efficient. Another important area of application is quantifier elimination and deciding the theory of the reals. So, ideally, we would like to have a sparse resultant for every problem, which calls for a *general algorithm* to construct them. This is the main contribution of this thesis. Moreover we discuss specific applications of our algorithm in specific cases to highlight its efficiency in practice.

1.2 Classical Elimination Theory

Elimination theory is a central piece of algebraic geometry that studies conditions and methods for eliminating variables in equivalent formulations of a given problem. The area was very active in the previous century, culminating in constructive solutions of most problems by the early 1900's, but then falling in a state of inactivity. Characteristically, the third edition of van der Waerden's *Modern Algebra* has a chapter on elimination which has disappeared from later editions. This was a casualty of the polemic between abstract and constructive mathematicians, the former apparently led by A. Weyl. The school of constructive mathematics has since gained much influence, especially with the advent of computational power which has rekindled interest in elimination methods.

The main object of interest is the *resultant* of n *homogeneous* polynomials $f_1, \ldots, f_n$ in n variables. If we consider all coefficients c_{ij} as indeterminates, the resultant is a single polynomial in the c_{ij}. It has integer coefficients and vanishes exactly when the system has a nonzero solution in $\overline{K}^n$, where $\overline{K}$ is the algebraic closure of the base field K. If the total number of coefficients is k, then the resultant can be viewed as a projection operator from $\overline{K}^{n+k-1}$ to $\overline{K}^k$. Moreover, the resultant is irreducible and is homogeneous in the coefficients of each polynomial f_i with degree

$$\prod_{j=1, j\neq i}^{n} d_j,$$

where d_j is the degree of f_j.

Since the resultant eliminates n input variables, it is also called the eliminant. Some authors have used the term *multivariate resultant* to distinguish it from the more widely known case for $n = 2$, whose resultant is named after Sylvester. In order to emphasize the difference of this resultant from the sparse resultant, which is the main focus of this thesis, we refer to it as the *homogeneous*, or classical, *resultant*. This terminology stresses the fact that the homogeneous resultant is defined for homogeneous polynomials.

The homogeneous resultant provides an exact condition for the solvability of homogeneous systems. However, it may vanish when no affine solution exists due to some component at infinity; this is most serious when this component is of excess dimension in which case the homogeneous resultant vanishes identically. This problem has been addressed in [Ren87, Can90]. Sparse elimination takes a

different approach by restricting attention to a subset of affine space and obtaining an exact condition for solvability in this subset. For sufficiently generic coefficients, we shall see that the homogeneous resultant is derived from the sparse resultant as a special case.

The resultant of two homogeneous polynomials was first studied by Euler and Bezout, though it bears the name of Sylvester [Sal85]. He defined it as the determinant of a matrix whose entries are either zero or some polynomial coefficient. In this case, and whenever the resultant is expressible as the determinant of a single matrix, we have a *Sylvester-type* formula. For a few special cases with $n \leq 6$ Sylvester-type formulae are given in [Dix08, MC27, Jou91].

For higher n Sylvester-type formulae are not generally possible though most existing formulations involve determinants of matrices in the coefficients. Specifically, Cayley [Cay48] defined the resultant via a series of n alternating divisions of determinants and more recently his formula was related to a Koszul complex associated to the given polynomials [Cha93]. Hurwitz [Hur13] introduced *inertia forms*, which are resultant multiples such that their the Greatest Common Divisor (GCD) equals the resultant. Probably the best exposition of most of this material is found in [vdW50], where the u-resultant is suggested as a tool for solving algebraic systems. Macaulay [Mac02] gave the most compact expression of the resultant as the quotient of a determinant by one of its minors. This expression has been improved for the case when all polynomials have the same degree in [Mac21].

The earliest of the contemporary constructive approaches was Lazard's [Laz81], which used the u-resultant to solve polynomial systems. Canny [Can88b] designed a parallelizable algorithm for computing the resultant of specialized polynomials out of inertia forms. The resultant construction was a building block for an optimal motion planning algorithm for the general "piano-movers" problem, while the u-resultant was also used in demonstrating gap theorems on the magnitude of roots. Macaulay's construction was used by Renegar [Ren92], who gave a more efficient algorithm for finding all common solutions and relied on the resultant in designing algorithms for deciding the theory of the reals. The asymptotic complexity of these algorithms is singly exponential in n, which is optimal.

The main drawback of the homogeneous resultant is that it may be identically singular when the variety of the homogenized system has higher dimensional components, even if the original affine system is zero-dimensional. Grigoryev, Chistov and Vorobjov in deciding the theory of the reals [CG84, GV87] as well as Canny, by means of the characteristic polynomial [Can90], deal with this issue. Further work [Cha93] has indicated ways to recover isolated roots when the system's variety has positive dimension. Auzinger and Stetter [AS88] reduced polynomial system solving to an eigenproblem; more on resultant-based methods for system solving in chapter 4.

1.3 Sparse Elimination Theory

Sparse elimination generalizes several results of classical elimination theory on multivariate polynomial systems by considering the structure of the given polynomials, namely their Newton polytopes. This leads to stronger algebraic and combinatorial results in general. The compactification $\mathbb{P}^n = \mathbb{P}^n_{\mathbb{C}}$ used by the classical theory accounts for several polynomial solutions with no physical significance, whose cardinality often dominates that of the affine roots. Sparse elimination concentrates on $(\mathbb{C}^*)^n$

$$\mathbb{C}^* = \mathbb{C} \setminus \{0\} \subset \mathbb{C} \subset \mathbb{P}^n.$$

Assume that the number of variables is n; roots in $(\mathbb{C}^*)^n$ are called *toric*. Consult [Ful93] for an introduction to toric varieties and their application to proving a series of combinatorial results, including certain properties of mixed volume and the Aleksandrov-Fenchel inequality (used in sect. 2.2). Toric

varieties are also called upon in proving the necessity implication of McMullen's conjecture on the relationship among the face numbers of simplicial polytopes [Sta90].

By considering $\mathbb{C}^*$ we may, consequently, extend our scope to *Laurent polynomials*. We use x^e to denote the monomial $x_1^{e_1} \cdots x_n^{e_n}$, where $e = (e_1, \ldots, e_n) \in \mathbb{Z}^n$ is an exponent vector or, equivalently, an integer lattice point, and $n \in \mathbb{Z}_{\geq 1}$. For any Laurent polynomial f_i,

$$f_i \in K[x_1, x_1^{-1}, \ldots, x_n, x_n^{-1}] = K[x, x^{-1}]$$

where K is a field. Formally, consider the following multiplicative subset S and define the Laurent polynomial ring as a ring of quotients:

$$S = \{x^a \mid a \in \mathbb{Z}_{\geq 0}^n\} \subset K[x] = K[x_1, \ldots, x_n], \qquad \text{then } K[x, x^{-1}] = S^{-1}K[x].$$

Laurent polynomials can be multiplied by monomials without affecting their zeros. Assume that this has been done so that no negative exponents appear. Then the degree of monomial x^e is $e_1 + e_2 + \cdots + e_n$ and the *total degree* of the original Laurent polynomial is the highest degree of any nonzero term in the new expression.

Let $\mathcal{A}_i = \text{supp}(f_i) = \{a_{i1}, \ldots, a_{i\mu_i}\} \subset \mathbb{Z}^n$ denote the set, with cardinality μ_i, of exponent vectors corresponding to monomials in f_i with nonzero coefficients. This set is the *support* of f_i and

$$f_i = \sum_{j=1}^{\mu_i} c_{ij} x^{a_{ij}}, \qquad c_{ij} \neq 0,\ j = 1, \ldots, \mu_i, \tag{1.1}$$

so that $\mathcal{A}_i$ is uniquely defined given f_i.

A *polytope* is a nonempty compact convex set with finitely many extremal points, called *vertices*.

Definition 1.3.1 *The Newton polytope of f_i is the convex hull of support $\mathcal{A}_i$, denoted $Q_i = \text{Conv}(\mathcal{A}_i) \subset \mathbb{R}^n$.*

The Newton polytope provides the bridge from the algebraic to the combinatorial geometric setting, by modeling sparseness in geometric terms. The planar construct, namely the Newton polygon, was introduced by Newton [New60, pp. 110-163] in using Puiseux series for studying the local behavior of plane curves; see also [BK86, ch. 8].

Some combinatorial geometry is required in the sequel; all concepts of interest are surveyed in [Sch93]. For arbitrary sets in $\mathbb{R}^n$ there is a natural associative and commutative addition operation called Minkowski addition.

Definition 1.3.2 *The* Minkowski sum $A + B$ *of sets A and B in $\mathbb{R}^n$ is*

$$A + B = \{a + b \mid a \in A, b \in B\} \subset \mathbb{R}^n.$$

Equivalently,

$$A + B = \bigcup_{a \in A} (a + B).$$

If A and B are convex polytopes then $A + B$ is a convex polytope.

The second "kinematic" formula in the definition interprets $A + B$ as the set covered by translating B by all vectors in A.

The Minkowski sums of convex polytopes can be computed as the Convex Hull of all sums $(a + b)$ of *vertices* of A and B respectively. The commutativity of this operation implies that translating A or B is equivalent to translating $A + B$.

Definition 1.3.3 *The* scalar multiple *of a set $A \subset \mathbb{R}^n$ by a real number $\lambda \in \mathbb{R}$ is*

$$\lambda A = \{\lambda a \,|\, a \in A\} \subset \mathbb{R}^n.$$

We introduce a new operation on subsets of $\mathbb{R}^n$ that specializes the usual Minkowski subtraction operation.

Definition 1.3.4 *Let A and B be sets in $\mathbb{R}^n$, then their* Minkowski difference *is*

$$A - B = \{x \in \mathbb{R}^n \,|\, x + B \subset A\} \subset \mathbb{R}^n.$$

If A, B are convex polytopes, then $A - B$ is a convex polytope.

We restrict B so that it always contains the origin. Then

$$A - B = (A - B) \cap A = \{a \in A \,|\, a + B \subset A\}.$$

Even when $A - B$ lies in A it does not define an inverse of the addition operation, since it does not equal $A + (-B)$ and, in general $B + (A - B) \overset{\subset}{\neq} A$. However, when A is itself a Minkowski sum $B + C$, then $(B + C) - B = C$, for any convex polytope C. We also state some identities used later: $A - (B + C) = (A - B) - C$ and $(A + U) - B = (A - B) + U$, where U is a one-dimensional polytope.

Let $\mathrm{Vol}(A)$ denote the Lebesgue measure of A in n-dimensional euclidean space, for polytope $A \subset \mathbb{R}^n$.

Definition 1.3.5 *Given convex polytopes $A_1, \ldots, A_n \subset \mathbb{R}^n$, there is a unique, up to multiplication by a scalar, real-valued function $MV(A_1, \ldots, A_n)$, called the* mixed volume *of $A_1, \ldots, A_n$ which is multilinear with respect to Minkowski addition and scalar multiplication. In other words, for $\mu, \rho \in \mathbb{R}_{\geq 0}$ and convex polytope $A'_k \subset \mathbb{R}^n$*

$$MV(A_1, \ldots, \mu A_k + \rho A'_k, \ldots, A_n) = \mu MV(A_1, \ldots, A_k, \ldots, A_n) + \rho MV(A_1, \ldots, A'_k, \ldots, A_n).$$

To define mixed volume exactly we require that

$$MV(A_1, \ldots, A_n) = n!\ \mathrm{Vol}(A_1), \qquad \text{when } A_1 = \cdots = A_n.$$

This definition differs from the classical mixed volume by $n!$. This scaling guarantees that the mixed volume of polytopes with integer vertices is integer. Despite this factor, it is clear that mixed volume generalizes polytope volume. An equivalent definition is

Definition 1.3.6 *For $\lambda_1, \ldots, \lambda_n \in \mathbb{R}_{\geq 0}$ and convex polytopes $A_1, \ldots, A_n \subset \mathbb{R}^n$, the* mixed volume *$MV(A_1, \ldots, A_n)$ is the coefficient of $\lambda_1 \lambda_2 \cdots \lambda_n$ in $\mathrm{Vol}(\lambda_1 A_1 + \cdots + \lambda_n A_n)$ expanded as a polynomial in $\lambda_1, \ldots, \lambda_n$.*

An explicit expression for the mixed volume is obtained by the Exclusion-Inclusion principle:

$$MV(A_1, \ldots, A_n) = \sum_{I \subset \{1, \ldots, n\}} (-1)^{n-|I|} \mathrm{Vol}(\sum_{i \in I} A_i),$$

where $|\cdot|$ denotes set cardinality and the second sum denotes Minkowski addition. In chapter 2 we shall examine more efficient ways for computing mixed volumes.

We now pass to *well-constrained* polynomial systems i.e., systems with the same number of polynomials and variables. A system of polynomials is *unmixed* if all supports are identical, otherwise it is *mixed*.

We are now ready to state the Bernstein bound on the number of roots in $(\mathbb{C}^*)^n$ for a system of n polynomials. This is the cornerstone of sparse elimination theory and has been generalized in several ways since its first statement. It relied on work by Kushnirenko [Kus75], while a subsequent proof and generalization to the genus of complete intersections and to intersection numbers was given by Khovanskii [Kho78]. For this it has also been called the BKK bound.

Theorem 1.3.7 [Ber75] *For polynomials $f_1, \ldots, f_n \in K[x, x^{-1}]$ with Newton polytopes $Q_1, \ldots, Q_n \subset \mathbb{R}^n$ the number of common solutions in $(\mathbb{C}^*)^n$ is either infinite or does not exceed $MV(Q_1, \ldots, Q_n)$. Equality holds when all coefficients are generic.*

Canny and Rojas have substantially weakened the requirements for equality [CR91] by requiring only the coefficients corresponding to Newton polytope vertices to be generic. A further combinatorial specification of which coefficients must be generic was proposed in [Roj94].

Fulton states a slight generalization of Khovanskii's version that applies to arbitrary varieties, including infinite ones. Consider the intersection multiplicity of the hypersurfaces defined by the given polynomials at an isolated common root α. This number is a positive integer and equals one exactly when the hypersurfaces meet transversally.

Theorem 1.3.8 [Ful93, sect. 5.5] *Given are polynomials $f_1, \ldots, f_n \in K[x, x^{-1}]$ with Newton polytopes $Q_1, \ldots, Q_n$. For any isolated common zero $\alpha \in (\mathbb{C}^*)^n$, let $i(\alpha)$ denote the intersection multiplicity at this point. Then $\sum_\alpha i(\alpha) \leq MV(Q_1, \ldots, Q_n)$, where the sum ranges over all isolated roots.*

Of course, for generic polynomials all intersections are isolated and transversal and equality holds. Another recent extension to Bernstein's theorem provides an upper bound on the number of common roots in $\mathbb{C}^n$; see also [Kho78, Roj94] for related work.

Theorem 1.3.9 [LW94] *For polynomials $f_1, \ldots, f_n \in \mathbb{C}[x, x^{-1}]$ with supports $\mathcal{A}_1, \ldots, \mathcal{A}_n$ the number of common isolated zeros in $\mathbb{C}^n$, counting multiplicities, is upwards bounded by $MV(\mathcal{A}_1 \cup \{0\}, \ldots, \mathcal{A}_n \cup \{0\})$.*

A different direction of research is towards lower as well as upper bounds on the number of common real roots in $\mathbb{R}^n$. Sturmfels [Stu92] establishes such bounds for unmixed systems calculated through polyhedral subdivisions. The generalization of the lower bound to mixed systems should be straightforward.

Bernstein's bound is at most as high as Bezout's bound, which is simply the product of the total degrees, and is usually significantly smaller for systems encountered in engineering applications. The two bounds are equal when every Newton polytope is an n-dimensional unit simplex S with all vertices on the coordinate axes and scaled by the total degree of the polynomial. In other words when each f_i, given its total degree, contains all possible terms with nonzero coefficients. Indeed, in this case

$$MV(d_1 S, \ldots, d_n S) = \prod_1^n d_i MV(S, \ldots, S) = \prod_1^n d_i.$$

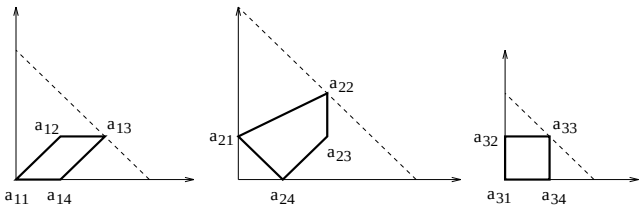

Figure 1.1: Newton polytopes for example 1.3.10.

Example 1.3.10 Here is a system of 3 polynomials in 2 unknowns

$$\begin{aligned} f_1 &= c_{11} + c_{12}xy + c_{13}x^2y + c_{14}x \\ f_2 &= c_{21}y + c_{22}x^2y^2 + c_{23}x^2y + c_{24}x \\ f_3 &= c_{31} + c_{32}y + c_{33}xy + c_{34}x \end{aligned} \tag{1.2}$$

with Newton polytopes shown in figure 1.1. Polynomial $f_2' = c_{21}y + c_{22}x^2y^2 + c_{23}x^2y + c_{24}x + c_{25}xy$ has different support but the same Newton polytope as f_2. The corresponding dense polynomials have Newton polytopes drawn with dashed lines.

The three pairs of polynomials have $MV(Q_1, Q_2) = 4$, $MV(Q_1, Q_3) = 3$ and $MV(Q_2, Q_3) = 4$, whereas the respective Bezout bounds are $12, 6$ and 8.

More striking examples, demonstrating that the Bezout bound can be exponential in the mixed volume, are provided by the simple and generalized eigenproblems:

Example 1.3.11 Let $A = [a_{ij}]$ be an $n \times n$ complex matrix, for which we wish to compute the eigenvalues λ and eigenvectors $v = (v_1, \ldots, v_n)$. This reduces to solving a system of n equations of the form $\sum_j a_{ij}v_j = \lambda v_i$, for $1 \leq i \leq n$, and equation $\sum_i v_i^2 = 1$, for unknowns $v_1, \ldots, v_n$ and λ. The Bezout bound for this polynomial system is 2^{n+1}. Since for every solution v, λ, there is a solution $-v, \lambda$ as well, the number of solutions is $2n$ which is precisely the mixed volume.

The *sparse* or *Newton resultant* provides a necessary and generically sufficient condition for the existence of toric roots for a system of $n+1$ polynomials in n variables; since it applies to mixed systems, it is sometimes called the *sparse mixed resultant*. To define the sparse resultant we regard a polynomial f_i as a generic point $c_i = (c_{i1}, \ldots, c_{im_i})$ in the space of all possible polynomials with the given support $\mathcal{A}_i$. It is natural to identify scalar multiples, so the space of all such polynomials contracts to the projective space $\mathbb{P}_K^{m_i-1}$ or, simply, $\mathbb{P}^{m_i-1}$. Then the input system (1.1) can be thought of as a point

$$c = (c_1, \ldots, c_{n+1}) \in \mathbb{P}^{m_1-1} \times \cdots \times \mathbb{P}^{m_{n+1}-1}.$$

Let $Z_0 = Z_0(\mathcal{A}_1, \ldots, \mathcal{A}_{n+1})$ be the set of all points c such that the system has a solution in $(\mathbb{C}^*)^n$ and let $Z = Z(\mathcal{A}_1, \ldots, \mathcal{A}_{n+1})$ denote the Zariski closure of Z_0 in the product of projective spaces. It is proven in [PS93] that Z is an irreducible variety.

Definition 1.3.12 *The* sparse resultant $R = R(\mathcal{A}_1, \ldots, \mathcal{A}_{n+1})$ *of system (1.1) is a polynomial in* $\mathbb{Z}[c]$. *If* $codim(Z) = 1$ *then* $R(\mathcal{A}_1, \ldots, \mathcal{A}_{n+1})$ *is the defining irreducible polynomial of the hypersurface* Z. *If* $codim(Z) > 1$ *then* $R(\mathcal{A}_1, \ldots, \mathcal{A}_{n+1}) = 1$.

Let $\deg_{f_i} R$ denote the degree of the resultant R in the coefficients of polynomial f_i and let

$$MV_{-i} = MV(Q_1, \ldots, Q_{i-1}, Q_{i+1}, \ldots, Q_{n+1}) \qquad \text{for } i = 1, \ldots, n+1.$$

A consequence of Bernstein's theorem is

Theorem 1.3.13 [PS93] *The sparse resultant is separately homogeneous in the coefficients c_i of each f_i and its degree in these coefficients equals the mixed volume of the other n Newton polytopes i.e.* $\deg_{f_i} R = MV_{-i}$.

The sparse resultant generalizes the homogeneous resultant; they coincide when all Newton polytopes are n-simplices scaled by the total degrees of the respective polynomials as described above and the polynomials are homogenized.

Example 1.3.10 (Continued) For the previous example, the sparse resultant has total degree $4+3+4 = 11$, whereas the homogeneous resultant has total degree $12+6+8 = 26$. The latter can be obtained as the sparse resultant when the Newton polytopes are the dashed triangles in figure 1.1. The matrices constructed by the algorithm in section 3.1 and its greedy version haves sizes 15 and 14, whereas the algorithm in section 3.2 reduces the matrix size to 12.

1.4 Related Work

This section is split in two parts, the first discussing results in sparse elimination theory while the second describes existing approaches to solving systems of equations.

The foundations of this line of work were laid in the work of Gelfand, Kapranov and Zelevinsky [GKZ91, GKZ94] on generalized hypergeometric functions, $\mathcal{A}$-discriminants and regular triangulations of Newton polytopes. The $\mathcal{A}$-discriminant is the discriminant of a Laurent polynomial with given support $\mathcal{A}$. From the connection between $\mathcal{A}$-discriminants and resultants, the sparse resultant was introduced for mixed systems and was called the $(\mathcal{A}_1, \ldots, \mathcal{A}_{n+1})$-resultant. For unmixed systems, the resultant was defined as the Chow form of a toric variety and expressed as the determinant of a Koszul complex [KSZ92]. Pedersen and Sturmfels [PS93] gave product formulae of Poisson type for general Chow forms and sparse resultants.

Newton polytopes are also instrumental in the work of Viro on real algebraic curves of prescribed topology [Vir84, Vir86]. Sturmfels extended this work to complete intersections by means of mixed subdivisions [Stu93b]. Another related and impressive line of work by Khovanskii explores a different model of sparseness based on the number of nonvanishing terms [Kho91].

The first constructive methods for computing and evaluating the sparse resultant were proposed by Sturmfels in [Stu93a], the most efficient having complexity super-polynomial in the degree of the resultant and exponential in n with a quadratic exponent. The present thesis describes two algorithms for this task that run in time polynomial in the sparse resultant's degree and exponential in n with a linear exponent, under certain mild conditions. The first algorithm uses a lifting of the given Newton polytopes to $n+1$ dimensions in order to define a unique subdivision of their Minkowski sum. The assumption on the type of lifting used has been removed by Sturmfels [Stu94].

For solving systems of polynomial equations, certain properties of the sparse resultant must be established. This was essentially accomplished in [PS94], where it was shown that certain monomials defined during the computation of mixed volume constitute a basis of the coordinate ring associated to

the variety of the given polynomials. Section 4.1 offers a simpler proof based on our sparse resultant algorithm.

Special interest has been exhibited in multihomogeneous systems, where for certain cases exact *Sylvester-type* formulae are obtained, i.e. matrices whose determinant equals the resultant exactly [SZ94]. These results are applied in section 3.3 and our second algorithm is shown to construct these formulae. A generalization of these results [WZ92], covering a wider class of systems, has yet to offer a constructive approach.

There exist several methods for solving polynomial systems, usually distinguished into algebraic methods, where root finding is viewed as a special case of eliminating variables, and numerical methods that typically rely on some iterative technique. A helpful survey of classical elimination methods is [KY92] whereas standard techniques from numerical analysis are presented in [Dre78, AG90].

The principal numerical techniques are continuation or homotopy methods which trace the paths of the roots of a parametrized system as it varies, by small steps, from an easy to solve starting system to the given one. Many a time these methods fail to find all roots, may suffer from numerical inaccuracy or are computationally inefficient because they have to trace a large number of paths that diverge to infinity. Homotopies that follow the optimal number of paths have been proposed for multihomogeneous systems [MSW]. A promising development in solving very large polynomial systems comes from *sparse homotopies* which exploit the structure of polynomials as modeled by Newton polytopes [HS, VVC94, VG94].

Gröbner, or standard, bases provide an algebraic method for computing special bases of ideals and thus studying their coordinate rings [Buc85, Buc89]. Operations between polynomial ideals are simplified and calculation of common isolated roots for an arbitrary number of polynomials can be carried out over exact arithmetic, in contrast to numerical methods that use floating point arithmetic. Gröbner bases constitute a widely applied approach and several implementations exist. However, the rational coefficients of the basis polynomials are in general very large, the degree of the intermediate polynomials may be doubly exponential in the number of variables and the worst-case complexity is d^{2^n}, where d is the maximum total polynomial degree and n is the number of variables.

For zero-dimensional affine varieties a tight upper bound on the complexity is $d^{\mathcal{O}(n)}$ [Y.N90], while the bit complexity is $d^{\mathcal{O}(n^2)}$; see also [CGH89]. For zero-dimensional homogeneous varieties the arithmetic complexity is again $d^{\mathcal{O}(n)}$ [HRS93]. Although Gröbner bases perform significantly better than the worst-case bounds in several cases, they fail to take advantage of the polynomial structure. Polynomial system solving can be reduced to an eigenproblem via the construction of multiplication maps constructed by a Gröbner basis computation. Multiplicities complicate this approach and have been recently addressed in [MS94].

Another approach pioneered by Ritt and generalized by Wu [Rit50, Wu84] consists in computing the characteristic set of the given polynomials. It has led to positive results in elementary *geometric theorem proving* where the polynomial system is triangular or nearly so. The characteristic set is easy to solve and its zero set is roughly equivalent to the original one. The method has not been as successful for more intricate or general problems and does not exploit the polynomial structure. Its complexity has been recently shown to be exponential in n and polynomial in the degrees whereas the parallel time complexity is polynomial. A survey can be found in [Mis93, ch. 5]. A related approach is Hong's [Hon86], which suffers from the same shortcomings.

Recent advances indicate that resultants may be the method of choice in geometric theorem proving. As part of the algebraic toolkit implementation in Berkeley, Ashutosh Rege [Reg94] has applied resultants with impressive speedups to nontrivial geometric theorems.

machine	clock rate [MHz]	memory [MB]	SpecInt92	SpecFP92
DEC Alpha 3000/300	150	64	67	77
Sun Sparc 20/61	60	32	95	93
Sun Sparc 10/40	40	32	50	60

Table 1.1: Hardware specifications

Resultant-based methods have a long history in system solving: the u-resultant method is the primary means of solving well-constrained systems for it splits into linear factors, each expressing an isolated root [vdW50, Laz81]. Typically, the cost of explicitly computing this polynomial is exorbitant therefore strategies have been devised to derive the root coordinates by determinant computation and interpolation from a matrix formula [Can88b, MC93]. Alternatively the roots may be represented as the values of rational functions at the zeros of a resultant polynomial [Ren87]. The method applies to zero-dimensional varieties but fails when the homogenized system has excess components at infinity. This has been remedied by the generalized characteristic polynomial approach of Canny [Can90] which parallels work by Chistov, Grigoryev and Vorobjov [CG84, GV87].

1.5 Computational Model and Hardware

Our model is the real Random Access Machine (RAM) [PS85]. The *input* is organized as a set of n vectors in $\mathbb{R}^d$, where $n \geq d > 0$ and the i-th vector is $x_i = (x_{i,1}, \ldots, x_{i,d})$ for $1 \leq i \leq n$, $1 \leq j \leq d$. The four basic operations $\{+, -, \times, /\}$ are assumed to be exact between real numbers, where the operands are constants, input quantities or have been computed previously. Branching occurs at tests against zero of an input or computed quantity and is three-way depending on the sign. The set of arithmetic operations computing a branch expression together with the corresponding test is referred to as a *primitive*.

We make use of two complexity measures. Under the *arithmetic model* the total cost of a program equals the number of instructions in the longest execution path. More realistically, we must keep track of the operands' bit size: Under the *bit model* only the input, output and branching instructions are assigned unit cost. For integers of size $\mathcal{O}(b)$, addition and subtraction, multiplication and division and, lastly, the Greatest Common Divisor (GCD) operation require respectively $\mathcal{O}(b)$, $\mathcal{O}(b \log b \log\log b)$ and $\mathcal{O}(b \log^2 b \log\log b)$ bit operations [AHU74]. We shall use $M(b) = \mathcal{O}(b \log^2 b \log\log b)$ as an upper bound on the bit complexity of any arithmetic or GCD operation between any two rational numbers with numerator and denominator of size $\mathcal{O}(b)$. Then the total bit complexity of a real RAM program equals the sum of the costs of every instruction on an execution path, maximized over all paths. For either measure we use $\mathcal{O}^*(\cdot)$ to express an upper bound on the asymptotic complexity ignoring polylogarithmic factors. Hence $M(b) = \mathcal{O}^*(b)$.

We report on several experiments conducted on the machines of table 1.1.

1.6 Thesis Overview

The next chapter examines algorithmic issues in computing mixed volumes and the more general problem of mixed subdivisions. From the complexity standpoint, a crucial question is the relation of mixed volume to the volume of Minkowski sum. The former represents the inherent difficulty of

elimination problems whereas the latter typically characterizes the complexity of our algorithms. We study this question and use the results in the subsequent complexity analysis. We propose an improved version of Sturmfels' Lifting algorithm that significantly reduces its empirical complexity. We report on its implementation and performance and discuss further applications in addition to computing Bernstein's bound.

Chapter 3 proposes two algorithms for constructing the sparse resultant and analyzes their asymptotic complexity. Both produce determinantal formulae i.e., matrices in the given coefficients whose determinants are nontrivial multiples of the resultant, so a measure of their success is the degree of the extraneous factor. We present theoretical as well as empirical bounds on this degree.

The first algorithm is based on a mixed subdivision of the Minkowski sum. We also give a greedy version that outputs smaller matrices and removes some preprocessing requirement. The subdivision algorithm leads to a direct proof of a special case of a variant of the effective Nullstellensatz in terms of Newton polytopes. We also conjecture that it should lead to an exact quotient formula for the resultant like the quotient formula of Macaulay's for the homogeneous resultant.

Section 3.1.7 describes an example and section 3.1.8 analyses the algorithm's complexity: it is exponential in the dimension with a linear exponent. Lastly, section 3.1.9 describes how to obtain the actual sparse resultant of a specialized system.

The second algorithm is described in section 3.2 and its complexity analyzed in section 3.2.2. This is essentially a heuristic, yet it constructs Sylvester-type formulae for all systems where this is known to be possible. Section 3.3 applies it this class systems and examines its practical performance on several instances.

The chapter on Solving Polynomial Systems starts with a simple proof on which monomial sets constitute a basis of the coordinate ring of the given variety. This proof follows form the subdivision-based resultant construction. We offer an algorithm to compute generic monomial bases and use this discussion to solve arbitrary polynomial systems by means of constructing multiplication maps. Two methods are suggested for applying sparse resultants in order to reduce the solution of well-constrained systems to computing all eigenvalues and eigenvectors of a square matrix. The complexity analysis shows that this approach is singly exponential with a linear exponent in n. The chapter concludes with our implementation of a solver and certain issues arising from using numerical linear algebra computations.

Chapter 5 discusses applications of sparse elimination theory to concrete problems in robot kinematics, computer vision as well as structural molecular biology. It also sketches the potential of these methods in game theory and computational economics. We examine the performance of our solver in terms of accuracy, running times and numerical stability and show that sparse elimination provides a general approach that is very efficient for moderate-size systems.

The chapter on generic position looks at the problem of input degeneracy in the context of geometric as well as algebraic algorithms. Given a class of primitives we define computationally optimal perturbation schemes from the viewpoint of incurred bit complexity. We apply one scheme in order to obtain a convex hull implementation for arbitrary dimension. Although the perturbation is defined as a function of a symbolic infinitesimal, the computation is purely numeric.

We conclude with a summary of our work and some directions of research.

Chapter 2

Mixed Subdivisions and Mixed Volumes

This chapter examines mixed subdivisions and mixed volumes of a set of n convex, n-dimensional polytopes and presents efficient algorithms for both problems. Intuitively, the mixed subdivision of the Minkowski sum of a set of Newton polytopes contains all the information required to treat these polytopes from the point of view of sparse elimination theory and can serve as a means of computing the mixed volume. By Bernstein's theorem, mixed volume lies at the heart of sparse elimination since it models the notion of sparseness and provides a bound for the number of isolated roots.

The mixed volume computation reduces to finding a monomial basis for the coordinate ring of the corresponding irreducible variety. Furthermore, it serves as a starting point for continuation methods that exploit the structure modeled by the Newton polytopes; these are called sparse homotopies and have recently been studied by different authors [HS, VVC94, VG94].

Subdivisions of unmixed systems are instrumental in the work of Viro [Vir84, Vir86] on constructing real algebraic curves of prescribed topology. Using mixed subdivisions of mixed systems Sturmfels [Stu93b] generalized Viro's theorem for hypersurfaces into constructive results on complete intersections.

Extending the discussion from n to $n+1$ polytopes offers the necessary background for the sparse resultant construction and the solution of the monomial basis problem in chapter 3 by the algorithms presented here. While mixed volume expresses the inherent complexity of sparse elimination methods, most of our algorithms go through a Minkowski sum construction and hence exhibit complexity proportional to the volume of the latter. It thus becomes central to the complexity analysis to obtain bounds on the volume of the Minkowski sum as a function of the mixed volume.

Section 2.1 introduces mixed subdivisions and explains how mixed volumes can be computed if the subdivision is given. The second part shifts attention to sets of $n+1$ polytopes in n dimensions and relates their mixed subdivision to those of the respective n-subsets. A theorem is established that identifies that part of the subdivision of $n+1$ polytopes which corresponds to the subdivision of a given subset of n polytopes. This result is called upon in chapter 4 to show that the mixed volume algorithm can also compute a monomial basis of a coordinate rings of the ideal defined by n polynomials in n variables.

Section 2.2 establishes some bounds on the volume of the Minkowski sum of n or $n+1$ polytopes as a function of the mixed volume of n polytopes. These bounds are used in the complexity analysis of subsequent algorithms. The general approach for computing mixed subdivisions and mixed volumes

is presented in section 2.3 and Sturmfels' lifting algorithm for the mixed volume is described. For the problem of computing the entire mixed subdivision, we have designed and implemented a simple procedure based on the Lifting algorithm.

We propose an improved variant of Sturmfels' algorithm for the mixed volume and also describe its implementation in section 2.4. The asymptotic complexity is not lower than other methods but experimental results suggest that this is the fastest method to date in terms of practical complexity. The existing lower bounds suggest that this is a reasonable approach. Randomized algorithm as well as approximation algorithms for the mixed volume have been suggested in [DGH94].

Lastly, section 2.5 touches upon a standard computer algebra benchmark, namely the problem of cyclic n-roots. We show how our implementation established the first upper bounds for certain unknown cases of high n and examine its practical complexity. Lastly, there is a comparison to other approaches to the problem of counting isolated roots.

2.1 Induced Subdivisions

Given convex polytopes $Q_1, \ldots, Q_n$ in n-dimensional space we define their Minkowski sum

$$Q = Q_1 + \cdots + Q_n \quad \subset \mathbb{R}^n.$$

By abuse of terminology a subdivision of polytopes $Q_1, \ldots, Q_n$ is a subdivision of their Minkowski sum. To define the latter we recall certain concepts from polytope theory [Grü67].

A *polyhedral complex* is a finite collection of convex polytopes in $\mathbb{R}^n$ such that the intersection of any two is a face of both and belongs to the collection. Let $R_i \subset \mathbb{R}^n$ be the vertex set of Q_i and $R = R_1 + \cdots + R_n$ be the set of all vector sums of n vertices where one summand comes from each polytope. A *polyhedral subdivision* or simply a *subdivision* of Q is a collection of subsets of R, called *cells*, such that the collection of convex hulls of all cells is a polyhedral complex and the union of these convex hulls equals Q. In what follows we restrict attention to *maximal* cells, in other words cells of full dimension. Let Δ denote the set of all maximal cells or equivalently their convex hulls; Δ defines a subdivision of polytope Q.

There exist several subdivisions of Q. To specify a particular one we follow the approach of [BS92, Stu94]. Let $l_1, \ldots, l_n$ be *lifting functions*

$$l_i : R_i \to \mathbb{R}, \qquad i = 1, \ldots, n.$$

They define *lifting* $l = (l_1, \ldots, l_n)$ that associates each Q_i to *lifted polytope*

$$\widehat{Q}_i = \{(q, l_i(q)) | \, q \in Q_i\} \quad \subset \mathbb{R}^{n+1}, \qquad i = 1, \ldots, n.$$

We define the *lifted Minkowski sum* as the Minkowski sum of the lifted polytopes $\widehat{Q} = \widehat{Q}_1 + \cdots + \widehat{Q}_n \quad \subset \mathbb{R}^{n+1}$.

Definition 2.1.1 *Given a convex polytope in* $\mathbb{R}^{n+1}$, *its* lower envelope *is the union of all* n*-dimensional faces, or facets, whose inner normal vector has positive* $(n+1)$*-st coordinate.*

The canonical projection π *induces* a subdivision Δ of Q when restricted to the lower envelope of $\widehat{Q}$ by defining the cells of Δ to be the projections of the lower envelope facets,

$$\pi : \mathbb{R}^{n+1} \to \mathbb{R}^n \; : F_\sigma \mapsto \sigma, \qquad \text{for all facets } F_\sigma \text{ on the lower envelope of } \widehat{Q}.$$

Definition 2.1.2 *Consider vertex* $\sum_{i=1}^{n}(p_i, l_i(p_i))$ *on the lower envelope of* $\widehat{Q}$ *defined by vertex set* $\{p_1, \ldots, p_n\}$ *where* $p_i \in Q_i$ *and let* $\{q_1, \ldots, q_n\}$ *be a distinct vertex set with* $q_i \in Q_i$ *such that* $\sum_{i=1}^{n} p_i = \sum_{i=1}^{n} q_i$. *The lifting defined by functions* $l_1, \ldots, l_n$ *is* sufficiently generic *if and only if the lifted images of any two points in* Q *are distinct, namely*

$$\sum_{i=1}^{n}(p_i, l_i(p_i)) \neq \sum_{i=1}^{n}(q_i, l_i(q_i)) \qquad \textit{or equivalently} \qquad \sum_{i=1}^{n} l_i(p_i) \neq \sum_{i=1}^{n} l_i(q_i).$$

The unique sum expressing a vertex of the lower envelope is called *optimal* since it minimizes the aggregate lifting function. The projections of the summands form a unique optimal sum that expresses the respective point in Q.

Induced subdivision Δ is *coherent* if there exists linear functional L on $\mathbb{R}^{n+1}$ such that for every cell $\sigma \in \Delta$, $L(\pi^{-1}(x))$ is maximized at $\pi^{-1}(x) \cap F_\sigma$, for all $x \in \sigma$. Here F_σ is the face projected to σ. To ensure the subdivision is coherent, we may start with $\widehat{Q}$ and choose to project onto Q those faces that maximize L in $\widehat{Q}$. Coherence guarantees that in moving from cell to cell, and in particular between maximal cells, the optimal sum changes in a continuous manner.

We have chosen L to be the inner product with a vector along the negative $(n+1)$-st axis. Coherent induced subdivisions can be defined by the upper as well as the lower envelope of $\widehat{Q}$ and instead of the $(n+1)$-st coordinate axis any direction in $\mathbb{R}^{n+1}$ may be used, as long as the lifting is sufficiently generic with respect to the chosen direction. Moreover, the lifting is not constrained to map $\mathbb{R}^n$ to $\mathbb{R}^{n+1}$, although this is computationally more efficient. One of the earliest approaches [Emi93] suggested lifting to $\mathbb{R}^{n^2}$, thus avoiding most of the genericity conditions.

There is one more condition in order to be able to apply subdivisions in the context of mixed volumes. Clearly, the dimension of any cell σ in Δ is n, as is the dimension of the associated facet $\widehat{F}_\sigma$ on the lower envelope. The latter can be expressed as the Minkowski sum of faces $\widehat{F}_i$ from $\widehat{Q}_i$ and their sum of dimensions is $\sum_{i=1}^{n} \dim \widehat{F}_i \geq n$. An induced subdivision is *tight* if

$$\forall \sigma \in \Delta, \;\; \widehat{F}_\sigma = \widehat{F}_1 + \cdots + \widehat{F}_n \quad \text{and} \;\; \dim \widehat{F}_1 + \cdots + \dim \widehat{F}_n = n, \qquad \text{where } \widehat{F}_i \text{ is a face of } \widehat{Q}_i.$$

For sufficiently generic liftings an induced subdivision is tight. We define a *mixed subdivision* of Q to be a tight coherent subdivision.

In what follows we concentrate on mixed subdivisions. All maximal cells are also uniquely expressed as the Minkowski sum

$$\sigma = F_1 + \cdots + F_n, \qquad \text{where } F_i \text{ is a face of } Q_i,$$

and this sum is again called *optimal*. The sequence

$$(\dim F_1, \ldots, \dim F_n) \text{ is the } \textit{type} \text{ of } \sigma,$$

where $\dim F_i \geq 0$ is the dimension of face F_i in $\mathbb{R}^n$. This is also the type of the respective facet $\widehat{F}_\sigma$.

Definition 2.1.3 *In a mixed subdivision of* Q, *(maximal) cells of type* $(1, \ldots, 1)$ *are called* mixed.

Mixed cells are the Minkowski sum of edges, hence they are *parallelotopes* in n dimensions and their volume is the determinant of a matrix whose rows are the edges defining the cell. It is straightforward to prove the following fact given the multilinearity of the mixed volume from Definition 1.3.5.

Proposition 2.1.4 *Given convex polytopes $Q_1, \ldots, Q_n$ and a mixed subdivision Δ of their Minkowski sum*

$$MV(Q_1, \ldots, Q_n) = \sum_{\text{mixed } \sigma \in \Delta} \text{Vol}(\sigma).$$

We extend this discussion to systems of $n+1$ convex polytopes $Q_0, Q_1, \ldots, Q_n$ in n-dimensional space. Let Q^0 denote the Minkowski sum of all input Newton polytopes

$$Q^0 = Q_0 + Q_1 + \cdots + Q_n \subset \mathbb{R}^n.$$

Let R_0 be the vertex set of Q_0 and let l_0 be a sufficiently generic lifting function $l_0 : R_0 \to \mathbb{R}$. The genericity condition is analogous to Definition 2.1.2. Then define the *lifted* Newton polytopes

$$\widehat{Q}_i = \{(p_i, l_i(p_i)) : p_i \in Q_i\} \subset \mathbb{R}^{n+1}, \qquad 0 \le i \le n$$

and take their Minkowski sum

$$\widehat{Q}^0 = \widehat{Q}_0 + \cdots + \widehat{Q}_n \subset \mathbb{R}^{n+1}.$$

A mixed subdivision Δ^0 of Q^0 is induced by projecting the lower envelope of $\widehat{Q}^0$ onto Q^0 so that lower facets map onto maximal cells. By genericity Δ^0 is coherent, tight and every vertex on the lower envelope is uniquely expressed as a Minkowski sum of $n+1$ vertices from the lifted polytopes. Moreover, each cell σ in Δ^0 is uniquely expressed as a Minkowski sum

$$\sigma = F_0 + \cdots + F_n \subset \mathbb{R}^n \ : \qquad F_i \text{ is a face of } Q_i,\ i = 0, \ldots, n.$$

This is the optimal sum for σ under the specific subdivision. Maximal cell σ is the projection of facet $\widehat{F}_\sigma$ which is uniquely expressed as the Minkowski sum of those faces in $\widehat{Q}_i$ corresponding to F_i. Again we define the type of σ or $\widehat{F}_\sigma$ to be the $(n+1)$-sequence $(\dim F_0, \dim F_1, \ldots, \dim F_n)$. Clearly, $\sum_{i=0}^n \dim F_i \ge n = \dim \sigma$ and for any $\sigma \in \Delta^0$ at least one face is zero-dimensional. Cells with exactly one such face are characterized as follows.

Definition 2.1.5 *Maximal cells in mixed subdivision Δ^0 whose type is $(1, \ldots, 1, 0, 1, \ldots, 1)$ are called* mixed. *Such a cell is i-*mixed *if, in its expression as an optimal sum of faces, the summand from Q_i is some vertex a_{ij}:*

$$\sigma = E_0 + \cdots + E_{i-1} + a_{ij} + E_{i+1} + \cdots + E_n, \qquad \textit{where } E_k \textit{ is an edge of } Q_k.$$

Lemma 2.1.6 *Assume Q_0 has a vertex $0^n = (0, \ldots, 0)$. Consider the mixed subdivision Δ of Q induced by lifting functions $l_1, \ldots, l_n$. There exists lifting function l_0 such that, if Δ^0 is induced by f_0 and and the same $l_1, \ldots, l_n$, then every 0-mixed cell of Δ^0 is of the form $0^n + \sigma$ for some mixed cell s in Δ.*

Proof Any point on the lower envelope of $\widehat{Q}^0$ is of the form $\widehat{p} + \widehat{a}_{0j}$, where $\widehat{p}$ is on the lower envelope of $\widehat{Q}$ and $\widehat{a}_{0j} \in \widehat{Q}_0$. We wish to show that every such point, for appropriate l_0, has a unique summand from $\widehat{Q}_0$, namely the lifted image of 0^n.

Consider points $\widehat{p}, \widehat{q}$ on the lower envelope of $\widehat{Q}$ and assume that $\widehat{p} + (0^n, l_0(0^n))$ and $\widehat{q} + \widehat{a}_{0j}$ lie on the same vertical, for some $a_{0j} \ne 0^n$. We can pick l_0 sufficiently large on the vertices of Q_0 except 0^n so that $\widehat{p} + (0^n, l_0(0^n))$ is on the lower envelope whereas $\widehat{p} + \widehat{a}_{0j}$ is not. For this it suffices to require that

$$l_0(a_{0j}) > \sum_{i=1}^n l_i(a_{ij_i}), \qquad \forall\ a_{0j} \in Q_0, a_{0j} \ne 0^n,\ \forall a_{ij_i} \in Q_i. \tag{2.1}$$

Consider a lower envelope facet $\widehat{F}_\sigma$ of $\widehat{Q}$, where σ is a mixed cell in Δ. A similar argument shows that under (2.1), for every facet $\widehat{F}_\sigma$, the sum $(0^n, l_0(0^n)) + \widehat{F}_\sigma$ is a lower envelope facet on $\widehat{Q}^0$. Then the total volume of all cells in Δ^0 of the form $0^n + \sigma$, where σ is a mixed cell of Δ, is $MV(Q_1, \ldots, Q_n)$. All of these cells are 0-mixed by construction, hence there are no more 0-mixed cells in Δ^0. □

It follows from this lemma and Proposition 2.1.4 that

Proposition 2.1.7 *For $i = 0, \ldots, n$,*

$$MV_{-i} = MV(Q_0, \ldots, Q_{i-1}, Q_{i+1}, \ldots, Q_n) = \sum_{i\text{-mixed } \sigma} \mathrm{Vol}(\sigma).$$

2.2 Mixed Volumes and Minkowski Sums

A crucial question in the complexity analysis of algorithms computing mixed subdivisions, mixed volume as well as the sparse resultant in the following chapter is the relation between the mixed volume and the volume of the Minkowski sum Q of polytopes $Q_1, \ldots, Q_n$ in n-dimensional space. Before proposing algorithms and analyzing their complexities, then, we establish some results on the relation of these two quantities. We extend the discussion to systems of $n+1$ polytopes where we establish bounds in terms of the sum of n-fold mixed volumes. Most of these results first appeared in [Emi94].

We denote by e the exponential base.

Lemma 2.2.1 *For unmixed systems $Q_1 = \cdots = Q_n$. Then*

$$\mathrm{Vol}(Q) = \Theta(e^n/\sqrt{n}) MV(Q_1, \ldots, Q_n).$$

Proof Stirling's approximation is used throughout the thesis to provide asymptotic bounds:

$$n! = \Theta(\frac{\sqrt{2\pi n} n^n}{e^n}) = \Theta(\frac{n^{n+1/2}}{e^n}).$$

For unmixed systems of polytopes, $MV(Q_1, \ldots, Q_n) = n!\mathrm{Vol}(Q_1)$ which is, asymptotically, $\Theta(n^{n+1/2}/e^n)$ $\cdot$ $\mathrm{Vol}(Q_1)$. The Minkowski sum volume is $\mathrm{Vol}(Q) = n^n \mathrm{Vol}(Q_1)$ and the claim follows. □

To model mixed systems we have to express their difference in shape and volume. This is a hard problem in general so we restrict attention to the case where all polytopes have a nonzero n-dimensional volume. Define the following two objects: the polytope of minimum volume Q_μ, $1 \leq \mu \leq n$, such that

$$\mathrm{Vol}(Q_\mu) = \min\{\mathrm{Vol}(Q_i) \,|\, i = 1, \ldots, n\},$$

and the *system's scaling factor* $s \in \mathbb{R}$, $s \geq 1$ which satisfies the following condition:

$$\text{minimize } s \;:\; \exists\, \lambda_i \in \mathbb{R} \;:\; \lambda_i + Q_i \subset s\, Q_\mu, \qquad i = 1, \ldots, n.$$

The first step is a lower bound on the mixed volume as a function of a single polytope.

Lemma 2.2.2 *If $\mathrm{Vol}(Q_i) > 0$ for $i = 1, \ldots, n$ and Q_μ is the polytope of minimum volume, then $MV(Q_1, \ldots, Q_n) \geq n!\,\mathrm{Vol}(Q_\mu)$.*

Proof One of the most important inequalities in convexity theory is the Aleksandrov-Fenchel inequality [Fen36, Ale37] which states that

$$MV^2(Q_1, \ldots, Q_n) \geq MV(Q_1, Q_1, Q_3, \ldots, Q_n)\, MV(Q_2, Q_2, Q_3, \ldots, Q_n),$$

for arbitrary polytopes $Q_i \subset \mathbb{R}^n$. A consequence of this is

$$MV^n(Q_1, \ldots, Q_n) \geq (n!)^n \mathrm{Vol}(Q_1) \cdots \mathrm{Vol}(Q_n).$$

These results, along with an extensive treatment of the theory, can be found in [BZ88, Sch93]. Let $\overline{V}$ be the geometric mean of $\mathrm{Vol}(Q_1), \ldots, \mathrm{Vol}(Q_n)$, then, since volume and mixed volume are positive-valued functions, the inequality becomes

$$MV(Q_1, \ldots, Q_n) \geq n!\, \overline{V}. \tag{2.2}$$

The last inequality implies the claim. □

The tightness of bound (2.2) is considered in the context of a concrete benchmark in section 2.5, where we show that it is not always a good approximation to mixed volume.

Theorem 2.2.3 *Given polytopes $Q_1, \ldots, Q_n \subset \mathbb{R}^n$ such that $\mathrm{Vol}(Q_i) > 0$ for all i, define Q_μ and the system's scaling factor s as above. Then*

$$\mathrm{Vol}(Q) = \mathcal{O}(e^n s^n / \sqrt{n}) MV(Q_1, \ldots, Q_n).$$

Proof By definition,

$$Q \subset \sum_{i=1}^{n} s\, Q_\mu = ns Q_\mu,$$

hence $\mathrm{Vol}(Q) = (ns)^n \mathrm{Vol}(Q_\mu)$. By the previous lemma, $MV(Q_1, \ldots, Q_n) \geq n! \mathrm{Vol}(Q_\mu)$. Application of Stirling's approximation completes the proof. □

This bound generalizes the unmixed case in which $s = 1$. Moreover, it is asymptotically quite tight, as seen by the following example. Let

$$Q_1 = \cdots = Q_{n-1}, \quad Q_n = sQ_1,$$

where $s > 1$ and $Q_\mu = Q_1$. Then $Q = (s + n - 1)Q_1$, hence $\mathrm{Vol}(Q) > s^n \mathrm{Vol}(Q_1)$, and $MV(Q_1, \ldots, Q_n) = s\, n!\, \mathrm{Vol}(Q_1)$. Therefore

$$\frac{\mathrm{Vol}(Q)}{MV(Q_1, \ldots, Q_n)} = \Omega\left(\frac{e^n}{n^{3/2}} \left(\frac{s}{n} \right)^{n-1} \right).$$

For $s = n^2$ and $s = 2^n$ the lower bound becomes, respectively,

$$\Omega\left(e^n \sqrt{s}^{\,n-5/2} \right) \qquad \text{and} \qquad \Omega\left(e^n \frac{s^{n-1}}{(\log s)^{n+1/2}} \right).$$

We extend the result to the Minkowski sum Q^0 of $n+1$ polytopes compared to the sum of all n-fold mixed volumes D, i.e. the sum of mixed volumes of all subsets of n polytopes. Notice that from Theorem 1.3.13, D is the total degree of the sparse resultant.

Lemma 2.2.4 *For unmixed systems* $Q_0 = Q_1 = \cdots = Q_n$, $\mathrm{Vol}(Q^0) = \Theta(e^n/n^{3/2})D$.

Proof Each n-fold mixed volume is $n!\mathrm{Vol}(Q_0)$, hence $D = (n+1)n!\mathrm{Vol}(Q_0) = (n+1)!\mathrm{Vol}(Q_0)$ and, by Stirling's approximation, $D = \Theta(\sqrt{n}((n+1)/e)^{n+1}\mathrm{Vol}(Q_0))$. Now $\mathrm{Vol}(Q^0) = (n+1)^n\mathrm{Vol}(Q_0)$ and the bound follows. □

Let Q_μ be the polytope with minimum volume among all $n+1$ polytopes. Then the scaling factor s of $n+1$ polytopes is defined as the minimum positive real such that $\lambda_i + Q_i \subset sQ_\mu$ for some λ_i, for $i = 0, 1, \ldots, n$.

Theorem 2.2.5 *Given polytopes* $Q_0, Q_1, \ldots, Q_n \subset \mathbb{R}^n$, *such that* $\mathrm{Vol}(Q_i) > 0$ *for all* i, $\mathrm{Vol}(Q^0) = \mathcal{O}(s^n e^{n+1}/n^{3/2})D$, *where* D *is the sum of the* $n+1$ n*-fold mixed volumes and* s *is the system's scaling factor.*

Proof $Q^0 \subset s(n+1)Q_\mu$ hence $\mathrm{Vol}(Q^0) \leq s^n(n+1)^n\mathrm{Vol}(Q_\mu)$. The sum of all n-fold mixed volumes is bounded below by the sum of n mixed volumes, each on a set of polytopes containing Q_μ. Then, by Lemma 2.2.2, $D > n\,n!\ \mathrm{Vol}(Q_\mu)$ therefore $\mathrm{Vol}(Q_\mu) = \mathcal{O}(e^n/n^{n+3/2})D$. This implies $\mathrm{Vol}(Q^0) = \mathcal{O}(s^n e^n(1+1/n)^n/n^{3/2})D$ and the claim follows from $\lim_{n\to\infty}(1+1/n)^n = e$. □

2.3 Computing Mixed Subdivisions

This section looks at particular liftings that are useful from a computational viewpoint, examines the randomness requirement to guarantee genericity and proposes a simple algorithm for constructing mixed subdivisions. Linear lifting functions provide enough flexibility and are computationally efficient since the face structure of Q_i and $\widehat{Q}_i$ are identical, so we restrict attention to linear forms $l_i \in \mathbb{Q}[x_1, \ldots, x_n]$.

For a sufficiently generic lifting, the lower envelope of $\widehat{Q}$ is in bijective correspondence with the Minkowski sum Q of the original polytopes, since every vertex on the lower envelope is expressed uniquely as a Minkowski sum. The latter condition relies on the genericity of lifting l, by Definition 2.1.2.

Lemma 2.3.1 *If all* $2n^2$ *coordinates of* l_i *are chosen independently and uniformly from an interval of size* 2^{L_l}, *where* $L_l \in \mathbb{Z}_{>0}$, *then the probability that genericity fails is bounded by*

$$\mathit{Prob}[\mathit{failure}] \leq \frac{1}{2^{L_l}}\frac{1}{2}m^n(m-1)^{n-1}: \qquad m \text{ is the maximum vertex cardinality of } Q_i. \tag{2.3}$$

Proof Fix two distinct sets of points $\{p_1, \ldots, p_n\}$ and $\{q_1, \ldots, q_n\}$ with p_i, q_i vertices of Q_i, for which $\sum_i p_i = \sum_i q_i$. We must bound the probability that the lifting fails to distinguish between the lifted images of these two points, namely that $\sum_i l_i(p_i) = \sum_i l_i(q_i)$.

Assume, without loss of generality, that p_1 and q_1 differ in their first coordinate and fix all coefficients in the n lifting forms, except for the first one in l_1. In choosing this last coefficient, the probability of picking the single value that does not distinguish between the two lifted points is $\leq 1/2^{L_l}$. By multiplying this probability with the number of pairs $\{p_i\}$, $\{q_i\}$ the result is proven.

In counting the number of pairs the maximum number is found when, for every i, $p_i \neq q_i$, because otherwise the possible pairs are constrained. If the p_i's are chosen at will, there are at most $m-1$ choices for each q_i and no choice for q_n since the points must satisfy $\sum_i p_i = \sum_i q_i$. Hence the

number of pairs is at most $1/2\, m^n (m-1)^{n-1}$. □

For typical values $n < 10$, $m < 15$ and two-word integers i.e. $L_l = 64$, the probability of failure is about 45%, while for $L_l = 96$, the probability becomes smaller than 10^{-9}. The above bound is overly pessimistic. To check deterministically whether a particular choice of lifting functions is sufficiently generic we may compare the normals of adjacent facets on the lower envelope of the lifted Minkowski sum.

The lifting idea immediately yields an algorithm for the construction of mixed subdivisions and thus the calculation of mixed volume. We have implemented this algorithm [EC93], which is useful when the entire subdivision is desired; we shall see below that faster methods exist for computing only the mixed volume.

Algorithm 2.3.2 (Mixed Subdivisions)
Input: Convex polytopes $Q_1, \ldots, Q_n$ in $\mathbb{R}^n$.
Output: All maximal cells in the mixed subdivision of $Q_1 + \cdots + Q_n$.

Description The algorithm starts by defining a linear lifting and computing the vertices on the lower envelope of $\widehat{Q}$. This is done in an incremental fashion, by computing the lower envelope vertex set of

$$(\widehat{Q}_1 + \cdots + \widehat{Q}_{k-1}) + \widehat{Q}_k, \qquad k = 2, \ldots, n,$$

given the vertices on the lower envelope of the two summands. Deciding whether a point is a vertex or not reduces to a linear programming problem. A convex hull algorithm constructs the lower envelope facet structure of $\widehat{Q}$ from its vertices. The Beneath-Beyond algorithm with the perturbation scheme of chapter 6 may be used for arbitrary n. Each facet $\widehat{F}$ can be uniquely expressed as

$$\widehat{F} = \widehat{F}_1 + \cdots + \widehat{F}_n : \qquad \widehat{F}_i \text{ is a face of } \widehat{Q}_i.$$

To decide the type of each facet $\widehat{F}$ we need to compute $\dim \widehat{F}_i$ for every i, since $\dim \widehat{F}_i = \dim F_i$ due to the linearity of the lifting. If the convex hull is simplicial, as is the case with the implementation of section 6.7, it suffices to know the number of distinct vertices from Q_i appearing in the optimal sum of some vertex of $\widehat{F}$. This is achieved by keeping track, for each vertex $\widehat{p}$ of $\widehat{Q}$, of the vertices from $\widehat{Q}_i$ that define it in a vector

$$[j_1, \ldots, j_n] : \qquad \widehat{p} = \sum_{i=1}^{n} \widehat{a}_{ij_i} \quad : \quad \widehat{a}_{ij_i} \text{ is a vertex of } \widehat{Q}_i.$$

□

Once the type of every facet is determined, we may compute the mixed volume by summing up all volumes of mixed cells.

Theorem 2.3.3 *The algorithm described above computes the mixed subdivision of the Minkowski sum Q of convex polytopes $Q_1, \ldots, Q_n$. Its worst-case complexity is $\mathcal{O}(v^{\lfloor n/2 \rfloor} n(n^2 + v))$, where v is the vertex cardinality of the lifted Minkowski sum $\widehat{Q}$. Let m be the maximum vertex cardinality of any Q_i and consider worst-case bound $v = \mathcal{O}(m^n)$. Then the complexity is $\mathcal{O}(m^{n\lfloor 1+n/2 \rfloor} n)$.*

Proof The correctness of the algorithm follows from the previous discussion. The asymptotic complexity is dominated by the convex hull construction hence has overall complexity $f(n^3 + vn)$ [PS85, sect. 3.4], where f is the number of facets of $\widehat{Q}$. It is known that $f = \mathcal{O}(v^{\lfloor n/2 \rfloor})$ [Grü67] so the complexity becomes $\mathcal{O}(v^{\lfloor n/2 \rfloor}(n^3 + vn))$. The bound in terms of m and n is straightforward. □

It is possible to construct cases where v is $\Theta(m^n)$, though in practice v is considerably smaller.

Corollary 2.3.4 *Assume* $\mathrm{Vol}(Q_i) > 0$ *for* $i = 1, \ldots, n$. *The complexity of the algorithm is* $\mathcal{O}(v^{\lfloor (n+4)/2 \rfloor})$. *If* $s > 1$ *denotes the scaling factor of polytopes* $Q_1, \ldots, Q_n$ *as defined above and* e *is the exponential base, then the complexity is*

$$\mathcal{O}\left((es)^{\lfloor (n+4)/2 \rfloor n} \, MV(Q_1, \ldots, Q_n)^{\lfloor (n+4)/2 \rfloor} \right).$$

Proof The number of vertices v in $\widehat{Q}$ is clearly bounded by the number of integer lattice points in Q which is asymptotically equal to $\mathrm{Vol}(Q)$ by Ehrart's polynomial [Ehr67]. By Theorem 2.2.3, $v = \mathcal{O}(e^n s^n) MV(Q_1, \ldots, Q_n)$. Thus $n^k = \mathcal{O}(v)$, for any positive integer k and the complexity bound becomes $\mathcal{O}(v^{\lfloor n/2 \rfloor} v^2)$. □

If the number of vertices of the Minkowski sum grows linearly with the number of summands, in particular if $v = \mathcal{O}(nm)$, where m is the maximum number of vertices in any Q_i, then the complexity of the algorithm is $\mathcal{O}((nm)^{\lfloor n/2 \rfloor} n^2 (n+m))$ from Theorem 2.3.3. When $m \geq n$ the complexity becomes $\mathcal{O}((nm)^{\lfloor n/2 \rfloor} n^2 m)$.

Our implementation is on public domain in `ftp://robotics.eecs.Berkeley.edu/pub/MixedVolume/subdiv`. To compute the convex hull we use our general-dimension implementation of the Beneath-Beyond algorithm which copes efficiently with degenerate point sets and constructs the facet structure of a simplicial polytope which lies arbitrarily close to the true convex hull under the measure of symmetric difference volume (ch. 6).

It is clear that extending this method to computing the mixed subdivision Δ^0_δ for $n+1$ polytopes is straightforward. The worst-case asymptotic complexity is $\mathcal{O}(m^{n \lfloor (n+3)/2 \rfloor} n)$, where r is the maximum number of vertices in $Q_0, \ldots, Q_n$.

2.4 The Lift-Prune Algorithm

This section examines algorithms that compute the mixed volume without first computing the mixed subdivision. These algorithms identify the mixed cells and then sum up their volumes.

The method of Huber and Sturmfels [HS] takes advantage of repeated polytopes by generalizing mixed and unmixed systems into *semi-mixed* systems: A semi-mixed system with polytope cardinalities $k_1, \ldots, k_l \geq 1$, such that $k_1 + \cdots + k_l = n$, is a system where there are only l distinct polytopes and Q_i is repeated k_i times, for $i = 1, \ldots, l$. The method of Verschelde and Gatermann [VG94] exploits symmetry in Newton polytopes, while general implementations using the Newton polytopes to model structure have been described in [VVC94]. These algorithms have the same worst-case asymptotic complexity as does our own algorithm defined below, namely singly-exponential in n. Nonetheless, based on experimental results on general inputs, our algorithm appears to be the most efficient to date.

We call our algorithm the *Lift-Prune* algorithm because it relies on the Lifting paradigm while it employs a pruning heuristic in order to speed up the search of mixed cells. The idea is to test, for every combination of n edges from the given polytopes, whether their Minkowski sum lies on the lower envelope of $\widehat{Q}$. If so its volume is computed and added to the mixed volume. To prune the combinatorial search, we make use of

Proposition 2.4.1 Fix a lifting and let $J \subset \{1, \ldots, n\}$ be an index set such that e_j is an edge of Q_j for all $j \in J$. If the Minkowski sum of lifted edges $\sum_{j \in J} \widehat{e}_j$ lies on the lower envelope of $\sum_{j \in J} \widehat{Q}_j$ then, for any subset $L \subset J$, $\sum_{l \in L} \widehat{e}_l$ lies on the lower envelope of the Minkowski sum $\sum_{l \in L} \widehat{Q}_l$.

Our algorithm constructs n-tuples of edges from Q_i incrementally by starting with edge pairs and adding one edge from another polytope at a time. As each edge is added, the k-tuple for $2 \leq k \leq n$ is tested on whether it lies on the lower envelope of the Minkowski sum of the corresponding lifted polytopes or not; only k-tuples that pass the test continue to be augmented.

Further pruning is achieved by eliminating from the edge sets of polytopes not yet considered those edges that cannot extend the current k-tuple. This means that for index set J, we let $L = J \cup \{i\}$, where i ranges over $\{1, \ldots, n\} \setminus J$ and check the edge tuples corresponding to L. For any i and corresponding L there is typically a constant fraction of edges eliminated from the edge set of Q_i. This filtering process is step 6 below and is computationally beneficial because it employs several "small" tests to decrease the number of "large" and expensive tests that must be ultimately performed.

Every test for a k-tuple of edges $e_{i_1}, \ldots, e_{i_k}$ is implemented as a linear programming problem. Assume that each $\widehat{Q}_i$ has vertices with rational coordinates. Let $\widehat{p}_i \in \mathbb{Q}^{n+1}$ be the midpoint of the lifted edge $\widehat{e}_i$ of $\widehat{Q}_i$ and let $\widehat{p} = \widehat{p}_{i_1} + \cdots + \widehat{p}_{i_k} \in \mathbb{Q}^{n+1}$ be an interior point of their Minkowski sum. The test of interest is equivalent to asking whether $\widehat{p}$ lies on the lower envelope or not, which is formulated as follows:

$$\begin{aligned} \text{maximize} \qquad & s \in \mathbb{R}: \qquad \widehat{p} - sz = \sum_{l \in \{i_1, \ldots, i_k\}} \sum_{j=1}^{m_l} \lambda_{lj} \widehat{v}_{lj}; \\ & \sum_{j=1}^{m_l} \lambda_{lj} = 1, \ \lambda_{lj} \geq 0, \ \forall\, l \in \{i_1, \ldots, i_k\}, j = 1 \ldots m_l; \end{aligned}$$

where $z = (0, \ldots, 0, 1) \in \mathbb{Z}^{n+1}$, $\widehat{v}_{lj}$ are the vertices of $\widehat{Q}_l$ and m_l their cardinality. Then $\widehat{p}$ lies on the lower envelope if and only if the maximal value of s is zero, for s expresses the distance between $\widehat{p}$ and the lower envelope point that lies on the same vertical. The constraints ensure that $\widehat{p}$ lies on the same vertical as a variable point defined as the Minkowski sum of points from the lifted polytopes.

Algorithm 2.4.2 (Lift-Prune)
Input: Convex polytopes $Q_1, \ldots, Q_n \subset \mathbb{R}^n$ with rational vertices.
Output: $MV(Q_1, \ldots, Q_n) \in \mathbb{Z}$.

Description The mixed volume is invariant under permutation of the polytopes. For a given permutation the algorithm is:

1. Enumerate the edges of all polytopes $Q_1, \ldots, Q_n$ in sets $E_1, \ldots, E_n$.
2. Compute random lifting vectors $l_1, \ldots, l_n \in \mathbb{Q}^n$.
3. Compute lifted edge $\widehat{e}_i$ for every edge $e_i \in E_i$, $i = 1, \ldots, n$.
4. Initialize the mixed volume to 0.
5. If $E_1 = \emptyset$ then terminate.
   Otherwise, pick any edge $e_1 \in E_1$, remove it from E_1, create current tuple (e_1), let sets $E_2', \ldots, E_n'$ be copies of $E_2, \ldots, E_n$ and let $k = 1$.
6. Let i range from $k+1$ to n: For every $e_i \in E_i'$, if $\sum_{j=1}^{k} \widehat{e}_j + \widehat{e}_i$ does not lie on the lower envelope of $\sum_{j=1}^{k} \widehat{Q}_j + \widehat{Q}_i$ then e_i is removed from E_i'.
7. Increment k.
8. If $k > n$, then add the volume of the Minkowski sum of $(e_1, \ldots, e_n)$ to the mixed volume; continue at step 5.
9. If $k \leq n$ and $E_k' = \emptyset$ then continue at step 5.
   If $k \leq n$ and $E_k' \neq \emptyset$ then add some edge $e_k \in E_k'$ to the current tuple $(e_1, \ldots, e_{k-1})$, remove e_k from E_k' and go to step 6.

This procedure does not exploit the fact that mixed volume is invariant under permutation of the polytopes. In our implementation, we change the order of the polytopes, or rather their edge sets, in a dynamic fashion so that when the algorithm at step 9 picks a new edge set, it chooses the one with minimum cardinality.

Mixed volume provides upper bounds on the number of toric roots of polynomial systems. It is trivial to extend this algorithm to compute bounds on the number of common roots in $\mathbb{C}^n$ of n polynomial in n variables. By Theorem 1.3.9 it suffices to add, at a preprocessing step, the origin $(0,\ldots,0)$ to every input vertex set, recompute Q_i and then simply execute the same algorithm. □

There exists a deterministic check of the genericity of the random lifting chosen at step 2. Each maximal cell corresponds to a facet on the lower envelope of $\widehat{Q}$. The lifting is sufficiently generic if and only the outer normals of any two adjacent facets corresponding to mixed cells are not parallel. We have added this option to our implementation, although it is computationally expensive.

The Lift-Prune Algorithm is incremental in the sense that partial results are available at every stage of execution. This is particularly useful in long runs of the program, when a loose bound on mixed volume can be detected long before termination. In addition, the tree structure of the combinatorial search permits to restart the algorithm in the middle of a computation and allows for a distributed implementation.

Complexity

Ignoring the pruning, the algorithm has to test g^n combinations, where g is an upper bound on the number of edges in every Newton polytope. If m is an upper bound on the number of polytope vertices, $g \leq m^2$ and the number of tests is $\mathcal{O}(m^{2n})$. Note that m is bounded by the maximum number of monomials in any polynomial; the latter provides a different model of sparseness studied in [Kho91].

Linear programming may be solved by any polynomial-time algorithm; in what immediately follows as well as in later sections we use Karmarkar's algorithm to derive bounds [Kar84]. For linear programs with V variables, C constraints and B maximum bits per coefficient, the bit complexity is

$$\mathcal{O}^*(C^2V^{5.5}B^2),$$

where $\mathcal{O}^*(\cdot)$ indicates that we have ignored polylogarithmic factors in C, V, B.

The complexity here is $\mathcal{O}(n^7m^6(L_l + L_d))$, where L_l is the maximum bit-size of a coordinate in any lifting form l_i, $i = 1,\ldots,n$ and L_d is the bit-size of the maximum coordinate of any Newton polytope vertex. The maximum coordinate is bounded by d hence $L_d \leq \log d$.

Theorem 2.4.3 *Let m be the maximum vertex cardinality per polytope, bounded by the maximum number of monomials in any polynomial. Let d be the maximum degree in any variable and $\epsilon < 1$ be the probability of failure of the lifting scheme. The worst-case bit complexity of the Lift-Prune algorithm for computing $MV(Q_1,\ldots,Q_n)$ is $m^{\mathcal{O}(n)}(\log d - \log \epsilon)$. For a constant probability ϵ and systems with maximum degree $d \leq 2^{m^{\mathcal{O}(n)}}$ the complexity is $m^{\mathcal{O}(n)}$.*

Proof There are m^{2n} edge tests at most, each reducing to a linear programming application with bit complexity $\mathcal{O}(n^7m^6(L_l + L_d))$. From (2.3), $m^{2n}/(2 \cdot 2^{L_l})$ is the probability ϵ that the lifting fails, then $L_l = \mathcal{O}(n \log m - \log \epsilon)$. The general bound is now immediate. □

This algorithm leads to substantial savings in the worst-case complexity versus the mixed subdivision algorithm analyzed in Theorem 2.3.3. The latter bound was proportional roughly to $m^{n^2/2}$, while here the exponent is linear. Under more reasonable assumptions, that capture better the average-case behavior, we see below that the Lift-Prune algorithm is again much faster. This has been verified experimentally.

The Lift-Prune algorithm puts mixed volume in complexity class #P because a non-deterministic machine could guess an edge combination that leads to a mixed cell with positive volume, then spend polynomial time to check this guess. If we subdivide each edge to unit-length segments and require that all combinations contain these segments then each guess leads to a subcell of unit volume and there are mixed volume distinct guesses.

Proposition 2.4.4 *Computing the mixed volume is in #P.*

The complexity of the algorithm is rather satisfactory given that the mixed volume problem is equivalent, for the special case of unmixed systems, to the convex hull volume. The latter is known to be #P-hard [DF88, Kha93]. Moreover, it has recently been shown that mixed volume is #P-complete [Ped94] by a reduction of computing the permanent.

For most practical applications the extra hypotheses are satisfied and the tighter bound $m^{\mathcal{O}(n)}$ applies. Nevertheless, the complexity is asymptotically comparable to the naive algorithm of section 2.3. Yet the Lift-Prune algorithm performs significantly better on all non-trivial problems (dimension larger than 2). To account for the effects of pruning we should estimate the number of edge combinations that pass the test at the various stages. We assume that this number is proportional to the maximum number of facets of any lifted polytope $\widehat{Q}_i$ and grows linearly with the polytope cardinality in the partial Minkowski sum $\widehat{Q}_1 + \cdots + \widehat{Q}_k$, so we define

$$\rho_k = \mathcal{O}\left(\frac{k m^{\lfloor (n+1)/2 \rfloor}}{m^{2k}}\right), \qquad k = 2, \ldots, n.$$

This assumption is supported by experimental evidence from the cyclic problem specified below.

Corollary 2.4.5 *Assume that the probability of failure ϵ for the lifting scheme is constant and that the maximum coordinate of any polytope vertex d obeys $d = 2^{(mn)^{\mathcal{O}(1)}}$. Then, under the hypothesis above on the number of valid edge combinations, the Lift-Prune algorithm has complexity $\mathcal{O}(m^{\lfloor (n+1)/2 \rfloor})(mn)^{\mathcal{O}(1)}$.*

Proof Since the number of edges per polytope is bounded by m^2, the number of linear programming problems is bounded by

$$\begin{aligned} m^4 + \rho_2 m^4 m^2 + \cdots + \rho_{n-1} m^{2(n-1)} m^2 &= m^4 + 2m^{\lfloor (n+1)/2 \rfloor + 2} + \cdots + (n-1) m^{\lfloor (n+1)/2 \rfloor + 2} \\ &= \mathcal{O}(n^2 m^{\lfloor (n+1)/2 \rfloor + 2}). \end{aligned}$$

As analyzed above the cost of every linear programming run is at most $\mathcal{O}(n^7 m^6 (\log d - \log \epsilon))$ which, under the current hypothesis, can be written $(nm)^{\mathcal{O}(1)}$. The claim follows. □

Our implementation is available on `ftp://robotics.eecs.Berkeley.edu/pub/MixedVolume`. Parallelization of our algorithm is straightforward.

A related problem is the computation of all $n+1$ n-fold mixed volumes. This is actually required, in section 3.2, in the construction of sparse resultant matrices. One method is to examine

all cells of Δ_δ^0 and add up the volumes of the i-mixed cells to compute the i-th mixed volume, yet the complexity is exponential in n, whereas the Lift-Prune algorithm has polynomial complexity under some reasonable assumptions. Therefore it is preferable to use the Lift-Prune algorithm $n+1$ times; this can be improved by combining knowledge about edge combinations from one computation to the next for any two subsets of n polytopes differ only in a single polytope.

2.5 Cyclic n-Roots

We make use of this standard benchmark for computer algebra packages to discuss empirical results. At the end of the section we return to bound (2.2) and examine its tightness for this problem. This leads to some interesting observations on the Newton polytopes of the cyclic n-roots problem.

Table 2.1 displays the running times of our implementation on the problem of cyclic n-roots for the DEC ALPHA 3000 of table 1.1, rounded to the nearest integer number of seconds. This problem is encountered in Fourier analysis and serves as a benchmark for computer algebra programs. Here we are interested just in bounding the number of isolated solutions.

It is known that if the prime factorization of n includes a square then the number of roots is infinite. This is the case for $n = 4, 8$, where there is a one-dimensional variety causing the mixed volume to give loose bounds. Nonetheless, in light of theorem 1.3.8 this is an upper bound on the number of 0-dimensional roots.

For small values of n the exact bounds were derived in a series of articles. Björck in [Bjö90] credits Lovász with settling the case $n = 5$. The same article states that $n = 6$ was essentially solved in [BF91b] except for an error corrected by Lazard. The problem for $n = 7$ was solved in [BF91a] with the help of Gröbner bases calculations on J. Backelin's program BERGMAN. For $n = 8$ there are 1152 isolated roots in addition to the one-dimensional variety [BF94].

For $n \geq 9$ the precise number of isolated roots is still unknown and for $n = 10$ even finiteness is open. Our bound for $n = 11$ verifies the conjecture by Fröberg and Björck that the number of roots for prime n is $\binom{2n-2}{n-1}$. The polynomial system is the following:

$$\begin{aligned} x_1 + x_2 + \cdots + x_n &= 0 \\ x_1x_2 + x_2x_3 + \cdots + x_nx_1 &= 0 \\ &\vdots \\ x_1 \cdots x_{n-1} + x_2 \cdots x_n + \cdots + x_nx_1 \cdots x_{n-2} &= 0 \\ x_1x_2 \cdots x_n &= 1 \end{aligned} \tag{2.4}$$

We compare running times with the Gröbner Bases package GB by Faugère, since it outperformed BERGMAN. GB was executed on a SUN SPARC 10/40. All running times should be solely viewed as rough indications of the problem's intrinsic complexity and the algorithms' performances. It should be noted that for $n = 7$, a running time of 50min on a SUN SPARC 2 has been reported by an anonymous referee of an article containing the present results.

The mixed volume computation constructs a monomial basis for the coordinate ring which is the first step towards solving the polynomial system, as explained in chapter 4. Still, Gröbner Bases provide significantly more information, including a tighter root count in general as well as solutions in the varieties of positive dimension. For $n = 8$ the Gröbner Basis was computed over a field of a large

n	#isolated roots	Lift-Prune Algorithm mixed volume	time	GB package #roots	time
3	6	6	0s		
4	0	16	0s		
5	70	70	0s		
6	156	156	2s	156	3s
7	924	924	27s	924	6h 0m 4s
8	1152	2560	4m 19s	infinite	(char> 0) 3h 5m 12s
9	unknown	11016	40m 59s	–	–
10	unknown	35940	4h 50m 14s	–	–
11	unknown	184756	38h 26m 44s	–	–

Table 2.1: Lift-prune algorithm performance for the cyclic n-roots problem; timings are on a DEC Alpha 3000. Timings for GB are on a Sun Sparc 20/61.

prime characteristic, while for larger n the problem was infeasible [Fau94]. Exploiting the symmetry, the algorithm of [VG94] requires 16.4 seconds on an DEC Alpha 5000/240 for the cyclic 5-root problem.

Following a suggestion by J. Canny, the geometric mean bound (2.2) is considered as an approximation to mixed volume. As it is, the system's Newton polytopes are all lower-dimensional, so an equivalent formulation is adopted by dividing the k-th polynomial in (2.4) by x_n^k for $k = 1, \ldots, n$. This formulation drops the effective dimension and is the preferred one for solving via sparse resultants. Let $y_i = x_i/x_n$, $i = 1, \ldots, n-1$, substitute variables $x_1, \ldots, x_{n-1}$, then the new system is

$$\begin{aligned} y_1 + y_2 + \cdots + 1 &= 0 \\ y_1y_2 + y_2y_3 + \cdots + y_1 &= 0 \\ &\vdots \\ y_1 \cdots y_{n-1} + y_2 \cdots y_{n-1} + \cdots + y_1 \cdots y_{n-2} &= 0 \\ y_1y_2 \cdots y_{n-1} &= x_n^{-n} \end{aligned}$$

The last equation is the only one dependent on x_n, hence we may restrict attention to the well-constrained system defined by the first $n-1$ equations in variables $y_1, \ldots, y_{n-1}$. Their Newton polytopes Q_i' have nontrivial volume and a bound on the roots of the original system is $nMV(Q_1', \ldots, Q_{n-1}')$.

Surprisingly, for $n = 3, \ldots, 11$, all Q_i' have volume equal to $1/(n-1)!$, hence lower bound (2.2) is trivially 1 and the associated bound for the original system is n. However, mixed volume grows at least exponentially with n, since the ratio of any two consecutive mixed volumes exceeds 2, as seen in table 2.1.

Chapter 3

Algorithms for the Sparse Resultant

Sylvester-type formulae provide the most compact way of expressing resultants, yet they are not generally available for arbitrary polynomial systems. Instead, we concentrate on determinantal formulae, defined by matrices whose determinant is a nontrivial multiple of the sparse resultant. These matrices are called *sparse resultant matrices*.

We consider Laurent polynomials $f_i \in K[x, x^{-1}]$ with $\mathcal{A}_i$ and Q_i being, respectively, their support and Newton polytope, for $i = 1, \ldots, n+1$. Coefficient field K is arbitrary, though we concentrate on $\mathbb{C}$ for the sake of simplicity. For the most part of this chapter the input polynomials are assumed to have indeterminate coefficients.

To exploit sparseness and achieve the degree bounds of theorem 1.3.13 we must work on the sublattice of $\mathbb{Z}^n$ generated by the Minkowski sum of all input supports $\sum_i \mathcal{A}_i$. In other words, the coarsest common refinement of the sublattices generated by each $\mathcal{A}_i$. Suppose this sublattice has rank n and is thus identified with $\mathbb{Z}^n$; in what follows, it is assumed that this has already been done by means of the Smith normal form [Stu94]. The improved version of the first algorithm in section 3.1.6 removes this hypothesis.

The first algorithm uses a mixed subdivision and was proposed in [CE93]. As in the previous chapter, a sufficiently generic lifting induces the subdivision of the Minkowski sum. In the present context, the lifting resembles the monomial ordering imposed in defining Gröbner bases. The matrix construction is presented in the next section, then the properties of the resultant matrix are established, namely that it is a nontrivial multiple of the resultant (sect. 3.1.1) and that has appropriate degree (sect. 3.1.2) to allow the computation of the actual resultant. Among the consequences of our construction is a simple derivation of a set of monomials which constitute a basis for the coordinate ring of the ideal of the input polynomials.

The algorithm is improved in section 3.1.6 where a greedy variant that yields smaller matrices is sketched. More general liftings can be used for our construction as discussed in section 3.1.3. This section also argues about the possibility of defining an exact rational Macaulay-type expression of the sparse resultant and establishes a special case of the sparse effective Nullstellensatz. Section 3.1.9 proposes two ways to compute the actual sparse resultant. For the example system of the Introduction the full construction is then shown; the complexity analysis concludes the first part. This is the object of section 4.1.

The second part of this chapter proposes an incremental algorithm to build more economical sparse resultant matrices and analyzes its complexity in section 3.2.2. Section 3.3 focuses on multihomogeneous systems for a class of which the algorithm constructs Sylvester-type formulae; the practical

performance of the algorithm is examined.

3.1 Subdivision Algorithm

This algorithm relies on the mixed subdivision of the Minkowski sum and is characterized by the fact that the size of the constructed matrix is known in advance. The characteristic feature of this algorithm is that it has a priori knowledge of the matrix dimension, in contrast to the incremental algorithm below for which the final matrix dimension is initially unknown. Matrix M is constructed by assigning a special role to polynomial f_1; any polynomial can assume this special role and an analogous construction will define the respective matrix.

Let Q denote the Minkowski sum of all $n+1$ input Newton polytopes

$$Q = Q_1 + Q_2 + \cdots + Q_{n+1} \subset \mathbb{R}^n.$$

Consider the function

$$(\mathbb{R}^n)^{n+1} \to \mathbb{R}^n : (p_1, \ldots, p_{n+1}) \mapsto p_1 + \cdots + p_{n+1}, \tag{3.1}$$

then Q may be thought of as the image of $Q_1 \times \cdots \times Q_{n+1}$ under this function. This is clearly a many-to-one mapping. We wish, for each $q \in Q$, to define a unique inverse $(p_1, \ldots, p_{n+1})$ under (3.1) in $Q_1 \times \cdots \times Q_{n+1}$.

To this end a method from [Stu94] is employed: choose $n+1$ sufficiently generic linear *lifting* forms $l_1, \ldots, l_{n+1} \in \mathbb{Z}[x_1, \ldots, x_n]$. Genericity is typically implemented by randomly choosing the l_i as in section 2.3. This section analyzes the probability of success, which can be set arbitrarily high, and the number of random bits needed. Then define, for $1 \le i \le n+1$, *lifted* Newton polytopes

$$\widehat{Q}_i = \{(p_i, l_i(p_i)) : p_i \in Q_i\} \subset \mathbb{R}^{n+1}.$$

Let the Minkowski sum of the lifted Newton polytopes be

$$\widehat{Q} = \widehat{Q}_1 + \cdots + \widehat{Q}_{n+1} \subset \mathbb{R}^{n+1}.$$

The induced subdivision construction of section 2.1 is again used. Let $\pi : \mathbb{R}^{n+1} \to \mathbb{R}^n$ denote projection on the first n coordinates. Let $h : \mathbb{R}^{n+1} \to \mathbb{R}$ denote projection on the $(n+1)$-st which essentially expresses the height of the lifting for each point. Both $\widehat{Q}$ and its lower envelope project, under π, to Q, but the first mapping is many-to-one while the second is one-to-one. Hence the following function is well-defined:

$$s : \mathbb{R}^n \to \mathbb{R}^{n+1} : q \mapsto \widehat{q} \in \pi^{-1}(q) \cap \widehat{Q}, \qquad \text{such that } h(\widehat{q}) \text{ is minimized.}$$

The lower envelope of $\widehat{Q}$ is then $s(Q)$. The genericity requirement on l_i is that every point $\widehat{q}$ on the lower envelope can be uniquely expressed as a sum of points $\widehat{q}_1 + \cdots + \widehat{q}_{n+1}$ with $\widehat{q}_i \in \widehat{Q}_i$. Hence a *unique inverse* is defined for mapping (3.1) by associating to every $q \in Q$ the $(n+1)$-tuple of points summing to $s(q)$; by construction, the n-vector sum of these points is q. This unique sum is called the *optimal* Minkowski sum of $q \in Q$.

Let $\widehat{\Delta}$ denote the natural coarsest polyhedral subdivision of the lower envelope of $\widehat{Q}$ into facets. Each facet of $\widehat{\Delta}$ is expressed as a Minkowski sum $\hat{F}_1 + \cdots + \hat{F}_{n+1}$ with $\hat{F}_i$ a face of $\widehat{Q}_i$, where $\sum_{i=1}^{n+1} \dim(\hat{F}_i) = n$. The image of $\widehat{\Delta}$ under π induces a tight coherent subdivision Δ of Q whose (maximal) cells have a unique expression $F_1 + \cdots + F_{n+1}$, where F_i is the face of Q_i which corresponds to

$\widehat{F}_i$ and at least one of the F_i is zero-dimensional i.e. a vertex. Recall that a *mixed cell* of the induced subdivision Δ has type $(1,\ldots,1,0,1,\ldots,1)$, in other words in its optimal sum where exactly one F_i is a vertex and the remaining F_j are edges.

For selecting the matrix entries in a well-defined manner, we must perturb Q slightly so that each integer lattice point lies in the *interior* of a cell of Δ. Let $\delta \in \mathbb{Q}^n$ be an infinitesimal vector in sufficiently generic position so that all integer lattice points lie in the interior of cells. Vector δ does not have to be infinitesimal; it suffices that it be small enough so that every point $p+\delta$ lies in a maximal cell that cobounds p. δ is typically a random vector and the probability that it fails to perturb all points to cell interiors is determined in section 3.1.8. Then, define

$$\mathcal{E} = \mathbb{Z}^n \cap (Q+\delta)$$

to be the set of exponent vectors that indexes the rows and columns of M. If Δ_δ denotes the subdivision obtained by shifting all cells of Δ by δ, the choice of δ is satisfactory if every $p \in \mathcal{E}$ lies in the interior of a cell of Δ_δ. We can now define our selection rule for elements of M based on the following function.

Definition 3.1.1 (Row content function) *Let $p \in \mathcal{E}$ lie in the interior of a cell $\delta + F_1 + \cdots + F_{n+1}$ of Δ_δ. Let i be the* maximum *integer such that F_i is a vertex, so $F_i = a_{ij} \in \mathcal{A}_i$ for some j. Then*

$$RC : \mathcal{E} \to \mathbb{Z}^2 : p \mapsto (i,j).$$

The row content function definition assigns a special role to f_1 by avoiding to pick an optimal sum with a vertex from $\mathcal{A}_1$ whenever possible. Essentially, it assigns a total ordering on the f_i so that it can uniquely associate a polynomial to every optimal sum.

Definition 3.1.2 *M is a matrix whose rows and columns are indexed by elements of $\mathcal{E}$. The element of M at row p and column q, for arbitrary $p,q \in \mathcal{E}$ with $RC(p) = (i,j)$, is*

$$M_{pq} = \begin{cases} c_{ik}, & \text{if } q-p+a_{ij} = a_{ik} \in \mathcal{A}_i, \text{ for some } k, \\ 0, & \text{if } q-p+a_{ij} \notin \mathcal{A}_i. \end{cases}$$

The lemma below proves this is a well-defined square matrix. A diagonal element is $M_{pp} = c_{ij}$ where $(i,j) = RC(p)$. The definition implies that the row of M indexed by $p \in \mathcal{E}$ contains the coefficients of f_i à la Macaulay, and represents the following multiple of f_i:

$$x^{(p-a_{ij})} f_i, \qquad \text{where } (i,j) = RC(p).$$

The number of columns is at least as large as the number of rows, which is $|\mathcal{E}|$. It is obvious that $p - a_{ij} + a_{ik} \in Q \cap \mathbb{Z}^n$ for any $a_{ik} \in \mathcal{A}_i$; but to show that the matrix is well-defined, we need to put the latter in $Q+\delta$. This is demonstrated by the following

Lemma 3.1.3 *Under the above notation, $p - a_{ij} + a_{ik} \in \mathcal{E}$, for all $a_{ik} \in \mathcal{A}_i$.*

Proof Since $p \in \mathcal{E}$, we have $p \in \sigma + \delta$, for some cell σ which can be expressed as a Minkowski sum $F_1 + \cdots + F_{i-1} + a_{ij} + F_{i+1} + \cdots + F_{n+1}$, where F_k is a face of Q_k. This implies that $p - a_{ij} + a_{ik}$ lies in

$$\sigma - a_{ij} + a_{ik} + \delta = F_1 + \cdots + F_{i-1} + a_{ik} + F_{i+1} + \cdots + F_{n+1} + \delta,$$

which is a subset of $Q+\delta$, for any $a_{ik} \in \mathcal{A}_i$. □

Theorem 3.1.4 *Matrix M is well-defined by definition 3.1.2. It is square and has dimension $|\mathcal{E}|$.*

To compute the row content function on $\mathcal{E}$ an explicit mixed subdivision may be constructed as in section 2.3. Yet this is an overkill and linear programming may be used instead to find only the optimal sums. In keeping with earlier notation, μ_i denotes the cardinality of $\mathcal{A}_i$ and μ is the maximum of $\mu_1, \ldots, \mu_{n+1}$.

Algorithm 3.1.5 (RC Computation)
Input: Supports $\mathcal{A}_1, \ldots, \mathcal{A}_{n+1}$, lifting functions $l_1, \ldots, l_{n+1}$ and perturbation vector δ.
Output: Vertex sets of Newton polytopes $Q_1, \ldots, Q_{n+1}$ and the value $RC(p)$ at every $p \in \mathcal{E}$.

Description The first stage constructs the vertex sets of Q_i by repeated application of linear programming. For a point p_{i1} in $\mathcal{A}_i$, we test whether

$$p_{i1} = \sum_{j=1}^{\mu_j} \lambda_j p_{ij}, \qquad \lambda_j \geq 0, \ \sum_{j=1}^{\mu_j} \lambda_j = 1.$$

The optimization function may simply be l_2. Feasibility implies p_{i1} is not a vertex and can be dropped from the right-hand sum used in testing later candidates. Infeasibility implies p_{i1} is a vertex of Q_i.

Now assume Q_i has vertex set $\{a_{i1}, \ldots, a_{im_i}\}$ for $m_i \leq \mu_i$. $p \in \sigma + \delta$ is equivalent to $p - \delta \in \sigma$, for some maximal cell σ. To reduce to linear programming we introduce constraints

$$p - \delta = \sum_{i=1}^{n+1} p_i = \sum_{i=1}^{n+1} \sum_{j=1}^{m_i} \lambda_{ij} a_{ij}$$

where

$$\lambda_{ij} \geq 0, \text{ for } 1 \leq j \leq m_i, \qquad \text{and} \quad \sum_{j=1}^{m_i} \lambda_{ij} = 1, \qquad \forall i \in \{1, \ldots, n+1\}.$$

The objective function forces the lifted point corresponding to p to lie on the lower envelope of $\widehat{Q}$ by requiring that

$$\sum_{i=1}^{n+1} \sum_{j=1}^{m_i} \lambda_{ij} l_i(a_{ij}) \text{ be minimized,}$$

where the l_i's are the generic lifting linear forms. For every i, the λ_{ij} that are positive in the optimal solution correspond to vertices a_{ij} defining the face F_i in the unique Minkowski sum expressing p. □
This procedure is analyzed in section 3.1.8.

If all polynomials are linear, a deterministic choice for $\delta = (\epsilon, \ldots, \epsilon)$ yields a Sylvester-type matrix formula for the sparse resultant, where $\epsilon > 0$ is sufficiently small.

For systems of $n+1$ dense polynomials in n variables, the sparse resultant coincides with the homogeneous resultant. For certain deterministic choices of the lifing forms, the row content function and the perturbation vector, the subdivision algorithm yields Macaulay's matrix whose determinant is an inertia form, as defined by Hurwitz (see sect. 1.2); the respective algorithms are presented in [KY92, sect. 2.3] and [vdW50, sect. 81]. Specifically, let

$$\begin{aligned} \delta &= (\epsilon, \ldots, \epsilon), \\ l_i &= L_{i1}x_1 + \cdots + L_{i(i-1)}x_{i-1} + x_i + L_{i(i+1)}x_{i+1} + \cdots + L_{in}x_n, \qquad i = 1, \ldots, n, \\ l_{n+1} &= L_{(n+1)n}x_1 + \cdots + L_{(n+1)n}x_n, \end{aligned} \tag{3.2}$$

where $\epsilon > 0$ is sufficiently small and every $L_{ij} \in \mathbb{Z}_{\gg 0}$ is sufficiently large, for $i = 1, \ldots, n+1$, $j = 1, \ldots, n$, so that it exceeds any sum of $n+1$ coordinates of Newton polytope vertices.

There is one Macaulay matrix for every polynomial f_i in the coefficients of which the determinant's degree is tight i.e., it equals the resultant's degree in f_i. Expositions of Macaulay's construction usually assign this distinguished role to f_{n+1}. To produce this matrix, we should modify RC to select the *minimum* possible index. As defined originally the subdivision algorithm will produce the Macaulay matrix of f_1 and analogous choices for RC produce inertia forms with exact degrees in the other polynomials. To keep with notation in [KY92, Can88b] we do use the modified RC that selects the minimum polynomial index.

We demonstrate that the corresponding Macaulay matrix equals the constructed matrix M. The degree of M in f_{n+1} is $MV_{-(n+1)} = \prod_{i \leq n} d_i$, where d_i is the total degree of f_i. This equals the degree of the Macaulay determinant in f_{n+1}. So it suffices to prove that the same holds for the other polynomials. The degree of the Macaulay determinant in f_1 equals the cardinality of

$$E_1 = \{e = (e_1, \ldots, e_n) \,|\, e_i \geq 0, \|e\| = D, e_1 \leq d_1\}, \qquad \text{where } D = 1 + \sum_{i=1}^{n+1} (d_i - 1) \tag{3.3}$$

and $\|\cdot\|$ denotes 1-norm, hence $\|e\| = \sum_{i=1}^{n+1} e_i$. The choice of δ implies that

$$p_1, \ldots, p_n > 0 \text{ and } \|p\| \leq \sum_{i=1}^{n+1} d_i \qquad \forall\, p = (p_1, \ldots, p_n) \in \mathcal{E}.$$

Lemma 3.1.6 *The number of rows in M containing coefficients of f_1 equal the cardinality of E_1 in (3.3).*

Proof Consider any point $p = (p_1, \ldots, p_n) \in \mathcal{E}$ with $p_1 > d_1$. Let $p = a_1 + \cdots + a_{n+1}$ be its optimal sum with $a_i \in Q_i$. By the choice of the lifting, a_1 accounts for as much as p_1 as possible. Hence the first coordinate of $a_1 \in Q_1$ equals d_1 and $a_1 = (d_1, 0, \ldots, 0)$. To see that this is feasible consider that, since $p_1 > d_1$, we have $p_2 + \cdots + p_n < d_2 + \cdots + d_{n+1}$.

Clearly, all points p with $p_1 > d_1$ will have $a_1 = (d_1, 0, \ldots, 0)$ in their optimal sum, so $RC(p) = (1, j)$, where a_1 is the j-th vertex in Q_1. The remaining points in $\mathcal{E}$ can be partitioned in sets where the second coordinate is larger that d_2, then the third larger than d_3 and so on. This partition imitates the one classical way of defining the Macaulay matrix and produces much the same result. A recursive argument shows that the points p with $p_1 \leq d_1$ will have $RC(p) = (i, j)$, with $i > 1$ and some j. We conclude that the number of rows in M containing coefficients of f_1 equals the cardinality of

$$\left\{ p = (p_1, \ldots, p_n) \,|\, p_i > 0, \|p\| \leq \sum_{i=1}^{n+1} d_i, p_1 > d_1 \right\}.$$

By setting $p_1 = e_1 + 1$ and $p_i = e_i - 1$ for $i > 1$ we prove the result. □

Application of this lemma to the points in $\mathcal{E}$ not indexing f_1 rows shows that the number of f_2 rows equals the degree of the Macaulay matrix. By repeating the same argument $n-1$ times in total we can prove

Theorem 3.1.7 *Assume that the given polynomial system is completely dense. Then there exist choices for lifting l, row content function RC, and perturbation δ such that the subdivision algorithm constructs the Macaulay matrix.*

3.1.1 A Nontrivial Multiple of the Resultant

First we prove that the determinant of M is a multiple of the sparse resultant and then that this multiple is not identically zero for any specialization of the coefficients. Regarding polynomial rings as vector spaces over the coordinate field, with respect to some fixed monomial basis, has been a fruitful viewpoint in the study of resultants. Similarly, an $(n+1)$-tuple of polynomials can be written as the concatenation of the $n+1$ respective vectors. Letting M denote the endomorphism represented by matrix M, we have

$$M : \mathbb{C}^{|\mathcal{E}|} \to \mathbb{C}^{|\mathcal{E}|} : (g_1, \ldots, g_{n+1}) \mapsto g = g_1 f_1 + \cdots + g_{n+1} f_{n+1} \tag{3.4}$$

where the support of every g_i is defined to be $\{p - a_{ij} \,|\, p \in \mathcal{E},\ RC(p) = (i,j)\}$. The support of g_i coincides with the set of monomials that multiply f_i in defining the rows of M. There is a bijective correspondence between the points $p \in \mathcal{E}$ and the points $p - a_{ij}$ in the support of g_i, hence the supports of g_i define a partition of $\mathcal{E}$. Moreover, the support of g corresponds exactly to the monomials indexing the columns of M and equals $\mathcal{E}$.

Endomorphism (3.4) is expressed as premultiplication of M by a row vector indexed by $\mathcal{E}$ since the supports of g_i define a partition of $\mathcal{E}$:

$$[g_1, \ldots, g_{n+1}]M = [g_1 f_1 + \cdots + g_{n+1} f_{n+1}].$$

Now linear algebra proves the first property of M.

Lemma 3.1.8 *If there exists $\xi \in (\mathbb{C}^*)^n$ such that $f_1(\xi) = \cdots = f_{n+1}(\xi) = 0$, then $\det M = 0$.*

Proof Assume that M is nonsingular. Then endomorphism M is surjective and we can choose polynomials $g_1, \ldots, g_{n+1}$ such that g in (3.4) is a monomial. This monomial is the sum of all products $g_i f_i$ so it must be zero at every solution ξ, which is impossible for $\xi \in (\mathbb{C}^*)^n$. Hence there can be no solution in $(\mathbb{C}^*)^n$, which contradicts the hypothesis. □

Theorem 3.1.9 *The sparse resultant R divides the determinant of M.*

Proof The lemma implies that $\det M = 0$ on the set Z_0 of specializations of c_{ij} such that the system has a solution in $(\mathbb{C}^*)^n$. Thus it is zero on the closure Z of Z_0, which is exactly the zero set of the resultant $R(\mathcal{A}_1, \ldots, \mathcal{A}_{n+1})$. Since the resultant is irreducible or 1 it must divide $\det M$. □

To exclude the possibility that $\det M$ is identically zero, we show that it does not vanish under the specialization of the coefficients

$$c_{ij} \mapsto t^{l_i(a_{ij})}, \tag{3.5}$$

where l_i are the lifting forms of the previous section. Each c_{ij} is substituted by an integral power of t, where t is a new indeterminate. Observe that the Newton polytope of the specialized f_i as a polynomial in $\mathbb{C}[x_1, x_1^{-1} \ldots, x_n, x_n^{-1}, t, t^{-1}]$ is precisely $\widehat{Q}_i$. Let $M(t)$ denote the matrix M under this specialization, and $\det M(t)$ denote its determinant, which is a polynomial in t with integer coefficients.

Lemma 3.1.10 (Tile balancing) *Let $\widehat{p}$ be a point in the interior of some facet of the subdivision $\widehat{\Delta}$ of the lower envelope $s(Q)$. By construction, $\widehat{p}$ has a unique (optimal) expression as a sum of points from $\widehat{Q}_1, \ldots, \widehat{Q}_{n+1}$, and one of these is a vertex $\widehat{a}_{ij} = (a_{ij}, l_i(a_{ij}))$. Then $(\widehat{p} - \widehat{a}_{ij} + \widehat{Q}_i) \cap s(Q) = \widehat{p}$.*

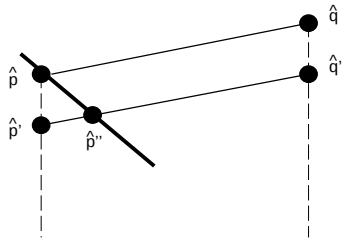

Figure 3.1: Proof of the tile balancing lemma.

Proof It suffices to show that every other point $\widehat{q} \in \widehat{p} - \widehat{a}_{ij} + \widehat{Q}_i$ lies above the lower envelope. It is easy to see that $\widehat{p} - \widehat{a}_{ij} + \widehat{Q}_i$ is contained in $\widehat{Q}$, because it consists of sums of $(n+1)$-tuples of points, one from each polytope. So all points in it are either on or above the lower envelope.

Now displace both $\widehat{p}$ and $\widehat{q}$ by decreasing their $(n+1)$-st coordinate by some ϵ, thus defining points $\widehat{p}', \widehat{q}' \in \mathbb{R}^{n+1}$ (see fig. 3.1). The displacement should be small enough so that the line $(\widehat{p}', \widehat{q}')$ intersects the lower envelope in the facet that contains $\widehat{p}$. This is always possible because $\widehat{p}$ lies in the interior of a facet. If $\widehat{p}''$ is this intersection point it is clear that $\widehat{Q}$ also contains $\widehat{p}'' - \widehat{a}_{ij} + \widehat{Q}_i$.

By convexity, vector $\widehat{q}' - \widehat{p}''$ is smaller than $\widehat{q} - \widehat{p}$ and is also in the same direction, see fig. 3.1. Now $\widehat{q} - \widehat{p}$ is contained in the convex set $\widehat{Q}_i - \widehat{a}_{ij}$. It follows that $\widehat{q}' - \widehat{p}''$ is also contained in $\widehat{Q}_i - \widehat{a}_{ij}$: $\widehat{q}' \in \widehat{p}'' - \widehat{a}_{ij} + \widehat{Q}_i \subset \widehat{Q}$. We have demonstrated a point $\widehat{q}'$ such that $\pi(\widehat{q}) = \pi(\widehat{q}')$ but $h(\widehat{q}') < h(\widehat{q})$. Thus $\widehat{q}$ is not on the lower envelope. □

Consider each $\widehat{Q}_i$ as the Newton polytope of a polynomial in the specialized system, where t is the $(n+1)$-st variable. More precisely, the Newton polytope of the polynomial in row p is $\widehat{Q}_i$ shifted so that its vertex $\widehat{a}_{ij}$ lies on the lower envelope, over p. The lemma simply states that the rest of the polytope will lie above the lower envelope. Figuratively the polytope resembles a tile, balancing on the lower envelope surface on a point.

Define a matrix $M'(t)$ by scaling the rows of $M(t)$:

$$M'_{pq} = t^{h(\widehat{p}) - l_i(a_{ij})} M_{pq}, \qquad \forall q \in \mathcal{E}, \quad \text{where} \quad (i,j) = RC(p), \widehat{p} = s(p).$$

Then the previous lemma leads to an inequality on the degree in t of the M' entries.

Lemma 3.1.11 *For all nonzero elements M'_{pq} with $p \neq q$, $\deg(M'_{pq}) > \deg(M'_{qq})$.*

Proof Let $\widehat{p}$ and $\widehat{q}$ be the points on the lower envelope $s(Q)$ such that $\pi(\widehat{p}) = p$ and $\pi(\widehat{q}) = q$. Let $(\iota, \gamma) = RC(q)$ and, since $\deg(M_{qq}(t)) = l_\iota(a_{\iota\gamma})$, we have

$$\deg(M'_{qq}(t)) = h(\widehat{q}) - l_\iota(a_{\iota\gamma}) + \deg(M_{qq}(t)) = (h(\widehat{q}) - l_i(a_{ij})) + l_i(a_{ij}) = h(\widehat{q}).$$

Let $\widehat{\Delta}_\delta$ denote the perturbed subdivision of the lower envelope; $\widehat{p}$ lies in the interior of a facet of $\widehat{\Delta}_\delta$. Let $M_{pq} = c_{ik} \neq 0$, then $q = p - a_{ij} + a_{ik}$, for some $a_{ik} \in \mathcal{A}_i$, and let $\widehat{q}_0 = \widehat{p} - \widehat{a}_{ij} + \widehat{a}_{ik}$. Then

$$\deg(M'_{pq}(t)) = h(\widehat{p}) - l_i(a_{ij}) + l_i(a_{ik}) = h(\widehat{q}_0).$$

From the previous lemma $\widehat{q}_0$ cannot lie on the lower envelope, hence $h(\widehat{q}_0) > h(\widehat{q})$. □

In the language of lifted Newton polytopes, the row-scaling of $M(t)$ by powers of t corresponds to translating the Newton polytope of row p so that vertex a_{ij} touches the lower envelope at $\widehat{p}$. The lemma considers elements in a column q of M' and compares the last coordinate of points in various lifted Newton polytopes that lie over q. By lemma 3.1.10, there is a unique point of minimum $(n+1)$-st coordinate on the lower envelope over q which must correspond to element M'_{qq}. Therefore the other entries in column q have higher degree in t than M'_{qq}.

Theorem 3.1.12 *Let* $\det N(t)$ *be any principal minor of* $M(t)$*, where* N *is the corresponding square submatrix, and let* $\mathcal{E}' \subset \mathcal{E}$ *denote the row and column monomials of* N*. Then the lowest degree term of* $\det N(t)$ *is the product of the leading diagonal elements of* $N(t)$*, it has coefficient* -1 *or* $+1$ *and integer exponent*

$$\sum_{p \in \mathcal{E}'} l_i(a_{ij}), \qquad \text{where } (i,j) = RC(p), \quad \forall p \in \mathcal{E}'.$$

Therefore this minor is non-vanishing.

Proof Let $N'(t)$ be the matrix obtained from N after multiplying the rows of M to define M'. If N'_{pq} is the entry indexed by $p, q \in \mathcal{E}'$ then

$$\det(N')(t) = \sum_{\sigma \in S(\mathcal{E}')} (-1)^{\text{sign}(\sigma)} \prod_{q \in \mathcal{E}'} N'_{\sigma(q)q}$$

where $S(\mathcal{E}')$ is the symmetric group on $\mathcal{E}'$ and $\text{sign}(\sigma)$ stands for the parity of permutation σ. For every σ not equal to the identity, we have $\sigma(q) \neq q$ for some q, so $\deg(N'_{\sigma(q)q}) > \deg(N'_{qq})$ by the previous lemma. Thus

$$\deg(\prod_{q \in \mathcal{E}'} N'_{qq}) < \deg(\prod_{q \in \mathcal{E}'} N'_{\sigma(q)q})$$

for every permutation σ other than the identity. This implies that the product of leading diagonal entries is a unique lowest power of t. Every term in $\det N'(t)$ is the product of the respective term in $\det N(t)$ and a fixed integral power of t, denoted by t^e, which equals the product of all scale factors: $\det N'(t) = t^e \det N(t)$. Hence

$$\det N(t) = \prod_{q \in \mathcal{E}'} N_{qq} + \text{higher order terms in } t = \prod_{q \in \mathcal{E}'} t^{l_i(a_{ij})} + \text{higher order terms in } t, \tag{3.6}$$

where $(i,j) = RC(p)$ for all $p \in \mathcal{E}'$. There exists some value $t_0 \neq 0$ of t for which this product is not canceled hence $\det N(t) \neq 0$. □

Corollary 3.1.13 *The lowest degree term in* t *of* $\det M(t)$ *is the product of all diagonal entries, has coefficient* -1 *or* $+1$ *and exponent*

$$\sum_{p \in \mathcal{E}} l_i(a_{ij}), \qquad \text{where } (i,j) = RC(p), \ \ \forall p \in \mathcal{E}.$$

Therefore $M(t)$ *is not identically singular, and moreover* M *is not identically singular*

Proof Set $\mathcal{E}'$ to $\mathcal{E}$ which implies that $N = M$. The previous theorem establishes the claim since there exists specialization (3.5) such that $c_{ij} \mapsto t^{l_i(a_{ij})}$ and a value of t so that $\det M(t)$ is nonzero. Hence $\det M$ is not identically zero. □

3.1.2 The Determinant Degree

This section examines the degree of $\det M$ in the coefficients of each input polynomial f_i. Given fixed integer vectors $e_1, \ldots, e_n \in \mathbb{Z}^n$, define an n-dimensional *half-open integral parallelotope HO*:

$$HO = \{\sum_{i=1}^{n} r_i e_i \mid \ r_i \in [0,1)\}$$

Lemma 3.1.14 *The number of integer lattice points in a half-open integral parallelotope equals its volume.*

Proof It follows from [Sta80, p.335] that the number of these lattice points is $n!\mathrm{Vol}(S)$ where S is the simplex $\mathrm{Conv}(0, e_1, \ldots, e_n)$. The volume of the parallelotope *HO* is known to be $n!\mathrm{Vol}(S)$. □

Corollary 3.1.15 *For any $\delta \in \mathbb{R}^n$, the number of integer lattice points in $HO + \delta$ equals* $\mathrm{Vol}(HO)$.

Proof Imagine that *HO* is displaced by $t\delta$ as t varies from 0 to 1. Observe that for each facet of *HO* that is open or closed, the opposite facet is closed or open respectively, and that the opposite facet is displaced from the first by an integral vector v. Thus as *HO* moves, whenever a lattice point p enters *HO*, a corresponding point at $p + v$ exits and vice versa. Thus the number of lattice points inside *HO* remains constant. By lemma 3.1.14 this number is the parallelotope's volume. □

A mixed facet of the subdivision $\widehat{\Delta}_\delta$ is the Minkowski sum of n edges and a vertex, hence a parallelotope in $\mathbb{R}^n$, and the same holds for every mixed cell in Δ_δ. The perturbation by generic δ guarantees that all integer points lie in the interior of a cell, therefore the number of rows associated with a mixed cell equals its volume.

The row content function RC chooses f_1 if there is no other possibility, which happens precisely at the mixed cells to which Q_1 contributes a vertex. By the multilinearity of mixed volume it can be shown that the total volume of these cells equals $MV(Q_2, \ldots, Q_{n+1})$. So the number of rows containing coefficients of f_1 is precisely $MV(Q_2, \ldots, Q_{n+1})$.

For every f_j, $j > 1$, the number of rows containing a multiple of f_j is at least as large as the number of points in the mixed cells to which Q_j contributes a vertex, because for these cells RC has no choice but to pick f_j. For a fixed $j > 1$ this number is greater or equal to $MV(Q_1, \ldots, Q_{j-1}, Q_{j+1}, \ldots, Q_{n+1})$. We have proved

Theorem 3.1.16 *The degree of* $\det M$ *in the coefficients of f_1 equals $MV(Q_2, \ldots, Q_{n+1})$, which equals the degree of $R(\mathcal{A}_1, \ldots, \mathcal{A}_{n+1})$ in the coefficients of f_1. The degree of* $\det M$ *in the coefficients of f_j, for $j > 1$, is $\geq MV(Q_1, \ldots, Q_{j-1}, Q_{j+1}, \ldots, Q_{n+1})$, in other words at least as large as the respective degree of $R(\mathcal{A}_1, \ldots, \mathcal{A}_{n+1})$.*

3.1.3 Generalized Lifting

Our algorithm has been generalized by B. Sturmfels [Stu94, sect. 2, 3] by generalizing the way the lifting functions l_i are defined. Consider any monomial $\prod c_{ij}^{\nu_{ij}}$ in the input coefficients; the exponent vector ν lies in $\mathbb{Z}^\mu$, where $\mu = \mu_1 + \cdots + \mu_{n+1}$ is the total number of coefficients. Following Sturmfels' notation we define a linear functional

$$\omega : \mathbb{Z}^\mu \to \mathbb{R} : \nu = (\ldots, \nu_{ij}, \ldots) \mapsto \mathbb{R}.$$

This linear functional defines a weight function on the monomials of the resultant R, hence we can speak of the *leading* and *trailing* monomials in R with respect to some ω which are those with maximum and minimum weight. Define the *extreme* monomials as those corresponding to vertices of the Newton polytope of R. Then the leading monomial is extreme, for generic ω. Further, the restriction of ω to vectors $(\ldots, 0, 1, 0, \ldots)$ with a single nonzero entry at some position between $\mu_1 + \cdots + \mu_{i-1} + 1$ and $\mu_1 + \cdots + \mu_{i-1} + \mu_i$ defines a lifting function ω_i on $\mathcal{A}_i$. As before, in constructing a mixed subdivision and defining the resultant matrix there is a genericity requirement that for all facets $\widehat{F}$ on $\widehat{Q}$,

$$\dim \widehat{F} = \sum_i^{n+1} \dim \widehat{F}_i, \qquad \text{where } F = \sum_i^{n+1} F_i.$$

The lifting functions l_i we have used are restrictions of linear functions on $\mathbb{R}^n$, each distinct for every given Newton polytope. The overall function l can be regarded as a linear function on $\mathbb{R}^{n(n+1)}$ and defines a weight function on resultant monomials:

$$l : (\ldots, \nu_{ij}, \ldots) \mapsto \sum_{i,j} \nu_{ij} l_i(a_{ij}),$$

where a_{ij} is the exponent vector in $\mathcal{A}_i$ associated to c_{ij}.

We can now show that the product of diagonal entries in resultant matrix M is divisible by the leading monomial of R. From the proof of theorem 3.1.12, the product of diagonal entries in the specialized matrix $M'(t)$ is of minimum degree and weight $l(\ldots, \nu_{ij}, \ldots) = \sum \nu_{ij} l_i(a_{ij})$, where $\prod c_{ij}^{\nu_{ij}}$ is the product of leading diagonal in M. This implies that the latter monomial is of minimum weight under l.

Lemma 3.1.17 [Stu94, thm. 2.1] *The product of the leading diagonal entries of M is a multiple of the trailing monomial of R with respect to l, which equals, for some constant c,*

$$c \prod_F c_{ij}^{\mathrm{Vol}(F)}, \qquad \textit{over all mixed facets } F \textit{ where } F = \cdots + a_{ij} + \cdots.$$

If M is defined through a mixed subdivision induced by the upper envelope of $\widehat{Q}$ instead of the lower envelope, then the product of the diagonal entries is a multiple of the leading monomial.

Sturmfels' generalization allows the generation of a resultant matrix for every different ω in much the same way as previously. Define the weight of a sum of Newton polytope vertices to be

$$\omega\left(\sum_{i=1}^{n+1} p_{ij_i}\right) = \sum_{i=1}^{n+1} \omega_i(0, \ldots, 0, 1, 0, \ldots, 0),$$

where the unit appears at the unique position corresponding to the coefficient c_{ij} of p_{ij} and ω_i is the restriction of ω to $\mathcal{A}_i$ specified above. Then a lifting is defined by using ω_i as the lifting function on $\mathcal{A}_i$ and the Minkowski sum of the lifted Newton polytopes may be constructed. This induces a mixed subdivision on Q and a row content function will construct matrix $M = M_\omega$ indexed by the integer lattice points in $Q + \delta$ for vector δ as above. Again, the product of diagonal entries is divisible by the extreme monomial in R with respect to ω, namely the leading or trailing monomial depending on whether the upper or lower envelope of $\widehat{Q}$ is used in defining the mixed subdivision of Q. In fact, any extreme monomial becomes the leading one for an appropriate choice of ω.

The determinant of M and its irreducible factor R have integer coefficients therefore, by Gauss' lemma, their quotient has integer coefficients. Relation (3.6) in the proof of theorem 3.1.12 and the previous lemma imply

Theorem 3.1.18 [GKZ91, thm. 3A.2.b] [Stu94, cor. 3.1] *All extreme monomials of the sparse resultant have coefficient* -1 *or* $+1$.

3.1.4 Towards an Exact Quotient Expression

A natural next step is to establish a formula for the sparse resultant as the quotient of two determinants, in the fashion of Macaulay [Mac02]. Let M be the matrix constructed in section 3.1, with rows and columns indexed by the integer points $\mathcal{E}$ in the subdivision Δ_δ of Minkowski sum Q. Define $\mathcal{E}^{nm}$ to be the subset of these points that do not lie in mixed cells of Δ_δ and denote by M^{nm} the square submatrix of M that includes all entries whose row and column index lie in $\mathcal{E}^{nm}$. Then the diagonal elements of M^{nm} constitute a subset of the diagonal elements of M and theorem 3.1.12 implies that M^{nm} is generically nonsingular.

Conjecture 3.1.19 *There exist vectors* δ *and forms* $l_1, \ldots, l_{n+1}$ *for which the determinant of matrix* M^{nm} *divides exactly the determinant of* M *and, hence, the sparse resultant of the given polynomial system is* $R = \det M / \det M^{nm}$.

It is clear that the proposed rational expression has the same degree as R in the coefficients of every polynomial f_i.

For $n = 2, 3$ several empirical results confirm the conjecture. We have experimented with different systems and found that for most choices of perturbation δ and lifting l the ratio of the determinants is a polynomial of the proper degree. For random δ and l about 90% of the choices lead to positive results.

The conjecture clearly holds in the dense case. A specialization of definitions (3.2) yields a submatrix M^{nm} identical to the submatrix defined by Macaulay. In particular, we require that

$$\begin{aligned} \delta &= (\epsilon, \ldots, \epsilon), \\ l_i &= L_{i1}x_1 + \cdots + L_{i(i-1)}x_{i-1} + x_i + L_{i(i+1)}x_{i+1} + \cdots + L_{in}x_n, \qquad i = 1, \ldots, n, \\ l_{n+1} &= L_{(n+1)n}x_1 + \cdots + L_{(n+1)n}x_n, \end{aligned}$$

where $\epsilon > 0$ is sufficiently small. All $L_{ij} \in \mathbb{Z}_{\gg 0}$ are sufficiently large and moreover $L_{(n+1)k} \gg L_{ij}$ so that every $L_{(n+1)k}$ is larger than the product of any L_{ij} with any sum of $n+1$ coordinates of Newton polytope vertices. *RC* is once more modified to pick the minimum possible index. Then the set of integer points in nonmixed cells corresponds exactly to the set of monomials indexing the Macaulay minor that defines the denominator in the exact expression for the homogeneous resultant [KY92].

Interestingly, there exist values of δ and l that do not lead to a reducible rational expression. Here is an example, with $n = 2$, $\delta = (\epsilon, \epsilon)$ for $1 \gg \epsilon > 0$, for which $\det M$ is not divisible by the denominator minor: The polynomials are completely dense and have degrees $1, 2, 3$; the bad lifting is

$$l_1 = 10^4 x_1 + 10^3 x_2, \qquad l_2 = 10^5 x_1, \qquad l_3 = 10^2 x_1 + 10 x_2.$$

3.1.5 Sparse Effective Nullstellensatz

The resultant matrix construction directly implies a sparse version of the effective Nullstellensatz, albeit only for ideals generated by $n + 1$ sufficiently generic polynomials. In other words, it is possible to bound the Newton polytopes of the polynomial coefficients in the ideal membership formula in terms of the Newton polytopes of the ideal generators. It is interesting to note how Newton polytopes

provide a more precise characterization of a polynomial by taking over the role that degree holds in classical elimination.

An effective version of Hilbert's Nullstellensatz is crucial in computational algebraic geometry. For the classical Nullstellensatz, the problem was settled by Brownawell [Bro87] and Kollár [Kol88] who showed

Theorem 3.1.20 (Effective Nullstellensatz) *Suppose $\mathcal{I} = (f_1, \ldots, f_r)$ is an ideal in polynomial ring $K[x_1, \ldots, x_n]$, where K is an arbitrary field and $\deg f_i \leq d$, for $i = 1, \ldots, r$. Let $h \in K[x_1, \ldots, x_n]$ have total degree $\deg h$. Polynomial h vanishes at the common zeros of $\mathcal{I}$ i.e. $h \in \sqrt{\mathcal{I}}$, if and only if there exist polynomials $g_1, \ldots, g_r \in K[x_1, \ldots, x_n]$ such that*

$$h^q = \sum_{i=1}^{r} g_i f_i, \qquad \text{for some } q \leq 2(d+1)^n, \text{ where } \deg(g_i f_i) \leq 2(\deg h + 1)(d+1)^n, \ i = 1, \ldots, r.$$

If $\mathcal{I} = (1)$ then

$$1 = \sum_{i=1}^{r} g_i f_i, \qquad \text{where } \deg g_i f_i \leq 2(d+1)^n, \ i = 1, \ldots, r.$$

For the special case of sets of $n+1$ generic polynomials, ideal $\mathcal{I}$ equals the entire ring and the associated variety is empty. By corollary 3.1.13, matrix M is nonsingular and map (3.4) $M : (g_1, \ldots, g_{n+1}) \mapsto \sum_i g_i f_i$ is surjective. The sparse resultant algorithm then implies a sparse, or Newton, effective Nullstellensatz:

Theorem 3.1.21 *Suppose $f_1, \ldots, f_{n+1}$ are generic Laurent polynomials in $K[x_1, x_1^{-1}, \ldots, x_n, x_n^{-1}]$ with Newton polytopes Q_i, where K is an arbitrary field and $\mathcal{I} = K[x, x^{-1}]$ is the generated ideal. Then there exist Laurent polynomials $g_1, \ldots, g_{n+1} \in K[x, x^{-1}]$, with Newton polytopes Q_i', such that*

$$1 = \sum_{i=1}^{n+1} g_i f_i : \qquad Q_i' \subset Q_1 + \cdots + Q_{i-1} + Q_{i+1} + \cdots + Q_{n+1}.$$

Proof By definition, the support of g_i contains vector differences of the form $p - p_i$, where $p \in \mathcal{E}$ and p_i is a summand of p and a vertex of Q_i. Therefore

$$\operatorname{supp}(g_i) \subset (Q_1 + \cdots + Q_{n+1}) - Q_i = Q_1 + \cdots + Q_{i-1} + Q_{i+1} + \cdots + Q_{n+1}$$

by definition 1.3.4 of Minkowski difference and the discussion following. This proves the result. □

For the dense case where $\mathcal{I} = (1)$, this implies $\deg g_i \leq nd$. It should be possible to generalize this result to an effective sparse Nullstellensatz for arbitrary systems with no common roots.

Conjecture 3.1.22 (Effective sparse Nullstellensatz) *Suppose $f_1, \ldots, f_r$ are arbitrary Laurent polynomials in $K[x_1, x_1^{-1}, \ldots, x_n, x_n^{-1}]$ with Newton polytopes Q_i, where K is an arbitrary field and $\mathcal{I} = K[x, x^{-1}]$ is the generated ideal. Then there exist Laurent polynomials $g_1, \ldots, g_r \in K[x, x^{-1}]$, with Newton polytopes Q_i', such that*

$$1 = \sum_{i=1}^{r} g_i f_i : \qquad Q_i' \subset Q_1 + \cdots + Q_{i-1} + Q_{i+1} + \cdots + Q_r.$$

We have proven above a special case of this conjecture. Once the latter is established, further generalization to nontrivial ideals would be straightforward.

3.1.6 Improved Algorithm

We now discuss an improved version of the algorithm, first proposed by Canny and Pedersen. We call it a *greedy* variant, since it proceeds in a greedy fashion to construct a matrix N which is at most as large as M and still possesses all essential properties of M.

Definition 3.1.23 *A* closed *submatrix N of M is any square proper submatrix of M such that the sets of monomials indexing the rows and columns of N are identical and, for every row included in N, all nonzero entries of this row are in N.*

The closedness condition is strong enough to lead to the following

Theorem 3.1.24 *For any closed submatrix N of M that includes at least $MV(f_1, \ldots, f_{i-1}, f_{i+1}, \ldots, f_{n+1})$ rows of f_i, the determinant $\det N$ is a nontrivial multiple of the resultant. Moreover, the degree of $\det N$ in the coefficients of f_i lies between the respective degree of the resultant and of $\det M$ in these coefficients.*

Proof By theorem 3.1.12, N is generically nonsingular. Theorem 3.1.9 also shows $R \,|\, \det N$. The relation on the degrees follows from the divisibility of $\det N$ by R and the fact that the rows of N containing f_i form a subset of those in M. □

Here is the greedy algorithm to construct a closed submatrix N. Assume that we have defined a row content function RC. Let RN and CN denote, respectively, the set of row and column monomials indexing N. Both RN and CN are subsets of $\mathcal{E}$ so their cardinality is at most that of $\mathcal{E}$, where $\mathcal{E} = (Q + \delta) \cap \mathbb{Z}^n$ as previously. In the original algorithm we have $RN = CN = \mathcal{E}$, while here we define RN and CN dynamically and expect that their cardinality be smaller than $|\mathcal{E}|$. Notice that we do not require the entire subdivision Δ_δ of Q but need to apply RC only on a subset of $\mathcal{E}$, which we do dynamically. In short, Δ_δ is never explicitly computed but instead optimal sums are found by linear programming for those points in RN.

Algorithm 3.1.25 (Greedy Algorithm)
Input: Supports $\mathcal{A}_1, \ldots, \mathcal{A}_{n+1}$, lifting functions $l_1, \ldots, l_{n+1}$ and perturbation vector δ.
Output: Monomial sets RN and CN indexing the rows and columns of N.

Description Let $a_i \in \mathcal{A}_i$ maximize δ and let $a = \sum_i a_i$. Initialize RN to contain a and let CN be empty; RN and CN will be incremented at successive steps until RN and CN are identical. At every round, given RN, we compute

$$\mathcal{B}_i = \{p - a_{ij} \,|\, p \in RN, a_{ij} = RC(p)\}, \quad i = 1, \ldots, n+1 \qquad \text{and } CN = \bigcup_{i=1}^{n+1} (\mathcal{B}_i + \mathcal{A}_i). \tag{3.7}$$

The computation of RC for points $p \in \mathcal{E}$ is carried out by linear programming as in algorithm 3.1.5. The sets of $p \in RN$ corresponding to each $\mathcal{B}_i$ induce a partition of RN and index the rows of the matrix filled with f_i coefficients. Equivalently, $\mathcal{B}_i$ is the support of g_i in endomorphism (3.4).

If $RN \neq CN$ then we set RN equal to CN and we repeat operations (3.7). The enlargement of RN and CN stops when the two sets coincide after an execution of (3.7). □

RN defines N; the diagonal entries of the matrix are again of the form $N_{pp} = c_{ij}$ where $(i, j) = RC(p)$, hence this matrix has no empty row or column. It is obvious that N is closed and a submatrix of M. The previous theorem then implies

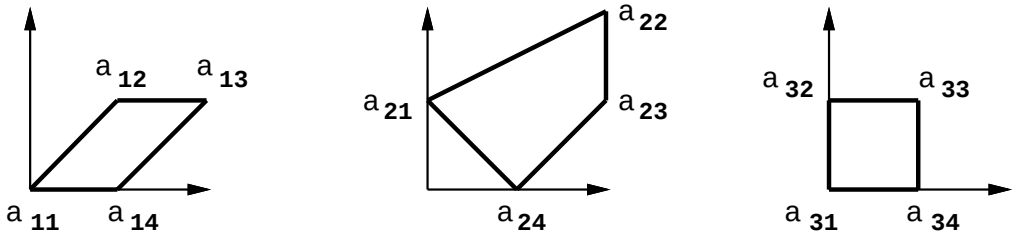

Figure 3.2: The Newton polytopes and the exponents a_{ij}.

Corollary 3.1.26 *Matrix N defined by RN and CN is a sparse resultant matrix, in other words its determinant is a nontrivial multiple of R. Moreover, its determinant satisfies* $\deg_{f_1} \det N = \deg_{f_1} R$ *and* $\deg_{f_i} \det N \geq \deg_{f_i} R$, *for* $i = 2, \ldots, n+1$.

In implementing this some bookkeeping allows us to increment sets $\mathcal{B}_i$ and CN at every round instead of recomputing them in their entirety. The main advantage of this algorithm is that it typically produces smaller matrices. A concrete example is given in section 3.1.7, where the two algorithms are compared. Another merit of this algorithm is that it removes the requirement on the integer lattice generated by the given supports being $\mathbb{Z}^n$. Even when this lattice is of lower density this algorithm works.

On the downside, we have less control over the subdivision that is implicitly used, which means that the exact quotient formula of conjecture 3.1.19 would not be directly applicable. Indeed, the difference of the dimension of M and the number of rows corresponding to non-mixed cells is not always equal to the degree of the sparse resultant.

3.1.7 Example

The construction is illustrated for the system of 3 polynomials in 2 unknowns (1.3.10) which we show here again:

$$\begin{aligned} f_1 &= c_{11} + c_{12}xy + c_{13}x^2y + c_{14}x \\ f_2 &= c_{21}y + c_{22}x^2y^2 + c_{23}x^2y + c_{24}x \\ f_3 &= c_{31} + c_{32}y + c_{33}xy + c_{34}x. \end{aligned} \tag{3.8}$$

The Newton polytopes are shown in figure 3.2; the mixed volumes are $MV(Q_1, Q_2) = 4$, $MV(Q_2, Q_3) = 4$, $MV(Q_3, Q_1) = 3$. Pick generic functions

$$\begin{aligned} l_1(x, y) &= Lx + L^2y \\ l_2(x, y) &= -L^2x - y \\ l_3(x, y) &= x - Ly \end{aligned}$$

where L is a sufficiently large positive integer. The three lifted Newton polytopes are shown in figure 3.3 from a perspective view in 3-dimensional space. This figure also depicts the lower envelope $s(Q)$ of the Minkowski sum $\widehat{Q}$ of the three lifted polytopes. The data in figure 3.3 is produced by our convex hull program (see sect. 6.7), and is visualized by GEOMVIEW. The program automatically triangulates all

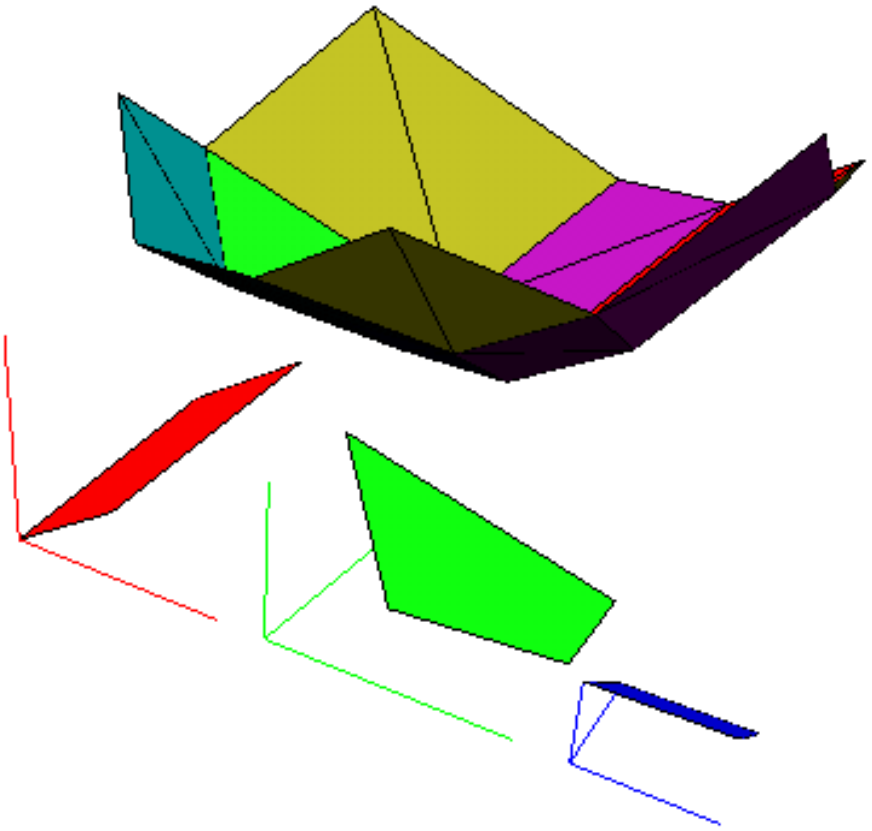

Figure 3.3: The lifted Newton polytopes and a triangulation of the lower envelope of their Minkowski sum.

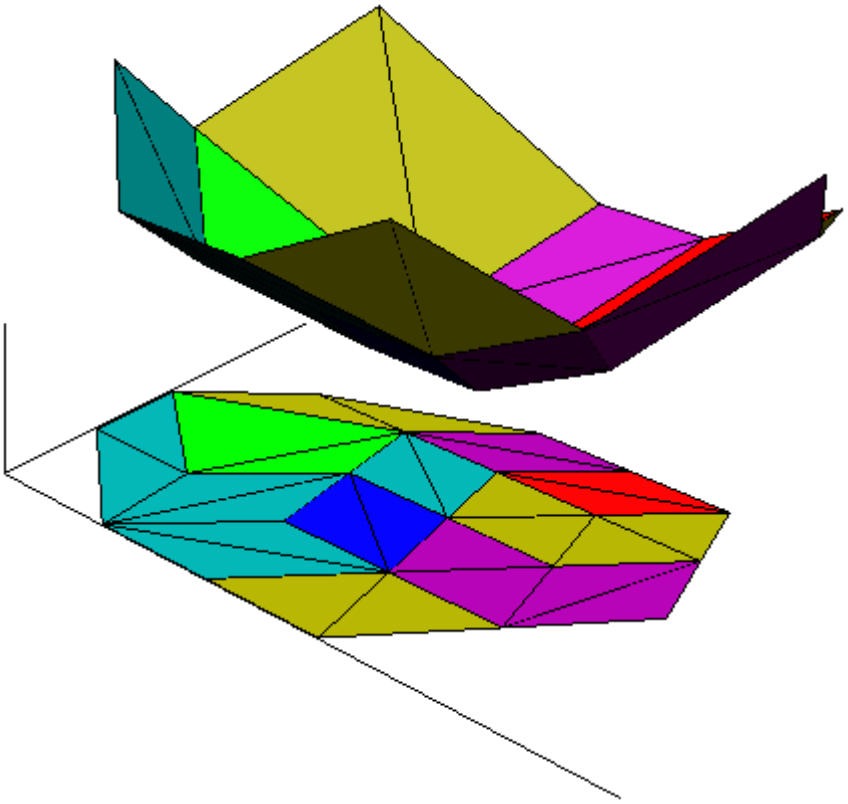

Figure 3.4: The lower envelope of the Minkowski sum of all lifted Newton polytopes and its planar projection; the latter equals the Minkowski sum of the original Newton polytopes.

output facets; this triangulation is immaterial and actually ignored in inducing a mixed subdivision of Q.

Figure 3.4 depicts the projection of the lower envelope, which lies in 3-dimensional space, to the 2-dimensional Minkowski sum of the original polytopes. The perspective view shows the bijective correspondence of facets on the envelope to maximal cells in the subdivision. Again, facets and cells are triangulated as a by-product of our convex hull code, though this is not relevant to our construction here. The mixed subdivision of $Q + \delta$ into 2-dimensional cells and the indices of the Newton polytope vertices in the optimal sums for each cell are shown in figure 3.5.

Matrix M_1 appears below with rows and columns indexed by exponent vectors from $\mathcal{E}$ and has dimension 15. M_1 contains the minimum number of f_1 rows, namely 4. Matrices corresponding to f_2 and f_3 are formed similarly.

The greedy algorithm produces a resultant matrix of size 14, whereas the total degree of the sparse resultant is the sum of the three mixed volumes $4 + 3 + 4 = 11$. The degree of the homogeneous resultant is 26.

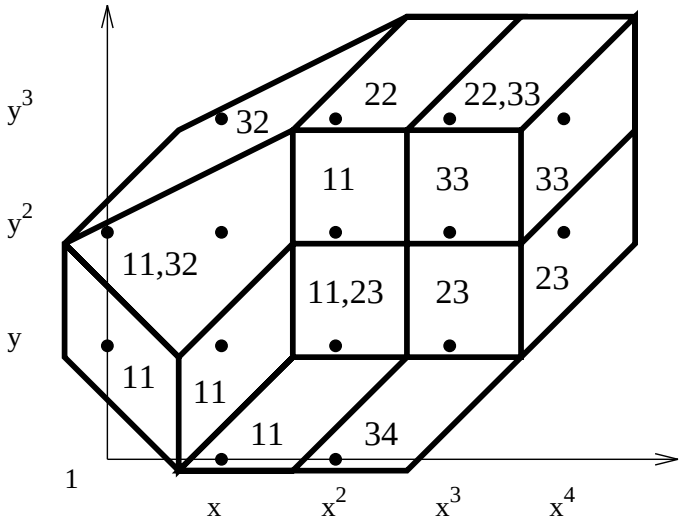

Figure 3.5: The induced subdivision Δ_δ of $Q + \delta$. Each cell is labeled with the indices of the Newton polytope vertices that contribute to the optimal sum expressing it: ij denotes vertex a_{ij}.

	$1,0$	$2,0$	$0,1$	$1,1$	$2,1$	$3,1$	$0,2$	$1,2$	$2,2$	$3,2$	$4,2$	$1,3$	$2,3$	$3,3$	$4,3$
$1,0$	c_{11}	c_{14}	0	0	c_{12}	c_{13}	0	0	0	0	0	0	0	0	0
$2,0$	c_{31}	c_{34}	0	c_{32}	c_{33}	0	0	0	0	0	0	0	0	0	0
$0,1$	0	0	c_{11}	c_{14}	0	0	0	c_{12}	c_{13}	0	0	0	0	0	0
$1,1$	0	0	0	c_{11}	c_{14}	0	0	0	c_{12}	c_{13}	0	0	0	0	0
$2,1$	c_{24}	0	c_{21}	0	c_{23}	0	0	0	c_{22}	0	0	0	0	0	0
$3,1$	0	c_{24}	0	c_{21}	0	c_{23}	0	0	0	c_{22}	0	0	0	0	0
$0,2$	0	0	c_{31}	c_{34}	0	0	c_{32}	c_{33}	0	0	0	0	0	0	0
$1,2$	0	0	0	c_{31}	c_{34}	0	0	c_{32}	c_{33}	0	0	0	0	0	0
$2,2$	0	0	0	0	0	0	0	0	c_{11}	c_{14}	0	0	0	c_{12}	c_{13}
$3,2$	0	0	0	0	c_{31}	c_{34}	0	0	c_{32}	c_{33}	0	0	0	0	0
$4,2$	0	0	0	0	0	c_{24}	0	0	c_{21}	0	c_{23}	0	0	0	c_{22}
$1,3$	0	0	0	0	0	0	0	c_{31}	c_{34}	0	0	c_{32}	c_{33}	0	0
$2,3$	0	0	0	c_{24}	0	0	c_{21}	0	c_{23}	0	0	0	c_{22}	0	0
$3,3$	0	0	0	0	0	0	0	0	c_{31}	c_{34}	0	0	c_{32}	c_{33}	0
$4,3$	0	0	0	0	0	0	0	0	0	c_{31}	c_{34}	0	0	c_{32}	c_{33}

3.1.8 Complexity

First we consider the randomization complexity in order for $\delta \in \mathbb{Q}^n$ to be a vector that perturbs every point in $\mathcal{E}$ to the interior of some maximal cell. Assume the entries of δ are rationals, where the numerators are distributed uniformly and independently in a set of S integer values and the denominators are all equal to some sufficiently large integer. The condition that a particular δ fails is equivalent to perturbing one point within some cell boundary.

Every cell boundary is a cell in Δ of dimension $n-1$. The random choice is not generic if δ lies in any of these boundaries. For each, the probability of failure is at most $1/S$ by Schwartz's

lemma [Sch80, lem. 1]. Now we have to bound the number of $(n-1)$-dimensional cells in Δ. We know that each maximal cell has volume at least $1/n!$. The number of integer lattice points in a polytope is asymptotically equal to its volume [Ehr67]. Therefore the total volume of Q is asymptotically $|\mathcal{E}|$. A rough bound on the number of maximal cells is then $|\mathcal{E}|\, n!$. Since every $(n-1)$-dimensional cell is cobounded by two maximal cells, the number of the former is at most $(|\mathcal{E}|\, n!)^2$.

Hence the total probability of failure for the random choice of δ is at most $(|\mathcal{E}|\, n!)^2/S$. This implies that the δ integers should be two-word integers for large-size problems, though in practice much fewer bits are required.

We now examine the complexity of constructing M. As mentioned, this is the crucial step in polynomial system solving (see the next chapter). Moreover, it is the first phase in computing the sparse resultant. In complexity bounds we sometimes ignore polylogarithmic factors in the involved quantities; this is denoted by $\mathcal{O}^*(\cdot)$.

The change of variables that may be required to ensure that $\sum_i \mathcal{A}_i$ generates $\mathbb{Z}^n$ involves the computation of a Smith normal form [HM91]. Typically, this step is not computationally expensive so we ignore it in the analysis.

Identifying the vertices of all Newton polytopes may be reduced to linear programming as in the first phase of algorithm 3.1.5. We can apply either Khachiyan's Ellipsoid or Karmarkar's algorithm. To bound the bit size of the input exponents a_{ij} we assume that the Newton polytopes have been translated to the first orthant so that they touch the positive half-axes, thus every exponent is bounded by $|\mathcal{E}|$.

In the case of Karmarkar's algorithm [Kar84] the bit complexity for a linear program of V variables, C constraints and B maximum bits per coefficient is

$$\mathcal{O}^*(C^2 V^{5.5} B^2).$$

To compute the vertices of each Q_i algorithm 3.1.5 applies linear programming with $\mathcal{O}(n)$ constraints and $\mathcal{O}(\mu_i)$ variables. If d is the highest degree of any input polynomial in a single variable, the maximum size of any coefficient is $\log d$. Then the bit complexity per Newton polytope is $\mu_i \mathcal{O}^*(n^2 \mu_i^{5.5} \log^2 d)$. Let the output be vertex set $\{a_{i1}, \ldots, a_{im_i}\}$ with $m_i \leq \mu_i$. Let $\mu = \max_i\{\mu_i\}$ be the maximum number of support points, then the total complexity for all Newton polytopes is $\mathcal{O}(n^3 \mu^{6.5} \log^2 d)$.

The most expensive step of the algorithm is to associate a unique optimal sum of points $p_i \in Q_i$ with every $p \in \mathcal{E}$. This is accomplished by the second phase of algorithm 3.1.5. The bit size of $l_i(a_{ij})$ is constant, once the desired probability of success is fixed. Let $m = \max_i\{m_i\}$ be the maximum vertex cardinality, then there are $\mathcal{O}(n)$ constraints and $\mathcal{O}(nm)$ variables λ_{ij}. The coefficients in these linear programming problems are coordinates of points in Q, hence have magnitude nd. Hence the bit complexity per point is $\mathcal{O}^*(n^{7.5} m^{5.5} \log^2(nd))$. Therefore the total bit complexity to compute all optimal sums is $\mathcal{O}^*(|\mathcal{E}| n^{7.5} m^{5.5} (\log^2 n + \log^2 d))$.

Finding all optimal sums offers a deterministic test for the genericity of δ. First, it lets us compute the optimal sum for each maximal cell. For points in the same cell σ, the summand face F_i is defined by all distinct vertices of Q_i contributing to optimal sums in σ. Assuming the lifting is generic, the dimension of F_i exceeds the number of points defining it by 1; this property always holds for simplicial Q_i. If $\sum_i \dim F_i < n$, then $p - \delta$ does not lie in the interior of any maximal cell, therefore the chosen δ is not sufficiently generic and a new one must be chosen.

Let e denote the exponential base and s the scaling factor of the system. The latter was formally defined as the minimum real such that $\lambda_i + Q_i \subset sQ_{\min}$ for some $\lambda_i \in \mathbb{R}$, where $Q_{\min}$ minimizes volume among the Newton polytopes.

Theorem 3.1.27 *Given polynomials $f_1, \ldots, f_{n+1}$, the algorithm computes sparse resultant matrix M*

with worst-case time complexity $\mathcal{O}^*(n^3\mu^{6.5}\log^2 d + |\mathcal{E}|n^{7.5}m^{5.5})$. *Let the system's scaling factor be* s *and the total degree of the sparse resultant be* D. *If all Newton polytopes have positive volume then the bit complexity is*

$$\mathcal{O}^*\left(n^3\mu^{6.5}\log^2 d + s^n e^n D n^6 m^{5.5}\right).$$

Proof The first bound follows from the previous discussion. It is simplified by applying $|\mathcal{E}| = \mathcal{O}(s^n e^{n+1} D/n^{3/2})$ from theorem 2.2.5. □

Typically the second term dominates, as seen in the next bound. By strengthening the hypotheses we provide bounds closer to the average-case performance of the algorithm.

Corollary 3.1.28 *Assume that the conditions set by the previous theorem hold,* s *is constant and* $\mu = \mathcal{O}(\sqrt{n}\sqrt{m}|\mathcal{E}|^{1/7})$. *Then the complexity for constructing* M *is* $e^{\mathcal{O}(n)}\mathcal{O}^*(Dn^6m^{5.5})$.

3.1.9 Computing the Resultant

This section discusses efficient ways for computing the sparse resultant from a set of matrices constructed by the algorithm. We could construct $n+1$ matrices $M_1, \ldots, M_{n+1}$, where each M_i has the minimum number of rows containing coefficients of f_i. These are obtained by modifying the row content function so that it never returns i when there is another choice. Let $D_1, \ldots, D_{n+1}$ be the determinants formed in this way. The GCD of $D_1, \ldots, D_{n+1}$ has the correct degree in all f_i's and, since the GCD is divisible by R, it equals R. Unfortunately, this method does not always work when the coefficients of the f_i are specialized. It can be used after a suitable perturbation of the specialized system, but there is a more economical method, adapted from [Can88b], of which two variants are sketched below.

Division Method

Let $g_1, \ldots, g_{n+1}$ be the given specialized polynomials. First we choose polynomials $h_1, \ldots, h_{n+1}$ with *random* integer coefficients, such that h_i has support $\mathcal{A}_i$. Then the input to the algorithm is specialized system

$$(f_1, \ldots, f_{n+1}) \mapsto (g_1 + u_1 h_1, \ldots, g_{n+1} + u_{n+1} h_{n+1})$$

where each u_i is a new indeterminate that belongs to the coefficient field of the new system. Define the extraneous factor b_i of each D_i via

$$D_i = b_i R$$

and notice that b_i will be independent of u_i since D_i and R have the same degree in the coefficients of $g_i + u_i h_i$.

Definition 3.1.29 *Suppose a polynomial* $p(u_1, \ldots, u_{n+1})$ *has maximum degree* d_i *in* u_i. *Then* p *is* rectangular *if it contains a monomial of the form* $u_1^{d_1}u_2^{d_2}\cdots u_{n+1}^{d_{n+1}}$.

Under the specialization above, note that R as well as $D_1, \ldots, D_{n+1}$ will be rectangular, because the coefficient of $u_1^{d_1}u_2^{d_2}\cdots u_{n+1}^{d_{n+1}}$ will be the resultant or the respective determinant, obtained from system $h_1, \ldots, h_{n+1}$.

Let $R^{(j)}(u_1, \ldots, u_j)$ be the leading coefficient, with respect to total degree, of R considered as a polynomial in $u_{j+1}, \ldots, u_{n+1}$. Define $D_i^{(j)}(u_1, \ldots, u_j)$ and $b_i^{(j)}(u_1, \ldots, u_j)$ analogously and notice that

all these polynomials are rectangular. Then $D_i^{(j)} = b_i^{(j)} R^{(j)}$ for all i and j. But since b_i is independent of u_i, $b_i^{(i)} = b_i^{(i-1)}$, which we can use to eliminate b_i:

$$R^{(i)} = \frac{D_i^{(i)}}{D_i^{(i-1)}} R^{(i-1)}, \qquad i = 1, \ldots, n+1. \tag{3.9}$$

$R^{(n+1)}$ is exactly the resultant of the f_i, so setting $u_1 = \cdots = u_{n+1} = 0$ in $R^{(n+1)}$ will give the resultant of the g_i. Recurrence (3.9) has initial term $R^{(0)}$ which is some integer that we may set to 1, thus obtaining $R^{(n+1)}$ equal to a scalar multiple of the resultant.

Identity (3.9) is valid for specializations of u_i's as long as no denominator vanishes. So we take $u_1 = u_2 = \cdots = u_{n+1} = u$, so that all $D_i^{(j)}$ become univariate polynomials in u. Since they are all rectangular, they have a unique term of highest total degree in the u_i's which cannot cancel, so none of them will vanish under this specialization. Each $D_i^{(j)}(u)$ is easily seen to be the determinant of M under the specialization

$$(f_1, \ldots, f_{n+1}) \mapsto (g_1 + uh_1, \ldots, g_j + uh_j, h_{j+1}, \ldots, h_{n+1})$$

and the leading coefficient of $D_i^{(j)}$ is once again nonzero for almost all choices of h_i. Thus we have an almost guaranteed method of constructing the resultant at the cost of adding the single variable u. More precisely, to bound the probability of failure by $1/c$ for some arbitrary $c \gg 1$ it suffices [Sch80, lem. 1], to pick the coefficients of each h_i uniformly distributed and independently, each with $\log(c|\mathcal{E}|)$ bits. It is possible to detect failure deterministically, in which case new randomized variables must be chosen.

If the g_i's are sufficiently generic, which means that no $D_i^{(j)}$ vanishes, we may compute $D_i^{(j)}$ as the determinant of M_i under the specialization

$$(f_1, \ldots, f_{n+1}) \mapsto (g_1, \ldots, g_j, h_{j+1}, \ldots, h_{n+1}).$$

GCD Method

This method requires that the coefficients of the given specialized system $g_1, \ldots, g_{n+1}$ be nonzero and chosen from some polynomial ring over $\mathbb{Q}$. Again we choose polynomials h_i with random coefficients, whose size is given in [Sch80, lem. 1], and specialize

$$(f_1, \ldots, f_{n+1}) \mapsto (g_1 + uh_1, \ldots, g_{n+1} + uh_{n+1}).$$

By Hilbert's irreducibility theorem, R will remain irreducible over $\mathbb{Q}[u]$ after almost all such specializations. Let $D_1(u)$ be the determinant of M_1 with this specialization and let $b(u)$ be the extraneous factor, $D_1(u) = b(u)R(u)$.

By lemma 2.1.6 we may suppose, without loss of generality, that M_1 is defined using l_1 whose values $l_1(a_{1j})$ are so much larger than values $l_i(a_{ij})$, for $i > 1$, that whenever a vertex a_{1j} of Q_1 appears in the optimal Minkowski sum of a point in $\mathcal{E}$, a_{1j} is the one minimizing l_1. Thus in every row containing coefficients of f_1, the leading diagonal element will be c_{1j}. Now let $D_2(u)$ be the determinant of M_1 under the specialization

$$(f_1, f_2, \ldots, f_{n+1}) \mapsto (x^{a_{1j}}, g_2 + uh_2, \ldots, g_{n+1} + uh_{n+1}),$$

with $D_2(u) = b(u)R'(u)$, where $R'(u)$ is the resultant under this new specialization. Therefore

$$R(u) = \frac{D_1(u)}{\gcd(D_1(u), D_2(u))}$$

and specializing $u = 0$ gives the resultant of $g_1, \ldots, g_{n+1}$. It is worth remarking that the degree of $b(u)$ is known in advance, namely it is the number of elements of $\mathcal{E}$ that do not lie in mixed cells. Thus the GCD computation is branch-free and reduces to calculation of the appropriate minors of the Sylvester matrix of D_1 and D_2 [Loo82].

Once again if the given g_i are generic enough then R can be computed simply as $D_1(0)/\gcd(D_1(0), D_2(0))$. The explicit condition here is that the specialized resultant under $f_i \mapsto g_i$ is irreducible and the determinants $D_1(0)$ and $D_2(0)$ do not vanish.

Complexity

The analysis of computing the sparse resultant R from resultant matrices reduces to analyzing certain standard polynomial algorithms [Loo82, DST88, Mis93]. For example, using the second approach reduces to computing the determinants of two resultant matrices by interpolation, finding their GCD by a subresultant polynomial sequence and lastly computing a polynomial quotient. The arithmetic complexity is polynomial in $|\mathcal{E}|$. Since both the matrix order and the bit size of the input exponents are bounded by $|\mathcal{E}|$, the overall complexity is polynomial in $|\mathcal{E}|$.

3.2 Incremental Algorithm

In this section we describe how to obtain a matrix such that some maximal minor is a nontrivial multiple of the sparse resultant. The entries of this resultant matrix are chosen among the indeterminate coefficients of the original polynomials. The present approach is direct in building the monomial sets indexing the matrix without computing a mixed subdivision, thus aiming at improving the complexity of the construction by making it depend on the size of the actual matrix rather than the size of the Minkowski sum. On the other hand, the construction is incremental in the sense that there is no a priori knowledge of the size of the matrix produced.

Let $\mathcal{P}(\mathcal{A}) \subset \mathbb{C}[x, x^{-1}]$ be the set of all Laurent polynomials in n variables with support $\mathcal{A} \subset \mathbb{Z}^n$. Clearly, $f_i \in \mathcal{P}(\mathcal{A}_i)$. Now fix supports $\mathcal{B}_1, \ldots, \mathcal{B}_{n+1} \subset \mathbb{Z}^n$ and consider the following linear transformation:

$$M : \mathcal{P}(\mathcal{B}_1) \times \cdots \times \mathcal{P}(\mathcal{B}_{n+1}) \to \mathcal{P}\left(\bigcup_{i=1}^{n+1} \mathcal{B}_i + \mathcal{A}_i\right) : (g_1, \ldots, g_{n+1}) \mapsto g = \sum_{i=1}^{n+1} g_i f_i, \tag{3.10}$$

where addition between supports stands for Minkowski Addition. The matrix we are constructing is precisely the matrix of this transformation and to define it fully we specify supports $\mathcal{B}_i$ at the end of this section. Every row of M is indexed by an element of some $\mathcal{B}_i$ and every column by an element of $\mathcal{B}_i + \mathcal{A}_i$ for some i; equivalently, the rows and columns are indexed respectively by the monomials of g_i and the monomials of g.

We fill in the matrix entries *à la* Macaulay as before: the row corresponding to monomial x^b of g_i contains the coefficients of polynomial $x^b f_i$ so that the coefficient of monomial x^q appears in the column indexed by x^q, where $b \in \mathcal{B}_i$, $q \in \mathrm{supp}(g)$. Columns indexed by monomials which do not explicitly appear in $x^b f_i$ have a zero entry.

Lemma 3.2.1 *If $f_1, f_2, \ldots, f_{n+1}$ have a common solution $\xi \in (\mathbb{C}^*)^n$ then M is singular.*

Proof If a common solution ξ exists, then it is a solution for all g in the image of linear transformation M. This implies that the image of M cannot contain any monomials and is, therefore, a proper subset

of the range. Since M is not surjective it is singular. □

The number of rows equals the sum of the cardinalities of supports $\mathcal{B}_i$ while the number of columns equals the cardinality of supp(g). Throughout this section we restrict ourselves to matrices M with *at least as many rows as columns.*

Theorem 3.2.2 *Every maximal minor D of M is a multiple of the sparse resultant $R(\mathcal{A}_1, \ldots, \mathcal{A}_{n+1})$.*

Proof By the lemma, M drops column rank on the set Z_0 of coefficient specializations such that $f_1, \ldots, f_{n+1}$ have a common solution. Any maximal minor D is zero on Z_0, thus it is zero on its (Zariski) closure Z which is the zero set of $R(\mathcal{A}_i, \ldots, \mathcal{A}_{n+1})$. Since the latter is irreducible or 1, it divides D in $\mathbb{Z}[c_1, \ldots, c_{n+1}]$ where c_i are the coefficients of f_i. □

Let $\deg_{f_i} D$ denote the degree in the coefficients of polynomial f_i of a maximal minor D of M. It is clear from theorem 1.3.13 that if R divides D and $D \neq 0$, then $\deg_{f_i} D \geq MV_{-i}$ for all i.

3.2.1 Matrix Construction

We now specify the construction of supports $\mathcal{B}_i$. Let $Q = Q_1 + \cdots + Q_{n+1} \subset \mathbb{R}^n$ be the Minkowski sum of the input Newton polytopes. Consider all n-fold partial Minkowski sums

$$Q_{-i} = Q - Q_i = \sum_{j \neq i} Q_j \subset \mathbb{R}^n \qquad \text{and let} \qquad T_i = Q_{-i} \cap \mathbb{Z}^n = (Q - Q_i) \cap \mathbb{Z}^n.$$

We shall restrict our choice of $\mathcal{B}_i$ by requiring that it be a subset of T_i; notice that this is the case in the subdivision algorithm above. One consequence is that the supports of all products $g_i f_i$ lie within the Minkowski sum Q, therefore supp(g) $\subset Q$.

Given a vector $v \in \mathbb{Q}^n$ and a nonzero real number β we define a one-dimensional polytope $V \subset \mathbb{R}^n$ as the convex hull of the origin and of point $\beta v \in \mathbb{R}^n$. The sign of β determines the direction in which V lies and its magnitude determines the length of V. For a fixed V we define

$$\mathcal{B}_i \;=\; (Q_{-i} - V) \cap \mathbb{Z}^n \;\subset T_i, \tag{3.11}$$

by using the Minkowski difference of two polytopes (def. 1.3.4). As the length of V decreases the cardinality of $\mathcal{B}_i$ tends to that of T_i. So for fixed v and β or, simply, for fixed V, matrix M is well-defined. The algorithm for constructing sets $\mathcal{B}_i$ and the resultant matrix can now be specified.

Definition 3.2.3 *Given convex polytope $P \subset \mathbb{R}^n$ and a vector $v \in \mathbb{Q}^n$, we define the* v-distance *of every point $p \in P \cap \mathbb{Z}^n$ to be the distance of p from the boundary of P on direction v, formally*

$$v\text{-}distance(p) = s \Leftrightarrow p + sv \in P \text{ and } s \in \mathbb{R}_{\geq 0} \text{ is maximized.}$$

Integer points on the boundary of polytope P which are extremal with respect to vector v have zero v-distance. Figure 3.6 shows different subsets of T_2 for system (1.3.10) with respect to v-distance, for $v = (11, 1)$. An equivalent definition of supports $\mathcal{B}_i$ is by ordering $Q_{-i} \cap \mathbb{Z}^n$ by v-distance, then selecting the points whose v-distance exceeds some bound. Vector v here and in (3.11) is the same and β can be used as the bound on v-distance. This is formalized in

Lemma 3.2.4 *For convex polytope P and one-dimensional polytope $V \in \mathbb{R}^n$,*

$$(P - V) \cap \mathbb{Z}^n \;=\; \{a \;\in\; P \cap \mathbb{Z}^n \mid v\text{-}distance(a) > \beta = \;\; length(V)\}.$$

We now turn to the question of enumerating all integer lattice points T_i in n-fold Minkowski sum Q_{-i}, for $1 \leq i \leq n+1$, together with their v-distances for some $v \in \mathbb{Q}^n$. Our Mayan Pyramid algorithm is recursive and computes, at every stage, the range of values for the k-th coordinate in T_i when the first $k-1$ coordinates are fixed.

When the algorithm is computing coordinate x_k for $1 < k \leq n$, all previous coordinates are set to $p_1, \ldots, p_{k-1}$.

Algorithm 3.2.5 (Mayan Pyramid)
Input: Integers i, k in $\{1, n\}$ and $v \in \mathbb{Q}^n$. If $k > 1$ a set of integer coordinates $p_1, \ldots, p_{k-1}$ is also given.
Output: $T_i \subset \mathbb{Z}^n$ together with the v-distance of each point in T_i.

Description The main loop of the point enumeration algorithm is the following:

1. Compute $mn, mx \in \mathbb{Z}$ which are, respectively, the minimum and maximum x_k-coordinates in Q_{-i} when the first $k-1$ coordinates are fixed to $p_1, \ldots, p_{k-1}$.
2. If $k < n$, for each $p_k \in [mn, mx]$ set $x_k = p_k$ and recursively call the algorithm with input $i, k+1$ and coordinates $p_1, \ldots, p_k$.
3. If $k = n$, for each $p_k \in [mn, mx]$ set $x_k = p_k$, compute the v-distance of point $(p_1, \ldots, p_n)$ and add it to T_i.

If $[mn, mx]$ is empty for any k the particular recursion terminates. Linear programming is used to compute mn, mx; here is how we find mn, for some $k > 1$. Note that the same setup with s maximized gives mx as the floor of the optimum.

$$\begin{aligned} \text{minimize} \quad s \in \mathbb{R}: \qquad & (p_1, \ldots, p_{k-1}, s) = \sum_{l=1, l\neq i}^{n+1} \sum_{j=1}^{m_i} \lambda_{lj} v_{lj}^k; \\ & \sum_{j=1}^{m_l} \lambda_{lj} = 1, \;\; \lambda_{lj} \geq 0, \;\; \forall l \neq i, j = 1 \ldots m_l; \end{aligned}$$

where v_{lj} are the vertices of Q_l, v_{lj}^k is the k-vector consisting of the first k coordinates of v_{lj} and m_l is the respective cardinality. Then mn is the ceiling of the optimal value of s.

Computing v-distances is accomplished by linear programming as well:

$$\begin{aligned} \text{minimize} \quad s \in \mathbb{R}: \qquad & (p_1, \ldots, p_n) + sv = \sum_{l=1, l\neq i}^{n+1} \sum_{j=1}^{m_i} \lambda_{lj} v_{lj}; \\ & \sum_{j=1}^{m_l} \lambda_{lj} = 1, \;\; \lambda_{lj} \geq 0, \;\; \forall l \neq i, j = 1 \ldots m_l. \end{aligned}$$

This procedure is used with $k < n$ coordinates for pruning the set of integer points T_i, since in practice it suffices to consider only points with a positive v-distance. Let v^k-distance be the analogue of v-distance for the projection of Q_{-i} and of v on subspace $x_{k+1} = \cdots = x_n = 0$. We can then test the point projections as they are constructed and eliminate all points $(p_1, \ldots, p_k)$ whose v^k-distance is zero. Further tests of this kind can achieve a more extensive pruning that decreases the empirical complexity. □

Incrementing the supports $\mathcal{B}_i$ is done either by decreasing the length of V or, equivalently, by lowering bound β on the v-distance. This is implemented by sorting every T_i on v-distance and choosing the subset with largest distances. The degree of maximal minor D in M must be at least $MV_{-i} = MV(Q_1, \ldots, Q_{i-1}, Q_{i+1}, \ldots, Q_{n+1})$. Hence we pick the initial sets $\mathcal{B}_i$ to be of that cardinality by choosing the MV_{-i} points of largest v-distance.

If D is generically nonzero then it equals the sparse resultant and we have obtained a Sylvester-type formula. Otherwise, points from T_i are added to $\mathcal{B}_i$ and they correspond to additional rows which are appended to the existing matrix; in general, more columns will have to be added as well. We summarize the matrix construction algorithm, under the assumption that a direction v has been chosen. It is an exercise to show that for $v = -\delta$, where δ is the perturbation vector in the subdivision algorithm, this algorithm constructs a matrix at most as large as the subdivision algorithm.

Algorithm 3.2.6 (Incremental Matrix Construction)
Input: $\mathcal{A}_i$, MV_{-i}, T_i with the v-distance of every point, for $i = 1, \ldots, n+1$.
Output: Maximal minor D of matrix M, such that D is a nontrivial multiple of the sparse resultant, or an indication that such a minor cannot be found.

Description The main loop is as follows:

1. Initialize supports $\mathcal{B}_i$ to include MV_i points from T_i with largest possible v-distance.
2. Construct matrix M with random coefficients.
3. If M has at least as many rows as columns and it is nonsingular then return any nonsingular maximal submatrix in it.
4. Otherwise, if $\mathcal{B}_i = T_i$ for $i = 1, \ldots, n+1$ i.e. the supports cannot be incremented, return with a message that the resultant matrix for this v is larger than that for $v = -\delta$.
5. Otherwise, let $\mathcal{B}_i = \{p \in T_i \,|\, v\text{-distance}(p) \geq \beta\}$ where $\beta \in \mathbb{R}$ is chosen so that the minimum number of new points are added to supports $\mathcal{B}_i$ and at least one $\mathcal{B}_i$ is incremented; go to step 2.

The nonsingularity test considers a generic matrix M whose nonzero entries are, in principle, symbolic coefficients. Genericity is simulated by picking uniformly distributed random coefficients from an interval of integers. The probability that this scheme fails is bounded in section 3.2.2; even then correctness is not affected though a larger matrix is obtained. □

There remains the question of determining v. In many situations, a deterministic v which guarantees the construction of a compact matrix formula can be found; such cases include systems whose structure is or resembles the multigraded structure, as demonstrated in section 3.3. For arbitrary systems, a random vector v is chosen. A last resort is to use $v = -\delta$ in order to obtain M at most as large as that by the subdivision algorithm.

In several applications, including polynomial system solving, an exact matrix formula for the resultant is not required. If the resultant must be found and $D \neq R$ there are two alternative ways to proceed in order to obtain the resultant under a specific specialization of the coefficients discussed in section 3.1.9. For both we fix the cardinality of $\mathcal{B}_1$ to MV_{-1} so that $\deg_{f_1} D = \deg_{f_1} R$. This will enable us to define R as the GCD of at most $n+1$ such minors.

Example 1.3.10 (continued from sect. 3.1.7) Figure 3.6 shows Q_{-2} in bold for system (3.8) and polytope V between the origin and $(2, 2/11)$ such that $Q_{-2} - V$ is the thin-line triangle with vertex set $\{(0,1), (1,1), (0,0)\}$; this is $\mathcal{B}_2$ for the final matrix M. Equivalently, this is the set of integer points in

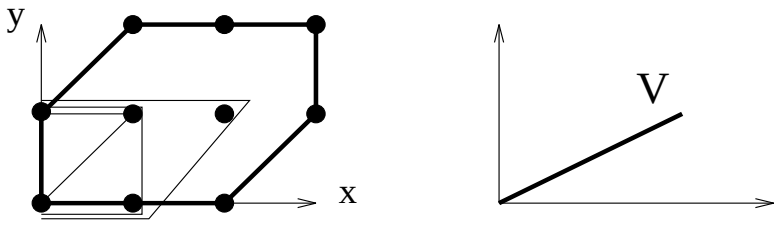

Figure 3.6: T_2 subsets with different v-distance bounds and segment V

Q_{-2} whose v-distance is larger than or equal to $5\sqrt{5}/11$, where $v = (2, 2/11)$. The thin polygonal lines in figure 3.6 define some subsets of T_2 for different cutoff values on the v-distance.

This v leads to a 13×12 nonsingular matrix M shown below with $\mathcal{B}_i$ cardinalities $5, 3, 5$, from which any 12×12 minor serves as D. The first line below displays the integer points indexing the columns. The subdivision algorithm gives a 15×15 matrix and the greedy one a 14×14 matrix, whereas the sparse resultant's total degree is the sum of mixed volumes $4 + 3 + 4 = 11$. The homogeneous resultant has total degree 26.

$$
\begin{array}{c}
\begin{array}{cccccccccccc}
3,2 & 4,3 & 5,3 & 4,2 & 3,1 & 5,2 & 4,1 & 2,2 & 3,3 & 2,1 & 5,4 & 4,4
\end{array} \\
\left[\begin{array}{cccccccccccc}
c_{11} & c_{12} & c_{13} & c_{14} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
c_{12} & 0 & 0 & c_{13} & c_{14} & 0 & 0 & 0 & 0 & c_{11} & 0 & 0 \\
c_{14} & c_{13} & 0 & 0 & 0 & 0 & 0 & c_{11} & c_{12} & 0 & 0 & 0 \\
0 & c_{14} & 0 & 0 & 0 & 0 & 0 & 0 & c_{11} & 0 & c_{13} & c_{12} \\
0 & 0 & 0 & c_{12} & c_{11} & c_{13} & c_{14} & 0 & 0 & 0 & 0 & 0 \\
c_{21} & 0 & c_{22} & 0 & 0 & c_{23} & c_{24} & 0 & 0 & 0 & 0 & 0 \\
0 & c_{22} & 0 & c_{23} & c_{24} & 0 & 0 & c_{2}1 & 0 & 0 & 0 & 0 \\
0 & 0 & c_{23} & c_{24} & 0 & 0 & 0 & 0 & c_{21} & 0 & c_{22} & 0 \\
c_{31} & c_{33} & 0 & c_{34} & 0 & 0 & 0 & 0 & c_{32} & 0 & 0 & 0 \\
c_{32} & 0 & 0 & c_{33} & c_{31} & 0 & c_{34} & 0 & 0 & 0 & 0 & 0 \\
c_{33} & 0 & 0 & 0 & c_{34} & 0 & 0 & c_{32} & 0 & c_{31} & 0 & 0 \\
0 & c_{31} & c_{34} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & c_{33} & c_{32} \\
0 & c_{32} & c_{33} & c_{31} & 0 & c_{34} & 0 & 0 & 0 & 0 & 0 & 0
\end{array}\right]
\end{array}
$$

We can now describe the full incremental algorithm. Speed and performance issues related to the size of the matrices are examined in the sections that follow.

Algorithm 3.2.7
Input: Supports $\mathcal{A}_1, \ldots, \mathcal{A}_{n+1}$ and direction vector $v \in \mathbb{Q}^n$.
Output: Maximal minor D of matrix M, such that D is a nontrivial multiple of the sparse resultant, or an indication that such a minor cannot be found.

Description Here is the main procedure.

1. Compute the vertex sets of Newton polytopes $Q_1, \ldots, Q_{n+1}$.
2. Use the Mayan Pyramid algorithm to compute sets $T_1, \ldots, T_{n+1} \in \mathbb{Z}^n$ and all v-distances.
3. Use the Lift-Prune algorithm to compute mixed volumes $MV_{-1}, \ldots, MV_{-(n+1)}$.
4. Use the Incremental Matrix Construction algorithm to construct matrix M whose maximal minor D is a nontrivial multiple of R and return D if found. Otherwise, return with an indication that minor D cannot be found.

Step 1 applies linear programming as explained in algorithm 3.1.5.

A useful feature here is that, as the matrix construction is incremental, the nonsingularity test is also incremental. We have implemented an incremental algorithm for LU decomposition of rectangular matrices which, given a partially decomposed matrix, will attempt to continue and complete the decomposition. It uses partial pivoting and stops when a pivot and the subcolumn below it are all zero, thus calling for a larger matrix M. Arithmetic is carried out over a large finite field, which allows for efficient and exact arithmetic. This has the disadvantage that the constructed matrix M may be larger than what would be possible over the integers, if the prime is unlucky. □

An available option is to limit the size of T_i or, alternatively, for the user to provide a cutoff v-distance. Points with smaller distances are not enumerated, and this speeds up the recursive Mayan Pyramid algorithm besides decreasing the space requirements. This is particularly useful with systems with multihomogeneous structure, discussed in section 3.3.

For linear programming, we use a publicly available implementation of the Simplex algorithm, since our goal is to release our software for distribution. It is evident that more efficient implementations of linear programming would significantly speed up our algorithm.

3.2.2 Complexity

Let μ be the maximum number of points per support $\mathcal{A}_i$, m and g be the maximum number of Newton polytope vertices and edges respectively, L_l be the maximum bit-size of a coordinate in any lifting vector l_i and L_d be the maximum coordinate size of any Newton polytope vertex. The latter is bounded by $\log d$, where d is the maximum polynomial degree in a single variable.

First a bound is derived on the probability that some intermediate nonsingular matrix M is singular due to the specialization of the coefficients. Assume they are uniformly and independently distributed in an integer set of cardinality S. If M is of dimension r, then the probability is bounded by r/S [Sch80, lem. 1].

In the main algorithm, computing one convex hull vertex set requires at most μ linear programming tests implying that the bit complexity is $(\mu n)^{\mathcal{O}(1)} L_d$. The cardinality of an integer point set is asymptotically bounded by the volume of their convex hull [Ehr67]. Hence μ is bounded by the maximum $\mathrm{Vol}(Q_i)$ which is bounded by $V = \max_i\{\mathrm{Vol}(Q_{-i})\}$.

The second step is to enumerate integer point sets T_i. Each linear programming problem in computing T_i has bit complexity $\mathcal{O}(n^6 m^5 L_d)$. An upper bound on the number of such problems is $|T_i|$ or, by the relation of point cardinality to volume, $\mathrm{Vol}(Q_{-i})$.

The third step is to compute all n-fold mixed volumes. It costs $m^{\mathcal{O}(n)}(L_d + L_l)$ by theorem 2.4.3 and its proof. Lastly, the complexity of the incremental matrix construction algorithm is dominated by the LU decomposition. This is carried out over a finite field defined by a preselected prime of larger yet constant size. Hence, the bit complexity is $\mathcal{O}(r^3)$, where r is the total number of rows in matrix M. Clearly r is at most equal to the total number of points in all sets T_i.

Theorem 3.2.8 *Given vector v, the worst-case bit complexity of the incremental algorithm equals $(nV)^{\mathcal{O}(1)}L_d + m^{\mathcal{O}(n)}(L_d + L_l)$, where V is the maximum of $\mathrm{Vol}(Q_{-i})$ over $i = 1, \ldots, n+1$. For systems with every $\mathrm{Vol}(Q_i) > 0$, scaling factor s and maximum n-fold mixed volume MV the bound becomes*

$$\mathcal{O}\left((es)^{\mathcal{O}(n)}MV^{\mathcal{O}(1)} + m^{\mathcal{O}(n)}(L_d + L_l)\right).$$

Proof By the preceding discussion, computing Newton polytope vertices is included in the first complexity term. The vertex cardinality of each polytope is bounded by its volume which is in turn bounded by V. Therefore, enumerating T_i and incrementing M, typically the most expensive steps, are captured by the first term as well. The second complexity term represents solely the cost of calculating all n-fold mixed volumes.

For the second expression we apply theorem 2.2.3. L_d is bounded by the maximum of n and V. Otherwise, the input system has a very special structure and the sparse resultant is trivially 1. □

This is a reasonable algorithm, taken into account the lower bounds available for the various subproblems solved. Namely, computing the integer lattice points is at least as hard as counting them which, by Ehrart's result, is at least as hard as computing a polytope volume. The latter problem is #P-complete. The same lower bound applies to mixed volume computation, as discussed earlier.

We now model average-case behavior.

Corollary 3.2.9 *Assume the hypotheses of the theorem hold and, additionally, s is constant, $d \leq 2^{m^{\mathcal{O}(n)}}$ and the probability of failure for the lifting is constant. Then the worst-case bit complexity is*

$$\mathcal{O}\left(e^{\mathcal{O}(n)}MV^{\mathcal{O}(1)} + m^{\mathcal{O}(n)}\right).$$

3.3 Multihomogeneous Systems

This section focuses on a special class of polynomial systems for which Sylvester-type formulae exist for the sparse resultant. It turns out that for the incremental algorithm to produce optimal resultant matrices, the input system does not need to be exactly multigraded (the concept is formalized below). So sparse systems approximating this structure usually lead to Sylvester-type formulae, as illustrated by concrete examples at the end of the section and in the next chapter.

A homogeneous polynomial is *multihomogeneous* if the set of variables can be partitioned into r subsets $x_1, \ldots, x_r$, so that the polynomial is homogeneous when considered as a polynomial in each subset. This is sometimes called an m-homogeneous polynomial, where m traditionally denotes the number of variable subsets. Suppose that the number of individual variables in x_k is $l_k + 1$, where one of them is the homogenizing variable, then the number of affine variables $n = l_1 + \cdots + l_r$. There is no relation between these l_i and the lifting functions of previous sections. If the total degree in variables x_k is d_k, then the polynomial is of

$$\textit{type } (l_1, \ldots, l_r; d_1, \ldots, d_r).$$

Here is a system with variable subsets x_{11}, x_{12} and x_{21}, x_{22}, x_{23} of type $(1, 2; 2, 1)$; the c_i are constant coefficients:

$$c_1x_{11}^2x_{21} + c_2x_{11}^2x_{22} + c_3x_{11}^2x_{23} + c_4x_{11}x_{12}x_{21}$$
$$+c_5x_{11}x_{12}x_{22} + c_6x_{11}x_{12}x_{23} + c_7x_{12}^2x_{21} + c_8x_{12}^2x_{22} + c_9x_{12}^2x_{23}.$$

Unmixed systems where each polynomial is multihomogeneous are called *multihomogeneous*. If the degree of polynomial i is d_{ij} in the j-th variable subset, then the number of common isolated solutions for the system of n polynomials is bounded by

$$\text{the coefficient of } \prod_{j=1}^{r} x_j^{l_j} \text{ in polynomial } \prod_{i=1}^{n} \left(\sum_{j=1}^{r} d_{ij} x_j \right).$$

On this topic consult [MS87] and for a generalization [MSW].

Unmixed systems where all polynomials have the same type are said to be of this type. So far we have considered dense systems, though for systems where some coefficients vanish the same approach may be used, as seen at the end of the section. In the context of resultants we are interested in overconstrained systems with $n+1$ equations. There should be no confusion from the fact that the polynomials given are multihomogeneous; to apply sparse elimination algorithms we dehomogenize each subset of variables by setting the (l_k+1)-st variable to one.

We may label the coordinates axes to correspond to the sequence $x_1, \ldots, x_r$ and consider each x_k as a subsequence of variables associated to an orthogonal subspace. The Newton polytope for every polynomial is then the Minkowski sum of r l_k-dimensional simplices, each defined on a disjoint set of coordinate axes and thus in orthogonal subspaces. Every simplex is denoted by $d_k S_{l_k}$ and is the convex hull of l_k segments of length d_k rooted at the origin and extending along each of the l_k axes corresponding to the variables in this group. Since we are in the unmixed case the n-fold Minkowski sum Q_{-i} is the same for any $i \in \{1, \ldots, n+1\}$. It is equal to integer polytope $P \subset \mathbb{R}^n$ which is simply a copy of the unique input Newton polytope scaled by n:

$$Q_1 = \cdots = Q_{n+1} = \sum_{k=1}^{r} d_k S_{l_k}, \qquad P = \sum_{k=1}^{r} n d_k S_{l_k} = n \sum_{k=1}^{r} d_k S_{l_k} \quad \subset \mathbb{R}^n.$$

Both summations express Minkowski addition of lower-dimensional polytopes in complementary subspaces, such that their sum is a full-dimensional polytope. Assume that all polytopes have been translated to the first orthant.

Sturmfels and Zelevinsky [SZ94] studied in particular the subclass of systems for which

$$l_k = 1 \text{ or } d_k = 1, \qquad k \in \{1, \ldots, r\}.$$

They showed that every such system has a number of Sylvester-type formulae for its sparse resultant, called *multigraded resultants*, one for every permutation of the indices $\pi \in S(1, \ldots, r)$. Polynomials and unmixed polynomial systems satisfying this condition will be also called multigraded. For the multigraded resultant matrix, all supports $\mathcal{B}_i$ are identical, of cardinality equal to the unique n-fold mixed volume. Let $B \subset \mathbb{R}^n$ be the convex hull of $\mathcal{B}_i$. Matrix M is defined by setting

$$B = \sum_{k=1}^{r} m_k S_{l_k} \subset \mathbb{R}^n \qquad \text{where } \; m_k = (d_k - 1) l_k + d_k \sum_{j:\pi(j)<\pi(k)} l_j, \qquad k \in \{1, \ldots, r\}.$$

For every permutation π the values of m_k define a unique vector v that the incremental algorithm may use to deterministically construct the Sylvester-formula. Specifically, partition the coordinates of $v \in \mathbb{Q}^n$ according to the subsequences x_k then

$$\text{set all coordinates in the } k\text{-th subset to } \frac{n d_k - m_k}{l_k} \in \mathbb{Q}. \tag{3.12}$$

Lemma 3.3.1 *Partition the n coordinates of vector $v \in \mathbb{Q}^n$ into r groups according to the partition of variables and set every coordinate in the k-th group equal to $(nd_k - m_k)/l_k \in \mathbb{Q}$. Then $P - V = B$.*

Proof By using the fact that $\sum_{k=1}^{r} l_k = n$ and that for every k we have $d_k = 1$ or $l_k = 1$, it follows that

$$\frac{nd_k - m_k}{l_k} = 1 + \frac{d_k}{l_k} \sum_{j:\pi(j)>\pi(k)} l_j > 0, \quad k = 1, \ldots, r.$$

Consider any point $p \in P - V$ with coordinates grouped in r groups, each of cardinality l_k. For k such that $l_k = 1$ we have two conditions on coordinate c from the fact that $p \in P$ and $p \in P - V$ respectively:

$$0 \leq c \leq nd_k \qquad \text{and} \qquad 0 \leq c + \frac{nd_k - m_k}{l_k} \leq nd_k$$

which are equivalent to $0 \leq c \leq m_k$, because $l_k = 1$. For k such that $l_k > 1$ and $d_k = 1$, we have two conditions on the sum s of the l_k coordinates in the k-th group of p, again from $p \in P$ and $p \in P - V$ respectively:

$$0 \leq s \leq n \qquad \text{and} \qquad 0 \leq s + l_k \frac{n - m_k}{l_k} \leq n$$

which are equivalent to $0 \leq s \leq m_k$. Now $p \in B$ if and only if every coordinate c of p corresponding to $l_k = 1$ lies in $[0, m_k]$ and every sum s of coordinates as above lies in $[0, m_k]$. Hence $p \in B$ if and only if $p \in P - V$. □

To see how this v was chosen, observe that B is a scaled-down copy of P, where the scaling has occurred by a different factor for every subsequence of l_k coordinates.

Theorem 3.3.2 *Given a multihomogeneous system of type $(l_1, \ldots, l_r; d_1, \ldots, d_r)$ such that $l_k = 1$ or $d_k = 1$ for $k = 1, \ldots, r$, we define $v \in \mathbb{Q}^n$ with the k-th group of coordinates equal to $(nd_k - m_k)/l_k$ as in (3.12). Then the first matrix constructed by our algorithm has determinant equal to the sparse resultant of the system.*

Proof It follows from the lemma that the first set of supports $\mathcal{B}_i$ constructed are all equal to $B \cap \mathbb{Z}^n$, since the cardinality of the latter is MV_{-i}, hence they are exactly those required to define a Sylvester-type formula for the resultant. Note that the formula obtained corresponds to the permutation π used in the definition of m_k and any permutation π may be used. □

We have been able to produce all multigraded resultants for different examples. Experimental results and preliminary running times on the Sun Sparc 10/40 of table 1.1 are displayed in table 3.1, rounded to the nearest integer number of seconds. $\deg R$ and $\deg D$ indicate respectively the total degree of the sparse resultant and the number of rows or columns of the maximal minor D that our algorithm constructs.

For systems that are not multigraded we have used v defined similarly and obtained resultant matrices of satisfactory size. For the second class of systems in table 3.1 for which there exists k such that $l_k > 1$ and $d_k > 1$, we have used formula (3.12) to calculate m_i and v, then have perturbed the latter to obtain the results shown. For type $(2, 1; 2, 1)$ the greedy implementation in section 3.1.6 produces a matrix of size 103. Notice that matrices of comparable dimension take significantly more time in this class of systems because of the incremental LU decomposition that requires several iterations, considerable

type	application	vector $u \in \mathbb{Z}^n$	$\deg R$	$\deg D$	CPU time
$(2,1,1;1,2,2)$		$(2,2,3,1)$	240	240	42s
$(1,1,1,1;2,2,1,1)$		$(7,5,2,1)$	480	480	1m 0s
$(1,1,1,1;3,3,1,1)$		$(10,7,2,1)$	1080	1080	2m 11s
$(1,1,1,1;3,3,2,1)$		$(10,7,3,1)$	2160	2160	4m 3s
$(1,1,1,1;3,3,3,1)$		$(10,7,4,1)$	3240	3240	6m 29s
$(2,1;2,1)$		$\sim(1,1,3)$	48	52	0s
$(2,1;2,2)$		$\sim(1,1,5)$	96	104	6s
$(2,1,1;2,1,1)$		$\sim(3,3,2,1)$	240	295	1m 43s
$(2,1,1;2,2,1)$		$\sim(3,3,3,1)$	480	592	8m 56s
$\sim(2,3,1,1)$	vision	$(5,5,2,2,2)$	60	60	12s
$\sim(3,4,2,2)$	kinematics	$\sim(11,11,11,3,3,3,3)$	246	745	25m 0s

Table 3.1: Sparse resultant algorithm performance on a Sun Sparc 20/61.

bookkeeping and memory page swaps. In the final version of the implementation this discrepancy should be minimized.

The last part of the table refers to specific applications, namely the motion from points problem in vision and the forward kinematics of the Stewart platform parallel robot. Neither is exactly a multihomogeneous system of the shown type but both approximate this structure. The kinematics problem is very sparse, which significantly lowers the sparse resultant degree; in actuality, we have used a perturbation of the shown v.

Specific results on system solving using resultant matrices are reported in chapter 5. In particular, the two problems above are studied in further detail, while section 5.4 discusses the relevance of multigraded systems to problems in game theory and computational economics.

Chapter 4

Solution of Polynomial Systems

The problem of computing all common zeros of a system of polynomials is of fundamental importance in a wide variety of scientific and engineering applications. This chapter studies efficient methods for computing monomial bases for generic polynomials and for finding all *isolated* solutions of an arbitrary system.

First we consider monomial bases of the coordinate ring defined by the polynomial ideal and show how their computation reduces to calculating mixed volume. The proof follows immediately form our resultant construction and is simpler than the first demonstration [PS94]. Monomial bases lead to *multiplication maps* which provide a standard method for calculating isolated solutions.

Depending on the lifting, different bases and multiplication maps (or tables) may be obtained. Similarly, these maps may be specified by a Gröbner basis with respect to a given term ordering. The comparative study of different approaches to the same problem can shed more light to the intriguing relation between liftings and orderings and, more importantly, resultants and Gröbner bases. A comprehensive account of alternative techniques for these problems and root finding in general is given in the Introduction: in addition to Gröbner bases we survey the Ritt-Wu method, the classical u-resultant approach as well as numerical continuation techniques.

The reduction of root finding to an eigenproblem was proposed in [AS88] and independently in the work of Manocha and Canny who obtained very fast methods for dealing with specific problems in modeling, graphics and robotics [Man92]. These implementations proved significantly faster than previous approaches relying on Gröbner bases and homotopies.

A merit of the resultant approach is that a large part of the computation can be executed in a preprocessing phase only once for every system with indeterminate coefficients. Our approach is similar to the methods based on the homogeneous resultant but uses the sparse resultant construction to obtain smaller matrices. Polynomials need not be homogenized and the explicit resultant polynomial is never computed. Instead a matrix formula is used to reduce root-finding to an eigenproblem, which allows the application of powerful techniques from linear algebra. We generalize the results in [Emi94] to minimize the a priori knowledge required by the algorithm, thus incorporating both resultant matrix construction algorithms.

The study of coordinate rings of varieties in K^n, where K is a field, has been shown to be particularly useful in studying systems of polynomial equations. For a 0-dimensional variety it is well known that the coordinate ring forms a finite-dimensional vector space and, actually, an algebra over K. An important algorithmic question is the construction of an explicit monomial K-basis for such a space. The reduction of this problem to identifying the mixed cells of a mixed subdivision has an easy

proof in section 4.1. A consequence is a Poisson formula for the sparse resultant which has theoretical interest since it verifies some important properties. Based on monomial bases, section 4.2 generates generic endomorphisms or multiplication maps for any given polynomial which allows us to compute in the coordinate ring.

We then generalize the discussion from generic to arbitrary polynomials. Root finding is reduced to an eigenproblem and then existing techniques are employed from numerical linear algebra which offer implementations of powerful and accurate algorithms. section 4.3 discusses this method in connection to both of our resultant algorithms for the case of adding an extra u-polynomial to obtain an overconstrained system.

A relatively novel approach that keeps the number of polynomials fixed is proposed in section 4.4 where one of n variables is *hidden* in the coefficient field thus producing an overconstrained system. We show how the calculation of all isolated roots again reduces to the computation of all eigenvalues and eigenvectors of a square matrix. The chapter concludes with a discussion of our implementation and the practical issues that arise thereof.

4.1 Monomial Bases

For n *generic* Laurent polynomials $f_1, \ldots, f_n$ in n variables, the definition of monomial bases from mixed subdivisions was first demonstrated by Pedersen and Sturmfels [PS94]. Their proof relies on reducing the general problem to binomial systems via Puiseux series. Theorem 4.1.3 verifies their result. However, we use a different proof which is considerably simpler once the construction of resultant matrix M is established and which leads to a constructive approach for finding the common zeros.

The genericity of the polynomials is equivalent to saying that all coefficients are generic. Let $\mathcal{I} = \mathcal{I}(f_1, \ldots, f_n)$ be the ideal that they generate and $V = V(f_1, \ldots, f_n) \in (\overline{K}^*)^n$ their variety, where $\overline{K}$ is the algebraic closure of field K. Assume that V has *dimension zero*. Then its coordinate ring $K[x, x^{-1}]/\mathcal{I}$ is an m-dimensional vector space over K by theorem 1.3.7, where

$$m = MV(f_1, \ldots, f_n) = MV(Q_1, \ldots, Q_n).$$

In addition, the ideal $\mathcal{I} = \mathcal{I}(f_1, \ldots, f_n)$ is assumed to be *radical*, or self-radical, i.e. $\mathcal{I} = \sqrt{\mathcal{I}}$, which is equivalent to saying that all roots in V are distinct.

We add a generic linear $f_0 \in K[x, x^{-1}]$ to the set $f_1, \ldots, f_n$ and define the Minkowski sum $Q^0 + \delta$ and its mixed subdivision Δ^0_δ via lifting $(l_0, l_1, \ldots, l_n)$. Without loss of generality we can choose f_0 such that it has the constant monomial 1 as one of its monomials and the zero exponent is a Q_0 vertex. Let $\mathcal{B} \subset \mathcal{E} \subset \mathbb{Z}^n$ be the set of all integer lattice points that lie in 0-mixed cells in Δ^0_δ. Equivalently, $\mathcal{B}$ is the set of all Laurent monomials with exponent vectors in the 0-mixed cells. By theorem 3.1.16, $|\mathcal{B}| = m$ and we can write $\mathcal{B} = \{b_1, \ldots, b_m\}$.

An important property of the subdivision-based matrix construction is that postmultiplication with certain column vectors expresses evaluation of the polynomials whose coefficients have filled in the rows of the matrix. More precisely, for an arbitrary $\alpha \in \overline{K}^n$,

$$M \begin{bmatrix} \vdots \\ \alpha^q \\ \vdots \end{bmatrix} = \begin{bmatrix} \vdots \\ \alpha^p f_{i_p}(\alpha) \\ \vdots \end{bmatrix}, \tag{4.1}$$

where $p \in \mathcal{E}$ indexes the row of M that contains the coefficients of $x^p f_{i_p}(x)$ and $q \in \mathcal{E}$ indexes the column corresponding to monomial x^q.

Since Q_0 contains $0^n \in \mathbb{Z}^n$ we can always pick, without loss of generality, lifting function l_0 such that Q_0 contributes only its zero vertex 0^n as a summand to the 0-mixed cells. The proof of lemma 2.1.6 formalizes the requirement on l_0 and proves the feasibility of this construction. By definition, every row indexed by a monomial in $\mathcal{B}$ contains the coefficients of $x^{b-0^n} f_0 = x^b f_0$, for some $b \in \mathcal{B}$.

The partition of $\mathcal{E}$ into $\mathcal{B}$ and $\mathcal{E} \setminus \mathcal{B}$ defines four blocks in M shown below, where the rightmost set of columns and bottom set of rows are indexed by $\mathcal{B}$. Submatrices M_{11} and M_{22} are square of size $|\mathcal{E} \setminus \mathcal{B}| = |\mathcal{E}| - m$ and $|\mathcal{B}| = m$ respectively, while M_{12} and M_{21} are rectangular. Let $\alpha \in V$ be a fixed common root. Relation (4.1) becomes

$$M \begin{bmatrix} \vdots \\ \alpha^{q_c} \\ \vdots \\ \alpha^{b_i} \\ \vdots \end{bmatrix} = \begin{bmatrix} & \\ M_{11} & M_{12} \\ & \\ M_{21} & M_{22} \\ & \end{bmatrix} \begin{bmatrix} \vdots \\ \alpha^{q_c} \\ \vdots \\ \alpha^{b_i} \\ \vdots \end{bmatrix} = \begin{bmatrix} \vdots \\ 0 \\ \vdots \\ \alpha^{b_i} f_0(\alpha) \\ \vdots \end{bmatrix} \tag{4.2}$$

where q_c ranges over $\mathcal{E} \setminus \mathcal{B}$ and b_i ranges over $\mathcal{B}$.

By theorem 3.1.12 every principal minor of M is generically nonzero, hence the inverse submatrix M_{11}^{-1} exists. Then, we can define $m \times m$ matrix

$$M' = M_{22} - M_{21} M_{11}^{-1} M_{12}.$$

Lemma 4.1.1 *All eigenvectors of M' are of the form $[\alpha^{b_1}, \ldots, \alpha^{b_m}]$ for some root $\alpha \in V$.*

Proof We premultiply both sides of (4.2) with the non-singular matrix

$$\begin{bmatrix} I & 0 \\ -M_{21} M_{11}^{-1} & I \end{bmatrix},$$

where I stands for the identity matrix of appropriate size, and obtain

$$\begin{bmatrix} & \\ M_{11} & M_{12} \\ & \\ 0 & M' \\ & \end{bmatrix} \begin{bmatrix} \vdots \\ \alpha^{q_c} \\ \vdots \\ \alpha^{b_i} \\ \vdots \end{bmatrix} = \begin{bmatrix} \vdots \\ 0 \\ \vdots \\ \alpha^{b_i} f_0(\alpha) \\ \vdots \end{bmatrix}. \tag{4.3}$$

The bottom part of this matrix equation is of interest:

$$M' \begin{bmatrix} \alpha^{b_1} \\ \vdots \\ \alpha^{b_m} \end{bmatrix} = \begin{bmatrix} \alpha^{b_1} f_0(\alpha) \\ \vdots \\ \alpha^{b_m} f_0(\alpha) \end{bmatrix}.$$

Let v'_α be the column vector $[\alpha^{b_1}, \ldots, \alpha^{b_m}]$, with $b_i \in \mathcal{B}$. Since $\alpha \in (\overline{K}^*)^n$, every v'_α is nonzero; furthermore, (4.3) yields an eigenvector equation

$$M' v'_\alpha = f_0(\alpha) v'_\alpha \Rightarrow \left(M' - f_0(\alpha) I\right) v'_\alpha = 0.$$

Since there are exactly m roots and we can construct one such vector per root, we obtain m such vectors. This is the largest possible number of eigenvectors, hence all eigenvectors of M' are of this form. □

Theorem 4.1.2 *Assume that variety $V = V(\mathcal{I})$ has dimension zero, ideal $\mathcal{I}$ is radical and $\mathcal{B}$ is the set of monomials corresponding to integer lattice points in 0-mixed cells in the subdivision Δ_δ^0 of Q^0. Then $\mathcal{B}$ forms a vector-space basis for the coordinate ring $K[x, x^{-1}]/\mathcal{I}$ over K.*

Proof The roots α are distinct and, by the genericity of f_0, all eigenvalues $f_0(\alpha)$ are distinct. This implies that all eigenvectors v'_α are linearly independent.

If the monomials in $\mathcal{B}$ are not independent in $K[x, x^{-1}]/\mathcal{I}$, then a nontrivial linear combination of them over K must belong to $\mathcal{I}$. Hence, there are elements $k_1, \ldots, k_m \in K$ not all zero such that, for every $\alpha \in V$, $\sum_{i=1}^m k_i \alpha^{b_i} = 0$. Construct now the square matrix below with $v'_{\alpha_j} = [\alpha_j^{b_1}, \ldots, \alpha_j^{b_m}]$ as the j-th column, where $V = \{\alpha_1, \ldots, \alpha_m\}$; this matrix has dependent rows:

$$\begin{bmatrix} \alpha_1^{b_1} & \alpha_2^{b_1} & \cdots & \alpha_m^{b_1} \\ \alpha_1^{b_2} & \alpha_2^{b_2} & \cdots & \alpha_m^{b_2} \\ & & \vdots & \\ \alpha_1^{b_m} & \alpha_2^{b_m} & \cdots & \alpha_m^{b_m} \end{bmatrix}. \tag{4.4}$$

This contradicts the independence of vectors v'_{α_j} and, since their cardinality equals the space dimension, $\mathcal{B}$ is a basis. □

In other words, we have defined a canonical surjective homomorphism

$$K[x, x^{-1}] \to K[x, x^{-1}]/\mathcal{I} \; : \; g \mapsto g \bmod \mathcal{I} = \sum_{b_i \in \mathcal{B}} c_{b_i} x^{b_i}, \; c_{b_i} \in K$$
$$\text{such that } g \in \mathcal{I} \; \Leftrightarrow \; c_{b_i} = 0, \; \forall\, b_i \in \mathcal{B}.$$

The forward direction of the last equivalence is shown above; the converse is evident. In words, every polynomial g is mapped to the canonical representative of its coset with respect to ideal $\mathcal{I}$.

It turns out that we can compute the basis without going through the resultant matrix because the set $\mathcal{B}$ is defined independently of f_0. Consider a *mixed subdivision* Δ_δ of the perturbed Minkowski sum

$$Q + \delta = Q_1 + \cdots + Q_n + \delta$$

induced by $l_1, \ldots, l_n$, where both l_i and $\delta \in \mathbb{Q}^n$ are the same as above. The subdivision is specified by defining Minkowski sum

$$\widehat{Q} = \widehat{Q}_1 + \cdots + \widehat{Q}_n \subset \mathbb{R}^{n+1}$$

of the lifted Newton polytopes $\widehat{Q}_i$ and projecting its lower envelope facets onto the maximal cells of Δ_δ. The maximal cells in the subdivision are either *mixed*, when they are the Minkowski sum of n edges, or *unmixed*. The sum of all mixed cell volumes is $m = MV(f_1, \ldots, f_n)$.

By lemma 2.1.6 there exists l_0 such that set $\mathcal{B}$ comprising the points in the 0-mixed cells of Δ_δ^0 equals the set of all points n mixed cells of Δ_δ. This immediately leads to an equivalent statement of theorem 4.1.2.

Theorem 4.1.3 *Assume that variety $V = V(\mathcal{I})$ has dimension zero, ideal $\mathcal{I}$ is radical and let $\mathcal{B}$ be the set of monomials corresponding to integer lattice points in mixed cells in the subdivision Δ_δ of Q. Then $\mathcal{B}$ forms a vector-space basis for the coordinate ring $K[x, x^{-1}]/\mathcal{I}$ over K.*

Hence the monomial basis construction is reduced to identifying the mixed cells in Δ_δ.

A Poisson formula for the sparse resultant was given in [PS93], where the extraneous factor was described. In this section we show how matrix M' can be used to obtain a Poisson formula for the sparse resultant and specify the extraneous factor in matrix M constructed by our subdivision algorithm. Again, we shall focus on the evaluations of the $\mathcal{B}$ monomials at the roots; let $v_i = [\alpha_i^{b_1}, \ldots, \alpha_i^{b_m}]$.

Lemma 4.1.4 *If the monomials in set $\mathcal{B} = \{x^{b_1}, \ldots, x^{b_m}\}$ are all linearly independent in $K[x, x^{-1}]/\mathcal{I}$ over K, then the vectors v_i are linearly independent over K.*

Proof If the vectors v_i are not independent, then the $m \times m$ matrix (4.4) is singular. Therefore there exist $k_1, \ldots, k_m$ in K which are not all zero, such that

$$\sum_{i=1}^{m} k_i [\alpha_1^{b_i}, \ldots, \alpha_m^{b_i}] = 0 \Rightarrow \sum_{i=1}^{m} k_i \alpha_j^{b_i} = 0, \forall \alpha_j \in V. \tag{4.5}$$

Let $g(x) = \sum_{i=1}^{m} k_i x^{b_i}$ be a polynomial in $K[x, x^{-1}]/\mathcal{I}$ which is not identically zero because the k_i cannot be all zero. On the other hand, (4.5) implies that g vanishes on V, thus $g \in \mathcal{I}$ and hence g should be identically zero in $K[x, x^{-1}]/\mathcal{I}$, under the canonical basis $\mathcal{B}$. Thus we arrive at a contradiction. □

Theorem 4.1.5 *Assume $\mathcal{I}$ is zero-dimensional and radical and M_{11} is nonsingular. Letting $V = \{\alpha_1, \ldots, \alpha_m\}$, we have*

$$\det M' = \prod_{i=1}^{m} f_0(\alpha_i).$$

Proof Recall that $\{x^{b_1}, \ldots, x^{b_m}\}$ is a vector space basis of $K[x, x^{-1}]/\mathcal{I}$ over K. We have seen that each root α_i corresponds to an eigenvector $v_i = [\alpha_i^{b_1}, \ldots, \alpha_i^{b_m}]$ of M', with associated eigenvalue $f_0(\alpha_i)$. By the previous lemma, all m vectors v_i are independent over K.

From linear algebra we know that if the eigenvectors span the domain and range of square matrix M', then M' is similar to a diagonal matrix D whose diagonal entries are the eigenvalues of M' [Hun74, Thm. VII.5.5]. The determinant of M' equals that of D which is equal to the product $f_0(\alpha_1) \cdots f_0(\alpha_m)$. □

4.2 Multiplication Maps

Given are Laurent polynomials

$$f_1, \ldots, f_n \in K[x_1, x_1^{-1}, \ldots, x_n, x_n^{-1}] = K[x, x^{-1}]$$

with Newton polytopes $Q_1, \ldots, Q_n \subset \mathbb{R}^n$, whose Minkowski sum Q has a mixed subdivision Δ. Let

$$\mathcal{I} = \{ \sum_{i=1}^{n} g_i f_i \,|\, g_i \in K[x, x^{-1}] \} \subset K[x, x^{-1}]$$

be the ideal generated by $f_1, \ldots, f_n$. Generically, this ideal is zero-dimensional and radical. In other words its variety contains only isolated points and by theorem 1.3.7 their cardinality is exactly $m = MV(Q_1, \ldots, Q_n)$. Then $K[x, x^{-1}]/\mathcal{I}$ is an m-dimensional vector space, and further an algebra, over K.

By the results of section 4.1 there exists a monomial basis $\mathcal{B}$ of $K[x,x^{-1}]/\mathcal{I}$ over K defined by the monomials in the mixed cells of Δ. Let $\mathcal{B}=\{b_1,\ldots,b_m\}\subset(Q\cap\mathbb{Z}^n)$, where the natural bijection between monomials and integer lattice points allows us to regard $\mathcal{B}$ equivalently as an integer point set. With respect to this basis polynomials in $K[x,x^{-1}]/\mathcal{I}$ have a canonical representation as the (row) vector in K^m of their coefficients.

Given an arbitrary polynomial $f_0\in K[x,x^{-1}]$ with Newton polytope Q_0 we wish to compute an endomorphism in $K[x,x^{-1}]/\mathcal{I}$ that expresses multiplication by f_0. This endomorphism is called the *multiplication map* of f_0 and its calculation provides a way for solving the well-constrained system $f_1=\cdots=f_n=0$. Let Q_0 be the Newton polytope of f_0 and $Q^0=Q_0+Q$ the Minkowski sum of the overconstrained system.

First a square resultant matrix M is computed by the subdivision resultant algorithm. The columns and rows of M are indexed by point set $\mathcal{E}\subset Q^0$. By lemma 2.1.6, there exists lifting function f_0 so that $\mathcal{B}$ corresponds to the 0-mixed cells of Δ^0. Following the derivation of section 4.1 there is a partition of M into four blocks of which M_{11} and M_{22} are square and indexed by $\mathcal{E}\setminus\mathcal{B}$ and $\mathcal{B}$ respectively.

There is a subtle point here. Since we do not restrict f_0 to have a constant term, we do not know in principle that $\mathcal{B}$ indexes both rows and columns of M_{22}. Yet we can fix any coefficient c_0 of f_0, whose corresponding exponent vector a_0 is extremal (i.e. a vertex) in Q_0 and choose l_0 such that this vertex is the only Q_0 summand in optimal sums for f_0 rows. This is possible by an argument analogous to the proof of lemma 2.1.6. Then there exists a permutation of the columns in M so that c_0 appears on every diagonal entry of M_{22} and for any row $x^{b_i-a_0}f_0$ of M_{22} there is a column indexed by b_i. Now the same arguments as before hold except that the eigenvalue corresponding to α is $f(\alpha)/\alpha^{a_0}$.

M_{11} is nonsingular so we can define square matrix $M'=M_{22}-M_{21}M_{11}^{-1}M_{12}$ indexed by $\mathcal{B}$ as follows:

$$M=\begin{bmatrix} M_{11} & M_{12}\\ M_{21} & M_{22}\end{bmatrix}\qquad\text{and}\qquad\begin{bmatrix} I & 0\\ -M_{21}M_{11}^{-1} & I\end{bmatrix}M=\begin{bmatrix} M_{11} & M_{12}\\ 0 & M'\end{bmatrix}. \tag{4.6}$$

This section shows how matrix M' is the matrix of the endomorphism in $K[x,x^{-1}]/\mathcal{I}$ which expresses multiplication by polynomial f_0.

Lemma 4.2.1 *The rows of M' contain the coefficients of polynomials $x^{b_i}f_0 \bmod \mathcal{I}$, for some $b_i\in\mathcal{B}$.*

Proof Premultiplication of M as in (4.6) has the effect of adding scalar multiples of the rows indexed by $\mathcal{E}\setminus\mathcal{B}$ to those indexed by $\mathcal{B}$. Hence, the row of M indexed by $b_i\in\mathcal{B}$ now contains the coefficients of

$$x^{b_i}f_0+\sum_{p\in\mathcal{E}\setminus\mathcal{B}}k_px^pf_{j_p},\qquad\text{for some } k_p\in K,\ j_p\in\{1,\ldots,n\}.$$

The right hand side of (4.6) shows that each row of M' corresponds to a polynomial g which is a linear combination of the monomials in $\mathcal{B}$, over K. The difference $g-x^{b_i}f_0$ belongs to $\mathcal{I}$, hence $g\equiv x^{b_i}f_0 \pmod{\mathcal{I}}$. □

Theorem 4.2.2 *Suppose $\mathcal{I}$ is zero-dimensional and radical and let M' denote both the matrix and the associated endomorphism in $K[x,x^{-1}]/\mathcal{I}$ with respect to basis $\mathcal{B}$. Then this endomorphism expresses multiplication by polynomial $f_0\in K[x,x^{-1}]/\mathcal{I}$,*

$$M':K[x,x^{-1}]/\mathcal{I}\to K[x,x^{-1}]/\mathcal{I}:g\mapsto gf_0 \bmod \mathcal{I}.$$

In other words, if (row) vector $v_g^T=[c_1,\ldots,c_m]$ represents polynomial $g\in K[x,x^{-1}]/\mathcal{I}$, with respect to basis $\mathcal{B}$, then (row) vector v_g^TM' represents polynomial $gf_0\in K[x,x^{-1}]/\mathcal{I}$ with respect to the same basis.

Proof From the previous lemma row b_i of M' contains the coefficients of polynomial $x^{b_i} f_0 \bmod \mathcal{I}$. Let $g = \sum_{i=1}^{m} c_i x^{b_i}$, then

$$g f_0 \bmod \mathcal{I} = \sum_{i=1}^{m} c_i (x^{b_i} f_0 \bmod \mathcal{I}) = \sum_{i=1}^{m} c_i \left(\sum_{j=1}^{m} M'_{ij} x^{b_j} \right) = \sum_{j=1}^{m} x^{b_j} \left(\sum_{i=1}^{m} c_i M'_{ij} \right).$$

If $b_j \in \mathcal{B}$ indexes the j-th column of M', then the last polynomial can be expressed as the row vector indexed by $\mathcal{B}$ with j-th entry $\sum_{i=1}^{m} c_i M'_{ij}$. By the definition of v_g we have

$$v_g^T M' = [c_1, \ldots, c_m] M' = \left[\sum_{i=1}^{m} c_i M'_{i1}, \ldots, \sum_{i=1}^{m} c_i M'_{im} \right]$$

and the claim is proven. □

4.3 Adding a Polynomial

The first part of this section, up to the first lemma, essentially reiterates the approach used to prove the monomial bases theorem above. We cast the discussion in the familiar terms of the u-polynomial and describe the exact method implemented.

The problem addressed here is to find all isolated roots $\alpha \in V$ where $V \subset (\overline{K}^*)^n$ is the zero-dimensional variety of

$$f_1, \ldots, f_n \in K[x_1, x_1^{-1}, \ldots, x_n, x_n^{-1}] \tag{4.7}$$

with cardinality $m = MV(Q_1, \ldots, Q_n)$. The present approach incorporates the ideas of the previous section but generalizes the discussion to allow M to be constructed by our incremental algorithm. An overconstrained system is obtained by adding extra polynomial f_0 to the given system. We choose f_0 to be linear with coefficients c_{0j} and constant term equal to indeterminate u.

$$f_0 = u + c_{01} x_1 + \cdots + c_{0n} x_n \ \in K[x, x^{-1}, u].$$

Recall that the resultant characterizes the solvability of the system, therefore the addition of artificial constraint f_0 may eliminate some of the solutions of $f_1 = \cdots = f_n = 0$ unless f_0 includes free variable u, which takes the value $-\sum_j c_{0j} \alpha_j$ at root $\alpha = (\alpha_1, \ldots, \alpha_n)$.

Coefficients c_{0j}, $j = 1, \ldots, n$, should define an *injective* function

$$f_0 : V \to \overline{K} : \alpha \mapsto f_0(\alpha).$$

There are standard deterministic strategies for selecting c_{0j} so that they ensure the injective property. In practice, coefficients c_{0j} may be randomly distributed in some range of integer values of size $S > 1$, and a bad choice for $c_{01}, \ldots, c_{0n}$ is one that will result in the same value of $f_0 - u$ at two distinct roots α and α'. Assume that α and α' differ in their i-th coordinate for some $i > 0$, then fix all choices of c_{0j} for $j \neq i$; the probability of a bad choice for c_{0i} is $1/S$, and since there are $\binom{m}{2}$ pairs of roots, the total probability of failure for this scheme is

$$\text{Prob[failure]} \leq \binom{m}{2} / S : \qquad c_{0j} \in \{1, \ldots, S\},\ j = 1, \ldots, n.$$

It suffices, therefore, to pick c_{0j} from a sufficiently large range in order to make the probability of success arbitrarily high. Moreover, it is clear that any choice of coefficients can be tested deterministically at the end of the algorithm.

Either algorithm for the resultant matrix may be used to build matrix M. As before, the vanishing of $\det M$ is a necessary condition for the overconstrained system to have common roots; for $\alpha \in V$, u is constrained to a specific value determined by the c_{0j} coefficients.

The construction of M is not affected by this definition of f_0. Let monomial set $\mathcal{E}$ index the columns of M and partition M so that the lower right square submatrix M_{22} depends on u and has size r. Suppose for now that the upper left square submatrix M_{11} is nonsingular. Clearly $r \geq m$ and equality holds if M is obtained by the subdivision resultant algorithm. For an arbitrary $\alpha \in (\overline{K}^*)^n$

$$\begin{bmatrix} M_{11} & M_{12} \\ M_{21} & M_{22}(u) \end{bmatrix} \begin{bmatrix} \vdots \\ \alpha^q \\ \vdots \end{bmatrix} = \begin{bmatrix} \vdots \\ \alpha^p f_{i_p}(\alpha) \\ \vdots \end{bmatrix} : \qquad q, p \in \mathcal{E}, i_p \in \{0, 1, \ldots, n\}, \tag{4.8}$$

Now $M'(u) = M_{22}(u) - M_{21}M_{11}^{-1}M_{12}$ is defined the same way as before, though now its diagonal entries are linear polynomials in u and

$$M'(u) \begin{bmatrix} \alpha^{b_1} \\ \vdots \\ \alpha^{b_r} \end{bmatrix} = \begin{bmatrix} \alpha^{p_1} f_0(\alpha, u) \\ \vdots \\ \alpha^{p_r} f_0(\alpha, u) \end{bmatrix}, \tag{4.9}$$

where $\mathcal{B} = \{b_1, \ldots, b_r\}$ and $\{p_1, \ldots, p_r\}$ index the columns and rows, respectively, of M_{22} and thus M'. For a root $\alpha \in V$ and for

$$u = -\sum_{j=1}^{n} c_{0j}\alpha_j$$

the right hand side vector in (4.9) is null. Let $v'_\alpha = [\alpha^{b_1}, \ldots, \alpha^{b_r}]$ and write $M'(u) = M' + uI$, where M' now is numeric and I is the $r \times r$ identity matrix. Then

$$(M' + uI)v'_\alpha = 0 \Rightarrow \left[M' - \left(\sum_j c_{0j}\alpha_{ij} \right) I \right] v'_\alpha = 0.$$

If the generated ideal $\mathcal{I}$ is radical then every eigenvalue has *algebraic multiplicity* one with probability greater or equal to $1 - \binom{m}{2}/S$. We can relax the condition on $\mathcal{I}$ by simply requiring that each eigenvalue has *geometric multiplicity* one. Algebraic multiplicity captures the usual notion of multiplicity, whereas geometric multiplicity expresses the dimension of the eigenspace associated with an eigenvalue. If there exist eigenvalues of higher geometric multiplicity we may exploit the fact that specializations of the u-resultant yield the root coordinates or we can define an overconstrained system by hiding a variable as in the next section and derive the root coordinates one by one.

In what follows we assume that all eigenvalues have unit geometric multiplicity. Hence it is guaranteed that among the eigenvectors of M' we shall find the vectors v'_α for $\alpha \in V$. By lemma 4.1.1 each eigenvector v'_α of M' contains the values of monomials $\mathcal{B}$ at some common root $\alpha \in (\overline{K}^*)^n$. By (4.8) we can define vector v_α as follows:

$$M_{11}v_\alpha + M_{12}v'_\alpha = 0 \Rightarrow v_\alpha = -M_{11}^{-1}M_{12}v'_\alpha. \tag{4.10}$$

The size of v_α is $|\mathcal{E}| - r$, indexed by $\mathcal{E} \setminus \mathcal{B}$. It follows that vectors v_α and v'_α together contain the values of every monomial in $\mathcal{E}$ at some root α.

Lemma 4.3.1 *Let $p_0, p_1, \ldots, p_s \in \mathcal{E}$, $s \geq n$, be a set of affinely independent points. Regard each p_i as a column vector and define $s \times n$ matrix P such that its i-th row equals $(p_i - p_0)^T$. Then, given v_α and v'_α, we can compute the coordinates of root $\alpha \in V(\mathcal{I})$. If $p_0, p_1, \ldots, p_s \in \mathcal{B}$ then v'_α suffices.*

Proof By the affine independence of the p_i it follows that P has rank n. Linear algebra tells us that there exists nonsingular $s \times s$ matrix Q such that QP is an upper-triangular matrix with nonzero diagonal $d_1, \ldots, d_n \in \mathbb{Z}$.

The concatenation vector $[v_\alpha, v'_\alpha]$ is indexed by $\mathcal{E}$ in the sense that every one of its entries is the value of α at the respective point. Consider the (column) subvector of $[v_\alpha, v'_\alpha]$ indexed by points $p_1, \ldots, p_s$. By construction of P, this subvector is in bijective correspondence with the rows of P. Now apply the sequence of elementary row operations specified by Q to the elements of this subvector as follows: a row swap is an exchange of vector entries, the scaling of a row by c corresponds to raising the respective entry to c and the addition of row i, multiplied by c, to row j corresponds to multiplication of vector entry j by the i-th entry raised to c. Let q denote this vector transformation. The resulting vector has the last $s - n$ entries equal to 1.

Let $\alpha = (\alpha_1, \ldots, \alpha_n)$ and $e_2, e_n, g_n \in \mathbb{Z}$, then this transformation can be written as follows:

$$QP = \begin{bmatrix} d_1 & e_2 & \cdots & \cdots & e_n \\ & & \vdots & & \\ 0 & \cdots & 0 & d_{n-1} & g_n \\ 0 & \cdots & 0 & 0 & d_n \\ 0 & & \cdots & & 0 \\ & & \vdots & & \\ 0 & & \cdots & & 0 \end{bmatrix} \quad \text{then} \quad q : \begin{bmatrix} \alpha^{p_1 - p_0} \\ \\ \\ \vdots \\ \\ \\ \alpha^{p_s - p_0} \end{bmatrix} \mapsto \begin{bmatrix} \alpha_1^{d_1} \alpha_2^{e_2} \cdots \alpha_n^{e_n} \\ \vdots \\ \alpha_{n-1}^{d_{n-1}} \alpha_n^{g_n} \\ \alpha_n^{d_n} \\ 1 \\ \vdots \\ 1 \end{bmatrix}.$$

The final step consists in reading off the coordinates of α from the modified vector. The value of coordinate n is obtained by taking the d_n-th root of the n-th entry; α_{n-1} is the d_{n-1}-th root of the vector's $(n-1)$-st entry divided by $\alpha_n^{g_n}$. The rest of the root coordinates are computed in an analogous fashion; this is in a sense the backwards substitution phase where the row elementary operations are transformed so that they apply to the exponents. □

Clearly, $n + 1$ points are necessary and sufficient, if affinely independent, to recover all root coordinates.

Theorem 4.3.2 *Assume $\mathcal{E}$ spans $\mathbb{Z}^n$. Then there exists an algorithm that finds a subset of $n+1$ affinely independent points in $\mathcal{E}$ with bit complexity $\mathcal{O}(|\mathcal{E}|n^2(\log d + \log n))$, where d is the maximum degree in any variable in the input polynomials.*

Proof $\mathcal{E}$ always includes $n+1$ such points because it spans a lattice of dimension n. A simple procedure to find such a set of points is the following: Select any set of n points from $\mathcal{E}$ and consider them as column vectors of a matrix. While this matrix does not have full rank, add the minimum number of points from $\mathcal{E}$ so that the matrix may achieve full rank. Continue until a full-rank matrix is obtained, which is guaranteed to happen after selecting at most $|\mathcal{E}|$ lattice points. This gives a set of n independent vectors; picking an additional distinct point comlpetes the set.

Enumerating the independent points has worst-case complexity $\mathcal{O}(|\mathcal{E}|n^2)$ since it reduces to a rank test on a $|\mathcal{E}| \times n$ matrix. The entries of this matrix are exponent vector entries of magnitude at

most dn. □

The last hypothesis concerned the nonsingularity of M_{11}. When it is singular the resultant matrix is regarded as a linear matrix polynomial in u and the values and vectors of interest are its singular values and right kernel vectors. Finding these for an arbitrary matrix polynomial is discussed in the next section.

Asymptotic Complexity

The principal steps are, given matrix M, to compute matrix M' and find its eigenvectors. We assume that $n+1$ affinely independent points have been computed offline for the given polynomials. If some lie in $\mathcal{E} \setminus \mathcal{B}$ the algorithm will have to compute the respective entries of vector v_α of (4.10).

Let $MM(\cdot)$ be the asymptotic complexity of matrix multiply as a function of the matrix size; currently $MM(k) = \mathcal{O}(n^{2.376})$ [CW90]. We shall loosely bound $MM(k)$ by $\mathcal{O}(k^3)$. It is known that inverting a matrix and computing its determinant and characteristic polynomial are all computationally equivalent to matrix multiply [vzG88].

We may occasionally compromise the tightness of the bounds in order to reduce the intricacy of the complexity formulae. In practice, most of the operations described here are not implemented with (exact) rational arithmetic but are instead carried out over floating point numbers of fixed size. An important aspect of this computation is numerical error, which we discuss in the last section and in the particular context of specific applications later.

Lemma 4.3.3 *Suppose that system (4.7) generates a zero-dimensional radical ideal, matrix M has been computed such that M_{11} is nonsingular and $n+1$ affinely independent points in $\mathcal{E}$ have been computed. Let r be the size of M' and d the maximum polynomial degree in a single variable. Then all common zeros of the polynomial system are approximated in time*

$$\mathcal{O}(|\mathcal{E}|^3 + rn^2\mu \log d).$$

Proof There are four matrix operations, namely inverting M_{11}, computing $X = M_{11}^{-1}M_{12}$, $M' = M_{21}X$ and eigenvectors v' with total cost $\mathcal{O}((|\mathcal{E}| - r)^3 + (|\mathcal{E}| - r)^2 r + (|\mathcal{E}| - r)r^2 + r^3)$. Computing at most $n+1$ entries of v has lower complexity than that. At each candidate root the input polynomials are evaluated with cost $\mathcal{O}(n\mu \log d)$ per polynomial for a total of $\mathcal{O}(rn^2\mu \log d)$. Since $\mu \geq n$ this dominates the process of recovering the coordinates. □

It is clear that for most systems the arithmetic complexity is dominated by the first term, namely the complexity of matrix operations and in particular the eigendecomposition.

A more concise expression for the complexity is obtained by some additional yet reasonable assumptions which have been observed to hold for most systems encountered in practice. Recall the definition of the scaling factor s of an overconstrained system from section 2.2 and let D stand for the total degree of the sparse resultant in the polynomial coefficients which equals the sum of all n-fold mixed volumes.

Theorem 4.3.4 *Assume that the scaling factor s of the overconstrained system is constant and the sum of all n-fold mixed volumes is $D = \Theta(nm)$, where $m = MV(f_1, \ldots, f_n)$. Under the hypotheses of the previous lemma the worst-case complexity of approximating all common roots of (4.7) is*

$$2^{\mathcal{O}(n)}m^3.$$

Proof By theorem 2.2.5 and under the hypotheses we can bound $|\mathcal{E}|$ as follows.

$$|\mathcal{E}| = \mathcal{O}\left(\frac{s^n e^n}{n} D\right) = 2^{\mathcal{O}(n)} m.$$

By bounding r, d and μ by $|\mathcal{E}|$ we arrive at the result. □
As expected, the complexity is single exponential in n and polynomial in the number of roots.

4.4 Hiding a Variable

An alternative way to obtain an overconstrained system from a well-constrained one is by *hiding* one of the original variables in the coefficient field. Hiding a variable instead of adding an extra polynomial possesses the advantage of keeping the number of polynomials constant. Our experience in solving polynomial systems in robotics and vision suggests that this usually leads to smaller eigenproblems. The resultant matrix is regarded as a matrix polynomial in the hidden variable and finding its singular values and kernel vectors generalizes the u-polynomial construction of the previous section.

We use slightly different notation with $n+1$ expressing the number of input polynomials. Formally, given

$$f_1, \ldots, f_{n+1} \in K[x_1, x_1^{-1}, \ldots, x_{n+1}, x_{n+1}^{-1}] \tag{4.11}$$

we can view this system as

$$f_1, \ldots, f_{n+1} \in K[x_{n+1}][x_1, x_1^{-1}, \ldots, x_n, x_n^{-1}], \tag{4.12}$$

which is a system of $n+1$ Laurent polynomials in variables $x_1, \ldots, x_n$. Notice that we have multiplied all polynomials by sufficiently high powers of x_{n+1} in order to avoid dealing with denominators in the *hidden variable* x_{n+1}. This does not affect the system's roots in $(\overline{K}^*)^{n+1}$.

The sparse resultant of this system is a univariate polynomial in x_{n+1}. We show below how this formulation reduces the solution of the original well-constrained system to an eigenproblem. It is clear that our sparse resultant algorithms can be applied in this case without modification and that all properties of the constructed resultant matrices hold.

Theorem 4.4.1 *Assume that M is a sparse resultant matrix for (4.12), with the polynomial coefficients specialized. Let $\alpha = (\alpha_1, \ldots, \alpha_n) \in (\overline{K}^*)^n$ such that $(\alpha, \alpha_{n+1}) \in (\overline{K}^*)^{n+1}$ is a solution of $f_1 = \cdots = f_{n+1} = 0$. Then $M(\alpha_{n+1})$ is singular and column vector $w = [\alpha^{q_1}, \ldots, \alpha^{q_c}]$ lies in the right kernel of $M(\alpha_{n+1})$, where $\mathcal{E} = \{q_1, \ldots, q_c\} \subset \mathbb{Z}^n$ are the exponent vectors indexing the columns of M.*

Proof For specialized polynomial coefficients, $M(\alpha_{n+1})$ is singular by definition. Recall that right multiplication by a vector of the column monomials specialized at a point produces a vector of the values of the row polynomials at this point. Let $\{p_1, \ldots, p_c\}$ index the rows of M, then

$$M(\alpha_{n+1}) w = M(\alpha_{n+1}) \begin{bmatrix} \alpha^{q_1} \\ \vdots \\ \alpha^{q_c} \end{bmatrix} = \begin{bmatrix} \alpha^{p_1} f_{i_1}(\alpha, \alpha_{n+1}) \\ \vdots \\ \alpha^{p_c} f_{i_c}(\alpha, \alpha_{n+1}) \end{bmatrix} = \begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix} : i_1, \ldots, i_c \in \{0, 1, \ldots, n\}. \tag{4.13}$$

□

Computationally it is preferable to have to deal with as small a matrix as possible. To this end we partition M into four blocks so that the upper left submatrix M_{11} is square, nonsingular and independent of x_{n+1}. Row and column permutations do not affect the matrix properties so we apply them to obtain maximal M_{11}.

Gaussian elimination of the leftmost set of columns is now possible and expressed as matrix multiplication, where I is the identity matrix of appropriate size.

$$\begin{bmatrix} I & 0 \\ -M_{21}(x_{n+1})M_{11}^{-1} & I \end{bmatrix} \begin{bmatrix} M_{11} & M_{12}(x_{n+1}) \\ M_{21}(x_{n+1}) & M_{22}(x_{n+1}) \end{bmatrix} = \begin{bmatrix} M_{11} & M_{12}(x_{n+1}) \\ 0 & M'(x_{n+1}) \end{bmatrix}, \tag{4.14}$$

where

$$M'(x_{n+1}) = M_{22}(x_{n+1}) - M_{21}(x_{n+1})M_{11}^{-1}M_{12}(x_{n+1}).$$

Let $\mathcal{B} \subset \mathcal{E}$ index M'. We do not have *a priori* knowledge of the sizes of M_{11} and M' whether the subdivision or the incremental algorithm has constructed M.

Corollary 4.4.2 *Let* $\alpha = (\alpha_1, \ldots, \alpha_n) \in (\overline{K}^*)^n$ *such that* $(\alpha, \alpha_{n+1}) \in (\overline{K}^*)^{n+1}$ *is a common zero of* $f_1 = \cdots = f_{n+1} = 0$. *Then* $\det M'(\alpha_{n+1}) = 0$ *and, for any vector* $v' = [\cdots \alpha^q \cdots]$, *where* q *ranges over* $\mathcal{B}$, $M'(\alpha_{n+1})v' = 0$.

Proof Immediate by (4.13) and (4.14). □

To recover the root coordinates $\mathcal{B}$ must affinely span $\mathbb{Z}^n$, otherwise we have to compute the kernel vector of matrix M which equals the concatenation of vectors v and v', where v' is the kernel vector of M' and v is specified from (4.14):

$$\begin{bmatrix} M_{11} & M_{12}(\alpha_{n+1}) \\ 0 & M'(\alpha_{n+1}) \end{bmatrix} \begin{bmatrix} v \\ v' \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad \Rightarrow \quad M_{11}v + M_{12}(\alpha_{n+1})v' = 0$$

$$\Leftrightarrow \quad v = -M_{11}^{-1}M_{12}(\alpha_{n+1})v',$$

since M_{11} is defined to be the maximal nonsingular submatrix. The concatenation $[v, v']$ is indexed by $\mathcal{E}$ which always includes an affinely independent subset unless all n-fold mixed volumes are zero and no roots exist. In general the proof of lemma 4.3.1 recovers all root coordinates by taking ratios of the vector entries.

We now concentrate on matrix polynomials and their companion matrices; certain results may be found in [GLR82]. Denote the hidden variable by x, then the polynomial space is

$$f_i \in K[x][x_1, x_1^{-1}, \ldots, x_n, x_n^{-1}], \qquad i = 1, \ldots, n+1, \tag{4.15}$$

and we denote the univariate matrix $M'(x_{n+1})$ by $A(x)$. Let r be the size of A and $d \geq 1$ the highest degree of x in any entry. We wish to find all complex values for x at which matrix

$$A(x) = x^d A_d + x^{d-1}A_{d-1} + \cdots + xA_1 + A_0$$

becomes singular, where matrices $A_d, \ldots, A_0$ are all square, of order r and have numerical entries. We refer to $A(x)$ as a *matrix polynomial* with degree d and matrix coefficients A_i. The values of x that make $A(x)$ singular are its *eigenvalues*. For every eigenvalue λ, there is a basis of the kernel of $A(\lambda)$ defined

by the *right eigenvectors* of the matrix polynomial associated to λ. This is the eigenproblem for matrix polynomials, a classic problem in linear algebra.

If A_d is nonsingular then the eigenvalues and right eigenvectors of $A(x)$ are the eigenvalues and right eigenvectors of a *monic* matrix polynomial. Notice that this is the case with the u-resultant formulation in the previous section.

$$A_d^{-1}A(x) = x^d I + x^{d-1}A_d^{-1}A_{d-1} + \cdots + xA_d^{-1}A_1 + A_d^{-1}A_0,$$

where I is the $m \times m$ identity matrix. The *companion matrix* of this monic matrix polynomial is defined to be a square matrix C of order rd.

$$C = \begin{bmatrix} 0 & I & \cdots & 0 \\ \vdots & & \ddots & \\ 0 & 0 & \cdots & I \\ -A_d^{-1}A_0 & -A_d^{-1}A_1 & \cdots & -A_d^{-1}A_{d-1} \end{bmatrix}.$$

It is known that the eigenvalues of C are precisely the eigenvalues of the monic polynomial, whereas its right eigenvectors contain as subvectors the right eigenvectors of $A_d^{-1}A(x)$. We offer a brief demonstration of this fact.

Lemma 4.4.3 *Let $w = [v_1, \ldots, v_d] \in \overline{K}^{md}$ be a (nonzero) right eigenvector of C, where each $v_i \in \overline{K}^m$, $i = 1, \ldots, d$. Then v_1 is a (nonzero) right eigenvector of $A_d^{-1}A(x)$ and $v_i = \lambda^{i-1}v_1$, for $i = 2, \ldots, d$, where λ is the eigenvalue of C corresponding to w.*

Proof From the basic relation

$$\begin{bmatrix} 0 & I & \cdots & 0 \\ \vdots & & \ddots & \\ 0 & 0 & \cdots & I \\ -A_d^{-1}A_0 & -A_d^{-1}A_1 & \cdots & -A_d^{-1}A_{d-1} \end{bmatrix} \begin{bmatrix} v_1 \\ \vdots \\ v_{d-1} \\ v_d \end{bmatrix} = \lambda \begin{bmatrix} v_1 \\ \vdots \\ v_{d-1} \\ v_d \end{bmatrix}$$

it follows that $v_i = \lambda v_{i-1}$ for $i \geq 2$, hence $v_i = \lambda^{i-1}v_1$, for $i = 2, \ldots, d$. The last row of the relation gives

$$-A_d^{-1}A_0v_1 - A_d^{-1}A_1v_2 - \cdots - A_d^{-1}A_{d-1}v_d = \lambda v_d$$

thus, substituting for v_i,

$$-A_d^{-1}A_0v_1 - A_d^{-1}A_1\lambda v_1 - \cdots - A_d^{-1}A_{d-1}\lambda^{d-1}v_1 = \lambda^d v_1$$

which proves that v_1 is the right eigenvector of the matrix polynomial corresponding to λ. Clearly, v_1 cannot be the zero vector because otherwise w would be the zero vector, contradicting the hypothesis. □

We now address the question of a singular leading matrix in a non-monic polynomial. The following transformation is also applied in order to improve the conditioning of the leading matrix.

Lemma 4.4.4 *Assume matrix polynomial $A(x)$ is not identically singular for all x and let d be the highest degree in x of any entry. Then there exists a transformation $x \mapsto (t_1y + t_2)/(t_3y + t_4)$ for some $t_1, t_2, t_3, t_4 \in \mathbb{Z}$, that produces a new matrix polynomial $B(y)$ of the same degree and with matrices of the same dimension, such that $B(y)$ has a nonsingular leading coefficient matrix.*

Proof By hypothesis, the univariate polynomial $\det A(x)$ has only a finite number of complex roots. Multiply by $(t_3 y + t_4)^d$ to obtain $B(y)$ with leading coefficient

$$t_1^d A_d + t_1^{d-1} t_3 A_{d-1} + \cdots + t_3^d A_0.$$

Now fix $t_3 = 1$, then the matrix polynomial above is singular exactly when t_1 takes those values of x that make $A(x)$ singular. Hence, it suffices for t_1 to avoid a finite number of complex values. Similarly, t_4 must avoid a finite number of singular values for y. □

It is easy to see that the resulting polynomial $B(y)$ has matrix coefficients of the same rank, for sufficiently generic scalars t_1, t_2, t_3, t_4, since every matrix is the sum of $d+1$ scalar products of A_i. Thus this transformation is often referred to as *rank balancing*.

Theorem 4.4.5 *If the values of the hidden variable in the solutions of $f_1 = \cdots = f_{n+1} = 0$, which correspond to eigenvalues of the matrix polynomial, are associated to eigenspaces of unit dimension and $A(x)$ is not identically singular then we can reduce root-finding to an eigenproblem and some evaluations of the input polynomials at candidate roots.*

Proof By the previous lemma the new matrix polynomial $B(y)$ has a nonsingular leading coefficient and by lemma 4.4.3 finding its eigenvalues and eigenvectors is reduced to an eigenproblem of the companion matrix C. By hypothesis, the eigenspaces of C are one-dimensional therefore for every root there is an eigenvector that yields n coordinates of the root by lemma 4.3.1. The associated eigenvalue may be either simple or multiple and yields the value of the hidden variable at the same root. Notice that extraneous eigenvectors and eigenvalues may have to be rejected by direct evaluation of the input polynomials at the candidate roots and a zero test. The right eigenvectors of $B(y)$ are identical to those of $A(x)$ but any eigenvalue λ of the former yields $(t_1\lambda + t_2)/(t_3\lambda + t_4)$ as an eigenvalue of $A(x)$. □

The condition that all eigenspaces are unit-dimensional is equivalent to the solution coordinate at the hidden variable having unit geometric multiplicity. For this it suffices that the algebraic multiplicity of these solutions is one i.e., all hidden coordinates are distinct. Since there is no restriction in picking which variable to hide, it is enough that one out of the original $n+1$ variables has unit geometric multiplicity. If none can be found, we can use each value of the hidden variable to solve the resulting $n \times n$ system.

Asymptotic Complexity

Our complexity bounds shall occasionally ignore the logarithmic terms; this is expressed by the use of $\mathcal{O}^*(\cdot)$.

Let μ the maximum number of monomials in any (n-variate) polynomial of (4.12) and d the maximum polynomial degree in a single variable; this is typically larger than the highest degree of the hidden variable but in the worst case they are equal. For analyzing the bit complexity analysis we would consider c and β, respectively, to be the largest polynomial coefficient and the largest eigenvalue or eigenvector entry.

Inverting M_{11} costs $MM(|\mathcal{E}| - r)$. Computing matrix $X = M_{11}^{-1} M_{12}$ costs $\mathcal{O}((|\mathcal{E}| - r)^2 r d)$ and computing $M' = M_{22} - M_{21} X$ costs $\mathcal{O}((|\mathcal{E}| - r) r^2 d)$, where the factor d in both cases is due to the fact that these matrices have polynomial entries in the hidden variable of degree d and $2d$ respectively. Inversion of A_d costs $MM(r)$ and computation of the coefficients $A_d^{-1} A_i$ costs $MM(r)d$. This complexity

asymptotically dominates the quadratic cost of transforming $A(x)$ to $B(y)$ with nonsingular leading coefficient. The total complexity so far is bounded by $\mathcal{O}(d|\mathcal{E}|^3)$.

Computing the eigenvalues and eigenvectors of companion matrix C takes $MM(dr)$ steps and computing at most n entries of $v_\alpha = Xv'_\alpha$ takes $\mathcal{O}(nr)$ steps. The latter operation is dominated by the recovering of the candidate roots which costs, together with evaluation at all polynomials, $\mathcal{O}^*(drMM(n)+drn^2\mu)$. This leads to

Lemma 4.4.6 *Suppose that the ideal of (4.12) is zero-dimensional, the coordinates of the hidden variable are distinct, matrix M has been computed such that M_{11} is nonsingular and the resulting matrix polynomial $A(x)$ is regular. In addition, a set of affinely independent points in $|\mathcal{E}|$ has been computed. Then all common isolated zeros are computed with worst-case complexity*

$$\mathcal{O}^*(|\mathcal{E}|^3 d + MM(rd) + rdn^2\mu) = \mathcal{O}^*(|\mathcal{E}|^3 d^3 + |\mathcal{E}|dn^2\mu).$$

Proof To arrive at the arithmetic complexity bounds we apply $r \leq |\mathcal{E}|$ and $n \leq \mu \leq |\mathcal{E}|$. □

We now make some reasonable assumptions about the input that further simplify the above formulae. Eventually we shall impose a constraint on the Newton polytopes, namely that the scaling factor s of the overconstrained system, defined in section 2.2, is constant. Thus we derive a bound on $|\mathcal{E}|$ in terms of the sparse resultant total degree D which equals the sum of all n-fold mixed volumes.

Theorem 4.4.7 *Let s denote the scaling factor of (4.12), d the maximum degree in a single variable, D the total degree of the sparse resultant and $m = MV(f_1, \ldots, f_{n+1})$. Under the hypotheses of the previous lemma, if $n^2 = \mathcal{O}(|\mathcal{E}|)$ then the worst-case arithmetic complexity of computing all common roots of (4.11) is*

$$\mathcal{O}(|\mathcal{E}|^3 d^3) \qquad or \qquad 2^{\mathcal{O}(n)}\mathcal{O}(m^6) \quad if\ s = \mathcal{O}(1),\ D = \mathcal{O}(nm).$$

Proof We apply the results of the previous lemma and $n \leq \mu \leq |\mathcal{E}|$; moreover we use theorem 2.2.5 to exploit the hypothesis on the geometry of the Newton polytopes. □

It is clear by the first bound on arithmetic complexity that the most expensive part is the eigendecomposition of the companion matrix. Moreover, the problem of root-finding is exponential in n as expected.

Singular Matrix Polynomials

The second assumption requires that $A(x)$ be nonsingular for generic x. For specialized coefficients this is not always the case, as illustrated by the Stewart platform system in the next chapter. A first step to remedy this problem is to linearize the problem by introducing the *companion polynomial* $C(x)$ of order rd. Linear matrix polynomials are called *pencils*.

Lemma 4.4.8 *Let $A(x) = x^d A_d + x^{d-1}A_{d-1} + \cdots + xA_1 + A_0$ and let*

$$C(x) = \begin{bmatrix} I & & & \\ & \ddots & & \\ & & I & \\ & & & A_d \end{bmatrix} x + \begin{bmatrix} 0 & -I & & \\ & & \ddots & \\ & & & -I \\ A_0 & A_1 & \cdots & A_{d-1} \end{bmatrix}.$$

For every eigenvalue λ with associated right eigenvector $w = [v_1, \ldots, v_d]$ of $C(x)$, $v_i = \lambda^{i-1}v_1$ for $i = 2, \ldots, d$ and $A(x)$ has the same eigenvalue λ and has right eigenvector v_1.

Proof By the definition of eigenvectors, the first $(d-1)r$ rows, where r is the dimension of each A_i, imply

$$\lambda v_i - v_{i+1} = 0 \Rightarrow v_{i+1} = \lambda v_i \Rightarrow v_{i+1} = \lambda^i v_1, \qquad i = 1, \ldots, d-1.$$

The last row of $C(\lambda)w = 0$ now gives

$$A_d \lambda v_d + A_0 v_1 + \cdots + A_{d-1} v_d = 0 \Rightarrow A_d \lambda^d v_1 + A_{d-1} \lambda^{d-1} v_1 + \cdots + A_0 v_1 = 0.$$

Clearly $v_1 \neq 0$ because otherwise $w = 0$ contradicting the hypothesis. □

By lemma 4.4.4, C_1 is singular under any transformation $x \mapsto (t_1 y + t_2)/(t_3 y + t_4)$. In particular for $t_1 = t_4 = 0$, $t_2 = t_3 = 1$ and $y \neq 0$ the polynomial becomes $C(y) = C_0 y + C_1$ with $\det C_0 = 0$ hence solving for $C(x)$ and $C(y)$ are equivalent problems.

Linear algebra supplies powerful tools for analyzing the structure of singular pencils. One approach relies on the fact that there exist square orthogonal matrices P and Q such that $P(C_1 x + C_0)Q$ is in generalized Schur form i.e., the leading matrix is upper triangular and the trailing one is upper quasi-triangular. The eigenvalues of the new matrix are the desired ones, whereas for every eigenvector v of $C(x)$ the new matrix has eigenvector vP. We may, therefore, *deflate* the new matrix and solve for its eigenvalues and eigenvectors, which are strictly less than rd. This is a well-defined problem and yields the common roots of the original system [GL89, ch. 7].

Another cubic method with lower practical complexity yet numerically less stable is based on the Kronecker canonical form [Wil78] which generalizes the Jordan form of numeric matrices. There exist square nonsingular matrices P, Q such that $P(C_1 x + C_0)Q$ is null except for the following four kinds of diagonal blocks.

$$J_1 = \begin{bmatrix} x-\lambda & 1 & & \\ & \ddots & \ddots & \\ & & \ddots & 1 \\ & & & x-\lambda \end{bmatrix}, \qquad J_2 = \begin{bmatrix} 1 & x & & \\ & \ddots & \ddots & \\ & & \ddots & x \\ & & & 1 \end{bmatrix},$$

$$J_3 = \begin{bmatrix} x & & \\ 1 & \ddots & \\ & \ddots & x \\ & & 1 \end{bmatrix}, \qquad J_4 = \begin{bmatrix} x & 1 & & \\ & \ddots & \ddots & \\ & & x & 1 \end{bmatrix}.$$

Blocks of type J_1 are Jordan blocks corresponding to finite eigenvalue $\lambda \in \overline{K}$. The J_2 block is always present, implying that $\det C_1 = 0$, for it corresponds to an infinite eigenvalue of x or to a zero eigenvalue of $C_0 y + C_1$. The two blocks of type J_3 and J_4 have size, respectively, $(l+1) \times l$ and $l \times (l+1)$, $l \geq 0$, and capture the singularity of the pencil. The eigenvectors of $P(C_1 x + C_0)Q$ can be computed in a decoupled fashion by considering four types of subproblems and the associated four subvectors. Clearly, the subvector of interest corresponds to J_1 blocks, for which a well-defined eigenproblem is posed. Among these eigenvalue and eigenvector pairs there exist those associated to the isolated roots of (4.15).

The decompositions suggested here do not affect the overall asymptotic complexity since they are both cubic in the matrix dimension but do increase significantly the practical complexity. Efficient strategies combining speed and numerical stability are currently being explored; see also [MZW94].

4.5 Implementation and Numerical Issues

The present approach has been implemented by the author with the goal to provide, together with an eigenvalue solver, a self-contained and fast polynomial solver. It is intended to be part of the algebraic toolkit built by the author and A. Rege under the supervision of J. Canny [Can94]. One advantage over previous algebraic as well as numerical methods is that the resultant matrix need only be computed once for all systems with the same set of exponents. So this step can often be done offline, while the eigenvalue calculations to solve the system for each coefficient specialization are online.

Another offline operation is to permute rows and columns of the given resultant matrix in order to create a maximal square submatrix at the upper left corner which is independent of the hidden variable. In order to minimize the computation involving polynomial entries and to reduce the size of the upper left submatrix that must be inverted, the program concentrates all constant columns to the left and within these columns permutes all zero rows to the bottom.

To find the eigenvector entries that will allow us to recover the root coordinates it is typically sufficient to examine $\mathcal{B}$ indexing M_{22} and search for pairs of entries corresponding to exponent vectors v_1, v_2 such that $v_1 - v_2 = (0 \ldots, 0, 1, 0, \ldots, 0)$. This will let us compute the i-th coordinate, if the unit appears at the i-th position. For inputs where we need to use points in $\mathcal{E} \setminus \mathcal{B}$, only the entries indexed by these points must be computed in vector v_α. In any case the complexity of this step is dominated by the others.

A more detailed presentation of the implementation of the online solver follows. The input is the set of all supports, the associated coefficients and matrix M whose rows and columns are indexed by monomial sets. The output is a list of candidate roots and the values of each at the given polynomials.

The test for singularity of the matrix polynomial is implemented by substituting a few random values in the hidden variable and testing whether the resulting numeric matrix has full rank. For rational coefficients this is done in modular arithmetic so the answer is exact with very high probability.

Most of the online computation is numeric for reasons of speed; in particular we use double precision floating point numbers. We use the LAPACK package [ABB+92] because it implements state-of-the-art algorithms, including block algorithms on appropriate architectures, and provides efficient ways for computing condition numbers and error bounds. Moreover it is publicly available, portable, includes a C version and is soon to include a parallel version for shared-memory machines. Of course it is always possible to use other packages such as EISPACK [SBD+76] or LINPACK [BDMS79].

A crucial question in numerical computation is to predict the extent of roundoff error; see for instance [GL89]. To measure this we make use of the standard definition of matrix norm $\|A\|_p$ for matrix A in order to define the condition number

$$\kappa_p(A) = \|A\|_p \|A^{-1}\|_p, \quad p = 1, 2, \infty, F, \quad A \text{ square}, \qquad \text{and if } A \text{ singular } \kappa_p(A) = \infty,$$

where F denotes the Frobenius norm. For instance $\kappa_2(A)$ equals the ratio of the maximum over the minimum singular value of A. For a given matrix, the ratio of any two condition numbers with $p = 1, 2, \infty, F$ is bounded above and below by the square of the matrix dimension or its inverse. We also use $\|v\|_p$ for the p-norm of vector v.

As precise condition numbers are sometimes expensive to compute, LAPACK provides approximations of them and the associated error bounds. These computations are very efficient compared to the principal computation. These approximations run the risk of underestimating the correct values, though this happens extremely seldom in practice. Error bounds are typically a function of the condition number and the matrix dimension; a reasonable assumption for the dependence on the latter is

to use a linear function. Small condition numbers indicate well-behaved or *well-conditioned* matrices while orthogonal matrices are perfectly conditioned with $\kappa = 1$; large condition numbers characterize *ill-conditioned* matrices.

After permuting rows and columns so that the maximal upper left submatrix is independent of u or the hidden variable x, we apply an LU decomposition with column pivoting to the upper left submatrix. We wish to decompose the maximal possible submatrix so that it has a reasonable condition number. The maximum magnitude of an acceptable condition number can be controlled by the user; dynamically, the decomposition stops when the pivot takes a value smaller than some threshold.

To compute M' we do not explicitly form M_{11}^{-1} but use its decomposition to solve linear problem $M_{11}X = M_{12}$ and then compute $M' = M_{22} - M_{21}X$. Different routines are used, depending on $\kappa(M_{11})$, to solve the linear problem, namely the slower but more accurate `dgesvx` function is called when this condition number is beyond some threshold. Let $\widehat{x}_j$ denote some column of X and let x_j be the respective column if no roundoff error were present. Then the error is bounded [GL89] by

$$\frac{\|x - \widehat{x}\|_\infty}{\|x\|_\infty} \leqq 4 \cdot 10^{-15} (|\mathcal{E}| - r) \kappa_\infty(M_{11}),$$

where $|\mathcal{E}| - r$ is the size of M_{11} and we have used $2 \cdot 10^{-16}$ as the machine precision under the IEEE double precision floating point arithmetic.

For nonsingular matrix polynomials $A(x)$ we try a few random integer (t_1, t_2, t_3, t_4) quadruples in order to lower the probability that a less favorable quadruple may be chosen. We then redefine the matrix polynomial to be the one with lowest $\kappa(A_d)$. This operation of *rank balancing* is indispensable when the leading coefficient is nonsingular as well as ill-conditioned. Empirically we have observed that for matrices of dimension larger than 200, at least two or three quadruples should be tried since a lower condition number by two or three orders of magnitude is sometimes achieved. In any case the asymptotic as well as practical complexity of this stage is dominated by the other stages.

If we manage to find a matrix polynomial with well-conditioned A_d we compute the equivalent monic polynomial and call the standard eigendecomposition routine. There are again two choices in LAPACK for solving this problem with an iterative or a direct algorithm, respectively implemented in routines `hsein` and `trevc`. Experimental evidence points to the former as being faster on problems where the matrix size is at least 10 times larger than the mixed volume, since an iterative solver can better exploit the fact that we are only interested in real eigenvalues and eigenvectors.

If A_d is ill-conditioned for all linear t_i transformations we build the matrix pencil and call the *generalized eigendecomposition* routine `dgegv` to solve $C_1 x + C_0$. The latter returns pairs (α, β) such that matrix $C_1\alpha + C_0\beta$ is singular. For every (α, β) pair there is a nonzero right generalized eigenvector. For nonzero β we obtain the eigenvalues as α/β, while for zero β and nonzero α the eigenvalue tends to infinity and depending on the problem at hand we may or may not wish to discard it. The case $\alpha = \beta = 0$ occurs if and only if the pencil is identically zero within machine precision [GL89], thus providing a runtime check on the overall accuracy.

In recovering the eigenvector of matrix polynomial $A(x)$ from the eigenvector of its companion matrix, we can use any subvector of the latter. We choose the topmost subvector when the eigenvalue is smaller than the unit, otherwise we use one of the lower subvectors. Nothing changes in the algorithm, since ratios of the entries will still yield the root, yet the stability of these ratios is improved.

There are certain properties of the problem that have not been exploited yet. Typically, we are interested only in real solutions. We could concentrate, therefore, on the real eigenvalues and eigenvectors and choose the algorithms that can distinguish between them and complex solutions at the earliest stage. Moreover, bounds on the root magnitude may lead to substantial savings, as exemplified in [Man94a].

Chapter 5

Kinematic Applications

The techniques discussed so far find their natural application in problems arising in a variety of fields and modeling geometric and kinematic constraints. As an empirical observation, polynomial systems encountered in robot and molecular kinematics, motion and aspect ratio calculation in vision and the design of mechanisms are characterized by low mixed volume compared to their Bezout bound. Moreover, their sparse structure appears to be relevant to our methods, as opposed to Gröbner bases or homotopies.

We describe in detail three problems from different areas and conclude with pointers to further applications. The first problem, analyzed and solved in section 5.1, is a standard problem from computer vision, where information is available about a static scene seen by two positions of a camera, and the camera motion must be computed. When the minimum amount of information is available so that the problem is solvable, an optimal sparse resultant matrix is constructed and the numerical answers computed efficiently and accurately by our implementation.

The second application, in section 5.2, comes from computational biology and reduces to an inverse kinematics problem. The symmetry of the molecule at hand leads to high multiplicity of the common zeros, which leads us to compare the two approaches of defining an overconstrained system, either by adding a u-polynomial or by hiding one of the input variables. Both methods are used for various instances in order for all roots to be calculated accurately.

Lastly, section 5.3 discusses our progress towards solving the forward kinematics of the Stewart platform in real time, which is arguably "the major outstanding problem in all of manipulator direct and inverse kinematics" [Rot93]. Our resultant-based approach offers the first closed form of the solutions. Different algebraic formulations are suggested to compute the roots and their advantages and drawbacks considered.

5.1 Camera Motion from Point Matches

Camera motion reconstruction, or *relative orientation*, in its various forms is a basic problem in photogrammetry. We are interested in the problem of computing the displacement of a camera between two positions in a static environment. Given are the coordinates of certain points in the two views under perspective projection on calibrated cameras. This is a fundamental question in navigating robots equipped with a camera or in commanding mechanical manipulators using visual information. If two cameras photograph the same scene, the solution is the translation and rotation between the two cameras. When the distance between the cameras is known, this problem provides information about the shape

of the object by computing the 3-dimensional coordinates of the points with respect to the two frames of reference.

Equivalently, this problem consists in computing the displacement of a rigid body, whose identifiable features include only points, between two snapshots taken by a stationary camera. We consider, in particular, the case where the minimum number of 5 point matches is available. In this case the algebraic problem reduces to a well-constrained system of polynomial equations and we are able to give a closed-form solution.

Typically, computer vision applications use at least 8 points in order to reduce the number of possible solutions to 3 and, for generic configurations, to 1. In addition, computing the displacement reduces to a linear problem and the effects of noise in the input can be diminished [LH81, May85, LH88, Hor91]. Our approach shows performance comparable to these methods and is well-suited for detecting and eliminating *outliers* in the presence of noise. These are data points which are so much affected by noise that they should not be taken into account. If noise is not significant, our approach would be a natural choice.

5.1.1 Problem Definition

The problem has attracted attention by several researchers, as evidenced by the references throughout the section. Different mathematical formulations of the problem can be found in [FM90, Hor86, Hor91]. To formalize, let orthonormal 3×3 matrix $R \in \mathrm{SO}(3, \mathbb{R})$ denote the rotation i.e. $R^T R = RR^T = I$ and $\det R = 1$. Let (column) vector $t \in \mathbb{R}^3$ denote the camera translation in the original frame of reference. The 5 points in the two images are (column) vectors $a_i, a_i' \in \mathbb{P}^3_{\mathbb{R}}$, for $i = 1, \ldots, 5$ in the first and second frame of reference respectively. Usually, the points are treated as vectors in $\mathbb{R}^3$; equations (5.6) and the discussion thereafter explain why the two viewpoints are essentially equivalent here. It is clear that the magnitude of t cannot be recovered, hence there are 5 unknowns, 3 defining the rotation and 2 for the translation. This gives a well-defined algebraic problem as long as the 5 constraints, one per point correspondence, are independent. For almost all point sets the constraints are independent and this is the case of interest here. Degenerate configurations can be constructed by forcing, say, all points to be coplanar.

Figure 5.1 shows one point A viewed by two camera positions, with a and a' being the respective images. C and C' are the optical centers, the retinal planes are $z = -1$ and $z' = -1$ and t is the translation of frame x, y, z to frame x', y', z'. The condition that a_i, a_i' be the coordinates of the same point seen from two different positions of the camera is equivalent to the coplanarity of vectors a_i, t and $Ra_i' + t$ which is a_i' with respect to the first frame. This condition is expressed as the vanishing of the *triple* product of these vectors:

$$a_i^T \left(t \times (Ra_i' + t)\right) = a_i^T \left(t \times Ra_i'\right) = 0, \qquad i = 1, \ldots, 5, \tag{5.1}$$

where $\times$ denotes cross product.

Demazure [Dem88] demonstrated that this polynomial system has at most 20 complex solutions, whereas simplified proofs of this fact can be found in [FM90, Hor91]. He also exhibited a set of points compatible with 10 real camera displacements, which proves the bound is tight since every solution R, t gives rise to another solution R', t, where R' is the composition of R and a rotation of 180° about t. This is due to the symmetry introduced by the algebraic formulation and is discussed below.

As already mentioned, there are two equivalent problems that are treated here. In particular, R, t is the motion of a camera in an otherwise static environment if and only if $R^{-1}, R^{-1}t$ is the rigid motion of a body between two positions seen by a fixed camera.

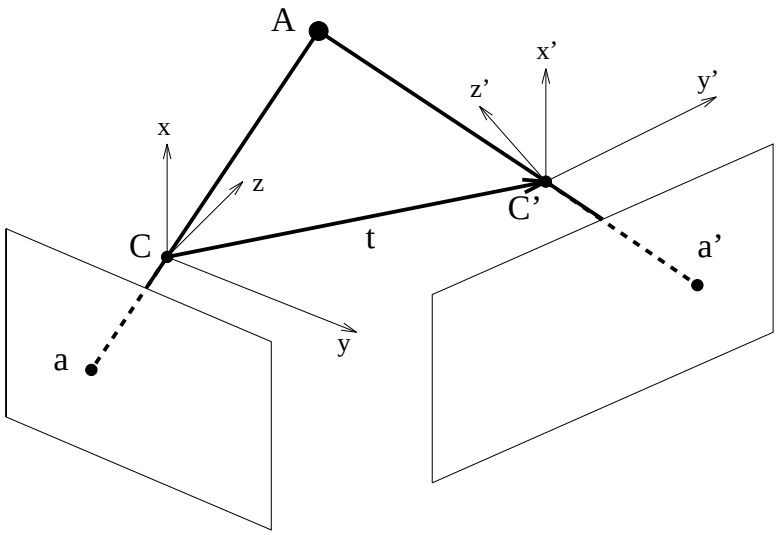

Figure 5.1: Single point seen by two camera positions.

5.1.2 Sparse Structure

This section represents a detour that illustrates a general technique for exploiting the structure of polynomial systems. The formulation here does not rely on the triple product above and turns out to be suboptimal, so it is abandoned in the following section. However, it is very interesting in our study of general and automatic techniques that bring out the structure of polynomial systems.

Surprisingly, we can make the structure manifest by *introducing new variables*. We assign these variables to the common subexpressions and replace each occurrence of the expression with the corresponding variable. The Bezout bounds will usually increase significantly as a result, but the Bernstein bounds decrease. This runs against the conventional wisdom for dealing with multivariate polynomials, which is to keep the number of variables as low as possible.

In [FM90, sect. 5.3] epipolar geometry is used to formalize the algebraic problem with unknowns (u_0, u_1, u_2) and (v_0, v_1, v_2) and inputs $e_i, e'_i, \delta_i, \delta'_i, \delta_{ij}, \delta'_{ij}$. This approach does not make use of the triple product above, but derives an equivalent system.

$$
\begin{gathered}
(u_2 - u_0)(v_2 - v_1) = (v_2 - v_0)(u_2 - u_1) \\
(u_2 e_2 - u_0 e_0)(v_2 e'_2 - v_1 e'_1) = (v_2 e'_2 - v_0 e'_0)(u_2 e_2 - u_1 e_1) \\
(\delta_{01} {u_2}^2 + \delta_{02} {u_1}^2 + 2\delta_0 u_2 u_1)(\delta'_2 v_0 v_1 - \delta'_{01} {v_2}^2 - \delta'_0 v_1 v_2 - \delta'_1 v_0 v_2) = \\
(\delta'_{01} {v_2}^2 + \delta'_{02} {v_1}^2 + 2\delta'_0 v_2 v_1)(\delta_2 u_0 u_1 - \delta_{01} {u_2}^2 - \delta_0 u_1 u_2 - \delta_1 u_0 u_2) \\
(\delta_{01} {u_2}^2 + \delta_1 2{u_0}^2 + 2\delta_1 u_2 u_0)(\delta'_2 v_0 v_1 - \delta'_{01} {v_2}^2 - \delta'_0 v_1 v_2 - \delta'_1 v_0 v_2) = \\
(\delta'_{01} {v_2}^2 + \delta_1 2p{v_0}^2 + 2\delta'_0 v_2 v_0)(\delta_2 u_0 u_1 - \delta_{01} {u_2}^2 - \delta_0 u_1 u_2 - \delta_1 u_0 u_2)
\end{gathered}
$$

The system is bihomogeneous in the two sets of variables (u_0, u_1, u_2) and (v_0, v_1, v_2). The main result of the paper [FM90] is that this system has 10 solutions. The Bernstein bound for the system as given is 18.

Inspection of the equations shows that there are common subexpressions in the last two equations. We can introduce new variables, x_1 and x_2, leading to an equivalent system including the first

two equations from above and

$$x_1 = (\delta_2' v_0 v_1 - \delta_{01}' {v_2}^2 - \delta_0' v_1 v_2 - \delta_1' v_0 v_2)$$
$$x_2 = (\delta_2 u_0 u_1 - \delta_{01} {u_2}^2 - \delta_0 u_1 u_2 - \delta_1 u_0 u_2)$$
$$(\delta_{01} {u_2}^2 + \delta_{02} {u_1}^2 + 2\delta_0 u_2 u_1) x_1 = (\delta_{01}' {v_2}^2 + \delta_{02}' {v_1}^2 + 2\delta_0' v_2 v_1) x_2$$
$$(\delta_{01} {u_2}^2 + \delta_{12} {u_0}^2 + 2\delta_1 u_2 u_0) x_1 = (\delta_{01}' {v_2}^2 + \delta_{12}' {v_0}^2 + 2\delta_0' v_2 v_0) x_2$$

which has mixed volume 12. Since Bernstein's theorem counts roots in $(\mathbb{C}^*)^n$, the introduction of x_1 and x_2 has forced the corresponding subexpressions to be nonzero for a valid solution. It was the vanishing of these expressions, together with the first two equations, that led to 6 spurious roots in the first system. In what follows we adopt a different approach that yields an optimal mixed volume.

5.1.3 Quaternion Formulation

The following quaternion formulation was independently suggested by J. Canny and is also proposed in [Hor91]. This allows a rational representation of rotations which is a promising approach for various problems in kinematics; a good reference is [McC90].

We shall make use of the following quaternion properties: If x, y, z are 3-vectors and $\dot{x} = [x_0, x], \dot{y} = [y_0, y], \dot{z} = [z_0, z] \in \mathbb{R}^4$ are arbitrary quaternions, then quaternion multiplication is defined through

$$\dot{x}\dot{y} = [x_0 y_0 - x^T y,\ x_0 y + y_0 x + x \times y]$$

and, if $\dot{z}^* = [z_0, -z]$ is the conjugate quaternion of $\dot{z}$

$$\dot{x}^T (\dot{y}\dot{z}) = (\dot{x}\dot{z}^*)^T \dot{y} \tag{5.2}$$

where quaternions are regarded as (column) 4-vectors, hence their inner product is well-defined.

Let quaternions $\dot{q} = [q_0, q]$, $\dot{t} = [t_0, t] \in \mathbb{R}^4$ represent the rotation and translation respectively. A rotation represented by angle ϕ and unit 3-vector s is expressed uniquely by quaternion $[\cos\phi/2, \sin\phi/2\, s]$, hence any rotation quaternion has unit *2-norm*, defined by

$$\dot{q}\,\dot{q}^* = \dot{q}^*\,\dot{q} = q_0^2 + q^T q = 1.$$

This may be regarded as the inner product of two 4-vectors. Conversely, a unit-norm quaternion, or simply unit quaternion, corresponds to two angle-axis representations, namely (ϕ, s) and $(-\phi, -s)$.

Vectors in 3-space map into quaternions with zero scalar part, therefore the scalar part of $\dot{t}$ vanishes:

$$t_0 = 0.$$

We introduce a new quaternion $\dot{d} = [d_0, d]$

$$\dot{d} \;=\; \dot{t}\dot{q} = [0, t][q_0, q] \in \mathbb{R}^4, \tag{5.3}$$

where $d \in \mathbb{R}^3$. One equation constrains $\dot{d}$ by requiring that the scalar part of $\dot{t}$ vanish:

$$\dot{t} = \frac{1}{\dot{q}\,\dot{q}^*}\,\dot{d}\,\dot{q}^* \;\Rightarrow\; (\dot{q}\,\dot{q}^*) t_0 \;=\; d_0\, q_0 - d^T q \Rightarrow d_0\, q_0 - d^T\, q = 0. \tag{5.4}$$

The latter equation guarantees that the quaternion recovered from $\dot{d}, \dot{q}$ via the first relation represents indeed a translation.

system	operation	CPU time
6×6	mixed volume	1m 16s
6×5	sparse resultant (offline)	12s
6×6 (first)	root finding (online)	0.2s (DEC ALPHA 3000)
6×6 (second)	root finding (online)	1s (SUN SPARC 20/61)

Table 5.1: Camera motion from point matches: all running times are measured on a DEC ALPHA 3000 except for the second system which is solved on a SUN SPARC 20/61.

If we let $\dot{a}_i = [0, a_i], \dot{a}'_i = [0, a'_i]$ for $i = 1, \ldots, 5$ be the vector quaternions representing the point coordinates, then $[0, Ra'_i] = \dot{q}\dot{a}_i\dot{q}^*$, where R is the rotation matrix from (5.1). By definition, $t \times Ra'_i$ is just the vector part of quaternion product $\dot{t}(\dot{q}\dot{a}'_i\dot{q}^*)$. Hence, equations (5.1) become

$$\dot{a}_i^T(\dot{t}(\dot{q}\dot{a}'_i\dot{q}^*)) = 0 \;\Leftrightarrow \dot{a}_i^T(\dot{d}\dot{a}'_i\dot{q}^*) = (\dot{a}_i\dot{q})^T(\dot{d}\dot{a}'_i) = 0, \qquad i = 1, \ldots, 5, \tag{5.5}$$

by the associativity of quaternion multiplication and (5.2).

Observe that the 6 equations (5.4,5.5) are *bilinear* in the two quaternions $\dot{q}, \dot{d}$ i.e. linear in each of the two sets of variables. Hence, we can dehomogenize by dividing by the product of a $\dot{q}$ and a $\dot{d}$ variable, say by $q_0 d_0$. What is the meaning of this division? The rotation quaternion is uniquely defined by the extra requirement that it has unit magnitude. For $\dot{d}$ no such requirement exists, which reflects the fact that $\dot{t}$ can be only determined up to a scalar factor. Consequently, we can fix one of the 4 parameters in each quaternion, and we choose

$$q_0 = 1, \qquad d_0 = 1.$$

This is equivalent to dividing by $q_0 d_0$ because the right hand side of equations (5.1) is zero. This leads to a well-constrained system of 6 dehomogenized polynomials in 6 variables organized in two 3-vectors. By abuse of notation, we denote these vectors by q, d.

$$\begin{aligned} (a_i^T q)(d^T a'_i) + a_i^T a'_i + (a_i \times q)^T a'_i + (a_i \times q)^T (d \times a'_i) + a_i^T (d \times a'_i) &= 0, \quad i = 1, \ldots, 5 \\ 1 - d^T q &= 0 \end{aligned} \tag{5.6}$$

These equations also demonstrate the bilinearity of the i-th polynomial in a_i, a'_i, for $i = 1, \ldots, 5$. It is possible then to divide throughout by the z-coordinate of a_i and the z'-coordinate of a'_i in order for the points to lie in projective space, as they will if they are obtained from actual images. In what follows we continue treating them as vectors in $\mathbb{R}^3$, keeping in mind this equivalence.

This system is well-constrained so we can apply Bernstein's bound to approximate the number of roots in $(\mathbb{C}^*)^6$. Generically, all roots are toric since zeroing one variable produces an overconstrained system. The mixed volume of this system is 20, which is an exact bound. The performance of our implementation on mixed volume is shown in table 5.1 for the DEC ALPHA 3000 of table 1.1. Notice that the system is multihomogeneous, as explained in section 3.3. The parameters have values $r = 2$, $n = 6$, all $d_{ij} = 1$ and

$$\text{the coefficient of } x_1^3 x_2^3 \text{ in } (x_1 + x_2)^6 \text{ is } \binom{6}{3} = 20.$$

So we obtain the same bound of 20.

5.1.4 Applying the Resultant Solver

System (5.6) is bilinear in $\{q_1, q_2, q_3\}$ and $\{d_1, d_2, d_3\}$. When we hide one variable, say q_1, in the coefficient field, the resulting overconstrained system of 6 polynomials is again bilinear in $\{q_2, q_3\}$ and $\{d_1, d_2, d_3\}$. For generic coefficients the first 5 supports in equations (5.6), are

$$\{(1,0,1,0,0),(1,0,0,1,0),(1,0,0,0,1),(1,0,0,0,0),$$
$$(0,1,1,0,0),(0,1,0,1,0),(0,1,0,0,1),(0,1,0,0,0),$$
$$(0,0,1,0,0),(0,0,0,1,0),(0,0,0,0,1),(0,0,0,0,0)\},$$

with respect to variables $(q_2, q_3, d_1, d_2, d_3)$. All monomials are extremal i.e., they correspond to vertices in Newton polytopes $Q_1, \ldots, Q_5$. The last equation of (5.6) has support

$$\{(1,0,0,1,0),(0,1,0,0,1),(0,0,1,0,0),(0,0,0,0,0)\}$$

again with all monomials extremal. $Q_1, \ldots, Q_5$ are precisely the Newton polytopes of a multigraded system of type $(2,3;1,1)$ for which exact matrix formulae exist as discussed in section 3.3. Q_6 deviates but this does not prevent our algorithm from constructing an optimal resultant matrix.

By theorem 3.3.2 we obtain vector $(5,5,2,2,2)$ on which to sort integer points; resultant matrix M is of dimension 60, while only 20 columns contain hidden variable q_1. 40×40 submatrix M_{11} is factorized and the resulting 20×20 pencil is passed to an eigendecomposition routine. The running times for the two main phases are reported in table 5.1. The rest of this section examines the accuracy of our procedure on two specific instances from [FM90]; we use error criterion

$$\sum_{i=1}^{5} \frac{1}{\|a_i\| \cdot \|a_i'\|} |a_i^T (t \times R a_i')|, \tag{5.7}$$

where $|\cdot|$ denotes absolute value. $\|\cdot\|$ is the vector 2-norm and t, R are the calculated translation and rotation. This expression vanishes when the solutions to R, t satisfy the coplanarity condition, otherwise it returns the absolute value of error normalized by the input norm.

First Instance

This example admits the maximum of 10 real solutions. Indeed, as shown by Demazure [Dem88], there are 10 solutions when

$$a_1 = a_1', a_2 = a_2', a_3 = a_3', a_4 = a_5', a_5 = a_4'.$$

Faugeras and Maybank slightly perturb these conditions to ensure genericity of the reconstructed scene and account for noise. They also impose

$$a_1^T a_2 = a_2^T a_3 = a_3^T a_1 = 0, \qquad a_1^T a_1 \simeq a_2^T a_2 \simeq a_3^T a_3.$$

The point coordinates are shown in table 5.2 and the results from our solver, to two or three decimals, in table 5.3. M_{11} is well-conditioned with $\kappa < 10^3$, so it is inverted to yield an optimal pencil of dimension 20. The latter has a well-conditioned leading coefficient with $\kappa < 10^4$, so it yields a monic polynomial on which the standard eigendecomposition is applied.

Recall that the translation is recovered only within a scalar factor. Additionally, pairs of solutions with opposite translation directions, e.g. solutions 1 and 14, differ by a rotation of 180° about

point	first frame	second frame
1	$(1000, 2000, 1000)$	$(1100, 1900, 900)$
2	$(1414, -1414, 1414)$	$(1314, -1514, 1314)$
3	$(-1732, 0, 1732)$	$(-1832, 100, 1632)$
4	$(2000, 1000, 3000)$	$(-1100, -900, 1900)$
5	$(-1000, -1000, 2000)$	$(2100, 1100, 2900)$

Table 5.2: Coordinates of matched points in first instance.

motion	rotation axis	rotation angle [°]	translation direction
1	$(0.697, 0.714, 0.073)$	179.23	$(0.325, -0.531, -0.783)$
2	$(-0.814, -0.578, 0.061)$	179.89	$(0.671, 0.726, 0.149)$
3	$(-0.561, -0.292, -0.774)$	177.01	$(0.545, 0.276, 0.792)$
4	$(0.415, 0.798, 0.438)$	177.34	$(0.419, 0.141, -0.897)$
5	$(0.693, -0.028, -0.720)$	174.93	$(0.627, 0.219, 0.748)$
6	$(-0.694, 0.030, 0.719)$	175.30	$(0.742, 0.138, 0.656)$
7	$(0.791, -0.555, 0.256)$	167.87	$(-0.419, -0.141, 0.897)$
8	$(0.248, 0.279, -0.928)$	176.85	$(-0.216, -0.277, 0.936)$
9	$(-0.684, -0.724, -0.084)$	171.77	$(0.487, -0.275, -0.829)$
10	$(-0.788, 0.558, -0.261)$	170.46	$(-0.420, -0.338, 0.842)$
11	$(-0.542, 0.588, -0.600)$	172.63	$(-0.487, 0.275, 0.829)$
12	$(-0.416, -0.801, -0.430)$	171.11	$(0.420, 0.338, -0.842)$
13	$(0.110, -0.983, 0.145)$	175.48	$(-0.742, -0.138, -0.656)$
14	$(0.534, -0.586, 0.610)$	155.74	$(-0.325, 0.531, 0.783)$
15	$(0.077, 0.170, -0.982)$	176.57	$(-0.068, -0.160, 0.985)$
16	$(-0.110, 0.984, -0.137)$	167.29	$(-0.626, -0.219, -0.748)$
17	$(-0.374, 0.149, 0.915)$	3.80	$(0.068, 0.160, -0.985)$
18	$(0.451, -0.555, 0.699)$	33.75	$(-0.671, -0.726, -0.149)$
19	$(-0.261, 0.552, 0.792)$	4.89	$(0.216, 0.277, -0.936)$
20	$(0.818, -0.392, 0.422)$	4.46	$(-0.545, -0.276, -0.792)$

Table 5.3: Real solutions of first instance.

point	first frame	second frame
1	$(1000, 2000, 1000)$	$(2232.04055, 134.92155, 1001)$
2	$(1414, -1414, 1414)$	$(-517.62265, -1931.54311, 1415)$
3	$(-1732, 0, 1732)$	$(-866.97555, 1499.98166, 1733)$
4	$(2000, 1000, 3000)$	$(1865.98044, -1232.09455, 3001)$
5	$(-1000, -1000, 2000)$	$(-1366.02976, 366.06377, 2001)$

Table 5.4: Coordinates of matched points in second instance.

	rotation axis	angle [°]	translation direction
1	$(-0.77894, -0.32703, -0.53507)$	145.66370	$(0.48803, 0.64380, 0.58936)$
2	$(-0.65776, 0.31415, -0.68459)$	136.86815	$(0.67599, 0.05269, 0.73503)$
3	$(0.59482, -0.68054, -0.42785)$	152.24447	$(-0.83075, 0.28440, 0.47851)$
4	$(0.67210, 0.39338, -0.62732)$	140.18127	$(-0.36236, -0.63645, 0.68090)$
5	$(-0.25655, 0.01373, -0.96643)$	121.67834	$(0.20008, 0.10167, 0.97449)$
6	$(0.31190, 0.23555, -0.92045)$	124.02482	$(-0.13458, -0.31792, 0.93852)$
7	$(0.04371, -0.60995, -0.79123)$	130.89260	$(-0.31197, 0.46065, 0.83095)$
8	$(0.01624, 0.00425, -0.99986)$	120.00554	$(-0.01035, -0.01021, 0.99989)$
9	$(-0.00249, -0.00106, 1.00000)$	59.92094	$(0.83075, -0.28440, -0.47851)$
10	$(-0.00186, 0.00182, 1.00000)$	60.08204	$(-0.48803, -0.64380, -0.58936)$
11	$(0.00036, 0.00025, 1.00000)$	59.98730	$(-0.67599, -0.05269, -0.73503)$
12	$(0.00031, -0.00036, 1.00000)$	60.00521	$(0.36236, 0.63645, -0.68090)$
13	$(-0.00029, -0.00012, 1.00000)$	60.00821	$(0.31197, -0.46065, -0.83095)$
14	$(0.00009, -0.00017, 1.00000)$	60.00004	$(0.13458, 0.31792, -0.93852)$
15	$(-0.00000, -0.00000, 1.00000)$	60.00146	$(0.01054, 0.00986, -0.99990)$
16	$(0.00001, 0.00016, 1.00000)$	60.00125	$(-0.20008, -0.10167, -0.97449)$

Table 5.5: Real solutions of second instance.

t. Only one of the two displacements is valid, while for the second one (sometimes called the twisted sister dual) the scene is behind the image plane, a condition which is not captured by the algebraic formulation. So both solutions represent one physical camera motion.

The maximum value of error criterion (5.7) is less than $1.3 \cdot 10^{-6}$, which is satisfactory. Only solutions $7, 10, 16$ are feasible in the sense that they reconstruct a 3-dimensional scene with all points in front of the camera for both views.

The input parameters are sufficiently generic to lead to the maximum number of real solutions. This explains the good conditioning of the matrices in the course of the program's execution. The next example looks at a less generic instance.

Second Instance

This is a nongeneric configuration as it has only 8 distinct real solutions that are, furthermore, clustered close together; this is manifest in the fact that matrix M_{11} is ill-conditioned. The matched point coordinates are constructed by applying a rotation of $60.00145558°$ about the z axis in the first frame

and a translation of $(.01, .01, -1)$. The point coordinates appear in table 5.4 and they are obtained by applying the above displacement to the original points of the first instance. The equations are obtained by rounding off the coefficients, since our current implementation reads in one-word integers; this proves sufficient.

Exactly the same procedure is used as before to produce a 60×60 resultant matrix, but now the upper leftmost 40×40 submatrix M_{11} has $\kappa > .6 \cdot 10^5$ so we choose not to invert it. Instead, the 60×60 pencil is considered: the leading matrix has $\kappa > 10^8$ in its original form and for any of 4 random transformations, hence it is not inverted and the generalized eigendecomposition is applied. The real solutions appear in table 5.5; a -0.00000 indicates a small negative value truncated to 5 decimal digits.

Applying error criterion 5.7, solution 8 yields the largest absolute value of $7.4 \cdot 10^{-5}$, hence all 16 solutions are acceptable. Again, they are paired into rigid motions with a common translation, within a scalar factor, and differing by a rotation of 180° about t, such as e.g. 1 and 10. The original motion is recovered as solution 15. There are another 4 complex solutions and 40 infinite ones. The total CPU running time for the online phase is, on the average, 1 second on the SUN SPARC 20/61 of table 1.1.

We have traded time for accuracy, as this problem leads to relatively ill-conditioned matrices compared to the first one. Of course, the approach of the first instance could have been applied to lead to a standard eigenproblem with higher speed, though less accuracy.

Discussion

It is interesting to observe what we have already mentioned from a theoretical standpoint in connection to roots with zero coordinates. In the second example our sparse resultant solver recovers roots in $\mathbb{C}^n \setminus (\mathbb{C}^*)^n$, namely we find camera motion 15 whose rotation quaternion $\dot{q}$ has $q_1 = q_2 = 0$. Such roots can be thought of as limits of roots in $(\mathbb{C}^*)^n$ as the system coefficients deform and approach some degenerate position. As long as the variety does not generically reside in $\mathbb{C}^n \setminus (\mathbb{C}^*)^n$, roots with zero coordinates will always be recovered. This is typically the case in practical applications. For the particular example, there is a stronger reason why all roots are recovered, namely all polynomials include a constant term.

Most implementations use more than the minimum number of point matches; the one in [FM90] is in MAPLE and is not intended for real-time applications. There exist various implementations of linear methods requiring at least 8 points, including [Hor91, Luo92]. We have been able to experiment with the latter which implements the least-squares method of [TH84], and found it faster on both instances but less accurate on the second one.

In particular, we choose the 10-th solution of the first instance and the 15-th of the second to generate an additional 3 matches for each of the problems above respectively. The CPU times on the two examples are 0.09 seconds and 0.06 seconds, respectively, on a SUN SPARC 20/61. On the first example (table 5.2) the output is accurate to at least 7 digits. On the second example, Luong's implementation returns a rotation of 60.0013° about $(-10^{-5}, 10^{-5}, 1)$ and a unit translation vector of $(-10^{-5}, -10^{-5}, -1)$ which differs significantly from t in solution 15, table 5.5.

5.2 Conformational Analysis of Cyclic Molecules

A relatively new branch of computational biology has been emerging as an effort to apply successful paradigms and techniques from geometry, robot kinematics and vision to predicting the structure of molecules, embedding them in euclidean space and finding the energetically favorable configurations [Con91, Hav91, SK91, SNW94, PC94]. The main premise for this interaction is the observation

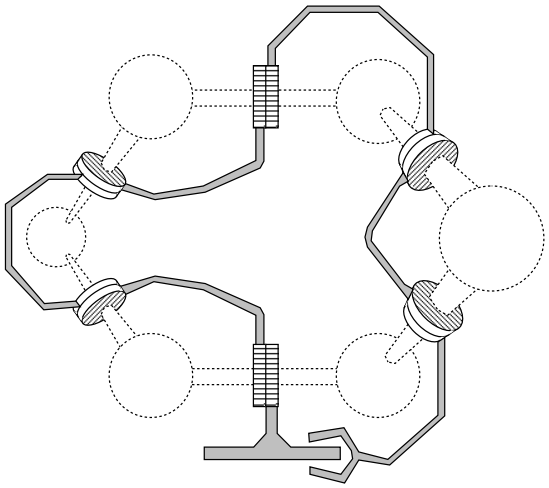

Figure 5.2: Correspondence of ring closure conformations to serial manipulator inverse kinematics.

that various structural requirements on molecules can be modeled as macroscopic geometric or kinematic constraints.

One example is the *docking problem* which examines the ways in which a ligand molecule attaches to a receptor site, see e.g. [SBK92]. If the docking match involves six pairs of atoms, then computing the ligand pose from the preferred bond distances is identical to the *forward kinematics* of the Stewart platform (see sect. 5.3).

This section examines the problem of computing all *conformations* of a *cyclic* molecule, which reduces to an *inverse kinematics* problem. Conformations specify the 3-dimensional structure of the molecule, thought of as a rigid body, modulo rotation and translation. Some authors reserve the term configuration for the ensemble of conformations during a time interval. It has been argued by Gō and Scherga [GS70] that energy minima can be approximated by allowing only the dihedral angles to vary, while keeping bond lengths and bond angles fixed. At a first level of approximation, therefore, solving for the dihedral angles under the assumption of *rigid geometry* provides information for the energetically favorable configurations.

The algebraic problem for six-atom molecules is well-constrained. Extending to molecules with more than six varying dihedrals has been approached by a grid search of the space of the two last dihedrals, each grid point giving rise to a six-dimensional subproblem [GS70, MZW94]. For molecules with six free dihedrals and an arbitrary number of atoms, the problem is identical to the inverse kinematics of a general serial manipulator with six rotational joints and has been solved in real time [MC92c].

We consider molecules of six atoms to illustrate our approach. Our contribution is to apply the theory of sparse elimination; we show that the corresponding algebraic formulation conforms to our model of sparseness. Our resultant solver is able to compute all solutions accurately even in cases where multiple solutions exist.

5.2.1 Algebraic Formulation

The molecule has a cyclic backbone of 6 atoms, typically of carbon. They determine primary structure, the object of our study. Carbon-hydrogen or other bonds outside the backbone are ignored. The bond lengths and angles provide the constraints while the six dihedral angles are allowed to vary.

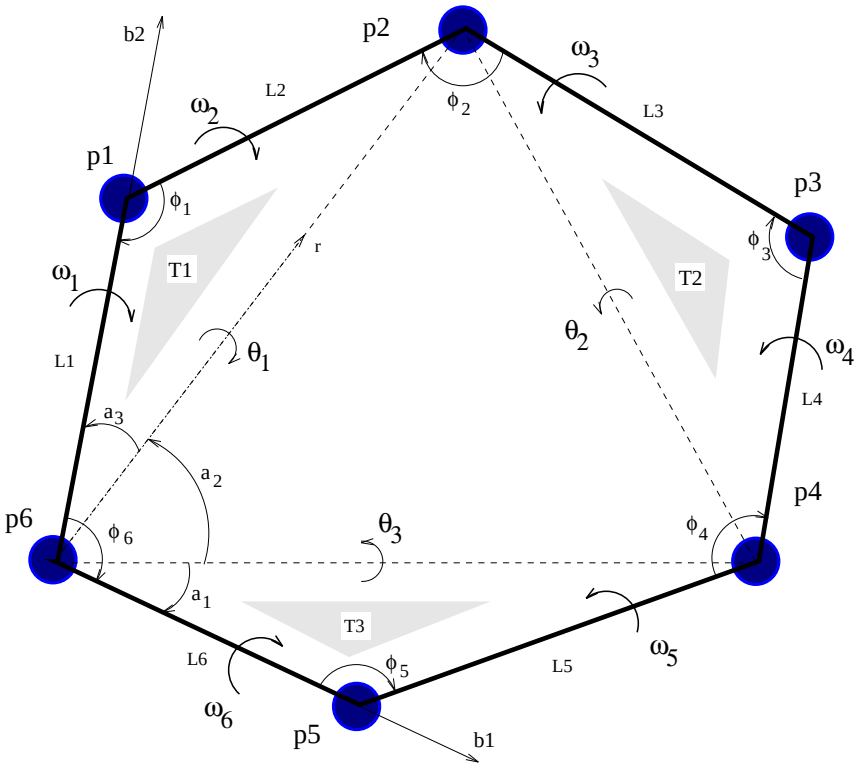

Figure 5.3: The cyclic molecule.

In kinematic terms, atoms and bonds are analogous to *links* and *joints* of a *serial mechanism* in which each pair of consecutive axes intersects at a link. This implies that the link offsets are zero for all six links which allows us to reduce the 6-dimensional problem to a system of 3 polynomials in 3 unknowns. The product of all link transformation matrices is the identity matrix, since the end-effector is at the same position and orientation as the base link. This analogy is illustrated in figure 5.2, courtesy of D. Parsons, where computing all conformations of the ring molecule is shown to be equivalent to the inverse kinematics of a serial manipulator with 6 rotational joints and known link geometry.

Given the link geometries we compute the rotation angles about the link axes. We adopt an approach proposed by D. Parsons [Par94]. Notation is defined in figure 5.3. Backbone atoms are regarded as points $p_1, \ldots, p_6 \in \mathbb{R}^3$; the unknown dihedrals are the angles $\omega_1, \ldots, \omega_6$ about axes (p_6, p_1) and (p_{i-1}, p_i) for $i = 2, \ldots, 6$. For readers familiar with the kinematics terminology, the Denavit-Hartenberg parameters α_i, d_i, a_i and θ_i equal respectively $180° - \phi_i$, L_i, zero and ω_i, according to the definition of [Cra89].

Each of triangles $T_1 = \triangle(p_1, p_2, p_6)$, $T_2 = \triangle(p_2, p_3, p_4)$ and $T_3 = \triangle(p_4, p_5, p_6)$ is fixed for constant bond lengths $L_1, \ldots, L_6$ and bond angles ϕ_1, ϕ_3, ϕ_5. Then the lengths of (p_2, p_6), (p_2, p_4) and (p_4, p_6) are constant hence *base* triangle $\triangle(p_2, p_4, p_6)$ is fixed in space, defining the xy-plane of a coordinate frame. Let θ_1 be the (dihedral) angle between the plane of $\triangle(p_1, p_2, p_6)$ and the xy-plane. Clearly, for any conformation θ_1 is well-defined. Similarly we define angles θ_2 and θ_3, as shown in figure 5.3. We call them *flap* (dihedral) angles to distinguish them from the bond dihedrals.

Conversely, given lengths L_i, angles ϕ_i for $i = 1, \ldots, 6$ and flap angles θ_i for $i = 1, \ldots, 3$ the coordinates of all points p_i are uniquely determined and hence the bond dihedral angles and the

associated conformation are all well-defined. For instance, dihedral ω_1 is given by

$$\omega_1 = s \ \arccos\left(\frac{v_{561}^T v_{612}}{\|v_{561}\| \ \|v_{612}\|}\right)$$
$$\text{where } v_{561} = (p_5 - p_6) \times (p_1 - p_6), \ v_{612} = (p_6 - p_1) \times (p_2 - p_1)$$
$$\text{and } s = \text{sign}\left((v_{561} \times v_{612})^T (p_1 - p_6)\right).$$

The sign function takes values in $\{-1, 0, 1\}$ and is positive when ω_1 is defined about direction $(p_1 - p_6)$ according to the right-hand rule; every p_i is assumed to be a position 3-vector. We have therefore reduced the problem to computing the three flap angles θ_i which satisfy the constraints on bond angles ϕ_2, ϕ_4, ϕ_6.

We show the explicit construction of the derivation for ϕ_6; the other two are analogous. Consider unit vectors b_1 and b_2 along rays (p_6, p_5) and (p_6, p_1), respectively. In the final conformation, on the plane of $\{p_1, p_5, p_6\}$, the following inner product expression holds:

$$(R_1 b_1)^T (R_2 b_2) = \cos\phi_6, \tag{5.8}$$

where R_1, R_2 express, respectively, rotation about (p_6, p_2) and (p_6, p_4) by θ_1 and θ_2. In particular, if we fix a coordinate frame at p_6 such that the x-axis coincides with (p_6, p_4) and the y-axis points upwards on the plane of $\{p_2, p_4, p_6\}$,

$$R_1(\theta_1) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta_1 & -\sin\theta_1 \\ 0 & \sin\theta_1 & \cos\theta_1 \end{bmatrix}$$

and

$$R_2(\theta_2) = \begin{bmatrix} \cos\theta_2 & r_x r_y(1-\cos\theta_2) & r_y \sin\theta_2 \\ r_x r_y(1-\cos\theta_2) & \cos\theta_2 & -r_x \sin\theta_2 \\ -r_y \sin\theta_2 & r_x \sin\theta_2 & \cos\theta_2 \end{bmatrix},$$

where $r = (r_x, r_y) = (\cos a_2, \sin a_2)$ is the unit vector on ray (p_6, p_2) and angles a_1, a_2, a_3 partition ϕ_6 as shown. Now constraint (5.8) expands to

$$\alpha_{21} + \alpha_{22}\cos\theta_1 + \alpha_{23}\cos\theta_2 + \alpha_{24}\cos\theta_1\cos\theta_2 + \alpha_{25}\sin\theta_1\sin\theta_2 = 0, \tag{5.9}$$

where the coefficients are

$$\begin{aligned} \alpha_{21} &= \cos a_1 \cos a_2 \cos a_3 - \cos\phi \\ \alpha_{22} &= \sin a_1 \sin a_2 \cos a_3 \\ \alpha_{23} &= -\cos a_1 \sin a_2 \sin a_3 \\ \alpha_{24} &= \sin a_1 \cos a_2 \sin a_3 \\ \alpha_{25} &= \sin a_1 \sin a_3. \end{aligned}$$

Coefficients α_{1i} and α_{3i} are computed analogously. Treating $\sin\theta_i$, $\cos\theta_i$ as independent variables we obtain polynomial system

$$\begin{aligned} \alpha_{11} + \alpha_{12}\cos\theta_2 + \alpha_{13}\cos\theta_3 + \alpha_{14}\cos\theta_2\cos\theta_3 + \alpha_{15}\sin\theta_2\sin\theta_3 &= 0, \\ \alpha_{21} + \alpha_{22}\cos\theta_3 + \alpha_{23}\cos\theta_1 + \alpha_{24}\cos\theta_3\cos\theta_1 + \alpha_{25}\sin\theta_3\sin\theta_1 &= 0, \\ \alpha_{31} + \alpha_{32}\cos\theta_1 + \alpha_{33}\cos\theta_2 + \alpha_{34}\cos\theta_1\cos\theta_2 + \alpha_{35}\sin\theta_1\sin\theta_2 &= 0, \\ \cos^2\theta_1 + \sin^2\theta_1 - 1 &= 0, \\ \cos^2\theta_2 + \sin^2\theta_2 - 1 &= 0, \\ \cos^2\theta_3 + \sin^2\theta_3 - 1 &= 0. \end{aligned} \tag{5.10}$$

This system has Bezout bound of 64 and mixed volume 16; the mixed volume is the exact number of complex roots generically as we shall prove below by demonstrating an instance with 16 real roots. Notice that 16 is also the exact number of solutions, generically, to the general inverse kinematics problem with 6 rotational joints [Pri86]. In other words, restricting the link offsets to zero does not simplify the algebraic problem.

For our resultant solver we prefer an equivalent formulation with a smaller number of polynomials, obtained by applying the standard transformation to half-angles that gives *rational* equations in the new unknowns t_i:

$$t_i = \tan\frac{\theta_i}{2} \;:\quad \cos\theta_i = \frac{1-t_i^2}{1+t_i^2},\ \sin\theta_i = \frac{2t_i}{1+t_i^2}, \qquad i = 1,2,3.$$

This transformation expresses automatically the last three equations in (5.10). By multiplying both sides of the i-th equation by $(1+t_j^2)(1+t_k^2)$, where (i,j,k) is a permutation in $S(1,2,3)$, the polynomial system becomes

$$\begin{aligned} f_1 = \beta_{11} + \beta_{12}t_2^2 + \beta_{13}t_3^2 + \beta_{14}t_2^2t_3^2 + \beta_{15}t_2t_3 &= 0 \\ f_2 = \beta_{21} + \beta_{22}t_3^2 + \beta_{23}t_1^2 + \beta_{24}t_3^2t_1^2 + \beta_{25}t_3t_1 &= 0 \\ f_3 = \beta_{31} + \beta_{32}t_1^2 + \beta_{33}t_2^2 + \beta_{34}t_1^2t_2^2 + \beta_{35}t_1t_2 &= 0 \end{aligned} \tag{5.11}$$

where

$$\begin{bmatrix} \beta_{i1} \\ \beta_{i2} \\ \beta_{i3} \\ \beta_{i4} \\ \beta_{i5} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 \\ 1 & -1 & 1 & -1 & 0 \\ 1 & 1 & -1 & -1 & 0 \\ 1 & -1 & -1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 4 \end{bmatrix} \begin{bmatrix} \alpha_{i1} \\ \alpha_{i2} \\ \alpha_{i3} \\ \alpha_{i4} \\ \alpha_{i5} \end{bmatrix} \qquad i = 1,2,3.$$

The new system has again Bezout bound of 64 and mixed volume 16.

5.2.2 Applying the Resultant Solver

This section examines three of the instances on which we have applied our solver. The input coefficients were computed as above by D. Parsons to include cases of varying randomness.

First Instance

This is a synthetic example for which we fix one feasible conformation with all flap angles equal to 90°; this corresponds to variables $t_1 = t_2 = t_3 = 1$. We shall see that root coordinates are repeated and our resultant solver can handle this. The bond lengths are all set to unity and the bond angles are set to one of two values:

$$\phi_1 = \phi_3 = \phi_5 = 2\pi/3 = 120^\circ, \qquad \phi_2 = \phi_4 = \phi_6 = 2\arcsin\left(\frac{\sqrt{3}}{4}\right) \simeq 51.32^\circ.$$

Notice that the bond lengths can be scaled without changing the problem. All polynomials are multiplied by 8 in order for the coefficients to be all integers, then β_{ij} is the (i,j)-th entry of matrix

$$\begin{bmatrix} -9 & -1 & -1 & 3 & 8 \\ -9 & -1 & -1 & 3 & 8 \\ -9 & -1 & -1 & 3 & 8 \end{bmatrix}.$$

The symmetry of the problem is bound to produce root coordinates of high multiplicity, so we decide to follow the first approach to solving the system (sect. 4.3) and add polynomial

$$f_0 = u + 31t_1 - 41t_2 + 61t_3$$

with randomly selected coefficients to obtain an overconstrained system. The alternative approach of hiding a variable could be used, as in the third instance below, to produce smaller matrices and thus lead to a faster solution. However, adding a u-polynomial distinguished the roots in a clearer fashion; in any case, it is interesting to compare results and performance for the two approaches.

We pick random vectors $(-5, 13, -91)$ and $(5, 3, 1)$ on which to sort the Minkowski sum integer points. We report the results from the first vector in detail; the second one gives equally accurate solutions. In the overconstrained system, the 3-fold mixed volumes are $12, 12, 12, 16$ hence the sparse resultant has total degree 52 and degree 16 in f_0. The resultant matrix is regular and has dimension 86, with 30 rows corresponding to f_0. This is the offline phase; the rest corresponds to the online execution of the solver.

The entire 56×56 upper left submatrix is decomposed and is relatively well-conditioned with $\kappa \simeq 10^2$. The leading matrix coefficient is singular within machine precision; two random transformations are used in order to produce a pencil with regular leading coefficient. Between the two new pencils, the best-conditioned leading matrix has $\kappa \simeq 3 \cdot 10^5$. It is still not considered to be well-conditioned therefore the generalized eigenproblem routine is called on the 30×30 pencil to produce 12 complex solutions, 3 infinite real solutions and 15 finite real roots which are evaluated on the original polynomials shown in table 5.6. Solution 1 is the one from which the example was constructed. The last two columns show the maximum and minimum absolute value of any of the four polynomials on the candidate values.

It is clear which candidates correspond to solutions; the true solutions computed by MAPLE V using Gröbner bases over the rationals are

$$\pm(1, 1, 1), \pm(5, -1, -1), \pm(-1, 5, -1), \pm(-1, -1, 5),$$

so our program computes the true roots to at least 5 digits. The interest of this example lies precisely in the degeneracy of the input coefficients that causes the roots to have repeated coordinates.

Transforming the solutions for the half-angle tangents to the solutions for the dihedral bond angles is straightforward; the values of the latter are shown in table 5.7. Notice that conformations appear in pairs of opposite dihedral angles, which is natural since for every conformation there is a symmetric one with the dihedrals negated. The average CPU time of the online phase on a SUN SPARC 20/61 is 0.4 second.

Conformations 1 and 2 are called *chair* conformations and are characterized by all flap triangles being on the same side of the base triangle. The rest are *boat* and *twisted boat* conformations; their difference is that only the former has the two flaps which lie on the same side of the base triangle at symmetrical positions.

Second Instance

Usually noise enters in the process that produces the coefficients; this example models this phenomenon. We consider the *cyclohexane* molecule which has 6 carbon atoms at equal distances and equal bond angles. Starting with the pure cyclohexane that has bond lenghts set to 1.526 Ångstrom and bond angles to 110.4°, we randomly perturb them by about 10% to obtain the input parameters in

conf.	t_1	t_2	t_3	u	max	min
1	1.0000	1.0000	1.0000	-51.00	$.4 \cdot 10^{-8}$	$.6 \cdot 10^{-9}$
2	-1.0000	-1.0000	-1.0000	51.00	$.2 \cdot 10^{-7}$	$.5 \cdot 10^{-8}$
3	-5.0000	1.0000	1.0000	135.00	$.3 \cdot 10^{-5}$	$.2 \cdot 10^{-6}$
4	5.0000	-1.0000	-1.0000	-135.00	$.2 \cdot 10^{-5}$	$.2 \cdot 10^{-6}$
5	-1.0000	5.0000	-1.0000	297.00	$.4 \cdot 10^{-4}$	$.6 \cdot 10^{-6}$
6	1.0000	1.0000	-5.0000	315.00	$.3 \cdot 10^{-4}$	$.1 \cdot 10^{-5}$
7	1.0000	-5.0000	1.0000	-297.00	$.6 \cdot 10^{-4}$	$.3 \cdot 10^{-6}$
8	-1.0000	-1.0000	5.0000	-315.00	$.3 \cdot 10^{-3}$	$.3 \cdot 10^{-4}$
9	$-1.2 \cdot 10^{4}$	$1.5 \cdot 10^{2}$	$-3.0 \cdot 10$	$3.7 \cdot 10^{5}$	$1.0 \cdot 10^{14}$	$8.0 \cdot 10^{3}$
10	$-4.6 \cdot 10^{4}$	$2.3 \cdot 10$	-2.8	$1.4 \cdot 10^{7}$	$3.0 \cdot 10^{12}$	$1.0 \cdot 10^{3}$
11	$4.6 \cdot 10^{4}$	$2.3 \cdot 10$	-2.8	$-1.4 \cdot 10^{7}$	$3.0 \cdot 10^{12}$	$1.0 \cdot 10^{3}$
12	$4.9 \cdot 10^{4}$	$2.6 \cdot 10$	-3.1	$-1.5 \cdot 10^{6}$	$4.0 \cdot 10^{12}$	$1.0 \cdot 10^{3}$
13	$-1.0 \cdot 10^{7}$	$1.2 \cdot 10$	$-6.5 \cdot 10^{-1}$	$3.1 \cdot 10^{8}$	$5.0 \cdot 10^{16}$	$4.0 \cdot 10$
14	$1.0 \cdot 10^{7}$	$1.3 \cdot 10$	$-6.9 \cdot 10^{-1}$	$-3.1 \cdot 10^{8}$	$5.0 \cdot 10^{16}$	7.0
15	$-2.0 \cdot 10^{10}$	$4.0 \cdot 10^{-1}$	$2.4 \cdot 10^{-1}$	$8.4 \cdot 10^{9}$	$3.0 \cdot 10^{20}$	8.0

Table 5.6: Candidate flap angles for the first problem.

conformation	$\omega_1[^\circ]$	$\omega_2[^\circ]$	$\omega_3[^\circ]$	$\omega_4[^\circ]$	$\omega_5[^\circ]$	$\omega_6[^\circ]$
1	106.10	-106.10	106.10	-106.10	106.10	-106.10
2	-106.10	106.10	-106.10	106.10	-106.10	106.10
3	-106.10	106.10	41.69	-106.10	106.10	-41.69
4	106.10	-106.10	-41.69	106.10	-106.10	41.69
5	-106.10	41.69	106.10	-106.10	-41.69	106.10
6	41.69	-106.10	106.10	-41.69	-106.10	106.10
7	106.10	-41.69	-106.10	106.10	41.69	-106.10
8	-41.69	106.10	-106.10	41.69	106.10	-106.10

Table 5.7: Dihedral angles for the first problem.

bond	length L_i [Å]	angle ϕ_i [°]
1	1.428180487	100.0532084
2	1.425160138	113.1588375
3	1.504810475	100.5419443
4	1.525644163	110.0976629
5	1.654323756	112.4040573
6	1.601390339	108.0302355

Table 5.8: Bond lengths and angles for the second problem.

conformation	t_1	t_2	t_3	max value
1	-0.712646432564	0.010384131235	0.623453274250	10^{-6}
2	0.368436394171	0.319725126853	0.296955936681	10^{-9}
3	-0.368436394171	-0.319725126853	-0.296955936681	10^{-9}
4	0.712646426008	-0.010384131256	-0.623453274250	10^{-5}

Table 5.9: Valid flap angles for the second problem.

conformation	$\omega_1[^\circ]$	$\omega_2[^\circ]$	$\omega_3[^\circ]$	$\omega_4[^\circ]$	$\omega_5[^\circ]$	$\omega_6[^\circ]$
1	-40.73	83.75	-40.05	-32.80	72.51	-26.00
2	63.57	-73.18	70.00	-57.98	60.30	-62.71
3	-63.57	73.18	-70.00	57.98	-60.30	62.71
4	40.73	-83.75	40.05	32.80	-72.51	26.00

Table 5.10: Dihedral angles for the second problem.

table 5.8, then multiply by 10^3 and round to the nearest integer to get β_{ij} as the entries of matrix

$$\begin{bmatrix} -310 & 959 & 774 & 1313 & 1389 \\ -365 & 755 & 917 & 1269 & 1451 \\ -413 & 837 & 838 & 1352 & 1655 \end{bmatrix}.$$

Interestingly, [GS73] proves that without noise this problem has an infinite number of solutions. We used the second approach to define an overconstrained system, namely by hiding variable t_3 in the coefficient field (sect. 4.4). For vectors $(3,1)$, $(1,1)$, $(100,1)$ and $(1,100)$ we obtained resultant matrices of dimension 16 in t_3 of quadratic degree, whereas the 2-fold mixed volumes are all 4 and the sparse resultant has degree $4+4+4=12$.

For the first vector, the matrix polynomial is regular and all rows and columns depend on the hidden variable, hence we have to deal with a 16×16 matrix polynomial. The leading coefficient is again singular, but a random transformation produces an equivalent polynomial whose leading coefficient has condition number $\kappa \simeq 10^3$ which is well-conditioned enough to invert. The monic quadratic polynomial reduces to a 32×32 companion matrix on which the standard eigendecomposition is applied. After rejecting false candidates, the recovered roots are shown in table 5.9, along with the maximum absolute value of the input polynomials at each one. We check the computed solutions against those obtained by a Gröbner bases computation over the integers and observe that each contains at least 8 correct digits.

The dihedral angles are shown in table 5.10. The total CPU time on a Sun Sparc 20/61 is 0.2 seconds on average for the online phase. Solutions 1 and 4 represent twisted boat conformations and solutions 2 and 3 represent chair conformations.

We have experimented with smaller amounts of perturbation on the pure cyclohexane parameters and have obtained similar qualitative results; in particular, chair conformations are always present. This observation verifies results from molecular biology indicating that this is indeed the favored conformation of cyclohexane.

Third Instance

Lastly we briefly report on an instance where the input parameters are sufficiently generic to produce 16 real roots. This input establishes a tight lower bound on the number of roots for the problem that matches the mixed volume computed earlier. We shall see that, since certain root coordinates are repeated, the user should explicitly examine the solutions in order to decide which coordinates make up the roots. To avoid this the approach used in the first instance may be used, although it is slower.

The β_{ij} coefficients are given by matrix

$$\begin{bmatrix} -13 & -1 & -1 & -1 & 24 \\ -13 & -1 & -1 & -1 & 24 \\ -13 & -1 & -1 & -1 & 24 \end{bmatrix}.$$

We achieve best stability by hiding t_3 and computing the roots that correspond to distinct values for t_3, then relying on the symmetry of the equations to derive the full set of 16 real roots. Alternatively, it is possible to add a u-polynomial as in the first example, yet this leads to larger matrices. Hiding t_3 yields a resultant matrix of dimension 16, quadratic in the hidden variable, whereas the sparse resultant has degree 12. The leading matrix coefficient has $\kappa \simeq 10^4$ for the second random transformation, leading to a companion matrix of dimension 32, which is solved by a standard eigendecomposition.

There are 16 real roots shown in table 5.11. For the triplets of roots where t_3 is fixed, we have filled in the t_1, t_2 coordinates by looking at the other solutions and relying on symmetry. For example, the coordinates of conformations 3-5 are specified by conformation 7 which implies that any permutation of $(.7795, .7795, 10.8577)$ is, approximately, a valid solution. Moreover, by the symmetry of the polynomials and the particular inputs, if $t_1 = t_2 = .7795$ make f_3 vanish, then $t_2 = t_3 = .7795$ is a zero of f_1 and $t_3 = t_1 = .7795$ is a zero of f_2. The solver does not produce any meaningful t_1, t_2 coordinates in these cases since the dimension of the eigenspace and the multiplicity of the eigenvalue is 3 (see sect. 4.4).

When all three coordinates are recovered, the maximum absolute value of the original polynomials is reported in the last column. The computed roots are correct to at least 7 decimal digits. The average CPU time of the online part is 0.2 seconds on a Sun Sparc 20/61.

5.3 Stewart Platform Kinematics

The Stewart platform, also called left hand, is a *parallel manipulator* with six prismatic joints connecting two rigid bodies, or platforms. The base platform is considered fixed while the top platform, or end-effector, is moving in 3-dimensional space, controlled by the lengths of the joints. The links are attached to the platforms by ball-joints that rotate freely. Parallel robots are especially useful when high stiffness and position precision are predominant requirements. Stewart [Ste65] introduced his platform to achieve these properties in flight simulation, while another fully parallel manipulator had already been proposed by Gough [Gou56]. Other applications exist including medical ones that rely on the accuracy of the mechanism to perform delicate operations [WS94].

The platform has one degree of freedom per joint; the position and orientation of the top platform is specified by six parameters, namely three for the orientation and three for the position in 3-space. Hence the problem is well-defined. In contrast to inverse kinematics, where the joint parameters must be computed from the end-effector position and orientation, this instance of *direct* or *forward kinematics* assumes that the leg lengths are known and the displacement of the top platform is to

conformation	t_1	t_2	t_3	max	$\theta_1[^\circ]$	$\theta_2[^\circ]$	$\theta_3[^\circ]$
1	4.625	4.625	0.332	10^{-6}	155.60	155.60	36.74
2	−4.625	−4.625	−0.332	10^{-6}	−155.60	−155.60	−36.74
3	0.780	0.780	0.780		75.88	75.88	75.88
4	0.780	10.858	0.780		75.88	169.48	75.88
5	10.858	0.780	0.780		169.48	75.88	75.88
6	−0.780	−0.780	−10.858	10^{-6}	−75.88	−75.88	−169.48
7	0.780	0.780	10.858	10^{-6}	75.88	75.88	169.48
8	−0.780	−0.780	−0.780		−75.88	−75.88	−75.88
9	−0.780	−10.858	−0.780		−75.88	−169.48	−75.88
10	−10.858	−0.780	−0.780		−169.48	−75.88	−75.88
11	4.625	4.625	4.625		155.60	155.60	155.60
12	4.625	0.332	4.625		155.60	36.74	155.60
13	0.332	4.625	4.625		36.74	155.60	155.60
14	−4.625	−4.625	−4.625		−155.60	−155.60	−155.60
15	−4.625	−0.332	−4.625		−155.60	−36.74	−155.60
16	−0.332	−4.625	−4.625		−36.74	−155.60	−155.60

Table 5.11: Candidate flap angles for the third problem.

be found. The algebraic problem reduces to the solution of a well-constrained system of polynomial equations and takes any of several forms, as we shall see below. An upper bound of 40 has been established in the past few years, which is exact because there exist particular instances with as many complex solutions.

Ronga and Vust [RV92] offered the first proof of 40 being an upper bound by applying intersection theory on vector bundles and using Chern classes. Simpler proofs were proposed in [Mou93, Laz93] and a probabilistic one in [Rag93]. A second proof by Mourrain [Mou94] views this problem as well as the computation of the camera motion from 5 point matches as special cases of a general problem on displacements. Wampler [Wam94] arrives at the upper bound by applying the multi-homogeneous Bezout bound to an appropriate polynomial system.

Different approaches have been examined in computing the displacement. A sample of work on special cases is [NWM90, NZAJ91, WLW], where one or both platforms are planar or some ball-joint attachments coincide. Faugère and Lazard [FL94] have used Gröbner bases on a number of cases, though running times for the general case are in the order of hours on a Sun Sparc 2. Innocenti and Parenti-Castelli [IPC91] arrive at a numerical solution by recasting the problem in inverse kinematics terms, for a mechanism of variable geometry. This allows them to decouple some of the variables and obtain a 4 or even 3-dimensional system, however of high degree. A fast numerical implementation by Wampler [Wam93] solves the general problem to satisfactory precision in about 14 seconds on an IBM RS/6000, after some preprocessing. Recently, the problem has been reduced to the solution of a univariate polynomial of degree 40 by Husty [Hus94]; however solution of such polynomials may be highly unstable. Although some of these approaches may lead to the fastest implementations for the particular problem, we are mostly interested in the performance of a general solver that may solve arbitrary kinematics applications in reasonable time.

5.3.1 Mixed Volume Formulation

This section describes an algebraic formulation of the forward kinematics that turns out to be most advantageous from the point of view of mixed volume. Again, we are interested in the performance of a mechanical solver that should be able to provide reasonable estimates for arbitrary problems exhibiting sparse structure. Ultimately, the case-specific transformations that go with generating the most amenable system will also be automated. This example serves as an important case study in this line of reseach.

Let $\dot{q} = [q_0, q_1, q_2, q_3] \in \mathbb{R}^4$ be the quaternion expressing the rotation, where q_0 is the scalar part and (q_1, q_2, q_3) is the vector part. In order to avoid having pairs of opposite values as solutions of $\dot{q}$ we set $q_0 = 1$ so $\dot{q} = [1, q_1, q_2, q_3]$. We introduce scalar variable

$$y_0 = 1 + q_1^2 + q_2^2 + q_3^2 \in \mathbb{R}^* \tag{5.12}$$

The unit quaternion expressing the rotation equals $\dot{q}/\sqrt{y_0}$.

Let the translation be represented by quaternion $\dot{t} = [0, t_1, t_2, t_3]$. Let $\dot{a}_i, \dot{b}_i$ represent the vector quaternions of the points at which joint i is attached at the bottom and top platform respectively. Points $\dot{a}_i$ and $\dot{b}_i$ are measured with respect to frames of reference attached at the stationary and the moving platforms respectively. If, for a minute, we forget the quaternion formulation and go back to 3-vectors t, a_i, b_i and rotation matrix R, the equations expressing lengths $L_1, \ldots, L_6$ become:

$$(-a_i + t + Rb_i)^T(-a_i + t + Rb_i) = L_i^2, \qquad i = 1, \ldots, 6.$$

It is clear that this is a well-constrained 6×6 system. Now in quaternion form the system is

$$(-\dot{a}_i y_0 + \dot{t} y_0 + \dot{q}\dot{b}_i\dot{q}^*)^T(-\dot{a}_i y_0 + \dot{t} y_0 + \dot{q}\dot{b}_i\dot{q}^*) = L_i^2 y_0^2, \qquad i = 1, \ldots, 6,$$

after multiplying both sides by y_0^2 in order to go from the unit rotation quaternions to $\dot{q}$. The system of 7 unknowns is well-constrained by adding equation (5.12).

In order to bring out the structure of the system we follow a strategy reminiscent of that in section 5.1.2. Intuitively we capture the dependence of certain subexpressions by introducing

$$\text{quaternion } \dot{z} = \frac{1}{y_0}\dot{q}^*\dot{t}\dot{q} \Rightarrow \dot{z} = [0, z_1, z_2, z_3] \in \mathbb{R}^4.$$

The implication is explained by the fact that $\dot{z}$ represents the rotated translation vector, hence it has zero scalar part.

In the new system of equations the first equation expresses the fact that the magnitude of the translation vector is constant. This follows from the first original constrain and the assumption, without loss of generality, that point $\dot{a}_1$ in the base platform and $\dot{b}_1$ in the top platform lie both at the respective origins: $\dot{a}_1 = \dot{b}_1 = 0$. Then

$$f_1 = \dot{t}^T\dot{t} - \alpha_1 = t_1^2 + t_2^2 + t_3^2 - \alpha_1,$$

where the $\alpha_1, \ldots, \alpha_6$ are scalar parameters determined by the inputs. In particular $\alpha_1 = L_1^2$, while $\alpha_2, \ldots, \alpha_6$ group all constant terms in the remaining polynomials.

The next 5 equations express the condition on the prismatic joint lengths between the other 5 points of the two platforms. By multiplying the polynomials by y_0 the roots are not altered and we obtain

$$f_i = \dot{b}_i^T\dot{z} - y_0\dot{a}_i^T\dot{t} - (\dot{q}\dot{b}_i\dot{q}^*)^T\dot{a}_i - y_0\alpha_i, \qquad i = 2, \ldots, 6,$$

The next three polynomials restrict z_1, z_2, z_3 by expressing $\dot{q}\dot{z} = \dot{t}\dot{q}$.

$$\begin{aligned} f_7 &= q_2 z_3 - q_3 z_2 + z_1 - (q_3 t_2 - q_2 t_3 + t_1), \\ f_8 &= q_3 z_1 - q_1 z_3 + z_2 - (q_1 t_3 - q_3 t_1 + t_2), \\ f_9 &= q_1 z_2 - q_2 z_1 + z_3 - (q_2 t_1 - q_1 t_2 + t_3). \end{aligned}$$

Lastly, variable y_0 is constraint by the vanishing of

$$f_{10} = \dot{q}^T \dot{q} - y_0 = 1 + q_1^2 + q_2^2 + q_3^2 - y_0.$$

The mixed volume of this 10×10 system is 54, computed by the Lift-Prune algorithm in 4 minutes, 53 seconds CPU time on the DEC ALPHA 3000 of table 1.1. Mixed volumes for the original 6-dimensional system, the 7-dimensional system of the next section as well as the 8-dimensional system mentioned in section 3.3 are, respectively, 160, 84 and 84. We can reiterate the moral of section 5.1.2: that increasing the number of variables does not necessarily increase the mixed volume.

5.3.2 Towards an Efficient Solution

For applying the resultant solver, we prefer a system of smaller dimension because dimension affects directly the size of the resultant matrix. We assume again $\dot{q}$ has a unit scalar entry and make use of variable y_0. Now we introduce new quaternion $\dot{x} = [x_0, x_1, x_2, x_3]$:

$$\dot{x} = \frac{1}{y_0}\, \dot{q}^*\, \dot{t}.$$

We define a well-constrained system of dimension 7 by eliminating y_0 via equation (5.12). The equations follow from the first 6 and the last equation of the previous system respectively.

$$\begin{aligned} f_1 &= \dot{x}^T \dot{x} - \alpha_1 (1 + q_1^2 + q_2^2 + q_3^2), \\ f_i &= \dot{b}_i^T (\dot{x}\dot{q}) - \dot{a}_i^T (\dot{q}\dot{x}) - (\dot{q}\, \dot{b}_i \dot{q}^*)^T \dot{a}_i - (1 + q_1^2 + q_2^2 + q_3^2)\alpha_i, \quad i = 2, \ldots, 6, \\ f_7 &= x_0 - x_1 q_1 - x_2 q_2 - x_3 q_3. \end{aligned}$$

The well-constrained system has mixed volume 84. We hide a quadratic variable, say q_1, and we arrive at an overconstrained system with 6-fold mixed volumes and sparse resultant degree given by $22+32+32+32+32+32+32 = 214$. We may consider the system as originating at a multihomogeneous system of type $(1, 1, 4; 2, 2, 2)$, where the variable subsets are (q_2), (q_3) and (d_1, d_2, d_3, d_4). Following the formula for multigraded systems we obtain vector $v = (2, 22, 3, 3, 3, 3)$ on which to sort the integer lattice points. In actuality we used perturbations of this vector, namely $v = (20, 220, 30, 35, 40, 45)$ and $v = (1, 1000, 50, 100, 150, 200)$. Applying the incremental algorithm produces for both vectors a 405×405 univariate matrix polynomial of quadratic degree in q_1. After permutation of rows and columns, the upper left submatrix M_{11} has dimension 116 and condition number $\kappa \simeq 10^6$. However, the univariate matrix polynomial is identically singular and its efficient eigendecomposition remains open.

If we add equation (5.12) and treat y_0 as an independent variable we obtain an 8-dimensional system with the same mixed volume. This is the system used in table 3.1. It provides an example of adding a variable without tangible advantage to the sparse structure: the mixed volume is constant and the resultant matrices increase size.

More work is needed in connection to the Stewart platform kinematics. We expect that it will significantly further our understanding of sparse methods in practical applications. More generally, it is a testbed for ideas on the matrix-based approach to system solving.

5.4 Game Theory

We conclude with a discussion of the relevance of multigraded systems to game theory, in particular to computing Nash equilibria. This question finds applications in economics, evolutionary biology and political science, where noncooperative games between several agents with alternative strategies model a variety of problems. The *Nash equilibrium* is a well-established concept that captures the notion of a stable or final state in a game hence the computation of all valid equilibria presents an interesting problem. Alternative concepts of equilibrium have been suggested in [AMR92], whose computation reduces to deciding the existential theory of the reals and may, therefore, be relevant to sparse elimination.

Here we illustrate the case of computing *mixed* Nash equilibria. Intuitively, *mixed strategies* are those where every agent, at any given stage of the game, chooses her next move among a fixed finite set of *pure strategies* with some prescribed positive probability. The condition of a Nash equilibrium may be expressed in terms of polynomials in these probabilities, where the coefficients are functions of certain measurable quantities.

Given a system of polynomial equations and inequalities that describe a *totally mixed* Nash equilibrium McKelvey and McLennan [MM94] propose a transformation that produces an equivalent system of equations and inequalities. The new system has the same maximal number of interesting, or *regular*, solutions. The inequalities restrict all variables to nonzero values, therefore we are solely interested in toric roots. Moreover, the system of equations is well-constrained and has an exact Bernstein bound [MM94].

It is clear that the system of equations conforms to our model of sparseness. Furthermore, all equations are multilinear in the unknowns. If we group the l_i affine variables describing the choices of agent i in $\sigma_i = (\sigma_{i1}, \ldots, \sigma_{il_i})$, then there are $n = l_1 + \cdots + l_r$ polynomials. The multihomogeneous form of every polynomial is, adopting the notation of [MM94],

$$\sum_{h_1 \cdots h_{i-1} h_{i+1} \cdots h_{r-1}} v^{ik}_{h_1 \cdots h_{i-1} h_{i+1} \cdots h_{r-1}} \prod_{j=1, j \neq i}^{r} \sigma_{j h_j} \qquad i = 1, \ldots, r, \;\; k = 1, \ldots, l_i,$$

where v are constant coefficients, σ are variables and the sum ranges over the strategies of all agents excluding i.

These systems are not exactly multigraded because some terms have zero coefficients, but we can nonetheless obtain rather compact resultant formulae. The simplest system with more than one solution has type $(1,1,1;\ 1,1,1)$ and is discussed in [MM94]. This system resembles the one modeling the inverse kinematics of the cyclic molecule in section 5.2 because the Newton polytopes here are scaled copies of those. Thus the same approaches can be used for solving the system. All 2-fold mixed volumes are 1 and the sparse resultant has degree 3; the incremental algorithm constructs a matrix of size 4. For practical results on constructing resultant matrices for multigraded and other systems refer to section 3.3.

Chapter 6

Generic Position

This thesis has considered powerful algebraic methods that apply readily to kinematic and geometric problems by exploiting their inherent structure. An important component of geometric and algebraic computation is achieving general position, as several algorithms presuppose it and because it brings out essential properties about the complexity of the problems at hand. The concepts developed in this chapter have appeared, in specialized and rather limited form, in previous chapters. Here we formalize the notion of general position and propose effective ways to achieve it algorithmically.

Quite often algorithms are designed under the assumption of input nondegeneracy. Although they can have many specific forms, most degeneracies in geometric or algebraic algorithms reduce to a division by zero, or to a sign determination for a value which is zero. In this chapter we describe efficient methods for systematically coping with such degeneracies using symbolic infinitesimal perturbations. Our methods apply to every algorithm that can be implemented on a real RAM.

This work is influenced by the treatment of the problem in [EM90] and, in a more general context, [Yap90b]. The main contribution of this article is to introduce the first general and efficient perturbations from the viewpoint of worst-case complexity. Previous methods incurred an extra computational cost that was exponential in some parameter of the input size.

The principal domains of applicability are geometric and also algebraic algorithms over an infinite ordered field. Take, for instance, a convex hull algorithm in arbitrary dimension over the reals. It is typically described under the hypothesis of general position which excludes several possible instances, such as more than k points lying on the same $(k-1)$-dimensional hyperplane, in m-dimensional euclidean space, $m \geq k$. For an algebraic algorithm, consider Gaussian elimination without pivoting that works under the hypothesis that the pivot never vanishes. Our perturbation scheme accepts a program written under this hypothesis and outputs a slightly longer program that works for all inputs.

The perturbations introduced change the original input instance into a nondegenerate one which is arbitrarily close, in the usual euclidean metric, to the original input. For algorithms that branch only on the sign of determinants, which includes several important geometric algorithms, we propose a deterministic method. It increases the worst-case arithmetic complexity of the algorithm by a multiplicative factor of $\mathcal{O}(\log d)$ and its worst-case bit complexity by a factor of $\mathcal{O}^*(d)$, where d is the dimension of the geometric space of the input objects and $\mathcal{O}^*(\cdot)$ ignores the polylogarithmic factor. An improved variant from [ECS94], which applies to Orientation, Transversality and a restricted form of Ordering, reduces the bit-complexity overhead to a logarithmic factor in the dimension. In addition to their efficiency, our schemes require no symbolic computation. For rational inputs, almost all arithmetic can be carried out over a finite field and all intermediate quantities computed grow in a quasi-linear

fashion with the dimension.

For general algorithms, we propose a randomized scheme that incurs a factor of $\mathcal{O}^*(D)$ on the arithmetic complexity, where D is the highest total degree in the input variables of any polynomial in the program. Under the bit model, the worst-case running time of the new program is asymptotically bounded by $\mathcal{O}^*(\phi^3)$, where ϕ is the original bit complexity of the original one.

All claims about the efficiency of our approach are based on worst-case complexities. Yet there exist other measures, such as output size, under which the perturbations cause a much more significant increase in running time. For a degenerate input the output may be of constant size while, under the perturbation, the given algorithm cannot do significantly better than what its worst-case performance predicts.

The next section proposes a syntactic definition of perturbations as applied to the degeneracy problem, based on Seidel's treatment [Sei94]. The key notion of valid perturbations is introduced to capture the desired properties of perturbations that will let algorithms defined on generic instances be applied to arbitrary ones. Section 6.2 describes previous work in the area, in particular on efficient geometric perturbations.

Section 6.3 and 6.4 define the three efficient schemes that we shall be applying and lay the foundations of linear perturbations and their evaluation in a general context by examining practical ways for establishing validity and efficiently executing a transformed program. Geometric algorithms with determinant primitives are studied in section 6.5 and algorithms with rational expression primitives in section 6.6.

The proposed schemes are simple to implement. To illustrate this claim, section 6.7 describes our implementation of the Beneath-Beyond algorithm that uses the bit-optimal scheme to construct the facet structure of convex hulls in arbitrary dimension and to compute their volume. This algorithm has provided the basis of the mixed subdivision computation in section 2.3. The issue of postprocessing is closely examined in this context and experimental results are reported.

6.1 Definitions

We adopt the real RAM computational model as defined in the Introduction of this thesis. In the present context we focus on the primitives used by an algorithm. A typical primitive, called Ordering, is the comparison of coordinates: when comparing the j-th coordinate of points x_i and x_k the branch polynomial is $f = x_{ij} - x_{kj}$ for $1 \leq i \neq k \leq n$. Another is the Orientation primitive; for the planar convex hull problem the branch polynomial is the determinant of

$$\Lambda_3 = \begin{bmatrix} 1 & x_{i_1,1} & x_{i_1,2} \\ 1 & x_{i_2,1} & x_{i_2,2} \\ 1 & x_{i_3,1} & x_{i_3,2} \end{bmatrix}$$

where $x_{i_1}, x_{i_2}, x_{i_3}$ are distinct input points; this test decides on which side of the line (x_{i_1}, x_{i_2}) point x_{i_3} lies. More primitives, including InSphere, are discussed below.

6.1.1 Perturbations

Geometric problems are defined in terms of maps associating any given input instance to a unique output instance.

Definition 6.1.1 *A* problem mapping *is a mapping* $\Pi : X \to Y$ *between topological spaces. The* input space X *has the standard euclidean topology. The* output space Y *is, generally, the product* $D \times R$ *of a finite space* D *with the discrete topology and the direct union* R *of real spaces with the euclidean topology.*

For geometric problems $X = \mathbb{R}^{nd}$ where n denotes the number of input objects, typically points, and d the space dimension. A running example will be the convex hull volume (CHV) problem, which maps point sets to the real number expressing the volume of their convex hull. The output space is $\mathbb{R}$ with the euclidean topology and the mapping is continuous.

Computational geometry is concerned with the effective computation of problem mappings. Often, however, the implementation of algorithms is impeded by certain conditions imposed by the algorithm designer on the input. Typically, "special" cases such as those where the mapping is discontinuous are assumed not to occur. To illustrate, consider the planar convex hull face-structure (CHF) problem where, given a point-set, the sequence of hull edges must be constructed. The output topology is the direct union of real euclidean topologies, each corresponding to a distinct combinatorial structure of the polygon. A variety of algorithms, including Beneath-Beyond, assume that three points are never collinear. This configuration represents a discontinuity in this map because it is arbitrarily close to two input instances which give rise to outputs in disjoint components and, hence, at infinite distance.

In a more general context, algorithms branching on arbitrary rational expressions may present the same difficulties for a special subset of inputs. A worse situation is encountered for the matrix inversion (INV) problem where an intrinsically degenerate input is a singular matrix for which the problem mapping is undefined. Perturbations supply a mechanism to allow programs to run and produce meaningful output even if they cannot handle these special configurations.

Definition 6.1.2 *For any input* $x \in X$*, a* perturbed instance *of* x *is a curve* $x()$ *rooted at* x*, i.e.the image of a continuous function* $x() : \mathbb{R}_{\geq 0} \to X$ *such that* $x(0) = x$*. A* perturbation scheme Q *defines a perturbed instance for every element of* X*.*

For the sake of simplicity, we do not explicitly show the dependence of $x()$ on the choice of Q. The intuitive notion of perturbations as very small changes to the input is formalized in

Definition 6.1.3 *Given a problem mapping* $\Pi : X \to Y$ *and a perturbation scheme* Q*, the* perturbed problem mapping $\overline{\Pi}$ *is a mapping from* X *to* Y *such that*

$$\overline{\Pi}(x) = \lim_{\epsilon \to 0^+} \Pi(x(\epsilon)),$$

assuming that every such limit exists.

Again, an explicit indication of the dependence of the derived mapping on the perturbation scheme is foregone.

The goal is that the new problem mapping be defined and continuous on a proper superset of the original domain, thus incorporating some or all of the special instances. We also hope that implementing an algorithm for $\overline{\Pi}$ will be easier than explicitly handling all special cases for which some given algorithm for Π is undefined. In short, we shall solve $\overline{\Pi}$ instead of Π and then argue that the output of $\overline{\Pi}$ can yield enough information to recover the output of Π at the same input. The latter constitutes the *postprocessing* phase, which is in general nontrivial. However, there is a restricted yet important case in which postprocessing is superfluous:

Proposition 6.1.4 *For any perturbation scheme, if mapping* Π *is continuous at* $x \in X$ *then* $\Pi(x) = \overline{\Pi}(x)$.

This is the case with CHV, discussed in detail in section 6.7. Things are less favorable for the planar CHF problem: Given a point set containing subsets of more than two collinear points on the hull boundary, the output polygon on perturbed input will contain edges split into more than one segments. Postprocessing then has to merge these segments by eliminating points in the interior of polygon edges. This process is analyzed for the general CHF problem in section 6.7. Postprocessing for the problem of polytope intersection is examined in [DMY93].

For INV, given a perturbed singular matrix, the algorithm should return its inverse and the perturbed input should be arbitrarily close to the original one under the standard euclidean metric. The first condition follows from the correctness of the algorithm on generic inputs once we establish that perturbed matrices are nonsingular. Restricted to nonsingular matrices, the problem mapping is continuous: On a perturbed nonsingular matrix the output is arbitrarily close to the exact solution; to recover the latter we can simply set the infinitesimal to zero. For singular matrices an additional specification is required for what constitutes a desired output and how to measure its distance from the perturbed output.

6.1.2 Computing with Perturbations

Given a program Φ that implements Π, the question is how to obtain another real RAM program $\overline{\Phi}$ that implements $\overline{\Pi}$. First, all arithmetic operations in Φ are transformed in order to handle perturbation curves. Memory locations in $\overline{\Phi}$ hold univariate functions in ϵ and a postprocessing stage eliminates ϵ from the output. Lastly, every branching operation of Φ is transformed to a branching operation that tests the limit of the sign of some ϵ-function, namely

$$\lim_{\epsilon \to 0^+} \text{sign} f(x(\epsilon))$$

if f is the respective function tested by Φ; $\text{sign}(\cdot)$ is a piecewise constant function with values in $\{-, 0, +\}$.

Problematic instances always include those where Π is discontinuous. In addition to these, a program may be undefined on other instances, for example two covertical points in the case of a planar CHF solved by a sweep-line algorithm. On the other hand if INV is solved by Gaussian elimination without pivoting, a singular maximal minor will cause the program to stop.

All inputs not dealt with by a program can be modeled by the vanishing of some polynomial in the input. Conforming to the standard viewpoint in the literature [EM90, Yap90b, EC95, Sei94] we have

Definition 6.1.5 *An input instance is* degenerate *with respect to some program, if and only if it causes some numerator or denominator polynomial* f *at a branch to vanish, where* f *is not identically zero. Alternatively, an input instance is* generic *with respect to this program if there is no such branch polynomial.*

Some authors distinguish between *problem-dependent* degeneracies i.e.those where Π is discontinuous, and *algorithm-induced* degeneracies, such as the covertical points for the sweep-line algorithm or the singular maximal minor for Gaussian elimination without pivoting.

Definition 6.1.6 *A perturbation scheme Q is* valid *with respect to a function f if and only if, for every input $x \in X$, the limit*

$$\lim_{\epsilon \to 0^+} \mathrm{sign} f(x(\epsilon))$$

exists and is nonzero. Perturbation Q is valid with respect to a set of functions if and only if it is valid with respect to every function in this set. Q is valid with respect to a given real RAM program if and only if it is valid with respect to the set of all branch polynomials in the program.

Clearly, under a valid perturbation no degenerate inputs arise, which implies that the zero branches in a program can be ignored:

Theorem 6.1.7 *Assume that Q is a valid perturbation scheme for a real RAM program Φ computing mapping Π and that $\overline{\Phi}$ is obtained by the transformation at the beginning of this section. Then $\overline{\Phi}$ computes the perturbed mapping $\overline{\Pi}$ and, for $x \in X$ such that Π is continuous, $\overline{\Phi}$ yields $\Pi(x)$. The statement holds if some, or all, of the zero branches of Φ are removed.*

Proof By validity all limits exist, hence the map $\overline{\Pi}$ is well-defined and computed by $\overline{\Phi}$. Proposition 6.1.4 establishes the second assertion. Since, by validity, no zero branches are taken in $\overline{\Phi}$, these branches might as well be pruned away. □

From this theorem, it is clear that the action of perturbations can be thought of as concentrated at the branches. The main advantage of the perturbation method is that some or all of the zero branches do not need to be implemented. This brings us to the original problem stated at the beginning of section 6.1.1. We now see how algorithms designed under the hypothesis of nondegeneracy can be used for solving the perturbed problem mapping, from which postprocessing can produce the output of Π.

It must be underlined that whenever a given program Φ is transformed to $\overline{\Phi}$ to reflect the application of some perturbation, all instructions should be changed according to the chosen scheme. It leads to severe inconsistencies to allow some instructions to be executed as if the perturbation were not implemented and, similarly, it is a grave error to try to use more than one scheme at a time. Imagine, for example, that in the planar CHF problem coordinate comparisons are not transformed under the perturbation, but the Orientation primitive is transformed. Then three covertical points may be detected to be so by coordinate comparisons, though for the Orientation test they are not even collinear.

6.2 Previous Work

The simplest approach in coping with degeneracies is to handle each special case separately, which is tedious for implementors and unattractive for theoreticians, though some recent work re-examines this common belief [DMY93, BMS94].

Dantzig's [Dan63] symmetry breaking rules in linear programming are regarded as the precursor of current systematic perturbations. The principal idea is to perturb the right-hand side of every constraint equation by an infinitesimal quantity that depends on the index of this equation. The i-th constraint then becomes

$$\sum_{j=1}^{n} a_{i,j} x_j + x_{n+i} = b_i(\epsilon) = b_i + \epsilon^i,$$

where $x_1, \ldots, x_n$ are the original variables, each x_i for $i > n$ is a slack variable and the $a_{i,j}$ and b_i are constants.

Edelsbrunner and Mücke generalize in [EM90] a technique called Simulation of Simplicity (SoS for short), already presented in [Ede86], which refines the above method. Every input coordinate $x_{i,j}$ is perturbed into

$$x_{i,j}(\epsilon) = x_{i,j} + \epsilon^{2^{i\delta - j}},$$

where $\delta > d$ and d is the dimension. The perturbation is infinitesimal due to symbolic variable ϵ; it is also conceptual in the sense that the computation remains numeric. Raising ϵ to such a high power intuitively distinguishes between any two coordinates which allows SoS to be applied to a wide range of geometric primitives, including the determinantal ones examined in this paper. One exception is an inconsistency in the case of the InSphere primitive discussed in section 6.5.3. Its main drawback is that it incurs an overhead to the arithmetic complexity of the algorithm which is exponential in d in the worst case: deciding the sign of a $d \times d$ perturbed determinant, although rather fast on the average, requires the calculation of $\Omega(2^d)$ minors in the worst case. To prove the latter bound for the case of the Orientation primitive it suffices to count all vectors $(v_1, \ldots, v_{d-2})$ such that d is the order of the matrix, $v_i \in \{1, \ldots, d\}$ and $i < j \Rightarrow v_i \leq v_j$. In [EM90] every such vector is associated with a minor that may have to be evaluated.

Yap in [Yap90b] deals with the more general setting in which branching occurs at arbitrary rational expressions and proposes a method which is equivalent to an infinitesimal perturbation, as proven in [Yap90a]. It has been extended to analytic test functions by Seidel [Sei94]. For input variables $x = (x_1, \ldots, x_N)$, the scheme considers a total ordering on all power products of the form

$$w \;=\; \prod_{i=1}^{N} x_i^{e_i}, \qquad e_i \geq 0.$$

This ordering, denoted by $\leq_A$, is *admissible* if, for all power products w, w', w'',

$$1 \leq_A w \qquad \text{and} \qquad w \leq_A w' \Rightarrow ww'' \leq_A w'w''.$$

If $w_1, w_2, \ldots$ is an admissible ordering of power products larger than one, then each polynomial $f(x)$ at the numerator or denominator of a branch expression is associated with the infinite list

$$S(f) \;=\; (f, f_{w_1}, f_{w_2}, \ldots)$$

where f_{w_k} is the partial derivative of f with respect to w_k. The sign of f is taken to be the sign of the first polynomial in $S(f)$ with a nonzero value, which can always be found after a finite number of evaluations. The worst-case complexity to evaluate this new test is exponential in N, although the average-case complexity is significantly lower. Consider sparse N-variate polynomials with degree in each variable bounded by m. If all variables are of the same maximum degree then f has at least m^N partial derivatives, and if all of them must be evaluated then the arithmetic complexity is $\Omega(m^N)$.

Dobrindt, Mehlhorn and Yvinec proposed an efficient scheme specifically for coping with degenerate intersections between a convex and a general polyhedron in three dimensions [DMY93]. It is noteworthy that the vertices of the convex polyhedron are guaranteed to be perturbed in a specific direction with respect to the given facets. Another merit of this work is that it discusses postprocessing in detail in order to recover the exact solution.

A *structural* perturbation for a motion-planning algorithm, in which the input objects are the semi-algebraic sets describing the obstacles, is given by Canny in [Can91]. He uses towers of infinitesimals to eliminate degeneracies while preserving essential properties of the sets, namely emptiness and number of connected components.

6.3 Linear Perturbations

The high worst-case complexity of SoS is essentially due to the high degree of the ϵ-factor. This section proposes *linear* perturbations in order to reduce the complexity of computing the preturbed primitives. Moreover, it provides a powerful validity criterion for a specific class of linear schemes.

Linear perturbations are of the following type:

$$x_i(\epsilon) = x_i + \epsilon b_i, \qquad 1 \leq i \leq n, \tag{6.1}$$

where $x_i = (x_{i,1}, \ldots, x_{i,d})$ and $b_i = (b_{i,1}, \ldots, b_{i,d}) \in \mathbb{R}^d$ are the i-th input and perturbation vectors respectively and multiplication is scalar. Let $f(x_1, \ldots, x_n)$ be any polynomial in n vector variables; its *initial form* is a homogeneous polynomial $\mathcal{I}(f)$ in the same variables, equal to the sum of all terms in f of maximum total degree. For homogeneous polynomials, as is typically the case in geometric algorithms, $f = \mathcal{I}(f)$.

Theorem 6.3.1 *Let $g(x_1, \ldots, x_n) = \mathcal{I}(f)$ be the initial form of f. For a linear perturbation (6.1) to be valid with respect to polynomial f, it suffices that $g(b_1, \ldots, b_n) \neq 0$.*

Proof Consider $f(x(\epsilon))$ as a univariate polynomial in ϵ. From definition 6.1.6, it is required that $f(\epsilon)$ never vanishes on perturbed input. If at least one coefficient is never zero, the polynomial is not identically zero and its zero set does not include all real numbers. The highest-order coefficient in $f(\epsilon)$ is $g(b_1, \ldots, b_n) \neq 0$, therefore the zero set of $f(\epsilon)$ is not fully dimensional which implies that $f(\epsilon)$ has a finite number of roots. It suffices now to assume that ϵ takes real values smaller than the minimum positive root of $f(\epsilon)$. □

This theorem can be readily generalized to nonlinear perturbations. Its significance is twofold. First, the validity requirement has to be tested only against the initial form of the branch polynomial. More importantly, the problem of designing an efficient perturbation scheme is reduced to finding a single set of input vectors $b_1, \ldots, b_n$ on which the branch polynomials do not vanish. In practice, one may use known point sets such as points on the moment curve, employed by our first perturbation scheme:

$$x_{i,j}(\epsilon) = x_{i,j} + \epsilon\, i^j, \qquad i = 1, \ldots, n,\ j = 1, \ldots, d, \tag{6.2}$$

where ϵ is a symbolic infinitesimal. An alternative is to use the points defined by the rows of a Cauchy matrix. This scheme has been improved by taking the residue $i^j \bmod q$ for some prime $q > n$, while for a wider class of algorithms (sect. 6.6) b_i is a random integer. In general, defining perturbation vectors $b_1, \ldots, b_n$ is a hard problem, a stronger version, in fact, of the zero avoidance problem [Zip90].

6.3.1 A Validity Criterion

This criterion was motivated by the application of scheme (6.2) to the InSphere primitive which decides, given points $x_{i_1}, \ldots, x_{i_{d+2}} \in \mathbb{R}^d$, whether $x_{i_{d+2}}$ lies in the interior of the hypersphere defined by the first $d+1$ points. This primitive is shown in section 6.5.3 to reduce to testing the sign of a $(d+2) \times (d+2)$ determinant which implies that validity rests, by theorem 6.3.1, upon the nonsingularity of matrix

$$W_{d+2} = \begin{bmatrix} 1 & i_1 & \ldots & i_1^d & \sum_{j=1}^d i_1^{2j} \\ 1 & i_2 & \ldots & i_2^d & \sum_{j=1}^d i_2^{2j} \\ \vdots & \vdots & & \vdots & \vdots \\ 1 & i_{d+2} & \ldots & i_{d+2}^d & \sum_{j=1}^d i_{d+2}^{2j} \end{bmatrix}.$$

The proof requires Descartes' rule of sign which we state here for completeness. Given a univariate polynomial in canonical form, the number of sign variations of its nonzero coefficients $u_1, \ldots, u_N$ is the number of consecutive pairs (u_k, u_{k+1}), $1 \le k < N$, such that the product $u_k u_{k+1}$ is negative.

Proposition 6.3.2 (Descartes' Rule of Sign) [CL82] *The number of sign variations of a polynomial's nonzero coefficients exceeds the number of positive zeros, multiplicities counted, by an even nonnegative integer.*

Proposition 6.3.3 *Matrix W_{d+2} is nonsingular for distinct positive i_j, $1 \le j \le d+2$.*

Proof If W_{d+2} is singular there is a nonzero vector $(q_1, \ldots, q_{d+2})$ in the kernel of the matrix, therefore the y-polynomial $\sum_{j=0}^{d} q_{j+1} y^j + q_{d+2} \sum_{j=1}^{d} y^{2j}$ has at least $d+2$ distinct positive zeros, namely $i_1, \ldots, i_{d+2}$. The polynomial has also at most $d+1$ sign variations, which contradicts Descartes' rule. □

Here we generalize the discussion to include potentially more primitives in addition to InSphere. The perturbation is restricted to the form:

$$x_{i,j}(\epsilon) = x_{i,j} + \epsilon \beta_i^{\gamma_j}, \qquad 1 \le i \le n, 1 \le j \le d, \tag{6.3}$$

with $\gamma = (\gamma_1, \ldots, \gamma_d) \in \mathbb{Z}^d$ fixed and $b_i = (\beta_i^{\gamma_1}, \ldots, \beta_i^{\gamma_d})$ as the i-th perturbation vector, where $\beta_i \in \mathbb{R}$ for $1 \le i \le n$. For a general polynomial $f(w_1, \ldots, w_t)$, the *support* of f is denoted $\mathrm{supp}(f)$. Consider t polynomials $f_j(x_i) = f_j(x_{i,1}, \ldots, x_{i,d})$, where $t \le n$, $1 \le i, j \le t$. Let $A_j = \mathrm{supp}(f_j) \subset \mathbb{Z}^d$, let the union of all singleton supports be $U = \bigcup_{|A_j|=1} A_j$, where $|\cdot|$ denotes cardinality, and define

$$B_j = A_j \setminus U \ \subset A_j \qquad 1 \le j \le t.$$

Denote inner product by $\langle \cdot, \cdot \rangle$ and let the set of inner products of exponent vectors for each B_j with a fixed vector $\gamma = (\gamma_1, \ldots, \gamma_d) \in \mathbb{Z}^d$ be

$$C_j = \{ \langle \gamma, a \rangle \in \mathbb{Z} \,|\, a \in B_j \}, \qquad 1 \le j \le t.$$

Each C_j has a minimum and maximum element denoted $\min C_j$ and $\max C_j$ respectively. We restrict attention to primitives expressed as determinants of order t, with (k,j)-th entry equal to $f_j(b_k)$.

Theorem 6.3.4 *Suppose that β_i is positive and distinct for every $1 \le i \le n$ in the perturbation scheme (6.3). Moreover each polynomial f_j has coefficients of the same sign and, possibly after reindexing, the non-empty sets C_j are ordered so that, for every $j > 1$, $\max C_{j-1} < \min C_j \le \max C_j < \min C_{j+1}$. Then the $t \times t$ matrix with (i,j)-th entry $f_j(b_i)$ is nonsingular.*

Lemma 6.3.5 *If $q_1, \ldots, q_t$ are any real values, y is a real variable, $\gamma \in \mathbb{Z}^d$ is fixed and polynomials f_j satisfy the hypothesis of theorem 6.3.4, then polynomial $F(y) = \sum_{j=1}^{t} q_j f_j(y^{\gamma_1}, \ldots, y^{\gamma_d})$ has fewer than t positive real roots.*

Proof By expanding $f_j(y^{\gamma_1}, \ldots, y^{\gamma_d}) = \sum_{a \in A_j} c_{j,a} y^{\langle \gamma, a \rangle}$, the univariate polynomial $F(y)$ can be written

$$\begin{aligned} F(y) = \sum_{j=1}^{t} q_j \sum_{a \in A_j} c_{j,a} y^{\langle \gamma, a \rangle} &= \sum_{j=1}^{t} \sum_{a \in A_j} q_j c_{j,a}\, y^{\langle \gamma, a \rangle} \\ &= \sum_{j : B_j \neq \emptyset} \sum_{a \in B_j} q_j c_{j,a}\, y^{\langle \gamma, a \rangle} + \sum_{j : |A_j| = 1} \sum_{a \in A_j} q'_j c_{j,a}\, y^{\langle \gamma, a \rangle} \\ &= \sum_{a \in \cup B_j} \left(\sum_{j : a \in B_j} q_j c_{j,a} \right) y^{\langle \gamma, a \rangle} + \sum_{j : |A_j| = 1} \sum_{a \in A_j} q'_j c_{j,a}\, y^{\langle \gamma, a \rangle}, \end{aligned}$$

where the q'_j are the result of combining all terms corresponding to singleton supports. The expression relies on the fact that the supports A_j are non-empty and are partitioned between non-singletons in the first summand and singletons in the second.

Now partition the first summand into sums of monomials whose exponents belong in a certain C_j. Since all B_j are distinct and by hypothesis the C_j are ordered, the coefficient of $y^{\langle \gamma, a \rangle}$ is a single product $q_j c_{j,a}$. Furthermore, there is no sign variation among the coefficients of each sum because all $c_{j,a}$ for a fixed j have the same sign. Hence, in the first summand the number of distinct coefficients is at most the number of nonempty B_j. In the second summand, the total number of coefficients is at most equal to the number of singleton supports. Therefore, there exist at most t distinct coefficient signs in $F(y)$, hence the number of sign variations is at most $t-1$. Application of Descartes' rule completes the proof. □

Proof of Theorem 6.3.4 If the matrix is singular, then there must exist a nonzero real vector $(q_1, \ldots, q_t)$ in the kernel of the linear transformation of the matrix, therefore the univariate polynomial $F(y)$ from lemma 6.3.5 has a distinct positive root β_i for all $1 \leq i \leq t$. The existence of t distinct positive roots contradicts lemma 6.3.5. □

For the InSphere primitive and matrix W_{d+2}, $t = d+2$, $f_1 = 1$, $f_j(b_k) = i_k^j$ for $2 \leq j \leq d+1$ and $f_{d+2}(b_k) = \sum_{l=1}^{d} i_k^{2l}$ where $1 \leq k \leq d+2$. Then proposition 6.3.3 follows as a corollary.

6.4 Evaluation of Branch Expressions

Some algebraic techniques are presented for the efficient evaluation of certain primitives in perturbed programs. An important consequence is that no computation in the derived program need involve the infinitesimal variable. Thus, although the perturbation is symbolic, all arithmetic is numeric.

We first discuss interpolation as a general method for computing univariate polynomials in ϵ from their values. The only assumption here is that the total degree of each polynomial is known; call it δ. The first step is to obtain a sequence of interpolation pairs, in other words pairs of ϵ specializations, usually at distinct primes, and the respective values of the polynomial. If $\delta + 1$ interpolation pairs are available, *dense interpolation* can be used to compute the coefficients in $\mathcal{O}(\delta \log^2 \delta)$ arithmetic operations by the Fast Fourier Transform (FFT) [AHU74, ch. 8].

If, furthermore, there is an *a priori* bound T on the number of nonzero terms that is significantly lower than the maximum number $\delta+1$, then *sparse interpolation* is preferable. There exists a probabilistic algorithm with arithmetic complexity $\mathcal{O}^*(\delta\tau)$ that requires $\mathcal{O}(\delta\tau)$ interpolation pairs, where $\tau \leq T$ is the actual number of nonzero terms in the polynomial. A deterministic algorithm has complexity $\mathcal{O}^*(T^2 \log \delta)$ and requires $2T$ interpolation pairs. Both algorithms are surveyed in [Zip90].

Traditionally, the cost of evaluating the unknown polynomial is of minor concern in the context of the interpolation problem, yet here this cost must be assessed. In general, computing the interpolation pairs takes time proportional to the number of pairs times the complexity of evaluating the polynomial. For the important case of determinantal tests discussed in this article, i.e.tests expressed as determinants, the complexity of the evaluation phase dominates the overall complexity. The rest of this section concentrates on determinantal tests of order t and develops more efficient ways for the combined problem of evaluation and sign determination.

Let $MM(t) = \mathcal{O}(t^{2.376})$ [CW90] be the arithmetic complexity of multiplying two $t \times t$ matrices and I (I_t) the identity matrix (of order t). Dense interpolation costs $\mathcal{O}(\delta MM(t))$, where $\delta \geq t$. An

improved technique for interpolating determinants whose entries are higher-degree polynomials in several variables appears in [MC92b]. Here, following [EC95], we generalize a near-optimal technique for the case when the entries are univariate polynomials.

Given $t \times t$ matrix $A(\epsilon)$ with polynomial entries in ϵ, we can express it as a matrix polynomial

$$A(\epsilon) = A_r \epsilon^r + A_{r-1}\epsilon^{r-1} + \cdots + A_1 \epsilon + A_0$$

where r is the maximum degree in ϵ of any matrix entry. The validity of the perturbation implies that there exists nonsingular matrix coefficient A_i. If A_r is nonsingular, we have

$$A_r^{-1}A(\epsilon) = \epsilon^r + A_r^{-1}A_{r-1}\epsilon^{r-1} + \cdots + A_r^{-1}A_1\epsilon + A_r^{-1}A_0$$

and the determinant of the right-hand side equals, by lemma 4.4.3, the characteristic polynomial of companion matrix

$$C = \begin{bmatrix} 0 & I_t & 0 & \cdots & 0 \\ 0 & 0 & I_t & \cdots & 0 \\ \vdots & \vdots & \vdots & & \vdots \\ 0 & 0 & 0 & \cdots & I_t \\ -A_r^{-1}A_0 & -A_r^{-1}A_1 & -A_r^{-1}A_2 & \cdots & -A_r^{-1}A_{r-1} \end{bmatrix}.$$

If A_r is singular we apply lemma 4.4.4 to obtain, with arbitrarily high probability, a new matrix polynomial $A(y)$ with regular leading coefficient. Then, every eigenvalue of $A(y)$ maps by a rational function to an eigenvalue of $A(\epsilon)$. We can apply lemma 4.4.4 because $A(\epsilon)$ is not identically singular by proposition 6.4.2 below; to prove this we start with the following

Lemma 6.4.1 *Any monic matrix polynomial is not identically singular. Any linear matrix polynomial that is identically singular has both coefficients singular.*

Proof Let monic $A(x) = Ix^d + \cdots + A_0$ with every A_i an $r \times r$ numeric matrix. $\det A(x)$ is a polynomial in x with leading term x^{dr}, i.e.the leading coefficient is 1, hence it does not vanish identically for all x.

Let linear $A(x) = A_1 x + A_0$. Since $A(0) = A_0$ it follows A_0 is singular. If A_1 is nonsingular, we write $\det A(x) = \det A_1 \det(Ix + A_1^{-1}A_0)$ and the monic polynomial must be identically singular which is infeasible, therefore A_1 must be singular. □

Proposition 6.4.2 *If matrix polynomial $A(x)$ has some nonsingular coefficient A_i there exists some value of x for which $A(x)$ is nonsingular.*

Proof If $A_i = A_0$ there is nothing to prove because $A(0) = A_0$ and the desired value of x may be zero. Otherwise assume $A(x)$ is indentically singular, denoted $\det A(x) \equiv 0$, and express it as $A(x) = B(x)x + A_0$, where $B(x)$ is a matrix polynomial of degree $d-1$, if d is the degree of $A(x)$. Clearly $\det A_0 = 0$. If moreover $\det B(x) \equiv 0$ we can apply the same argument on $B(x)$ and iteratively arrive at a linear polynomial with both coefficients singular, by the previous lemma. This contradicts the hypothesis, hence

$$\det B(x) \not\equiv 0 \;\Rightarrow\; A(x) = B(x)(Ix + B(x)^{-1}A_0).$$

This implies that the monic polynomial should be identically singular, thus violating the previous lemma. We have arrived at a contradiction by supposing $\det A(x) \equiv 0$, hence the result is proven. □

Although the primitives studied in this thesis lead to regular matrix polynomials, we conclude with a general result on determinantal primitives.

Theorem 6.4.3 *Let $A(\epsilon)$ be a matrix of order t, whose entries are univariate polynomials in ϵ such that there is some nonsingular coefficient. Determining the sign of* $\det A(\epsilon)$ *can be reduced to computing determinant* $\det A_1$ *and the characteristic polynomial of companion matrix C. The total arithmetic complexity is $MM(rt)\log(rt)$.*

Proof The discussion above reduces the case of singular and regular leading coefficient to the same problem, namely inverting A_r, computing C and its characteristic polynomial coefficients until the most significant nonzero coefficient is found. The transformation of a matrix polynomial with singular leading coefficient has complexity $\mathcal{O}(r^2t^2)$, which is dominated by computing the characteristic polynomial of C. □

A discussion of modular arithmetic is in order here because, in addition to being a common method for conducting arithmetic on computers, it is also the fastest with respect to bit complexity for evaluating the perturbed tests. Besides the classical application to integer arithmetic [AHU74], modular methods and the Chinese Remainder theorem can be used with rational data with the same asymptotic complexity [DST88].

The basic approach is as follows. First the given quantities are mapped to their residues modulo a set of primes, then the required computation is performed within each finite field defined by every one of these primes and, lastly, the integer answer is computed by the results in each finite field. This last step relies on the Chinese Remainder theorem. In order for the entire process to be deterministic, a bound on the value of the final answer must be known, which is used to calculate the number of different finite fields used.

Let k denote the number of finite fields $\mathbb{Z}_p$, for distinct primes p, necessary to carry out a particular computation, where k depends on the bit size of the final answer. The first stage of mapping each given quantity to its respective k residues, as well as the third stage of calculating the answer from its k residues, have each bit complexity $\mathcal{O}(M(k)\log k)$. The middle stage is the actual computation within each $\mathbb{Z}_p$ and its bit complexity is k times the arithmetic complexity of this computation. We have made the implicit assumption that each prime p has constant bit size and that choosing such a prime, from an existing and sufficiently long list, is a constant-time operation.

Corollary 6.4.4 The arithmetic complexity of computing $\det(A_1\epsilon + A_0)$ is $\mathcal{O}(MM(t)\log t)$, where t is the order of matrices A_0 and A_1. Let s be the maximum bit size of any entry in A_1 and A_0. Then the bit complexity of computing the above determinant is $\mathcal{O}((ts)^{1+\alpha} + tsMM(t)\log t)$, for some arbitrarily small positive constant α.

Proof The operations required are a matrix inversion, a matrix multiplication, calculation of a determinant and computation of the coefficients of a characteristic polynomial. Each takes $\mathcal{O}(MM(t))$ time, except from the last step which takes $\mathcal{O}(MM(t)\log t)$ time, for arbitrary matrices, due to an algorithm by Keller-Gehrig [KG85]. To establish the bit complexity bound, the transformation of theorem 6.4.3 is used. The bit size of the coefficients of the ϵ-polynomial representing the determinant is $\mathcal{O}(ts)$ since the original matrix entries have size s and its order is t. Hence modular arithmetic may be used over $k = \mathcal{O}(ts)$ fields. □

6.5 Some Common Geometric Primitives

This section deals with specific perturbation schemes for certain common determinantal primitives. Namely it applies

$$x_{i,j}(\epsilon) = x_{i,j} + \epsilon\, i^j, \qquad i = 1, \ldots, n,\ j = 1, \ldots, d, \tag{6.2}$$

where ϵ denotes a symbolic infinitesimal, to Ordering and InSphere. Furthermore, it applies the bit-optimal variant proposed in [ECS94]

$$x_{i,j}(\epsilon) = x_{i,j} + \epsilon(i^j \bmod q), \qquad i = 1, \ldots, n,\ j = 1, \ldots, d, \tag{6.4}$$

where q is the smallest prime that exceeds n, to Orientation, Transversality and a restricted form of Ordering. The bit size of the perturbation quantities is bounded by $\log n$, which is optimal because there must be at least n distinct such quantities. It is reasonable to suppose that at least a constant fraction of the n input vectors are distinct. Hence, if s denotes an upper bound on the bit size of the input data, $s \geq \log n$.

In practice, the emphasis is on designing efficient valid perturbations from a computational complexity point of view. The comparison is carried out between worst-case complexities of programs and their perturbed counterparts; this encourages the design of schemes efficient for the almost-generic cases, as opposed to very special cases. The reason is that the overhead incurred by perturbing is not output-sensitive. For instance, given n coincident points as input to an algorithm solving the CHF problem, the exact output is a single point whereas the perturbed output is a polytope with $n^{\lfloor d/2 \rfloor}$ facets. See [BMS94] for a discussion of limitations of this approach.

For avoiding very degenerate cases like this and for ensuring the lower bound on s a preprocessing phase can be used to detect and eliminate duplicates without affecting the asymptotic complexity of most algorithms.

6.5.1 Ordering

This primitive decides the order of two quantities expressing the k-th coordinate of the i_1-th and i_2-th input points. On input perturbed with (6.2) the primitive decides the sign of

$$x_{i_1,k}(\epsilon) - x_{i_2,k}(\epsilon) \;=\; x_{i_1,k} + \epsilon\, i_1^k - x_{i_2,k} - \epsilon\, i_2^k.$$

For degenerate inputs the factors of the infinitesimal must be compared, which comes down to comparing i_1 against i_2. Notice that this is the lexicographic ordering. Since all indices are distinct, the perturbation is valid by theorem 6.3.1. Perturbation (6.4), for $k = 1$, is valid too.

The evaluation requires in the worst case, an extra constant-time check. Under the bit model, the extra comparison adds a $\mathcal{O}(\log n)$ factor, which is upper bounded by the original bit complexity because each $x_{i,j}$ is $\Omega(\log n)$ bits long.

Theorem 6.5.1 *Perturbation (6.2) is valid with respect to the Ordering primitive and does not change the asymptotic running-time complexity of this primitive in the arithmetic as well as the bit model. The same holds for perturbation (6.4) for comparisons along the first coordinate.*

Unfortunately, if we compare along some general k-th coordinate, validity may not hold for the second scheme.

6.5.2 Orientation and Transversality

Given a query point $x_{i_{d+1}}$ and a hyperplane in $\mathbb{R}^d$ spanned by points $x_{i_1}, \ldots x_{i_d}$, Orientation decides in which one of the two halfspaces defined by this hyperplane the query point lies. A degeneracy occurs exactly when $x_{i_{d+1}}$ lies on the hyperplane. The primitive is formulated as a test of a determinant sign; the relevant matrix is Λ_{d+1} below. Transversality determines the orientation of d points in $\mathbb{R}^{d-1}$ given by their homogeneous coordinates and is expressed as the sign of $\det(\Delta_d)$:

$$\Lambda_{d+1} = \begin{bmatrix} 1 & x_{i_1,1} & x_{i_1,2} & \ldots & x_{i_1,d} \\ 1 & x_{i_2,1} & x_{i_2,2} & \ldots & x_{i_2,d} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_{i_{d+1},1} & x_{i_{d+1},2} & \ldots & x_{i_{d+1},d} \end{bmatrix}, \qquad \Delta_d = \begin{bmatrix} x_{i_1,1} & x_{i_1,2} & \ldots & x_{i_1,d} \\ x_{i_2,1} & x_{i_2,2} & \ldots & x_{i_2,d} \\ \vdots & \vdots & & \vdots \\ x_{i_d,1} & x_{i_d,2} & \ldots & x_{i_d,d} \end{bmatrix}.$$

Matrix Δ_d also comes up in a dual context, when the input objects are hyperplanes in $(d-1)$-dimensional space and Transversality decides on which side of the first hyperplane lies the intersection of the other $d-1$ hyperplanes as in [EG86], for example, where $d = 3$.

For completeness we state the following theorem from [EC95] which relies on theorem 6.3.1, the properties of Vandermonde matrices and corollary 6.4.4.

Theorem 6.5.2 *Perturbation (6.2) is valid with respect to algorithms that branch on determinants of $\Lambda_{\delta+1}$ and Δ_δ, for $\delta \leq d$, where d is the space dimension. The perturbation increases the asymptotic running-time complexity of evaluating the primitive, under the arithmetic model, by $\mathcal{O}(\log d)$. Under the bit model, the worst-case complexity is increased by a factor of $\mathcal{O}^*(d)$.*

Now consider scheme (6.4). By theorem 6.3.1 the nonsingularity of $\Lambda_{d+1}(\epsilon)$ is obtained by using the closed form expression of a Vandermonde determinant.

$$\det \overline{V_{d+1}} = \det \begin{bmatrix} 1 & i_1 \bmod q & \ldots & i_1^d \bmod q \\ 1 & i_2 \bmod q & \ldots & i_2^d \bmod q \\ \vdots & \vdots & & \vdots \\ 1 & i_{d+1} \bmod q & \ldots & i_{d+1}^d \bmod q \end{bmatrix} \equiv \prod_{k>l\geq 1}^{d} (i_k - i_l) \not\equiv 0 \pmod q.$$

Validity in the case of Transversality follows similarly. The crucial property for both schemes is that they define n vectors, every d of which are linearly independent.

The sign of $det\Lambda_{d+1}(\epsilon)$ and $det\Delta_{d+1}(\epsilon)$ is the sign of the least significant term in the respective polynomial. One way to compute it, adopted by SoS, is to calculate directly all terms, starting with the one of least degree, until finding one that does not vanish. Fortunately, our scheme lends itself to the more efficient technique of theorem 6.4.3. Both perturbed matrices satisfy the theorem's hypothesis; for $\Lambda_{d+1}(\epsilon)$ to do so, the first column is multiplied by ϵ, which does not affect the determinant sign.

Theorem 6.5.3 Perturbation (6.4) is valid with respect to Orientation and Transversality. It increases the worst-case arithmetic and bit complexities of the Orientation and Transversality primitives by a $\mathcal{O}(\log d)$ factor.

Proof Validity follows from theorem 6.3.1. The original arithmetic complexity is $\Theta(MM(d))$ from [vzG88]. From corollary 6.4.4, the complexity on perturbed input is $\mathcal{O}(MM(d)\log d)$. The original worst-case bit complexity depends on the size of the answer which is $\Theta(ds)$. Typically modular arithmetic is used, requiring $\Theta(ds)$ different finite fields, while on perturbed input the number of finite fields is $\mathcal{O}(d(s+\log n))$,

The assumption that $s \geq \log n$ finishes the proof. □

An important feature for implementors is that the growth of any computed quantity is quasi-linear in the dimension. For instance, in a 3-dimensional problem with input quantities of absolute magnitude less than 10^5, any computed quantity fits in a computer word.

6.5.3 InSphere

We apply (6.2) to this primitive test which decides, given $d+2$ points, whether the $(d+2)$-nd point lies in the interior of the higher-dimensional sphere defined by the first $d+1$ points in $\mathbb{R}^d$. It can be reduced to testing the sign of a determinant as follows. First, lift all points to the surface of a paraboloid in $\mathbb{R}^{d+1}$ by adding a $(d+1)$-st coordinate equal to the sum of the squares of the d coordinates defining each point. The original space is a d-dimensional hyperplane which the paraboloid touches at the origin. Let $x_{i_1}, x_{i_2}, \ldots, x_{i_{d+1}}$ be the points defining the sphere, their lifted images define a hyperplane H in $\mathbb{R}^{d+1}$. The query point $x_{i_{d+2}}$ lies within the sphere if and only if its lifted image lies below H, in other words to the same side of H as the original points. A degeneracy occurs exactly when $x_{i_{d+2}}$ lies on the sphere or, equivalently, on H, which happens exactly at the singularities of

$$\Gamma_{d+2} = \begin{bmatrix} 1 & x_{i_1,1} & x_{i_1,2} & \ldots & x_{i_1,d} & \sum_{j=1}^{d} x_{i_1,j}^2 \\ 1 & x_{i_2,1} & x_{i_2,2} & \ldots & x_{i_2,d} & \sum_{j=1}^{d} x_{i_2,j}^2 \\ \vdots & \vdots & \vdots & & \vdots & \vdots \\ 1 & x_{i_{d+1},1} & x_{i_{d+1},2} & \ldots & x_{i_{d+1},d} & \sum_{j=1}^{d} x_{i_{d+1},j}^2 \\ 1 & x_{i_{d+2},1} & x_{i_{d+2},2} & \ldots & x_{i_{d+2},d} & \sum_{j=1}^{d} x_{i_{d+2},j}^2 \end{bmatrix}.$$

Eliminating degeneracies for the particular matrix could be achieved by the "cheap trick" of [EM90] which perturbs the points on the higher-dimensional paraboloid, by perturbing the sum of squares as if it were an additional coordinate. However, this may lead to inconsistencies in some special configurations, if the same algorithm also uses another primitive such as Λ_{d+1}. Consider, for instance, deciding the relative position of a line and a circle which touch at two coincident points x_1 and x_2, by using the Orientation primitive on the line and x_1 and the InSphere primitive on the circle and x_2.

Validity reduces by theorem 6.3.1 to proving the nonsingularity of the Vandermonde-resembling matrix

$$W_{d+2} = \begin{bmatrix} 1 & i_1 & \ldots & i_1^d & \sum_{j=1}^{d} i_1^{2j} \\ 1 & i_2 & \ldots & i_2^d & \sum_{j=1}^{d} i_2^{2j} \\ \vdots & \vdots & & \vdots & \vdots \\ 1 & i_{d+2} & \ldots & i_{d+2}^d & \sum_{j=1}^{d} i_{d+2}^{2j} \end{bmatrix},$$

which follows from proposition 6.3.3 or theorem 6.3.4. Unfortunately, the hypothesis of the theorem is not readily satisfied by the second perturbation (6.4). A similar scheme, with residues taken mod q, with $q = \Omega(n^{d-1})$, has been recently shown [Thi93] to be valid, offering a slight improvement on complexity.

The perturbed determinant expands to the sum

$$det\Gamma_{d+2}(\epsilon) = \begin{vmatrix} 1 & x_{i_1,1}(\epsilon) & \ldots & x_{i_1,d}(\epsilon) & \epsilon^2 \sum_{j=1}^{d} i_1^{2j} + \epsilon(2\sum_{j=1}^{d} x_{i_1,j} i_1^j - i_1^{d+2}) \\ \vdots & \vdots & & \vdots & \vdots \\ 1 & x_{i_{d+2},1}(\epsilon) & \ldots & x_{i_{d+2},d}(\epsilon) & \epsilon^2 \sum_{j=1}^{d} i_{d+2}^{2j} + \epsilon(2\sum_{j=1}^{d} x_{i_{d+2},j} i_{d+2}^j - i_{d+2}^{d+2}) \end{vmatrix}$$

$$+ \begin{vmatrix} 1 & x_{i_1,1}(\epsilon) & \dots & x_{i_1,d}(\epsilon) & \sum_{j=1}^{d} x_{i_1,j}^2 + \epsilon i_1^{d+2} \\ & \vdots & & \vdots & \vdots \\ 1 & x_{i_{d+2},1}(\epsilon) & \dots & x_{i_{d+2},d}(\epsilon) & \sum_{j=1}^{d} x_{i_{d+2},j}^2 + \epsilon i_{d+2}^{d+2} \end{vmatrix}.$$

Computing each of the two determinants can be reduced to a characteristic polynomial computation by theorem 6.4.3. For the first determinant, it suffices to move an ϵ factor from the last to the first column, while for the second determinant the first column must be multiplied by ϵ.

Theorem 6.5.4 *Perturbation (6.2) is valid with respect to the InSphere primitive and increases its arithmetic complexity by a $\mathcal{O}(\log d)$ factor. Under the bit model, the worst-case complexity increases by a $\mathcal{O}^*(d)$ factor.*

Proof Validity is already established, based on theorem 6.3.4. The original arithmetic complexity is $\Theta(MM(d))$ and, by corollary 6.4.4, the new complexity is $\mathcal{O}(MM(d)\log d)$. In the worst case, the determinant has size $\Theta(ds)$. With modular arithmetic the original bit complexity is then

$$\Theta(d^2(ds)^{1+\alpha} + dsMM(d)),$$

where α denotes the smallest of several positive constants.

For the perturbed primitive modular arithmetic is used and the number of finite fields is $\mathcal{O}(ds + d^2 \log n)$ since this is the coefficient size in each of the two characteristic polynomials that must be computed. This bound follows from the fact that each coefficient is the sum of certain minors of a matrix of order $d+2$, whose entries have bit size bounded by the maximum of s, for the input data, and $d \log n$ for the perturbation quantities. Hence the bit complexity of evaluating the primitive is

$$\mathcal{O}(d^2(ds + d^2 \log n)^{1+\alpha} \; + \; (ds + d^2 \log n)MM(d) \log d).$$

The overhead now follows from $s \geq \log n$. □

6.5.4 Other Primitives

Our techniques directly apply to several other primitives including those presented in [EM90]. Primitives that decide on the relative position of derived objects may pose a limitation to our method. Consider, for instance, the two-dimensional ham-sandwich algorithm in [EW86] with lines on the plane being the input objects and their intersection points being the derived objects. The three primitives of the algorithm are: deciding whether a point lies above or below a line; comparing the first coordinate of two points; and comparing the distances of two points from a line.

Applying scheme (6.2) to the points removes all degeneracies but it is not clear that this does not create some inconsistent configuration. Applied to the input lines, SoS successfully perturbs them into general position; however, perturbation (6.2) fails for the second test. We considered a scheme using the first n primes, denoted as $q_1, \dots, q_n$:

$$x_{i,j}(\epsilon) \; = \; x_{i,j} \; + \; \epsilon(q_i^j). \tag{6.5}$$

The bit complexity of the perturbation quantities is then $\mathcal{O}(d \log n)$. Applied to the ham-sandwich algorithm, perturbation (6.5) is valid for the second but not for the third test. One should keep in mind that consistency requires that exactly one scheme is applied to all primitives of a specific algorithm.

6.6 Arbitrary Rational Functions

For the general case where the branching tests are arbitrary rational functions, we propose a randomized perturbation which is easy to implement and applies to algebraic problems such as INV and linear programming as well as geometric algorithms whose branching functions are not covered by those examined above.

Let f be an arbitrary rational function whose sign determines the direction taken at some branch and express it as p/q, where p, q are polynomials in the input variables, each of total degree bounded by D, where D is the maximum total degree in the input variables of any polynomial in the real RAM program. Suppose that the input variable vector x belongs to $\mathbb{R}^n$.

For a given input, define the perturbed instance $x(\epsilon) = (x_1(\epsilon), \ldots, x_n(\epsilon))$ as follows:

$$x_i(\epsilon) = x_i + \epsilon r_i, \qquad i = 1, \ldots, n, \tag{6.6}$$

where ϵ is an infinitesimal symbolic variable and r_i is a random integer. Each r_i is chosen uniformly over a range that depends on the desired probability that none of the branching polynomials vanishes. This probability of success can be fixed to be arbitrarily high. It is parameterized by a real constant $c \geq 1$; all claims in this section hold with probability at least $1 - 1/c$.

The total number of polynomials appearing at the numerator or denominator of a branch expression is at most $2 \cdot 3^T$, where T is the maximum number of branches on any execution path. Schwartz's lemma [Sch80] requires that the range of the random values contains at least as many values as the product of c and the total degree of the polynomial whose roots we wish to avoid. Here, this polynomial is the product of all branch polynomials, hence its degree is bounded by $2 \cdot 3^T D$. Therefore, the bit size of the perturbation quantities is

$$\lceil \lg c + \lg D + (\lg 3)\, T + 1 \rceil,$$

where lg denotes the logarithm of base 2.

It is feasible that for some set of random variables, $x(\epsilon)$ will still cause some branching polynomial to vanish. In this case the perturbation has failed, so the algorithm is restarted and new random variables are picked, independently and uniformly distributed over the same range. It is clear intuitively that any deterministic scheme avoiding the zeros of all polynomials would take time at least exponential in the number of variables. Intuitively, our method is faster because it randomly selects one n-dimensional perturbation vector instead of trying out all possible ones.

Lemma 6.6.1 *Let the entries of $r = (r_1, \ldots, r_n)$ be independently and uniformly chosen integers of $\lceil \lg c + \lg D + (\lg 3)\, T + 1 \rceil$ bits each, for any $c \geq 1$. Then, there exists with probability at least $1 - 1/c$, a positive real constant ϵ_0 such that, for every positive real $\epsilon < \epsilon_0$, every branching rational function $f(x + \epsilon r)$ is defined, nonzero and of constant sign.*

Proof Let $g(x + \epsilon r)$ be any polynomial appearing at the numerator or denominator of some branch expression and let $G(a + \epsilon r)$ be the product of all distinct polynomials g. By hypothesis, none of these polynomials is identically zero, therefore G also is not identically zero. For a moment, fix $\epsilon = 1$ and consider $G(a + 1r)$ as a polynomial in r, whose degree in x and r is the same. Since D bounds the total degree of any polynomial g, the total degree of G is at most $2 \cdot 3^T D$. Now we apply a lemma proven in [Sch80]. The probability that r, chosen uniformly at random with the given size, is a root of $G(a+1r)$ is at most $1/c$. All claims that follow concern the particular r and hold with probability at least $1 - 1/c$.

First observe that none of the polynomials $g(a + 1r)$ vanishes at r, hence every $g(x + \epsilon r)$ may be regarded as a polynomial in ϵ that is not identically zero. Consequently, its zero set is of positive codimension and, more specifically, a finite point set. Consider the minimum positive root for every g and let ϵ_0 be the minimum over all polynomials g. □

Theorem 6.6.2 *Perturbation (6.6) is valid with arbitrarily high probability, with respect to any algorithm that branches on rational functions in the input variables.*

Proof The perturbed instance is arbitrarily close to the original one as ϵ tends to zero. Branches decide on the sign of a perturbed rational expression, which is the sign of the lowest-order term in the ϵ-polynomial that does not vanish. By the previous lemma all polynomials have a constant nonzero sign for sufficiently small ϵ, hence $x(\epsilon)$ is in general position. For nondegenerate inputs all query polynomials have a non-vanishing real part, i.e.a term independent of ϵ which dominates the sign. □

What is the tradeoff in efficiency? The arithmetic complexity of the algorithm is increased by the time required to manipulate the ϵ-polynomials symbolically which depends on D, since the degree of every polynomial in ϵ is the same as its total degree in x. Here we make no assumptions about the structure of the polynomials, hence we are bound to use the results on the dense case in section 6.4. The bit complexity is also affected by D but not by c which is fixed.

Lemma 6.6.3 *Under perturbation (6.6) the arithmetic time complexity of the branching instructions and the arithmetic operations is $\mathcal{O}(D)$ and $\mathcal{O}(D \log^2 D)$ respectively.*

Proof Each operation in $\{+, -, \times, /\}$ involves multiplication of ϵ-polynomials and a Greatest Common Divisor computation to reduce to lowest terms so that the degree bound D is observed. The multiplication takes time $\mathcal{O}(D \log D)$ and the GCD $\mathcal{O}(D \log^2 D)$ by interpolation techniques based on FFT [AHU74, Ch. 8]. Branching instructions must find the lowest non-vanishing term in the corresponding ϵ-polynomial, which takes $\mathcal{O}(D)$ time. □

Degree D cannot be bounded in general by a polynomial in the arithmetic complexity of the original algorithm, which implies that the perturbation may be prohibitively expensive under the arithmetic model. Exponentiating a rational number, for instance, takes roughly a logarithmic number of steps in the exponent, while on perturbed input the worst-case arithmetic complexity is at least linear in it, which means the complexity overhead is exponential in the original complexity. However, we obtain better bounds by considering bit complexities.

Theorem 6.6.4 *Under the arithmetic model, the running-time increases due to perturbation (6.6) by a multiplicative factor of $\mathcal{O}^*(D)$, where D is the maximum total degree in the input variables of any polynomial in the real RAM program. Under the bit model, the overhead for the worst-case complexity is $\mathcal{O}^*(\phi^2(n, s))$, where $\phi(n, s)$ asymptotically bounds the original worst-case bit complexity of the algorithm, s is the maximum bit size of the input quantities.*

Proof By the previous lemma for every instruction the overhead is $\mathcal{O}(D \log^2 D)$. This establishes the arithmetic complexity overhead.

By the definition of T there exists an execution path with bit complexity $\Omega(T)$, hence $\phi(n, s) = \Omega(T)$. By the definition of D, there exists a path where a polynomial of degree D in the input variables

is computed. Since the only legal arithmetic operations lie in $\{+, -, *, /\}$, there must exist an earlier operation on this path computing a polynomial of degree at least $D/2$, hence computing values of bit size $sD/2$. The operation that uses these values as operands has bit cost $\Omega(sD/2) = \Omega(D)$. Hence $\phi(n, s) = \Omega(D)$.

After the perturbation, the same program operates on perturbed quantities; their starting bit size is multiplied by $\mathcal{O}(\log D + T)$, assuming the probability of success c in choosing the perturbation quantities is constant. Moreover, the arithmetic complexity has overhead $\mathcal{O}(D \log^2 D)$. Hence, the worst-case bit complexity overhead is

$$\mathcal{O}((\log D + T) D \log^2 D). \tag{6.7}$$

Recalling the two lower bounds on $\phi(n, s)$, we have $\phi(n, s)^{2+\alpha} = \Omega(T D^{1+\alpha})$, which bounds the bit complexity overhead (6.7), for some appropriate $\alpha > 0$. □

An immediate consequence is

Corollary 6.6.5 *Perturbation (6.6) does not affect the worst-case bit complexity class of the algorithm. In particular, if the original complexity lies in P or EXPTIME, then the complexity on perturbed input also lies in P or EXPTIME respectively.*

Taking up the running example of Gaussian elimination for INV, we observe that no checks for zero denominators have to be carried out on perturbed input. Perturbation thus eliminates the need for interchanging rows. Computation is symbolic, with GCD operations at every step in order to cancel common terms and thus prohibit the degree in ϵ of the symbolic polynomials from growing exponentially in the number of examined rows. For nonsingular matrices, the result is obtained by setting ϵ to zero at the end. For singular instances, this causes some denominator to vanish, so we take the limit as ϵ goes to zero. For singular matrices the result is some real matrix that approximates, in a sense, the inverse.

6.7 Computing Convex Hulls

This section discusses our implementation of the Beneath-Beyond algorithm [Ede87] which solves the convex hull volume (CHV) problem for finite input sets of integral points in arbitrary dimension, and reports on the running-time performance. The implementation produces approximate solutions to the general convex hull face-structure (CHF) problem, in a sense specified later. We examine the issue of postprocessing that arises when we wish to recover the exact facet structure from the output.

Convex hull computation in general dimension is a fundamental geometric problem with a wide variety of applications, such as visibility and illumination in modeling and graphics [TS91, TFFH94], collision detection in robotics and animation [LMC94], material identification in geology [Boa93], molecular docking in drug fabrication [Con91] as well as in the context of this thesis for computing mixed subdivisions and solving systems of nonlinear equations by sparse homotopies.

The algorithm is designed under the assumption of nondegeneracy; perturbation (6.4) is applied in order to allow arbitrary inputs, since the only two primitive tests needed are Ordering on the first coordinate and Orientation. The exact volume of the convex hull is possible to obtain without any postprocessing because, by proposition 6.1.4, the problem mapping

$$\mathrm{CHV} : \mathbb{Z}^{dn} \to \mathbb{Q}, \qquad n = \text{ point cardinality}, d = \text{ dimension},$$

d	n	user CPU time $\rho \simeq 1$ (random)	$\rho \simeq .5$	$\rho = 0$ (coincident)
4	54	4s		24s
4	100	8s	40s	1m 6s
4	200	19s	1m 11s	2m 8s
4	500	53s		7m 39s
3	100	0s		11s
4	100	8s	40s	1m 6s
5	100	43s	4m 7s	5m 19s
6	100	4m 18s		45m 15s
7	100	29m 1s		

Table 6.1: Performance of the convex hull volume implementation.

is continuous everywhere. The algorithm sorts all points on their first coordinate and then proceeds incrementally by adding each new point to the convex hull of all previous points. Due to the perturbation, each region between the new point and the existing hull can be partitioned into d-simplices, each defined by the new point and one of the existing facets. Then, it is straightforward to compute the exact volume by summing all simplex volumes whose expression as an ϵ-polynomial has a nonzero constant term. Note that extension to rational inputs is possible without affecting the asymptotic complexity; for the case of modular arithmetic see [DST88].

The *extreme* points of a given point-set are those that strictly maximize the inner product with some d-vector, i.e.they are not expressible as a convex combination of the other points; these are exactly the vertices of the convex hull. Perturbation (6.4) guarantees that the output polytope is simplicial; its vertex set is a superset of the extreme points because it may contain some points that are not extreme but simply *extremal*, i.e.they maximize the inner product with a certain vector. Hence the output vertex set is not always of minimum cardinality. Also, the number of facets may not be minimum because of the extremal points reported as vertices and because all facets are triangulated.

If a specific application required that the output polytope had the minimum number of facets regardless of whether they are simplices or not, then certain adjacent facets would have to be merged. This can be accomplished by comparing the normals of every two facets adjacent to a ridge, for every ridge. The normal of a facet can be computed in $\mathcal{O}(MM(d))$ and there are d tests per facet, hence this postprocessing does not affect the worst-case complexity of the program.

The implementation has benefited from code written by H. Rosenberger, E. Mücke and D. Manocha. The current version is in C and free for distribution. It includes about 1000 lines for the main combinatorial part, 600 lines for the perturbation part and 1400 lines for the modular and big-integer exact arithmetic package. The program is available from `ftp://robotics.eecs.Berkeley.edu/pub/ConvexHull`. The performance of the program on certain input instances is reported on table 6.1, for experiments on a Sun Sparc 10/40. See table 1.1 for this machine's ratings.

Each Orientation test comprises of a heuristic calculation of $\det \Lambda_{d+1}$; only if this vanishes the reduction to a characteristic polynomial is undertaken. As before, d and n stand for the dimension and the number of input points respectively and all coordinates are integers in $(-100, 100)$. The output of the program is the rational volume and a list of facets, each described by the defining input points; no

postprocessing was implemented. The user CPU running times are rounded down to an integer number of seconds.

For fixed d and n we have experimented on inputs of various degrees of degeneracy. The last three columns are headed by the approximate fraction ρ of Orientation tests whose evaluation is nonzero on the original input; these tests are carried out as determinant calculations. Thus, the first column corresponds to random inputs with practically all tests being generic. The other extreme has inputs with coincident points, constructed to test the program's performance when all Orientation tests reduce to a characteristic polynomial. The middle column corresponds to point sets comprised of random points, generated as above, and coincident points, at an appropriate ratio.

In analyzing these results it must be remembered that the program's complexity depends on the number of facets in the partial convex hulls. In the worst case, the hull of n points in d dimensions has $\mathcal{O}(n^{\lfloor d/2 \rfloor})$ facets. However, the expected number of facets for points selected randomly as above is proportional to $\log^{d-1} n$ [BKST78]; this is verified by our experimental results.

Chapter 7

Conclusions

This thesis proposes efficient algorithmic solutions to elimination problems in computer algebra and computational algebraic geometry. Root finding may be considered as a special case of elimination and is the main purpose of our algorithms. Given an arbitrary system of nonlinear multivariate polynomial equations, its resultant polynomial characterizes the solvability of the system. Moreover, it serves in eliminating variables and allows non-linear systems to be transformed to linear eigenproblems.

Our contribution is to describe the first efficient and general algorithms for computing the sparse resultant, which arises in the context of toric varieties and sparse elimination theory. Both algorithms define matrices whose determinants are divisible by the sparse resultant. The sparse resultant generalizes the classical homogeneous resultant and exploits the structure of the given polynomials. The sparse resultant is defined for any polynomial system, and is the simplest condition for solvability given the term structure. Its size is known to depend only on the geometry of the Newton polytopes of the input polynomials.

We describe two algorithms for computing sparse resultants via matrix determinants. The first is the subdivision algorithm, which uses a decomposition of the Minkowski sum of the newton polytopes to define a matrix. The size of this matrix is well-characterized and is approximately the Minkowski sum volume. The determinant of this matrix is a multiple of the resultant, and in general has more rows than the degree of the resultant. The resultant matrix helps establish a special case of the effective Nullstellensatz in terms of Newton polytopes. We also conjecture that it can be used as the numerator matrix in an exact quotient formula for the sparse resultant like Macaulay's for the classical one.

We can speak of an optimal, or Sylvester-type, matrix whose size equals the resultant degree, but for most systems, no such optimal matrix exists. Since the first method produces matrices which are sometimes much larger than optimal, we were motivated to improve it. The second algorithm uses a heuristic to produce smaller matrices than the first. We show that it produces optimal matrices in all the cases where they are known to exist. Furthermore, in experiments so far, the heuristically-constructed matrices are within a factor of 3 of optimal size.

The sparseness of the system is measured by the *mixed volume* of its newton polytopes, which gives an upper bound on the number of isolated roots. We describe a refinement of Sturmfels' subdivision algorithm for mixed volume which is very fast and has been used to derive new degree bounds for a popular class of algebraic benchmarks. Bounds are derived between mixed volume and the Minkowski sum, and these are used in the complexity analysis of the resultant algorithms.

Two methods based on the sparse resultant are suggested for calculating the common roots and then applied to concrete problems in robotics, vision and molecular kinematics. This illustrates our

claim that problems from these areas often exhibit structure that can be exploited in the context of sparse elimination. Our solver finds all isolated solutions to satisfactory accuracy and speed; in addition, its generality should establish it as the method of choice for systems of moderate size.

Lastly, we defined the notion of input perturbation and have concentrated on linear schemes, which are amenable to efficient computation techniques. In particular, we have proposed two such schemes that are valid for certain important geometric primitives. The merit of these schemes is twofold. First, their simplicity makes them attractive for practical use and, second, they are the most efficient to date. Moreover, our randomized scheme has arbitrarily high probability of success and trades randomization for efficiency; it applies to any algorithm with arbitrary rational branching functions.

7.1 Further Work

Of both theoretical as well as practical interest is to settle conjecture 3.1.19 about a rational formula for the sparse resultant. The classical work of Macaulay [Mac02] is a source of inspiration and, more concretely, of arguments that one may be able to translate from the projective case to the sparse case. An important question of theoretical interest is to extend theorem 3.1.21 to a full effective Nullstellensatz in terms of Newton polytopes.

A theoretical explanation of the incremental algorithms' efficiency should be possible through the theory of Koszul complexes and a generalization of the notion of degree to sparse polynomials based on Newton polytopes. The determination of favorable vectors v for different classes of systems would automate the process of constructing compact matrix formulae.

A longer-term challenge, which requires experience with both the empirical and the theoretical facet of this research, is to understand the relation and comparative advantage of different approaches to elimination. Namely, we would like to explain better the conditions under which sparseness can be exploited by resultant-based methods as opposed to Gröbner bases or numerical methods. Moreover, we should study the extent to which some combination of these approaches may be fruitful.

Invariably, resultants are used in the most efficient methods for quantifier elimination and decision procedures for the theory of the reals [Gri88, Can88a, Ren92, BPR94]. If the given polynomials are sparse in the current sense, the sparse resultant would be the natural choice in all of these methods. It is an interesting to characterize the polynomials in the final system in terms of Newton polytopes in order to decide whether they retain any structure.

An important merit of this work is its practical application in solving algebraic systems in robot kinematics, vision, modeling as well as computational biology. In this respect, one open question is the transformation of arbitrary systems to an equivalent form that is amenable to sparse elimination and in particular to the computation of mixed volumes and sparse resultants. A crucial question in this respect is the notion of genericity, in particular in relation to Bernstein's bound.

This leads us to generic position and the symbolic perturbations that achieve it. The basic existential question on the perturbation method is still open. After the flurry of papers proposing different perturbation schemes, some objections have recently been voiced against the general applicability of this method [DMY93, BMS94] motivated by the observation that the difficulty and complexity of postprocessing might dominate that of the entire program.

Bibliography

[ABB+92] E. Anderson, Z. Bai, C. Bischof, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, S. Ostrouchov, and D. Sorensen. *LAPACK Users' Guide.* SIAM, Philadelphia, 1992.

[AG90] E. Allgower and K. Georg. *Numerical Continuation Methods.* Springer Verlag, Berlin, 1990.

[AHU74] A.V. Aho, J.E. Hopcroft, and J.D. Ullman. *The Design and Analysis of Computer Algorithms.* Addison-Wesley, Reading, Mass., 1974.

[Ale37] A.D. Aleksandrov. On the Theory of Mixed Volumes of Convex Bodies, II. New Inequalities Between Mixed Volumes and Their Applications. *Math. Sb. N. S.*, 2:1205–1238, 1937. In Russian.

[AMR92] S. Azhar, A. McLennan, and J.H. Reif. Computation of equilibria in noncooperative games. In *Proc. Workshop for Computable Economics*, December 1992.

[AS88] W. Auzinger and H.J. Stetter. An elimination algorithm for the computation of all zeros of a system of multivariate polynomial equations. In *Proc. Intern. Conf. on Numerical Math., Intern. Series of Numerical Math., 86*, pages 12–30. Birkhäuser Verlag, Basel, 1988.

[BDMS79] J. Bunch, J. Dongarra, C. Moler, and G.W. Stewart. *LINPACK User's Guide.* SIAM, Philadelphia, 1979.

[Ber75] D.N. Bernstein. The number of roots of a system of equations. *Funct. Anal. and Appl.*, 9(2):183–185, 1975.

[BF91a] J. Backelin and R. Fröberg. How we proved that there are exactly 924 cyclic 7-roots. In *Proc. ACM Intern. Symp. on Symbolic and Algebraic Computation*, pages 103–111, Bonn, 1991.

[BF91b] G. Björck and R. Fröberg. A faster way to count the solutions of inhomogeneous systems of algebraic equations, with applications to cyclic n-roots. *J. Symbolic Computation*, 12:329–336, 1991.

[BF94] G. Björck and R. Fröberg. Methods to "divide out" certain solutions from systems of algebraic equations, applied to find all cyclic 8-roots. Manuscript, Dept. of Math., Stockholm University, 1994.

[BGW88] C. Bajaj, T. Garrity, and J. Warren. On the applications of multi-equational resultants. Technical Report 826, Purdue Univ., 1988.

[Bjö90] G. Björck. Functions of modulus 1 on z_n whose Fourier transforms have constant modulus, and "cyclic n-roots". In J.S. Byrnes and J.F. Byrnes, editors, *Recent Advances in Fourier Analysis and Its Applications*, pages 131–140. Kluwer Academic, 1990. NATO Adv. Sci. Inst. Ser. C.

[BK86] E. Brieskorn and H. Knörrer. *Plane Algebraic Curves.* Birkhäuser, Basel, 1986.

[BKST78] J.L. Bentley, H.T. Kung, M. Schkolnick, and C.D. Thompson. On the average number of maxima in a set of vectors. *J. ACM*, 25:536–543, 1978.

[BMS94] C. Burnikel, K. Mehlhorn, and S. Schirra. On degeneracy in geometric computations. In *Proc. 5th ACM–SIAM Symp. on Discr. Algorithms*, pages 16–23, 1994.

[Boa93] J.W. Boardman. Automated spectral unmixing of aviris data using convex geometry concepts. In *Proc. 4th Airborne Geoscience Workshop*, Washington, D.C., October 1993.

[BPR94] S. Basu, R. Pollack, and M.-F. Roy. On the combinarotial and algebraic complexity of quantifier elimination. In *Proc. IEEE Symp. Foundations of Comp. Sci.*, Sante Fe, New Mexico, 1994. To appear.

[Bro87] D. Brownawell. Bounds for the degree in the Nullstellensatz. *Annals of Math., Second Ser.*, 126(3):577–591, 1987.

[BS92] L.J. Billera and B. Sturmfels. Fiber polytopes. *Annals of Math.*, 135:527–549, 1992.

[Buc85] B. Buchberger. Gröbner bases: An algebraic method in ideal theory. In N.K. Bose, editor, *Multidimensional System Theory*, pages 184–232. Reidel Publishing Co., 1985.

[Buc89] B. Buchberger. Applications of Gröbner bases in non-linear computational geometry. In D. Kapur and J. Mundy, editors, *Geometric Reasoning*, pages 415–447. M.I.T. Press, 1989.

[BZ88] Y.D. Burago and V.A. Zalgaller. *Geometric Inequalities.* Grundlehren der mathematischen Wissenschaften, 285. Springer, Berlin, 1988.

[Can88a] J. Canny. Some algebraic and geometric computations in pspace. In *Proc. ACM Symp. Theory of Computing*, pages 460–467, 1988.

[Can88b] J.F. Canny. *The Complexity of Robot Motion Planning.* M.I.T. Press, Cambridge, Mass., 1988.

[Can90] J. Canny. Generalised characteristic polynomials. *J. Symbolic Computation*, 9:241–250, 1990.

[Can91] J.F. Canny. Computing roadmaps of semi-algebraic sets. In *Proc. Intern. Symp. Applied Algebra, Algebraic Algorithms and Error-Correcting Codes*, New Orleans, 1991.

[Can94] J. Canny. A toolkit for non-linear algebra. In J.-C. Latombe and K. Goldberg, editors, *Proc. Workshop on the Algorithmic Foundations of Robotics*, volume I, San Francisco, 1994. A.K. Peters. To appear in 3/95.

[Cay48] A. Cayley. On the theory of elimination. *Dublin Math. J.*, II:116–120, 1848.

[CE93] J. Canny and I. Emiris. An efficient algorithm for the sparse mixed resultant. In G. Cohen, T. Mora, and O. Moreno, editors, *Proc. Intern. Symp. Applied Algebra, Algebraic Algor. and Error-Corr. Codes, Lect. Notes in Comp. Science 263*, pages 89–104, Puerto Rico, 1993. Springer Verlag.

[CG84] A.L. Chistov and D.Y. Grigoryev. *Complexity of Quantifier Elimination in the Theory of Algebraically Closed Fileds.* Lect. Notes Comp. Sci. 176. Springer-Verlag, New York, 1984.

[CGH89] L. Caniglia, A. Galligo, and J. Heintz. Some new effective bounds in computational geometry. In *Proc. Intern. Symp. Applied Algebra, Algebraic Algor. and Error-Corr. Codes, Lect. Notes in Comp. Science 357*, pages 131–152. Springer Verlag, 1989.

[Cha93] M. Chardin. The resultant via a Koszul complex. In F. Eyssette and A. Galligo, editors, *Computational Algebraic Geometry*, volume 109 of *Progress in Mathematics*, pages 29–39. Birkhäuser, Boston, 1993. Presented at MEGA '92.

[CL82] G.E. Collins and R. Loos. Real zeros of polynomials. In B. Buchberger, G.E. Collins, and R. Loos, editors, *Computer Algebra: Symbolic and Algebraic Computation*, pages 83–94. Springer-Verlag, Wien, 2nd edition, 1982.

[Con91] M.L. Connolly. Molecular Interstitial Skeleton. *Computers Chem.*, 15(1):37–45, 1991.

[CR91] J. Canny and J.M. Rojas. An optimal condition for determining the exact number of roots of a polynomial system. In *Proc. ACM Intern. Symp. on Symbolic and Algebraic Computation*, pages 96–102, Bonn, July 1991.

[Cra89] J.J. Craig. *Introduction to Robotics: Mechanics and Control.* Addison-Wesley, Reading, Mass., 2nd edition, 1989.

[CW90] D. Coppersmith and S. Winograd. Matrix multiplication via arithmetic progressions. *J. Symbolic Computation*, 9:251–280, 1990.

[Dan63] G.B. Dantzig. *Linear Programming and Extensions.* Princeton University Press, Princeton, 1963.

[Dem88] M. Demazure. Sur deux problèmes de reconstruction. Technical Report 882, I.N.R.I.A., Rocquencourt, 1988.

[DF88] M. Dyer and A. Frieze. The complexity of computing the volume of a polyhedron. *SIAM J. Comput.*, 17:967–974, 1988.

[DGH94] M. Dyer, P. Gritzmann, and A. Hufnagel. On the complexity of computing mixed volumes. Manuscript, December 1994.

[Dix08] A.L. Dixon. The eliminant of three quantics in two independent variables. *Proceedings of London Mathematical Society*, 6:49–69, 209–236, 1908.

[DMY93] K. Dobrindt, K. Mehlhorn, and M. Yvinec. A complete framework for the intersection of a general polyhedron with a convex one. In *Proc. 3rd Workshop Algorithms Data Struct., Lect. Notes Comp. Science 709*, pages 314–324. Springer-Verlag, Berlin, 1993.

[Dre78] F.J. Drexler. A homotopy method for the calculation of zeros of zero dimensional ideals. In H.G. Wacker, editor, *Continuation Methods*. Academic Press, New Yprk, 1978.

[DST88] J.H. Davenport, Y. Siret, and E. Tournier. *Computer Algebra*. Academic Press, London, 1988.

[EC93] I. Emiris and J. Canny. A practical method for the sparse resultant. In *Proc. ACM Intern. Symp. on Symbolic and Algebraic Computation*, pages 183–192, Kiev, 1993.

[EC95] I.Z. Emiris and J.F. Canny. A general approach to removing degeneracies. *SIAM J. Computing*, 24(3), 1995. To appear. A preliminary version in *Proc. IEEE Symp. Foundations of Comp. Sci.,* 1991, pp. 405–413.

[ECS94] I.Z. Emiris, J.F. Canny, and R. Seidel. Efficient Perturbations for Handling Geometric Degeneracies. *Algorithmica, Spec. issue on comp. geometry in manufacturing*, 1994. Submitted.

[Ede86] H. Edelsbrunner. Edge-skeletons in arrangements with applications. *Algorithmica*, 1:93–109, 1986.

[Ede87] H. Edelsbrunner. *Algorithms in Combinatorial Geometry*. Springer-Verlag, Heidelberg, 1987.

[EG86] H. Edelsbrunner and L.J. Guibas. Topologically sweeping an arrangement. In *Proc. ACM Symp. Theory of Comp.*, pages 389–403, 1986.

[Ehr67] E. Ehrart. Sur un problème de géométrie diophantienne, i. polyèdres et réseaux. *J. Reine Angew. Math.*, 226:1–29, 1967.

[EM90] H. Edelsbrunner and E.P. Mücke. Simulation of simplicity: A technique to cope with degenerate cases in geometric algorithms. *ACM Trans. Graphics*, 9(1):67–104, 1990.

[Emi93] I. Emiris. An efficient computation of mixed volume. Technical Report 734, Computer Science Division, U.C. Berkeley, Berkeley, CA, 1993.

[Emi94] I.Z. Emiris. On the complexity of sparse elimination. Technical Report 840, Computer Science Division, U.C. Berkeley, 1994. Submitted for publication.

[EW86] H. Edelsbrunner and R. Waupotitsch. Computing a ham-sandwich cut in two dimensions. *J. Symbolic Comput.*, 2:171–178, 1986.

[Fau94] J.-C. Faugère, 1994. Personal Communication.

[Fen36] W. Fenchel. Inégalités quadratiques entre les volumes mixtes des corps convexes. *C. R. Acad. Sci. Paris*, 203:647–650, 1936.

[FL94] J.C. Faugère and D. Lazard. The combinatorial classes of parallel manipulators. *J. Mech. Mach. Theory*, 1994. To appear.

[FM90] O.D. Faugeras and S. Maybank. Motion from point matches: Multiplicity of solutions. *Intern. J. Comp. Vision*, 4:225–246, 1990.

[Ful93] W. Fulton. *Introduction to Toric Varieties*. Number 131 in Annals of Mathematics. Princeton University Press, Princeton, 1993.

[GCS91] Z. Gigus, J. Canny, and R. Seidel. Efficiently Computing and Representing Aspect Graphs of Polyhedral Objects. *IEEE Trans. on Pattern Analysis and Machine Intelligence*, 13(6):542–551, 1991.

[GKZ91] I.M. Gelfand, M.M. Kapranov, and A.V. Zelevinsky. Discriminants of polynomials in several variables and triangulations of Newton polytopes. *Leningrad Math. J.*, 2(3):449–505, 1991. (Translated from *Algebra i Analiz* 2, 1990, pp. 1–62).

[GKZ94] I.M. Gelfand, M.M. Kapranov, and A.V. Zelevinsky. *Discriminants and Resultants.* Birkhäuser, Boston, 1994.

[GL89] G.H. Golub and C.F. Van Loan. *Matrix Computations.* The Johns Hopkins University Press, Baltimore, Maryland, 1989.

[GLR82] I. Gohberg, P. Lancaster, and L. Rodman. *Matrix Polynomials.* Academic Press, New York, 1982.

[Gou56] V.E. Gough. Contribution to discussion to papers on research in automobile stability and control and in tyre performance by Cornell staff. In *Proc. Auto. Div. Instn. Mech. Engrs.*, pages 392–395, 1956.

[Gri88] D.Y. Grigoryev. The complexity of deciding Tarski algebra. *J. Symbolic Computation*, 5:65–108, 1988.

[Grü67] B. Grünbaum. *Convex Polytopes.* Wiley-Interscience, New York, 1967.

[GS70] N. Gō and H.A. Scheraga. Ring closure and local conformational deformations of chain molecules. *Macromolecules*, 3(2):178–187, 1970.

[GS73] N. Gō and H.A. Scheraga. Ring closure in chain molecules with c_n or s_{2n} symmetry. *Macromolecules*, 6(2):273–281, 1973.

[GV87] D.Y. Grigoryev and N.N. Vorobjov. Solving systems of polynomial inequalities in subexponential time. *J. Symbolic Computation*, Special Issue on Decision Algorithms for the Theory of Real Closed Fields, 1987.

[Hav91] T.F. Havel. An evaluation of computational strategies for use in the determination of protein structure from distance constraints obtained by nuclear magnetic resonance. *Pog. Biophys. molec. Biol.*, 56:43–78, 1991.

[HM91] J.L. Hafner and K.S. McCurley. Asymptotically fast triangularization of matrices over rings. *SIAM J. Computing*, 20(6):1068–1083, 1991.

[Hon86] J.W. Hong. Proving by example and gap theorem. In *Proc. IEEE Symp. Foundations of Comp. Sci.*, pages 107–116, 1986.

[Hor86] B.K.P. Horn. *Robot Vision.* M.I.T. Press, Cambridge, Mass., 1986.

[Hor91] B.K.P. Horn. Relative Orientation Revisited. *J. Opt. Soc. Am.*, 8(10):1630–1638, 1991.

[HRS93] J. Heintz, M.-F. Roy, and P. Solernò. Single exponential path finding in semi-algebraic sets, part ii: The general case. In C.L. Bajaj, editor, *Algebraic Geometry and its Applications*, pages 467–481. Springer Verlag, 1993.

[HS] B. Huber and B. Sturmfels. A polyhedral method for solving sparse polynomial systems. *Math. Comp.* To appear. A preliminary version presented at the Workshop on Real Algebraic Geometry, Aug. 1992.

[Hun74] T.W. Hungerford. *Algebra.* Graduate Texts in Mathematics, 73. Springer-Verlag, New York, 1974.

[Hur13] A. Hurwitz. Über die Trägheitsformen Eines Algebraischen Moduls. *Annali di Mat.*, Tomo XX(Ser. III):113–151, 1913.

[Hus94] M.L. Husty. An algorithm for solving the direct kinematics of Stewart-Gough-type platforms. Technical Report 94-1, McGill CIM, Montréal, 1994.

[IPC91] C. Innocenti and V. Parenti-Castelli. A new kinematic model for the closure equations of the generalized Stewart platform mechanism. In S. Stifter and J. Lenarčič, editors, *Advances in Robot Kinematics*, pages 457–464. Springer-Verlag, Wien, 1991.

[Jou91] J.-P. Jouanolou. Le formalisme du résultant. *Adv. in Math.*, 90(2), 1991.

[Kar84] N. Karmarkar. A new polynomial-time algorithm for linear programming. *Combinatorica*, 4:373–395, 1984.

[KG85] W. Keller-Gehrig. Fast algorithms for the characteristic polynomial. *Theor. Comp. Sci.*, 36:309–317, 1985.

[Kha93] L. Khachiyan. Complexity of polytope volume computation. In János Pach, editor, *New Trends in Discrete and Computational Geometry*, chapter IV. Springer-Verlag, 1993.

[Kho78] A.G. Khovanskii. Newton polyhedra and the genus of complete intersections. *Funktsional'nyi Analiz i Ego Prilozheniya*, 12(1):51–61, Jan.–Mar. 1978.

[Kho91] A.G. Khovanskii. *Fewnomials.* AMS Press, Providence, Rhode Island, 1991.

[Kol88] J. Kollár. Sharp effective Nullstellensatz. *J. Amer. Math. Soc.*, 1:963–975, 1988.

[KSZ92] M.M. Kapranov, B. Sturmfels, and A.V. Zelevinsky. Chow polytopes and general resultants. *Duke Math. J.*, 67(1):189–218, 1992.

[Kus75] A.G. Kushnirenko. The Newton polyhedron and the number of solutions of a system of k equations in k unknowns. *Uspekhi Mat. Nauk.*, 30:266–267, 1975.

[KY92] D. Kapur and Lakshman Y.N. Elimination methods: An introduction. In B. Donald, D. Kapur, and J. Mundy, editors, *Symbolic and Numerical Computation for Artificial Intelligence*, pages 45–89. Academic Press, 1992.

[Laz81] D. Lazard. Résolution des systèmes d'Équations algébriques. *Theor. Comp. Science*, 15:77–110, 1981.

[Laz93] D. Lazard. Generalized Stewart platform: How to compute with rigid motions? In *Proc. IMACS-SC*, 1993.

[LH81] H.C. Longuet-Higgins. A Computer Algorithm for Reconstructing a Scene from Two Projections. *Nature*, 293:133–135, 1981.

[LH88] H.C. Longuet-Higgins. Multiple Interpretations of a Pair of Images of a Surface. *Proc. Roy. Soc. London A*, 418, 1988.

[LMC94] M.C. Lin, D. Manocha, and J. Canny. Efficient contact determination for dynamic environments. In *Proc. IEEE Intern. Conf. Robotics and Automation*, pages 602–608, 1994.

[Loo82] R. Loos. Generalized polynomial remainder sequences. In B. Buchberger, G.E. Collins, and R. Loos, editors, *Computer Algebra: Symbolic and Algebraic Computation*, pages 115–137. Springer-Verlag, Wien, 2nd edition, 1982.

[Luo92] Q.-T. Luong. *Matrice Fondamentale et Auto-calibration en Vision par Ordinateur.* PhD thesis, Université de Paris-Sud, Orsay, Dec. 1992.

[LW94] T.Y. Li and X. Wang. The BKK root count in C^N. Manuscript, 1994.

[Mac02] F.S. Macaulay. Some formulae in elimination. *Proc. London Math. Soc.*, 1(33):3–27, 1902.

[Mac21] F.S. Macaulay. Note on the resultant of a number of polynomials of the same degree. *Proc. London Math. Soc.*, pages 14–21, 1921.

[Man92] D. Manocha. *Algebraic and Numeric Techniques for Modeling and Robotics.* PhD thesis, Computer Science Division, Department of Electrical Engineering and Computer Science, University of California, Berkeley, May 1992.

[Man94a] D. Manocha. Algorithms for computing selected solutions of polynomial equations. *J. Symbolic Computation*, 11:1–20, 1994.

[Man94b] D. Manocha. Solving systems of polynomial equations. *Comp. Graphics and Appl.*, pages 46–55, 1994. Special Issue on Solid Modeling.

[May85] S.J. Maybank. The angular velocity associated with the optical flow field due to a rigid moving body. *Proc. Roy. Soc. London A*, 401:317–326, 1985.

[MC27] F. Morley and A.B. Coble. New results in elimination. *American J. Math.*, 49:463–488, 1927.

[MC92a] D. Manocha and J. Canny. The impicit representation of rational parametric surfaces. *J. Symbolic Computation*, 13:485–510, 1992.

[MC92b] D. Manocha and J. Canny. Multipolynomial resultants and linear algebra. In *Proc. ACM Intern. Symp. on Symbolic and Algebraic Computation*, pages 96–102, 1992.

[MC92c] D. Manocha and J. Canny. Real time inverse kinematics of general 6R manipulators. In *Proc. IEEE Intern. Conf. Robotics and Automation*, pages 383–389, Nice, May 1992.

[MC93] D. Manocha and J. Canny. Multipolynomial resultant algorithms. *J. Symbolic Computation*, 15(2):99–122, 1993.

[McC90] J.M. McCarthy. *An Introduction to Theoretical Kinematics*. M.I.T. Press, Cambridge, Mass., 1990.

[Mis93] B. Mishra. *Algorithmic Algebra*. Springer Verlag, New York, 1993.

[MM94] R.D. McKelvey and A. McLennan. The maximal number of regular totally mixed Nash equilibria. Technical Report 865, Div. of the Humanities and Social Sciences, California Institute of Technology, Pasadena, Calif., July 1994.

[Mou93] B. Mourrain. The 40 "generic" positions of a parallel robot. In *Proc. ACM Intern. Symp. on Symbolic and Algebraic Computation*, pages 173–182, Kiev, July 1993.

[Mou94] B. Mourrain. Enumeration problems in geometry, robotics and vision. In *Proc. MEGA '94*, Santander, Spain, 1994. Birkhäuser. To appear in 1995.

[MS87] A.P. Morgan and A.J. Sommese. A homotopy for solving general polynomial systems that respects m-homogeneous structures. *Appl. Math. Comput.*, 24:101–113, 1987.

[MS94] H.M. Möller and H.J. Stetter. Multivariate polynomial equations with multiple zeros solved by matrix eigenproblems. Submitted for Publication, 1994.

[MSW] A.P. Morgan, A.J. Sommese, and C.W. Wampler. A product-decomposition theorem for bounding Bézout numbers. *SIAM J. Numerical Analysis*. To appear. Manuscript, 1993.

[MZW94] D. Manocha, Y. Zhu, and W. Wright. Conformational analysis of molecular chains using nano-kinematics. In *Proc. 1st IEEE Workshop on Shape and Pattern Recognition in Computational Biology*, pages 3–23, Seattle, 1994. Also Tech. Report 94-036, Dept. of Computer Science, Univ. of North Carolina.

[New60] I. Newton. *The Correspondence of Isaac Newton*, volume 2 (1676-1687). Cambridge Univ. Press, Cambridge, England, 1960.

[NWM90] P. Nanua, K.J. Waldron, and V. Murthy. Direct kinematic solution of a Stewart platform. *IEEE Trans. Robotics and Automation*, 6:438–444, 1990.

[NZAJ91] C.C Nguyen, Z.-L. Zhou, S.S. Antrazi, and C.E. Campbell Jr. Efficient computation of forward kinematics and Jacobian matrix of a Stewart platform-based manipulator. In *Proc. IEEE SOUTHEASTCON*, pages 869–874, Williamsburg, Virginia, April 1991.

[Par94] D. Parsons, 1994. Personal Communication.

[PC94] D. Parsons and J. Canny. Geometric problems in molecular biology and robotics. In *Proc. 2nd Intern. Conf. on Intelligent Systems for Molecular Biology*, pages 322–330, Palo Alto, CA, August 1994.

[Ped94] P. Pedersen. AMS–IMS–SIAM Summer Conference on Continuous Algorithms and Complexity. Mt. Holyoke, Mass., July 1994.

[Pri86] E.J.F. Primrose. On the input-output equation of the general 7R mechanism. *Mech. Mach. Theory*, 21:509–510, 1986.

[PS85] F.P. Preparata and M.I. Shamos. *Computational Geometry: An Introduction.* Springer-Verlag, New York, 1985.

[PS93] P. Pedersen and B. Sturmfels. Product Formulas for Resultants and Chow Forms. *Math. Zeitschrift*, 214:377–396, 1993.

[PS94] P. Pedersen and B. Sturmfels. Mixed Monomial Bases. In *Proc. MEGA '94*, Santander, Spain, 1994. Birkhäuser. To appear in 1995.

[Rag93] M. Raghavan. The Stewart platform of general geometry has 40 configurations. *J. Mech. Des.*, 115:277–282, 1993.

[Reg94] A. Rege. A practical algorithm for geometric theorem proving. Manuscript, U.C. Berkeley, December 1994.

[Ren87] J. Renegar. On the worst case arithmetic complexity of approximating zeros of systems of polynomials. Technical report, School of Operations Research, Cornell University, Ithaca, NY, 1987.

[Ren92] J. Renegar. On the computational complexity of the first-order theory of the reals, parts I, II, III. *J. Symbolic Computation*, 13(3):255–352, 1992.

[Rit50] J.F. Ritt. *Differential Algebra.* American Mathematical Society, New York, 1950.

[Roj94] J.M. Rojas. A convex geometric approach to counting the roots of a polynomial system. *Theor. Comp. Science*, 133(1):105–140, 1994.

[Rot93] B. Roth. Connections between robotic and classical mechanisms. In *Proc. 6th Intern. Symp. on Robot. Research*, Hidden Valley, Penn., October 1993. To appear.

[RV92] F. Ronga and T. Vust. Stewart platforms without computer. Manuscript, 1992.

[RvE93] M.-F. Roy and T. van Effelterre. Aspect Graphs of Algebraic Surfaces. In *Proc. ACM Intern. Symp. on Symbolic and Algebraic Computation*, pages 183–192, 1993.

[Sal85] G. Salmon. *Modern Higher Algebra.* G.E. Stechert and Co., New York, 1885. Reprinted 1924.

[SBD+76] B.T. Smith, J.M. Boyle, J.J. Dongarra, B.S. Garbow, Y. Ikebe, V.C. Klema, and C.B.Moler. *Matrix Eigensystem Routines – EISPACK Guide.* Lect. Notes in Comp. Science, 6. Springer-Verlag, Berlin, 1976.

[SBK92] B.K. Shoichet, D.L. Bodian, and I.K. Kuntz. Molecular Docking Using Shape Descriptors. *J. Computational Chem.*, 13(3):380–397, 1992.

[Sch80] J.T. Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *J. ACM*, 27(4):701–717, 1980.

[Sch93] R. Schneider. *Convex Bodies: The Brunn-Minkowski Theory.* Cambridge University Press, Cambridge, 1993.

[Sei94] R. Seidel. The nature and meaning of perturbations in geometric computing. In *Proc. 11th Symp. Theoret. Aspects Computer Science, Lect. Notes Computer Science 775*, pages 3–17. Springer-Verlag, 1994.

[SK91] B.K. Shoichet and I.K. Kuntz. Protein Docking and Complementarity. *J. Molec. Biol.*, 221:327–346, 1991.

[SNW94] B. Sandak, R. Nussinov, and H.J. Wolfson. 3-D flexible docking of molecules. In *Proc. 1st IEEE Workshop on Shape and Pattern Recognition in Computational Biology*, Seattle, 1994.

[Sta80] R.P. Stanley. Decompositions of Rational Convex Polyhedra. In J. Srivastava, editor, *Combinatorial Mathematics, Optimal Designs and Their Applications, Annals of Discrete Math. 6*, pages 333–342. North-Holland, Amsterdam, 1980.

[Sta90] R. Stanley. The number of faces of a simplicial convex polytope. *Adv. Math.*, 35:236–238, 1990.

[Ste65] D. Stewart. A platform with six degrees of freedom. In *Proc. Inst. Mech. Engs.*, volume 180, pages 371–386, London, 1965.

[Stu92] B. Sturmfels. The asymptotic number of real zeros of a sparse polynomial system. In *Proc. Workshop on Hamiltonian and Gradient Flows: Algorithms and Control*, Fields Institute, Waterloo, Ontario, 1992.

[Stu93a] B. Sturmfels. Sparse elimination theory. In D. Eisenbud and L. Robbiano, editors, *Proc. Computat. Algebraic Geom. and Commut. Algebra 1991*, pages 377–396, Cortona, Italy, 1993. Cambridge Univ. Press.

[Stu93b] B. Sturmfels. Viro's Theorem for Complete Intersections. Manuscript, 1993.

[Stu94] B. Sturmfels. On the Newton polytope of the resultant. *J. of Algebr. Combinatorics*, 3:207–236, 1994.

[SZ94] B. Sturmfels and A. Zelevinsky. Multigraded resultants of Sylvester type. *J. of Algebra*, 163(1):115–127, 1994.

[TFFH94] S. Teller, C. Fowler, T. Funkhouser, and P. Hanrahan. Partitioning and Ordering Large Radiosity Computations. *Computer Graphics (Proc. SIGGRAPH '94)*, 28, 1994.

[TH84] R.Y. Tsai and T.S. Huang. Uniqueness and estimation of three-dimensional motion parameters of rigid objects with curved surfaces. *IEEE Trans. on Pattern Analysis and Machine Intelligence*, 6:13–27, 1984.

[Thi93] T. Thiele, 1993. Personal Communication.

[TS91] S. Teller and C.H. Séquin. Visibility preprocessing for interactive walkthroughs. *Computer Graphics (Proc. SIGGRAPH '91)*, 25(4):61–69, 1991.

[vdW50] B.L. van der Waerden. *Modern Algebra*. F. Ungar Publishing Co., New York, 3rd edition, 1950.

[VG94] J. Verschelde and K. Gatermann. Symmetric Newton polytopes for solving sparse polynomial systems. Technical Report 3, Konrad-Zuse-Zentrum für Informationstechnik Berlin, 1994.

[Vir84] O.Ya. Viro. Gluing of plane real algebraic curves and construction of curves of degrees 6 and 7. In L.D. Faddeev and A.A. Malcev, editors, *Topology*, Lect. Notes in Math., pages 187–200. Springer-Verlag, 1984.

[Vir86] O.Ya. Viro. Progress in topology of real algebraic manifolds over the last six years. *Uspehki Math. Nauk.*, 41:45–67, 1986.

[VVC94] J. Verschelde, P. Verlinden, and R. Cools. Homotopies exploiting Newton polytopes for solving sparse polynomial systems. *SIAM J. Numerical Analysis*, 31(3):915–930, 1994.

[vzG88] J. von zur Gathen. Algebraic complexity theory. In J. Traub, editor, *Annual Review of Computer Science*, pages 317–347. Annual Reviews, Palo Alto, Cal., 1988.

[Wam93] C. Wampler. Polynomial continuation: A tutorial. Technical Report 8080, General Motors R & D, Warren, Mich., 1993. Presented at the Workshop on Solving Systems of Algebraic Equations, CIRM, Marseille, Nov. '93.

[Wam94] C. Wampler. Forward displacement analysis of general six-in-parallel SPS (Stewart) platform manipulators using soma coordinates. Technical Report 8179, General Motors R & D, Warren, Mich., 1994.

[Wil78] J.H. Wilkinson. Linear differential equations and Kronecker's canonical form. In C. de Boor and G.H. Golub, editors, *Recent Advances in Numerical Analysis*, pages 231–265. Academic Press, New York, 1978.

[WLW] G. Wang, T.Y. Li, and X. Wang. Solving the direct position kinematics of Stewart platforms by a homotopy continuation method. Manuscript.

[WS94] J.M. Wendlandt and S.S. Sastry. Design and control of a simplified Stewart platform for endoscopy. In *Proc. IEEE Conf. Decision Control*, Lake Buena Vista, Florida, December 1994. To appear.

[Wu84] W-T. Wu. Basic principles of mechanical theorem proving in geometries. *J. Sys. Sci. and Math. Sci.*, 4(3):207–235, 1984. Also in *J. Automated Reasoning.*

[WZ92] J. Weyman and A. Zelevinsky. Determinantal formulas for multigraded resultants. Manuscript, 1992.

[Yap90a] C.-K. Yap. A geometric consistency theorem for a symbolic perturbation scheme. *J. Comp. Sys. Sci.*, 40:2–18, 1990.

[Yap90b] C.-K. Yap. Symbolic treatment of geometric degeneracies. *J. Symbolic Comput.*, 10:349–370, 1990.

[Y.N90] Lakshman Y.N. *On the complexity of computing Gröbner bases for zero-dimensional polynomial ideals.* PhD thesis, Computer Science Department, Rensselaer Polytechnic Institute, Troy, New York, December 1990.

[Zip90] R. Zippel. Interpolating polynomials from their values. *J. Symbolic Computation*, 9:375–403, 1990.